

FEM Practices

Jan Tomek

August 5, 2026

Contents

1	Coordinate Transformations	5
1.1	Constructing Transformation Matrix	6
1.1.1	Rotation Matrix	6
1.1.2	Euler's angles of XYZ rotation	11
1.1.3	Euler's angles of ZYX rotation	13
1.1.4	Euler's angles of XZY rotation	14
1.1.5	Euler's angles of ZXY rotation	15
1.1.6	Euler's angles of YXZ rotation	17
1.1.7	Euler's angles of YZX rotation	18
1.1.8	Euler's angles of XYX rotation	19
1.1.9	Euler's angles of XZX rotation	20
1.1.10	Euler's angles of YXY rotation	21
1.1.11	Euler's angles of YZY rotation	22
1.1.12	Euler's angles of ZXZ rotation	22
1.1.13	Euler's angles of ZYZ rotation	23

1.2	Coordinate Transformation	24
1.3	Tensor Transformation	24
1.4	Moment of Inertia	42
1.5	Quaternions	45
1.5.1	Quaternion properties	45
1.5.2	Scalar and vector parts	47
1.5.3	Conjugation, the norm and reciprocal	48
1.5.4	Unit quaternion (or versor)	50
1.5.5	Quaternions and three-dimensional geometry	50
1.5.6	Matrix representations	52
1.5.7	Exponential, logarithm and power functions	54
1.5.8	Quaternions used for coordinate transformation	55
2	Boundary conditions not in GCS	61
2.1	Skew boundary conditions	61
2.2	Cartesian CSYS	63
2.3	Cylindrical CSYS	65
3	Multifreedom Constraints	71
3.1	Methods for Imposing MFCs	72
3.1.1	MFC Matrix Forms	73
3.1.2	The Example Structure	75
3.2	The Master-Slave Method	76

<i>CONTENTS</i>	5
3.2.1 A One-Constraint Example	76
3.2.2 Multiple Homogeneous MFCs	78
3.2.3 Nonhomogeneous MFCs	80
3.2.4 The General Case	81
3.2.5 Retaining the Original Freedoms	82
3.2.6 Model Reduction by Kinematic Constraints	83
3.2.7 Assesment of the Master-Slave Method	85
3.3 The Penalty Method	88
3.3.1 Choosing the Penalty Weight	89
3.3.2 The Square Root Rule	90
3.3.3 Penalty Elements for General MFCs	91
3.3.4 The Theory Behind the Penalty Method Coefficient Matrix	93
3.3.5 Assesment of the Penalty Method	94
3.4 Lagrange Multiplier Adjuction	95
3.4.1 Physical Interpretation	95
3.4.2 Lagrange Multipliers fro General MFCs	97
3.4.3 The Theory Behind	98
3.4.4 Assesment of the Lagrange Multiplier Method	98
3.4.5 The Augmented Lagrangian Method	99
3.5 Summary	100
4 Rigid Body Elements	101
4.1 RBE2	102

4.1.1	RBE2 Coefficients Example:	104
4.1.2	Editing the Master Stiffness Matrix	107
4.2	RBE3	112
5	Governing Equations	115
5.1	Material Matrices	115
6	Dynamics	117
6.1	Function properties	117
6.2	SDOF system	118
6.2.1	Definition	118
6.2.2	Equation of Motion for SDOF Systems	119
6.2.3	Time Solution for Unforced Undamped SDOF Systems . .	119
6.2.4	Time Solution for Unforced Damped SDOF Systems . . .	121
6.2.5	SDOF Systems under Harmonic Excitaion	128
6.3	Damping	134
6.3.1	Rayleigh Damping	134
6.4	Modal decomposition	140
6.4.1	Definition	140
6.4.2	Orthonormalised Modal Base	141
6.4.3	MDOF to SDOF	143
6.4.4	Initial Conditions	145

<i>CONTENTS</i>	7
7 Kinematics	147
7.1 Rigid Body Motion	147
8 Iterative Methods	149
8.1 Root finding problems	149
8.1.1 Incremental Search Method	149
8.1.2 Bisection Method	151
8.1.3 Methods Based on Linear Interpolation	153
8.2 Minimizing function of Multiple variables	167
8.3 second order ODE's	173
8.4 Explicit	174
8.4.1 Euler's method	174
8.4.2 Runge-Kutta 4th order method	181
8.5 Implicit	187
8.5.1 Newton-Raphson	188
9 Model reduction	193
9.1 Guyan reduction	193
9.2 Craig-Bampton reduction	195
9.2.1 Constraint modes	195
9.2.2 Fixed-interface normal modes	196
9.2.3 Craig-Bampton transformation	197
9.3 Flexible body modal reduction	199

10.6 TET4 Element	260
10.6.1 Element Formulation	260
10.6.2 Python Implementation	261
11 Numerics	269
11.1 Numerical Spaces	269
11.1.1 Hilbert Space	269
11.1.2 Sobolev Space	269
11.1.3 Banach Space	269
11.1.4 Krylov Space	269
11.2 Orthogonalisation	269
11.2.1 Gram-Schmidt process	269
11.2.2 Modified Gram-Schmidt process	272
11.3 Factorisation	274
11.3.1 Cholesky Factorisation	274
11.3.2 LU Factorisation	280
11.3.3 QR Factorisation	280
11.3.4 Conversion to Hessenberg tridiagonal form using Givens rotations	280
11.3.5 Conversion to Hessenberg tridiagonal form using House- holder reflections	280
11.3.6 Conversion to Hessenberg tridiagonal form using Jacobi rotations	280
11.3.7 Jacobi rotations to diagonal matrix	280

Chapter 1

Coordinate Transformations

1.1 Constructing Transformation Matrix

1.1.1 Rotation Matrix

From: wolfram

1. 2-D (in $\mathbb{R}^2$):

Rotating an object to a new coordinate system:

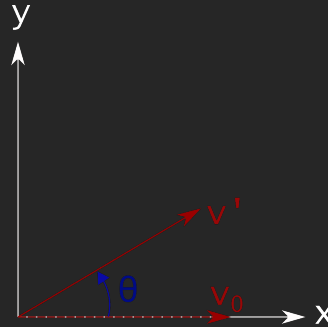


Figure 1.1: Rotating object

In $\mathbb{R}^2$, consider the matrix that rotates a given vector $\mathbf{v}_0$ by a counterclockwise angle θ in a fixed coordinate system. Then

$$\mathbf{T}_\theta = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}, \quad (1.1)$$

so

$$\mathbf{v}' = \mathbf{T}_\theta \mathbf{v}_0. \quad (1.2)$$

On the other hand, consider the matrix that rotates the *coordinate system* through a counterclockwise angle θ . The coordinates of the fixed vector $\mathbf{v}$ in the rotated coordinate system are now given by a rotation matrix which is the *transpose* of the fixed-axis matrix, as can be seen on the second figure, is equivalent to rotating the vector by a counterclockwise angle $-\theta$ relative to a fixed set of axes, giving:

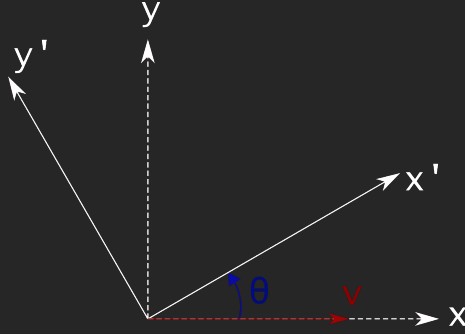


Figure 1.2: Rotating Axes

$$\mathbf{T}'_{\theta} = \begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix}. \quad (1.3)$$

This is the convention commonly used in textbooks.

2. 3-D ($\mathbb{R}^3$):

In $\mathbb{R}^3$, coordinate system rotations of the x -, y - and z -axis in a counter-clockwise direction when looking towards the origin give the matrices:

$$\mathbf{R}_x(\varphi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \varphi & \sin \varphi \\ 0 & -\sin \varphi & \cos \varphi \end{bmatrix}. \quad (1.4)$$

$$\mathbf{R}_y(\theta) = \begin{bmatrix} \cos \theta & 0 & -\sin \theta \\ 0 & 1 & 0 \\ \sin \theta & 0 & \cos \theta \end{bmatrix}. \quad (1.5)$$

$$\mathbf{R}_z(\psi) = \begin{bmatrix} \cos \psi & \sin \psi & 0 \\ -\sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix}. \quad (1.6)$$

It is best to think about a rotation around axis as about a **rotation in a plane**. Therefore rotation around $\mathbf{x}$ is rotation in $\mathbf{yz}$ plane, rotation around $\mathbf{y}$ is a rotation in $\mathbf{zx}$ plane and a rotation around $\mathbf{z}$ is a rotation in $\mathbf{xy}$ plane.

The defining the index of the axes such that $\mathbf{x} = 0$, $\mathbf{y} = 1$ and $\mathbf{z} = 2$ we can define the rotation matrix so that:

$$\mathbf{R} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (1.7)$$

then

$$\begin{aligned} \mathbf{R}_{i,i} &= \cos \theta \\ \mathbf{R}_{i,j} &= \sin \theta \\ \mathbf{R}_{j,i} &= -\sin \theta \\ \mathbf{R}_{j,j} &= \cos \theta \end{aligned} \quad (1.8)$$

where i is index of first plane axis and j is the index of the second plane axis.

The plane axes always cycle in the following manner: $xy, yz, zx, xy, \dots$

Any **rotation** can be given as a composition of rotations about three axes (**Euler's rotation theorem**), and thus be represented by a 3×3 matrix operating on a vector.

$$\begin{bmatrix} x'_1 \\ x'_2 \\ x'_3 \end{bmatrix} = \begin{bmatrix} t_{11} & t_{12} & t_{13} \\ t_{21} & t_{22} & t_{23} \\ t_{31} & t_{32} & t_{33} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}. \quad (1.9)$$

An **arbitrary rotation** around a unit vector $\mathbf{U} = (U_x, U_y, U_z)$ by an angle θ is given by:

$$R_U(\theta) = \mathbf{I} + (\sin \theta)S + (1 - \cos \theta)S^2 \quad (1.10)$$

where $\mathbf{I}$ is an identity matrix,

$$\mathbf{S} = \begin{bmatrix} 0 & -U_x & U_y \\ U_z & 0 & -U_x \\ -U_y & U_x & 0 \end{bmatrix} \quad (1.11)$$

and $\theta > 0$ indicates a counterclockwise rotation in the plane normal to $\mathbf{U}$. The observer is positioned on the side of the plane to which the vector points and looking back at the origin.

3. Alternate way of constructing **Rotation matrix** without specifying angles:

The rotation matrix to a new coordinate system can be constructed from its axes, where the rotation matrix can be defined as:

$$\mathbf{T}_R = \begin{bmatrix} x_1 & x_2 & x_3 \\ y_1 & y_2 & y_3 \\ z_1 & z_2 & z_3 \end{bmatrix}, \quad (1.12)$$

where $\mathbf{x} = [x_1, x_2, x_3]^T$, $\mathbf{y} = [y_1, y_2, y_3]^T$ and $\mathbf{z} = [z_1, z_2, z_3]^T$ are unit vectors in the direction of x -, y - and z -axis of the new coordinate system.

The **transformation matrix** can be defined by any two vectors $\mathbf{u}$ and $\mathbf{v}$, where $\mathbf{u}$ defines the direction of *x-axis* and $\mathbf{v}$ defines the *xy-plane*. To construct the **rotation matrix** use the following steps:

- (a) create x-axis of unit length

$$\mathbf{x} = \frac{\mathbf{u}}{\|\mathbf{u}\|}. \quad (1.13)$$

- (b) create z-axis of unit length

$$\mathbf{z} = \frac{\mathbf{u} \times \mathbf{v}}{\|\mathbf{u} \times \mathbf{v}\|}. \quad (1.14)$$

- (c) create y-axis of unit length

$$\mathbf{y} = \frac{\mathbf{z} \times \mathbf{x}}{\|\mathbf{z} \times \mathbf{x}\|}. \quad (1.15)$$

- (d) compose **rotation matrix**

$$\mathbf{T}_R = \begin{bmatrix} \mathbf{x}^T \\ \mathbf{y}^T \\ \mathbf{z}^T \end{bmatrix}. \quad (1.16)$$

The creation of the **rotation matrix** by any two other vectors is analogic, as **orthogonal** base must be constructed first, then the matrix is composed of its vectors.

To test, whether the transformation matrix is correct, one can use the orthogonality criterion:

$$\mathbf{T}^T = \mathbf{T}^{-1} \quad (1.17)$$

$$\mathbf{T}\mathbf{T}^T = \mathbf{T}^T\mathbf{T} = \mathbf{I} \quad (1.18)$$

or

$$\det(\mathbf{T}) = 1 \quad (1.19)$$

1.1.2 Euler's angles of XYZ rotation

With the order of *xyz* rotation the rotation matrix is composed as:

$$\begin{aligned} \mathbf{T} &= \mathbf{R}_z(\psi)\mathbf{R}_y(\theta)\mathbf{R}_x(\varphi) \\ &= \begin{bmatrix} \cos \psi & \sin \psi & 0 \\ -\sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \theta & 0 & -\sin \theta \\ 0 & 1 & 0 \\ \sin \theta & 0 & \cos \theta \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \varphi & \sin \varphi \\ 0 & -\sin \varphi & \cos \varphi \end{bmatrix} \\ &= \begin{bmatrix} \cos \theta \cos \psi & \cos \psi \sin \theta \sin \varphi - \cos \varphi \sin \psi & \cos \varphi \cos \psi \sin \theta + \sin \varphi \sin \psi \\ \cos \theta \sin \psi & \cos \varphi \cos \psi + \sin \theta \sin \varphi \sin \psi & \cos \varphi \sin \theta \sin \psi - \cos \psi \sin \varphi \\ -\sin \theta & \cos \theta \sin \varphi & \cos \theta \cos \varphi \end{bmatrix} \end{aligned} \quad (1.20)$$

Then the three *Euler angles* for a *xyz* rotation can be obtained:

$$\varphi = \text{atan2}(t_{32}, t_{33}), \quad (1.21)$$

$$\theta = \text{atan2} \left(-t_{31}, \sqrt{t_{32}^2 + t_{33}^2} \right), \quad (1.22)$$

$$\psi = \text{atan2}(t_{21}, t_{11}), \quad (1.23)$$

where $\text{atan2}(y, x)$ definition is:

$$\text{atan2}(y, x) = \left\{ \begin{array}{ll} \arctan\left(\frac{y}{x}\right) & \text{if } (x > 0), \\ \arctan\left(\frac{y}{x}\right) + \pi & \text{if } (x < 0) \wedge (y \geq 0), \\ \arctan\left(\frac{y}{x}\right) - \pi & \text{if } (x < 0) \wedge (y < 0), \\ \frac{\pi}{2} & \text{if } (x = 0) \wedge (y > 0), \\ -\frac{\pi}{2} & \text{if } (x = 0) \wedge (y < 0), \\ \text{undefined} & \text{if } (x = 0) \wedge (y = 0) \end{array} \right\}. \quad (1.24)$$

Note the angle limitation for values returned while doing an Euler angle decomposition of a **rotation** matrix:

$$\begin{aligned} \varphi &\in (-\pi, \pi) \\ \theta &\in \left(-\frac{\pi}{2}, \frac{\pi}{2}\right) \\ \psi &\in (-\pi, \pi) \end{aligned}. \quad (1.25)$$

Alternative way to obtain the angles:

$$\left\{ \begin{array}{l} \varphi = \text{atan2}(-t_{21}, t_{22}) \\ \theta = (t_{20}) \\ \psi = \text{atan2}(-t_{10}, t_{00}) \end{array} \right. \quad (1.26)$$

A python implementation with alternative components used to obtain the angles

```

1 import numpy as np
2
3 def EulerXYZ(R: np.ndarray) -> tuple:
4     if R[2,0] < 1.:
5         if R[2,0] > -1.:
6             b = np.arcsin(R[2,0])
7             a = np.arctan2(-R[2,1], R[2,2])
8             c = np.arctan2(-R[1,0], R[0,0])
9         else:
10            b = -np.pi / 2.
11            a = -np.arctan2(R[0,1], R[1,1])
12            c = 0.
```

```

13  else:
14      b = np.pi / 2.
15      a = np.arctan2(R[0,1], R[1,1])
16      c = 0.
17
18  return a, b, c

```

1.1.3 Euler's angles of ZYX rotation

For a rotation defined in *zyx* order the rotation matrix is:

$$\begin{aligned}
 \mathbf{T} &= \mathbf{R}_x(\varphi)\mathbf{R}_y(\theta)\mathbf{R}_z(\psi) \\
 &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \varphi & \sin \varphi \\ 0 & -\sin \varphi & \cos \varphi \end{bmatrix} \begin{bmatrix} \cos \theta & 0 & -\sin \theta \\ 0 & 1 & 0 \\ \sin \theta & 0 & \cos \theta \end{bmatrix} \begin{bmatrix} \cos \psi & \sin \psi & 0 \\ -\sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix} \\
 &= \begin{bmatrix} \cos \theta \cos \varphi & -\cos \theta \sin \varphi & \sin \varphi \\ \cos \psi \sin \varphi + \cos \varphi \sin \theta \sin \psi & \cos \varphi \cos \psi - \sin \theta \sin \varphi \sin \psi & -\cos \theta \sin \psi \\ \sin \varphi \sin \psi - \cos \varphi \cos \psi \sin \theta & \cos \psi \sin \theta \sin \varphi + \cos \varphi \sin \psi & \cos \theta \cos \psi \end{bmatrix}. \tag{1.27}
 \end{aligned}$$

Note that the angles φ , θ and ψ defined here are different from the *XYZ* rotation defined above! The rotation is performed by first rotating around *z-axis* counterclockwise by an angle ψ , then around *y-axis* counterclockwise by an angle θ and finally around *x-axis*, counterclockwise, by an angle φ .

Then the *Euler's angles* can be obtained as:

$$\begin{cases} \varphi = \arctan\left(\frac{-t_{12}}{t_{11}}\right) \\ \theta = \arcsin(t_{13}) \\ \psi = \arctan\left(\frac{-t_{23}}{t_{33}}\right) \end{cases}. \tag{1.28}$$

Alternative way to obtain the angles:

$$\begin{cases} \varphi = \text{atan2}(t_{12}, t_{22}) \\ \theta = (-t_{02}) \\ \psi = \text{atan2}(t_{01}, t_{00}) \end{cases} \tag{1.29}$$

A python implementation with alternative components used to obtain the angles

```

1 import numpy as np
2
3 def EulerZYX(R: np.ndarray) -> tuple:
4     if R[0,2] < 1.:
5         if R[0,2] > -1.:
6             b = np.arcsin(-R[0,2])
7             a = np.arctan2(R[0,1], R[0,0])
8             c = np.arctan2(R[1,2], R[2,2])
9         else:
10            b = np.pi / 2.
11            a = -np.arctan2(-R[2,1], R[1,1])
12            c = 0.
13    else:
14        b = -np.pi / 2.
15        a = np.arctan2(-R[2,1], R[1,1])
16        c = 0.
17
18    return a, b, c

```

1.1.4 Euler's angles of XZY rotation

For a rotation defined in xzy order the rotation matrix is:

$$\begin{aligned}
 \mathbf{T} &= \mathbf{R}_x(\varphi)\mathbf{R}_z(\psi)\mathbf{R}_y(\theta) \\
 &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \varphi & \sin \varphi \\ 0 & -\sin \varphi & \cos \varphi \end{bmatrix} \begin{bmatrix} \cos \psi & \sin \psi & 0 \\ -\sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \theta & 0 & -\sin \theta \\ 0 & 1 & 0 \\ \sin \theta & 0 & \cos \theta \end{bmatrix} \\
 &= \begin{bmatrix} \cos \theta \cos \psi & \sin \varphi \sin \theta + \cos \varphi \cos \theta \sin \psi & -\cos \varphi \sin \theta + \cos \theta \sin \varphi \sin \psi \\ -\sin \psi & \cos \varphi \cos \psi & \cos \psi \sin \varphi \\ \cos \psi \sin \theta & -\cos \theta \sin \varphi + \cos \varphi \sin \theta \sin \psi & \cos \varphi \cos \theta + \sin \varphi \sin \theta \sin \psi \end{bmatrix}.
 \end{aligned} \tag{1.30}$$

The the angles are:

$$\begin{cases} \varphi = \text{atan2}(t_{12}, t_{11}) \\ \theta = \text{atan2}(t_{20}, t_{00}) \\ \psi = (-t_{10}) \end{cases} \quad (1.31)$$

A python implementation used to obtain the angles

```

1 import numpy as np
2
3 def EulerXZY(R: np.ndarray) -> tuple:
4     if R[1,0] < 1.:
5         if R[1,0] > -1.:
6             b = np.arcsin(-R[1,0])
7             a = np.arctan2(R[1,2], R[1,1])
8             c = np.arctan2(R[2,0], R[0,0])
9         else:
10            b = np.pi / 2.
11            a = -np.arctan2(-R[0,2], R[2,2])
12            c = 0.
13     else:
14         b = -np.pi / 2.
15         a = np.arctan2(-R[0,2], R[2,2])
16         c = 0.
17
18     return a, b, c

```

1.1.5 Euler's angles of ZXY rotation

For a rotation defined in *zxy* order the rotation matrix is:

$$\begin{aligned}
\mathbf{T} &= \mathbf{R}_z(\psi)\mathbf{R}_x(\varphi)\mathbf{R}_y(\theta) \\
&= \begin{bmatrix} \cos \psi & \sin \psi & 0 \\ -\sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \varphi & \sin \varphi \\ 0 & -\sin \varphi & \cos \varphi \end{bmatrix} \begin{bmatrix} \cos \theta & 0 & -\sin \theta \\ 0 & 1 & 0 \\ \sin \theta & 0 & \cos \theta \end{bmatrix} \\
&= \begin{bmatrix} \cos \theta \cos \psi - \sin \varphi \sin \theta \sin \psi & \cos \psi \sin \varphi \sin \theta + \cos \theta \sin \psi & -\cos \varphi \sin \theta \\ -\cos \varphi \sin \psi & \cos \varphi \cos \psi & \sin \varphi \\ \cos \psi \sin \theta + \cos \theta \sin \varphi \sin \psi & -\cos \theta \cos \psi \sin \theta + \sin \theta \sin \psi & \cos \varphi \cos \theta \end{bmatrix}.
\end{aligned} \tag{1.32}$$

The the angles are:

$$\begin{cases} \varphi = \text{atan2}(t_{12}, t_{11}) \\ \theta = \text{atan2}(t_{20}, t_{00}) \\ \psi = (-t_{10}) \end{cases} \tag{1.33}$$

Python example of algorithm:

```

1 import numpy as np
2
3 def EulerZXY(R: np.ndarray) -> tuple:
4     if R[1,2] < 1.:
5         if R[1,2] > -1.:
6             b = np.arcsin(R[1,2])
7             a = np.arctan2(-R[1,0], R[1,1])
8             c = np.arctan2(-R[0,2], R[2,2])
9         else:
10            b = -np.pi / 2.
11            a = -np.arctan2(R[2,0], R[0,0])
12            c = 0.
13     else:
14         b = np.pi / 2.
15         a = np.arctan2(R[2,0], R[0,0])
16         c = 0.
17
18     return a, b, c

```

1.1.6 Euler's angles of YXZ rotation

For a rotation defined in yxz order the rotation matrix is:

$$\begin{aligned}
 \mathbf{T} &= \mathbf{R}_y(\theta)\mathbf{R}_x(\varphi)\mathbf{R}_z(\psi) \\
 &= \begin{bmatrix} \cos \theta & 0 & -\sin \theta \\ 0 & 1 & 0 \\ \sin \theta & 0 & \cos \theta \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \varphi & \sin \varphi \\ 0 & -\sin \varphi & \cos \varphi \end{bmatrix} \begin{bmatrix} \cos \psi & \sin \psi & 0 \\ -\sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix} \\
 &= \begin{bmatrix} \cos \theta \cos \psi + \sin \varphi \sin \theta \sin \psi & \cos \varphi \sin \psi & -\cos \psi \sin \theta + \cos \theta \sin \varphi \sin \psi \\ \cos \psi \sin \varphi \sin \theta - \cos \theta \sin \psi & \cos \varphi \cos \psi & \cos \theta \cos \psi \sin \varphi + \sin \theta \sin \psi \\ \cos \varphi \sin \theta & -\sin \varphi & \cos \varphi \cos \theta \end{bmatrix}.
 \end{aligned} \tag{1.34}$$

The the angles are:

$$\begin{cases} \varphi = (-t_{21}) \\ \theta = \text{atan2}(t_{20}, t_{22}) \\ \psi = \text{atan2}(t_{01}, t_{11}) \end{cases} \tag{1.35}$$

Python example of algorithm:

```

1
2 import numpy as np
3
4 def EulerYXZ(R: np.ndarray) -> tuple:
5     if R[2,1] < 1.:
6         if R[2,1] > -1.:
7             b = np.arcsin(-R[2,1])
8             a = np.arctan2(R[2,0], R[2,2])
9             c = np.arctan2(R[0,1], R[1,1])
10        else:
11            b = np.pi / 2.
12            a = -np.arctan2(-R[1,0], R[0,0])
13            c = 0.
14    else:
15        b = -np.pi / 2.
16        a = np.arctan2(-R[1,0], R[0,0])

```

```

17         c = 0.
18
19     return a, b, c

```

1.1.7 Euler's angles of YZX rotation

For a rotation defined in yzx order the rotation matrix is:

$$\begin{aligned}
 \mathbf{T} &= \mathbf{R}_y(\theta)\mathbf{R}_x(\varphi)\mathbf{R}_z(\psi) \\
 &= \begin{bmatrix} \cos \theta & 0 & -\sin \theta \\ 0 & 1 & 0 \\ \sin \theta & 0 & \cos \theta \end{bmatrix} \begin{bmatrix} \cos \psi & \sin \psi & 0 \\ -\sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \varphi & \sin \varphi \\ 0 & -\sin \varphi & \cos \varphi \end{bmatrix} \\
 &= \begin{bmatrix} \cos \theta \cos \psi & \sin \psi & -\cos \psi \sin \theta \\ \sin \varphi \sin \theta - \cos \varphi \cos \theta \sin \psi & \cos \varphi \cos \psi & \cos \theta \sin \varphi + \cos \varphi \sin \theta \sin \psi \\ \cos \varphi \sin \theta + \cos \theta \sin \varphi \sin \psi & -\cos \psi \sin \varphi & \cos \varphi \cos \theta - \sin \varphi \sin \theta \sin \psi \end{bmatrix}.
 \end{aligned} \tag{1.36}$$

The the angles are:

$$\begin{cases} \varphi = \text{atan2}(-t_{12}, t_{11}) \\ \theta = \text{atan2}(-t_{02}, t_{00}) \\ \psi = (t_{01}) \end{cases} \tag{1.37}$$

Python example of algorithm:

```

1  import numpy as np
2
3  def EulerYZX(R: np.ndarray) -> tuple:
4      if R[0,1] < 1.:
5          if R[0,1] > -1.:
6              b = np.arcsin(R[0,1])
7              a = np.arctan2(-R[0,2], R[0,0])
8              c = np.arctan2(-R[2,1], R[1,1])
9          else:
10             b = -np.pi / 2.

```



```

11         a = -np.arctan2(R[1,2], R[2,2])
12         c = 0.
13     else:
14         b = np.pi / 2.
15         a = np.arctan2(R[1,2], R[2,2])
16         c = 0.
17
18     return a, b, c

```

1.1.8 Euler's angles of XYX rotation

For a rotation defined in xyx order the rotation matrix is:

$$\begin{aligned}
 \mathbf{T} &= \mathbf{R}_x(\varphi_1) \mathbf{R}_y(\theta) \mathbf{R}_x(\varphi_0) \\
 &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \varphi_1 & \sin \varphi_1 \\ 0 & -\sin \varphi_1 & \cos \varphi_1 \end{bmatrix} \begin{bmatrix} \cos \psi & \sin \psi & 0 \\ -\sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \varphi_0 & \sin \varphi_0 \\ 0 & -\sin \varphi_0 & \cos \varphi_0 \end{bmatrix} \\
 &= \begin{bmatrix} \cos \psi & \sin \psi \cos \varphi_1 & \sin \psi \sin \varphi_1 \\ -\sin \psi \cos \varphi_0 & \cos \psi \cos \varphi_0 \cos \varphi_1 - \sin \varphi_0 \sin \varphi_1 & \cos \psi \cos \varphi_0 \sin \varphi_1 + \cos \varphi_1 \sin \varphi_0 \\ \sin \psi \sin \varphi_0 & -\cos \varphi_0 \sin \varphi_1 - \cos \psi \cos \varphi_1 \sin \varphi_0 & \cos \varphi_0 \cos \varphi_1 - \cos \psi \sin \varphi_0 \sin \varphi_1 \end{bmatrix}.
 \end{aligned} \tag{1.38}$$

The the angles are:

$$\begin{cases} \varphi_0 = \text{atan2}(t_{02}, -t_{10}) \\ \psi = (t_{00}) \\ \varphi_1 = \text{atan2}(t_{02}, t_{01}) \end{cases} \tag{1.39}$$

Python example of algorithm:

```

1 import numpy as np
2
3 def EulerXYX(R: np.ndarray) -> tuple:
4     if R[0,0] < 1.:
5         if R[0,0] > -1.:

```

```

6         b = np.arccos(R[0,0])
7         a = np.arctan2(R[0,1], -R[0,2])
8         c = np.arctan2(R[1,0], R[2,0])
9     else:
10        b = np.pi
11        a = -np.arctan2(-R[2,1], R[1,1])
12        c = 0.
13    else:
14        b = 0.
15        a = np.arctan2(-R[2,1], R[1,1])
16        c = 0.
17
18    return a, b, c

```

1.1.9 Euler's angles of XZX rotation

For a rotation defined in xzx order the rotation matrix is:

$$\begin{aligned}
 \mathbf{T} &= \mathbf{R}_x(\varphi_1)\mathbf{R}_z(\psi)\mathbf{R}_x(\varphi_0) \\
 &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \varphi_1 & \sin \varphi_1 \\ 0 & -\sin \varphi_1 & \cos \varphi_1 \end{bmatrix} \begin{bmatrix} \cos \theta & 0 & -\sin \theta \\ 0 & 1 & 0 \\ \sin \theta & 0 & \cos \theta \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \varphi_0 & \sin \varphi_0 \\ 0 & -\sin \varphi_0 & \cos \varphi_0 \end{bmatrix} \\
 &= \begin{bmatrix} \cos \theta & \sin \theta \sin \varphi_1 & -\sin \theta \cos \varphi_1 \\ \sin \theta \sin \varphi_0 & \cos \varphi_0 \cos \varphi_1 - \cos \theta \sin \varphi_0 \sin \varphi_1 & \cos \varphi_0 \sin \varphi_1 + \cos \theta \cos \varphi_1 \sin \varphi_0 \\ \sin \theta \cos \varphi_0 & -\cos \theta \cos \varphi_0 \sin \varphi_1 - \cos \varphi_1 \sin \varphi_0 & \cos \theta \cos \varphi_0 \cos \varphi_1 - \sin \varphi_0 \sin \varphi_1 \end{bmatrix}.
 \end{aligned} \tag{1.40}$$

The the angles are:

$$\begin{cases} \varphi_0 = \text{atan2}(t_{10}, t_{20}) \\ \theta = (t_{00}) \\ \varphi_1 = \text{atan2}(t_{01}, -t_{02}) \end{cases} \tag{1.41}$$

Python example of algorithm:

```

1 import numpy as np

```

```

2
3 def EulerXZX(R: np.ndarray) -> tuple:
4     if R[0,0] < 1.:
5         if R[0,0] > -1.:
6             b = np.arccos(R[0,0])
7             a = np.arctan2(R[0,2], R[0,1])
8             c = np.arctan2(R[2,0], -R[1,0])
9         else:
10            b = np.pi
11            a = -np.arctan2(R[1,2], R[2,2])
12            c = 0.
13     else:
14         b = 0.
15         a = np.arctan2(R[1,2], R[2,2])
16         c = 0.
17
18     return a, b, c

```

1.1.10 Euler's angles of YXY rotation

```

1 import numpy as np
2
3 def EulerYXY(R: np.ndarray) -> tuple:
4     if R[1,1] < 1.:
5         if R[1,1] > -1.:
6             b = np.arccos(R[1,1])
7             a = np.arctan2(R[1,0], R[1,2])
8             c = np.arctan2(R[0,1], -R[2,1])
9         else:
10            b = np.pi
11            a = -np.arctan2(R[2,0], R[0,0])
12            c = 0.
13     else:
14         b = 0.
15         a = np.arctan2(R[2,0], R[0,0])
16         c = 0.
17

```

```
18     return a, b, c
```

1.1.11 Euler's angles of YZY rotation

```
1  import numpy as np
2
3  def EulerYZY(R: np.ndarray) -> tuple:
4      if R[1,1] < 1.:
5          if R[1,1] > -1.:
6              b = np.arccos(R[1,1])
7              a = np.arctan2(R[1,2], -R[1,0])
8              c = np.arctan2(R[2,1], R[0,1])
9          else:
10             b = np.pi
11             a = -np.arctan2(-R[0,2], R[2,2])
12             c = 0.
13     else:
14         b = 0.
15         a = np.arctan2(-R[0,2], R[2,2])
16         c = 0.
17
18     return a, b, c
```

1.1.12 Euler's angles of ZXZ rotation

Note: One of the most used forms of the **Euler** angles.

```
1  import numpy as np
2
3  def EulerZXZ(R: np.ndarray) -> tuple:
4      if R[2,2] < 1.:
5          if R[2,2] > -1.:
6              b = np.arccos(R[2,2])
```

```

7         a = np.arctan2(R[2,0], -R[2,1])
8         c = np.arctan2(R[0,2], R[1,2])
9     else:
10        b = np.pi
11        a = -np.arctan2(-R[1,0], R[0,0])
12        c = 0.
13    else:
14        b = 0.
15        a = np.arctan2(-R[1,0], R[0,0])
16        c = 0.
17
18    return a, b, c

```

1.1.13 Euler's angles of ZYZ rotation

```

1 import numpy as np
2
3 def EulerZYZ(R: np.ndarray) -> tuple:
4     if R[2,2] < 1.:
5         if R[2,2] > -1.:
6             b = np.arccos(R[2,2])
7             a = np.arctan2(R[2,1], R[2,0])
8             c = np.arctan2(R[1,2], -R[0,2])
9         else:
10            b = np.pi
11            a = -np.arctan2(R[0,1], R[1,1])
12            c = 0.
13    else:
14        b = 0.
15        a = np.arctan2(R[0,1], R[1,1])
16        c = 0.
17
18    return a, b, c

```

1.2 Coordinate Transformation

- **Rotation:**

Let $\mathbf{A}^T = [a_x, a_y, a_z]$ be the original coordinates, $\mathbf{B}^T = [b_x, b_y, b_z]$ be the transformed coordinates, and $\mathbf{T}$ a tranformation matrix in the form:

$$\mathbf{T} = \begin{bmatrix} x_1 & x_2 & x_3 \\ y_1 & y_2 & y_3 \\ z_1 & z_2 & z_3 \end{bmatrix} = \begin{bmatrix} \mathbf{x} \\ \mathbf{y} \\ \mathbf{z} \end{bmatrix}, \quad (1.42)$$

where $\mathbf{x} = [x_1, x_2, x_3]$, $\mathbf{y} = [y_1, y_2, y_3]$ and $\mathbf{z} = [z_1, z_2, z_3]$ are the *x-axis*, *y-axis* and *z-axis* **unit vectors**, respectively, of the new coordinate system defined in the original one, while sharing their origin.

The following applies:

$$\mathbf{T} \times \mathbf{A} = \mathbf{B}, \quad (1.43)$$

and

$$\mathbf{T}^T \times \mathbf{B} = \mathbf{A}. \quad (1.44)$$

Note: Perpendicular vector is created using *cross-product*.

1.3 Tensor Transformation

From: source 1, source 2, source 3

Let real symmetric matrix $\mathbf{A}$ be the original tensor, real symmetric matrix $\mathbf{B}$ the transformed tensor, and orthogonal matrix $\mathbf{T}$ a tranformation matrix in the form:

$$\mathbf{A} = \begin{bmatrix} \sigma_{11}^A & \sigma_{12}^A & \sigma_{13}^A \\ \sigma_{21}^A & \sigma_{22}^A & \sigma_{23}^A \\ \sigma_{31}^A & \sigma_{32}^A & \sigma_{33}^A \end{bmatrix}, \quad (1.45)$$

$$\mathbf{B} = \begin{bmatrix} \sigma_{11}^B & \sigma_{12}^B & \sigma_{13}^B \\ \sigma_{21}^B & \sigma_{22}^B & \sigma_{23}^B \\ \sigma_{31}^B & \sigma_{32}^B & \sigma_{33}^B \end{bmatrix}, \quad (1.46)$$

$$\mathbf{T} = \begin{bmatrix} x_1 & x_2 & x_3 \\ y_1 & y_2 & y_3 \\ z_1 & z_2 & z_3 \end{bmatrix} = \begin{bmatrix} \mathbf{x} \\ \mathbf{y} \\ \mathbf{z} \end{bmatrix}, \quad (1.47)$$

where $\mathbf{x} = [x_1, x_2, x_3]$, $\mathbf{y} = [y_1, y_2, y_3]$ and $\mathbf{z} = [z_1, z_2, z_3]$ are the *x-axis*, *y-axis* and *z-axis* **unit vectors**, respectively, of the new coordinate system defined in the original one, while sharing their origin.

The transformation matrix $\mathbf{T}$ must satisfy $\mathbf{T}\mathbf{T}^T = \mathbf{T}^T\mathbf{T} = \mathbf{I}$, where $\mathbf{I}$ is an identity matrix. This means that transformation matrix must be orthogonal, therefore satisfying $\mathbf{T}^T = \mathbf{T}^{-1}$.

Then **tensor transformation** is performed as such:

$$\mathbf{T} \times \mathbf{A} \times \mathbf{T}^T = \mathbf{B}. \quad (1.48)$$

$$\mathbf{T}^T \times \mathbf{B} \times \mathbf{T} = \mathbf{A}. \quad (1.49)$$

- **Principal values:**

The principal values can be found as the **eigenvalues** of the tensor matrix.

Characteristic equation of tensor $\mathbf{\Sigma}$:

$$\mathbf{\Sigma}\mathbf{V} = \mathbf{\Lambda}\mathbf{V}, \quad (1.50)$$

therefore:

$$(\mathbf{\Sigma} - \mathbf{\Lambda}\mathbf{I})\mathbf{V} = 0, \quad (1.51)$$

and:

$$\det(\mathbf{\Sigma} - \mathbf{\Lambda}\mathbf{I}) = 0, \quad (1.52)$$

where $\mathbf{\Lambda}$ is a diagonal matrix of eigenvalues (principal values) and $\mathbf{V}$ is the eigenvector matrix.

The characteristic cubic equation can be also written as:

$$\lambda^3 - I_1\lambda^2 + I_2\lambda - I_3 = 0, \quad (1.53)$$

giving three roots equal to tensor eigenvalues.

First compute **Invariants** $\mathbf{I}_n$:

$$\begin{aligned} I_1 &= \sigma_{11} + \sigma_{22} + \sigma_{33} \\ I_2 &= \sigma_{11}\sigma_{22} + \sigma_{22}\sigma_{33} + \sigma_{33}\sigma_{11} - |\sigma_{12}|^2 - |\sigma_{13}|^2 - |\sigma_{23}|^2 \\ I_3 &= -\sigma_{11}|\sigma_{23}|^2 - \sigma_{22}|\sigma_{13}|^2 - \sigma_{33}|\sigma_{12}|^2 + \sigma_{11}\sigma_{22}\sigma_{33} + 2\text{Re}(\sigma_{12}\sigma_{13}\sigma_{23}) \end{aligned} \quad (1.54)$$

where $\text{Re}()$ means **real part** in case some of the values are complex.

In matrix form:

$$\begin{aligned} I_1 &= \text{tr}[\mathbf{\Sigma}] \\ I_2 &= \begin{vmatrix} \sigma_{11} & \sigma_{12} \\ \sigma_{12} & \sigma_{22} \end{vmatrix} + \begin{vmatrix} \sigma_{11} & \sigma_{13} \\ \sigma_{13} & \sigma_{33} \end{vmatrix} + \begin{vmatrix} \sigma_{22} & \sigma_{23} \\ \sigma_{23} & \sigma_{33} \end{vmatrix} \\ I_3 &= \det(\mathbf{\Sigma}) \end{aligned} \quad (1.55)$$

Then compute *help values*:

$$Q = \frac{3I_2 - I_1^2}{9}, \quad (1.56)$$

$$R = \frac{2I_1^3 - 9I_1I_2 + 27I_3}{54}, \quad (1.57)$$

$$\theta = \cos^{-1} \left(\frac{R}{\sqrt{-Q^3}} \right). \quad (1.58)$$

Lastly to get **principal values**:

$$\bar{\sigma}_1 = 2\sqrt{-Q} \cos\left(\frac{\theta}{3}\right) + \frac{1}{3}I_1. \quad (1.59)$$

$$\bar{\sigma}_2 = 2\sqrt{-Q} \cos\left(\frac{\theta + 2\pi}{3}\right) + \frac{1}{3}I_1. \quad (1.60)$$

$$\bar{\sigma}_3 = 2\sqrt{-Q} \cos\left(\frac{\theta + 4\pi}{3}\right) + \frac{1}{3}I_1. \quad (1.61)$$

The **principal values are not sorted**. The principal tensor is then:

$$\boldsymbol{\Sigma} = \begin{bmatrix} \sigma_1 & 0 & 0 \\ 0 & \sigma_2 & 0 \\ 0 & 0 & \sigma_3 \end{bmatrix}, \quad (1.62)$$

where $\bar{\sigma}_1, \bar{\sigma}_2$ and $\bar{\sigma}_3 \implies \sigma_1 > \sigma_2 > \sigma_3$.

Afterwards the **principal axes** are obtained as:

$$\begin{aligned} \boldsymbol{\Gamma}_1 &= \boldsymbol{\Sigma} - \sigma_1 \mathbf{I} \\ \boldsymbol{\Gamma}_2 &= \boldsymbol{\Sigma} - \sigma_2 \mathbf{I} , \\ \boldsymbol{\Gamma}_3 &= \boldsymbol{\Sigma} - \sigma_3 \mathbf{I} \end{aligned} \quad (1.63)$$

where $\boldsymbol{\Gamma}$ is a coordinate system corresponding to the principal value i and $\mathbf{I}$ is a 3×3 identity matrix.

The vectors corresponding to each **principal value** are obtained:

$$\boldsymbol{\Gamma}_1 = \begin{bmatrix} \bar{x}_{11} & \bar{x}_{12} & \bar{x}_{13} \\ \bar{y}_{11} & \bar{y}_{12} & \bar{y}_{13} \\ \bar{z}_{11} & \bar{z}_{12} & \bar{z}_{13} \end{bmatrix} = \begin{bmatrix} \bar{\mathbf{x}}_1 \\ \bar{\mathbf{y}}_1 \\ \bar{\mathbf{z}}_1 \end{bmatrix}, \quad (1.64)$$

$$\boldsymbol{\Gamma}_2 = \begin{bmatrix} \bar{x}_{21} & \bar{x}_{22} & \bar{x}_{23} \\ \bar{y}_{21} & \bar{y}_{22} & \bar{y}_{23} \\ \bar{z}_{21} & \bar{z}_{22} & \bar{z}_{23} \end{bmatrix} = \begin{bmatrix} \bar{\mathbf{x}}_2 \\ \bar{\mathbf{y}}_2 \\ \bar{\mathbf{z}}_2 \end{bmatrix}, \quad (1.65)$$

$$\boldsymbol{\Gamma}_3 = \begin{bmatrix} \bar{x}_{31} & \bar{x}_{32} & \bar{x}_{33} \\ \bar{y}_{31} & \bar{y}_{32} & \bar{y}_{33} \\ \bar{z}_{31} & \bar{z}_{32} & \bar{z}_{33} \end{bmatrix} = \begin{bmatrix} \bar{\mathbf{x}}_3 \\ \bar{\mathbf{y}}_3 \\ \bar{\mathbf{z}}_3 \end{bmatrix}, \quad (1.66)$$

where $\bar{\mathbf{x}}$, $\bar{\mathbf{y}}$ and $\bar{\mathbf{z}}$ are vectors of size 3.

$$\mathbf{x} = \frac{\bar{\mathbf{y}}_1 \times \bar{\mathbf{z}}_1}{|\bar{\mathbf{y}}_1 \times \bar{\mathbf{z}}_1|}, \quad (1.67)$$

$$\mathbf{y} = \frac{\bar{\mathbf{z}}_2 \times \bar{\mathbf{x}}_2}{|\bar{\mathbf{z}}_2 \times \bar{\mathbf{x}}_2|}, \quad (1.68)$$

$$\mathbf{z} = \frac{\bar{\mathbf{x}}_3 \times \bar{\mathbf{y}}_3}{|\bar{\mathbf{x}}_3 \times \bar{\mathbf{y}}_3|}. \quad (1.69)$$

The principal axes are also the **eigenvectors** of tensor Σ :

$$\mathbf{V} = [\mathbf{x} \quad \mathbf{y} \quad \mathbf{z}] = \begin{bmatrix} x_1 & y_1 & z_1 \\ x_2 & y_2 & z_2 \\ x_3 & y_3 & z_3 \end{bmatrix}. \quad (1.70)$$

- **Principal values, Cardano's formula derivation and python implementation:**

An efficient way to compute the **roots of the cubic equation** is the **Cardano's formula** or its variations. **Cardano's equation** works only for **depressed** 3rd degree polynomials.

Once again we start with a **real, symmetric** tensor Σ :

$$\Sigma = \begin{bmatrix} \sigma_{xx} & \tau_{xy} & \tau_{xz} \\ \tau_{yx} & \sigma_{yy} & \tau_{yz} \\ \tau_{zx} & \tau_{zy} & \sigma_{zz} \end{bmatrix}, \quad (1.71)$$

Then from the definition of eigenvalues and eigenvectors:

$$\Sigma \mathbf{V} = \Lambda \mathbf{V}, \quad (1.72)$$

therefore:

$$(\Sigma - \Lambda \mathbf{I}) \mathbf{V} = 0, \quad (1.73)$$

and:

$$\det(\Sigma - \Lambda \mathbf{I}) = 0, \quad (1.74)$$

where $\mathbf{\Lambda}$ is a diagonal matrix of eigenvalues (principal values) and $\mathbf{V}$ is the eigenvector matrix.

$$\mathbf{\Lambda}_i = \begin{bmatrix} \lambda_i & 0 & 0 \\ 0 & \lambda_i & 0 \\ 0 & 0 & \lambda_i \end{bmatrix}, \quad (1.75)$$

$$\mathbf{V}_i = \begin{bmatrix} \bar{x}_{i1} & \bar{x}_{i2} & \bar{x}_{i3} \\ \bar{y}_{i1} & \bar{y}_{i2} & \bar{y}_{i3} \\ \bar{z}_{i1} & \bar{z}_{i2} & \bar{z}_{i3} \end{bmatrix} = \begin{bmatrix} \bar{\mathbf{x}}_i \\ \bar{\mathbf{y}}_i \\ \bar{\mathbf{z}}_i \end{bmatrix}. \quad (1.76)$$

It needs to be noted that each **principal value** of a **tensor matrix** corresponds to three vectors that define a coordinate system where first vector is perpendicular to the principal plane.

Rewriting the characteristic cubic equation:

$$\begin{aligned} 0 = & \lambda^3 \\ & + \lambda^2 (-\sigma_{xx} - \sigma_{yy} - \sigma_{zz}) \\ & + \lambda (\sigma_{xx}\sigma_{yy} + \sigma_{xx}\sigma_{zz} + \sigma_{yy}\sigma_{zz} - \tau_{xy}^2 - \tau_{xz}^2 - \tau_{yz}^2) \\ & - \sigma_{xx}\sigma_{yy}\sigma_{zz} + \sigma_{xx}\tau_{yz}^2 + \sigma_{yy}\tau_{xz}^2 + \sigma_{zz}\tau_{xy}^2 - 2\tau_{xy}\tau_{xz}\tau_{yz} \end{aligned} \quad (1.77)$$

and setting I_1, I_2, I_3 as invariants:

$$\begin{aligned} I_1 &= -\sigma_{xx} - \sigma_{yy} - \sigma_{zz} \\ I_2 &= \sigma_{xx}\sigma_{yy} + \sigma_{xx}\sigma_{zz} + \sigma_{yy}\sigma_{zz} - \tau_{xy}^2 - \tau_{xz}^2 - \tau_{yz}^2 \\ I_3 &= \sigma_{xx}\tau_{yz}^2 + \sigma_{yy}\tau_{xz}^2 + \sigma_{zz}\tau_{xy}^2 - \sigma_{xx}\sigma_{yy}\sigma_{zz} - 2Re(\tau_{xy}\tau_{xz}\tau_{yz}) \end{aligned} \quad (1.78)$$

where $Re()$ means **real part** (contrary to $Im()$ which would mean imaginary part).

in **matrix** form:

$$\begin{aligned}
I_1 &= -tr[\mathbf{\Sigma}] \\
I_2 &= \begin{vmatrix} \sigma_{11} & \sigma_{12} \\ \sigma_{12} & \sigma_{22} \end{vmatrix} + \begin{vmatrix} \sigma_{11} & \sigma_{13} \\ \sigma_{13} & \sigma_{33} \end{vmatrix} + \begin{vmatrix} \sigma_{22} & \sigma_{23} \\ \sigma_{23} & \sigma_{33} \end{vmatrix} \\
I_3 &= -det(\mathbf{\Sigma})
\end{aligned} \tag{1.79}$$

substituting 1.78 into 1.77 we get:

$$\lambda^3 + I_1\lambda^2 + I_2\lambda + I_3 = 0, \tag{1.80}$$

where the invariants are:

Cardano's method requires first transforming 1.77 to the form:

$$x^3 - 3x = t \tag{1.81}$$

by defining:

$$\begin{aligned}
p &= I_1^2 - 3I_2 \\
q &= -\frac{27}{2}I_3 - I_1^3 + \frac{9}{2}I_1I_2 \\
t &= 2p^{-3/2}q \\
x &= \frac{3}{\sqrt{p}} \left(\lambda + \frac{1}{3}I_1 \right)
\end{aligned} \tag{1.82}$$

the the solution to 1.81 is given by:

$$x = \frac{1}{u} + u \tag{1.83}$$

with

$$u = \sqrt[3]{\frac{t}{2} \pm \sqrt{\frac{t^2}{4} - 1}} \tag{1.84}$$

We can then write $u = e^{i\phi}$, with:

$$\begin{aligned}
\phi &= \frac{1}{3} \arctan \frac{\sqrt{|t^2/4 - 1|}}{t/2} \\
\phi &= \frac{1}{3} \arctan \frac{\sqrt{p^3 - q^2}}{q} \\
\phi &= \frac{1}{3} \arctan \frac{\sqrt{27 \left[\frac{1}{4} I_2^2 (p - I_2) + I_3 * \left(q + \frac{27}{4} I_3 \right) \right]}}{q}
\end{aligned} \tag{1.85}$$

The last step of 1.85 is necessary for numerical stability and precision as the term $p^3 - q^2$ is highly sensitive.

The 1.85 equation can be for better stability and to enforce the angle to the domain $\phi \in \langle -\pi, \pi \rangle$:

$$\phi = \frac{1}{3} \arctan2 \left(\sqrt{27 \left[\frac{1}{4} I_2^2 (p - I_2) + I_3 * \left(q + \frac{27}{4} I_3 \right) \right]}, q \right) \tag{1.86}$$

When evaluating either 1.85 or 1.86, care must be taken to correctly resolve the ambiguity in the arctan by taking account of the sign of q . For $q > 0$, the solution must lie in the first quadrant, for $q < 0$, it must be located in the second. In contrast to this, solutions differing by multiples of 2π are equivalent, so x can take three different values:

$$\begin{aligned}
x_1 &= 2 \cos \phi \\
x_2 &= 2 \cos \left(\phi + \frac{2\pi}{3} \right) = -\cos \phi - \sqrt{3} \sin \phi \\
x_3 &= 2 \cos \left(\phi - \frac{2\pi}{3} \right) = -\cos \phi + \sqrt{3} \sin \phi
\end{aligned} \tag{1.87}$$

These values correspond to the three eigenvalues of Σ :

$$\lambda_i = \frac{\sqrt{p}}{3} x_i - \frac{1}{3} I_1 \tag{1.88}$$

A python implementation:

```

1 import numpy as np
2
3 def principal_cardano_equation(flat_tensor_matrix: np.ndarray) ->
  ↳ np.ndarray:
4     """Eigenvalues of a Hermitian 3x3 matrix using Cardano equations
5
6     source: https://www.mpi-hd.mpg.de/personalhomes/globes/3x3/index.html
7            https://en.wikipedia.org/wiki/Cubic\_equation
8
9     [ sx, txy, txz]   [a, d, f]
10    [txy,  sy, tyz] = [d, b, e]
11    [txz, tyz,  sz]   [f, e, c]
12
13    Args:
14        flat_tensor_matrix (np.ndarray): a symmetric, positive definite
15        ↳ tensor components
16
17                                     [[sx, sy, sz, txy, tyz, txz],
18                                     [sx, sy, sz, txy, tyz, txz]
19                                     .
20                                     .
21                                     .
22                                     [sx, sy, sz, txy, tyz, txz]]
23
24    Returns:
25        np.ndarray: principal stresses sorted from largest to smallest [s1,
26        ↳ s2, s3]
27
28    """
29    # store each component of the tensor into standalone vector
30    a, b, c, d, e, f = flat_tensor_matrix.T
31
32    # prepare values that are used multiple times
33    M_SQRT3 = np.sqrt(3)
34    de = d * e
35    dd = d * d
36    ee = e * e
37    ff = f * f
38
39    # tr(A)

```

```

36     m = a + b + c
37     # det(A23) + det(13) + det(12)
38     c1 = (a * b + a * c + b * c) - (dd + ee + ff)
39     # det(A)
40     c0 = c * dd + a * ee + b * ff - a * b * c - 2 * f * d * e
41
42     p = m * m - 3.0 * c1
43     q = m * (p - (3.0 / 2.0) * c1) - (27.0 / 2.0) * c0
44     sqrt_p = np.sqrt(np.abs(p))
45
46     phi = 27.0 * ( 0.25 * c1 * c1 * (p - c1) + c0 * (q + 27.0 / 4.0 * c0))
47     phi = (1.0 / 3.0) * np.arctan2(np.sqrt(np.abs(phi)), q)
48
49     c = sqrt_p * np.cos(phi)
50     s = (1.0 / M_SQRT3) * sqrt_p * np.sin(phi)
51
52     l2 = (1.0 / 3.0) * (m - c)
53     l3 = l2 + s
54     l1 = l2 + c
55     l2 -= s
56
57     # Sort the eigenvalues using a fast sorting network
58     l1, l2 = np.minimum(l1, l2), np.maximum(l1, l2)
59     l2, l3 = np.minimum(l2, l3), np.maximum(l2, l3)
60     l1, l2 = np.minimum(l1, l2), np.maximum(l1, l2)
61
62     return np.array([l3, l2, l1], dtype=float).T
63

```

- **Maximum Shear value:**

Maximum **shear stress** occurs at an angle of 45 degrees to principal axes. If principal stresses are aligned such that $\sigma_1 > \sigma_2 > \sigma_3$ and the stress tensor is:

$$\Sigma = \begin{bmatrix} \sigma_1 & 0 & 0 \\ 0 & \sigma_2 & 0 \\ 0 & 0 & \sigma_3 \end{bmatrix}, \quad (1.89)$$

then maximum shear stress τ_{max} can be obtained as follows:

$$\tau_{max} = \frac{\sigma_1 - \sigma_3}{2} \quad (1.90)$$

and is at 45 degrees angle in the 1-3 plane of the principal axes to the principal coordinate system.

More generally (in 2D):

$$\tau_{max} = \sqrt{\left(\frac{\sigma_x - \sigma_y}{2}\right)^2 + \tau_{xy}^2}. \quad (1.91)$$

When the principal tensor is rotated by 45 degrees to obtain **maximum shear stress**, the axial stress components corresponding to this shear stress are equal.

The value of maximal shear stress is obtained:

$$\tau_{max} = \frac{\sigma_{max} - \sigma_{min}}{2}. \quad (1.92)$$

The value of axial stresses corresponding to maximal shear stress is the average of maximal and minimal axial stresses:

$$\sigma_{\tau_{max}} = \frac{\sigma_{max} + \sigma_{min}}{2}. \quad (1.93)$$

- **Example:**

Let a stress tensor in principal coordinate system be:

$$\Sigma = \begin{bmatrix} 433 & 0 & 0 \\ 0 & 125 & 0 \\ 0 & 0 & 24 \end{bmatrix}, \quad (1.94)$$

then $\sigma_1 = \sigma_{max} = 433$ MPa and $\sigma_3 = \sigma_{min} = 24$ MPa.

Transformation matrix for rotating by 45 degrees in 3-1 plane is:

$$\mathbf{T} = \begin{bmatrix} \frac{\sqrt{2}}{2} & 0 & -\frac{\sqrt{2}}{2} \\ 0 & 1 & 0 \\ \frac{\sqrt{2}}{2} & 0 & \frac{\sqrt{2}}{2} \end{bmatrix}. \quad (1.95)$$

Rotating tensor Σ by transformation matrix $\mathbf{T}$ we get:

$$\Sigma_\tau = \mathbf{T}\Sigma\mathbf{T}^T = \begin{bmatrix} 228.5 & 0 & 204.5 \\ 0 & 125 & 0 \\ 204.5 & 0 & 228.5 \end{bmatrix}, \quad (1.96)$$

where:

$$\begin{aligned} \tau_{max} &= \sigma_{13} = \sigma_{31} = 204.5 \\ \sigma_{11} &= \sigma_{33} = 228.5 \\ \sigma_{22} &= \sigma_2 \end{aligned} \quad (1.97)$$

Check for correct values:

$$\tau_{max} = \frac{\sigma_1 - \sigma_3}{2} = \frac{433 - 24}{2} = \frac{409}{2} = 204.5 \text{ MPa} \implies \text{OK!}, \quad (1.98)$$

$$\sigma_{11} = \sigma_{33} = \frac{\sigma_1 + \sigma_3}{2} = \frac{433 + 24}{2} = 228.5 \text{ MPa} \implies \text{OK!}. \quad (1.99)$$

σ_2 has not changed as it is colinear with the axis of rotation.

The relationship between principal normal streses and maximum shear stresses can be better understood by examining a plot of the stresses as a function of the rotation angle.

Notice that there are multiple θ_p and $\theta_{\tau-max}$ angles because of the periodical nature of the equations. However, they will give the same absolute values.

At the principal stress angle, θ_p , the shear stress will always be zero, as shown on the diagram. And the maximum shear stress will occur when the two principal normal stresses, σ_1 and σ_2 , are equal.

When σ_x or σ_y are either max or min, the shear stress τ_{xy} is equal to 0. When shear stress τ_{xy} is max or min, the stresses σ_x and σ_y are equal to their average.

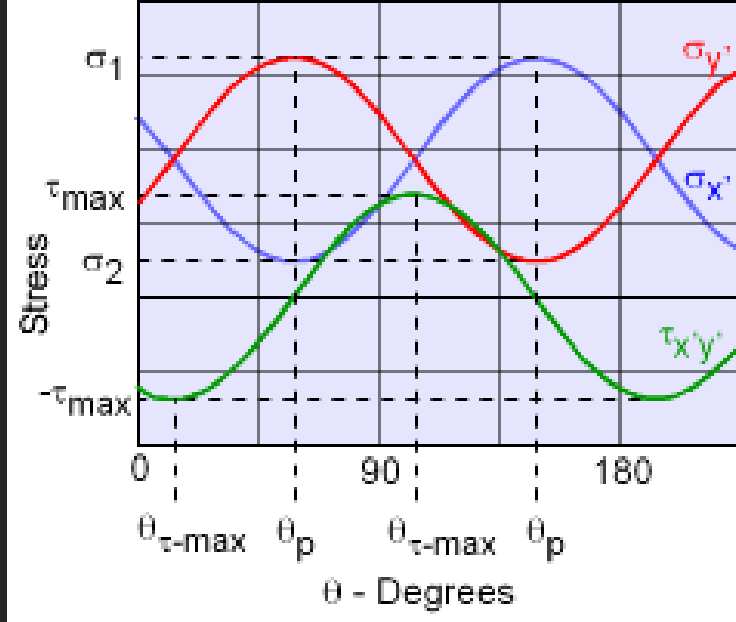


Figure 1.3: Stresses as a function of angle

- **Principal axes, obtaining principal axes with python implementation:**

The principal axes can be obtained from the following property

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v} \quad (1.100)$$

As the **principal axis** (eigenvector) of $\mathbf{A}$ is not changed by $\mathbf{A}$, only scaled, then it lies in the **nullspace** of the matrix $\mathbf{B}$ from the characteristic equation:

$$\mathbf{B} = (\mathbf{A} - \lambda\mathbf{I}) \quad (1.101)$$

The matrix $\mathbf{B}$ is **singular** with 2 independent rows and a third one being a linear combination of others. The vector that lies in the **nullspace** can be obtained by using a cross product of any two rows. This can be performed

for the first two principal values (eigenvalues λ_i and the third principal axis can be again computed using a cross product of the two previously extracted axes.

Python example:

```

1 import numpy as np
2
3 def gram_schmidt(vectors: np.ndarray) -> np.ndarray:
4     """Perform Gram-Schmidt orthogonalisation of vectors
5
6     Args:
7         vectors (np.ndarray) an array of row vectors to orthogonalise
8             ↪ against
9
10                each other, if the array is 3 dimensional then
11                preform orthogonalisation for each array of
12                row vectors separately
13
14     Returns:
15         (np.ndarray): an array of orthogonal vectors
16     """
17     ortho = np.copy(vectors)
18
19     if len(ortho.shape) == 3:
20         for i in range(1, ortho.shape[1]):
21             for j in range(i):
22                 # np.sum(A * B, axis=1) is equal to row by row dot product
23                 ↪ of two sets of vectors
24                 # written as a 2D matrix
25                 vec = ortho[:,j,:] / np.linalg.norm(ortho[:,j,:],
26                 ↪ axis=1)[:,None]
27                 ortho[:,i,:] -= vec * np.sum(ortho[:,i,:] * vec,
28                 ↪ axis=1)[:,None]
29
30     elif len(ortho.shape) == 2:
31         for i in range(1, ortho.shape[0]):
32             for j in range(i):
33                 # np.sum(A * B, axis=1) is equal to row by row dot product
34                 ↪ of two sets of vectors
35                 # written as a 2D matrix
36                 vec = ortho[j,:] / np.linalg.norm(ortho[j,:])
37                 ortho[i,:] -= vec * np.dot(ortho[i,:], vec)

```

```

30
31     return ortho
32
33
34 def normalise(vectors: np.ndarray) -> np.ndarray:
35     """Normalise supplied vectors to a length of 1"""
36     normalised = np.copy(vectors)
37     if len(normalised.shape) == 3:
38         for i in range(vectors.shape[1]):
39             normalised[:,i,:] /= np.linalg.norm(normalised[:,i,:],
39             ↪ axis=1)[:,None]
40
41     elif len(normalised.shape) == 2:
42         normalised /= np.linalg.norm(normalised, axis=1)[:,None]
43
44     elif len(normalised.shape) == 1:
45         normalised /= np.linalg.norm(normalised)
46
47     return normalised
48
49
50 def orthonormalise(vectors: np.ndarray) -> np.ndarray:
51     """Orthonormalise sets of supplied vectors using GramSchmidt method
52     first and then normalisation to a length of 1
53     """
54     return normalise(gram_schmidt(vectors))
55
56
57 def principal_axes(tensors: np.ndarray, principal_values: np.ndarray) ->
58 ↪ np.ndarray:
59     """Get the principal axes corresponding to the principal values by
60     getting the vector that is in the nullspace of the rank 2 homogeneous
61     equation:
62          $(A - \lambda I) = 0$ 
63     by setting  $\lambda$  to  $\lambda_1$ ,  $\lambda_2$  and  $\lambda_3$ , we can obtain three vectors that are
64     in the nullspace of the first, second and third eigenvalue by using
65     cross product of any two rows of the characteristic equation.
66
67     Args:
68         tensors (np.ndarray):          an array of tensors, one tensor
69                                     on each row

```

```

69         [[sx, sy, sz, txy, tyz, tzx], ... ]
70     principal_values (np.ndarray): an array of the same number of rows
71                                     as the tensors and 3 columns with
72                                     principal values
73
74     Returns:
75         (np.ndarray): the principal axes belonging to the supplied
76                       principal values
77
78     """
79     principal = np.copy(principal_values)
80     if len(principal.shape) == 1 and principal.shape[0] == 3:
81         principal = principal.reshape(1, 3)
82
83     if len(principal.shape) != len(tensor.shape):
84         raise ValueError(f"A mismatch in tensor and principal values
85         ↳ dimensions " +
86         f"{str(tensor.shape):s} != {str(principal.shape):s}")
87     elif principal.shape[0] != tensor.shape[0]:
88         raise ValueError(f"The number of principal values does not match
89         ↳ the number of tensors " +
90         f"{str(tensor.shape):s} != {str(principal.shape):s}")
91     elif principal.shape[1] != 3:
92         raise ValueError(f"The number of principal values is not 3:
93         ↳ {str(principal.shape):s}")
94
95     # sort principal stresses by amount from largest to smallest
96     amount_idx = np.argsort(np.abs(principal), axis=1)[::-1]
97     principal = np.take_along_axis(principal, amount_idx, axis=1)
98
99     # prepare reverse sort from largest to smallest
100     amount_idx = np.argsort(principal, axis=1)[::-1]
101
102     a, b, c, d, e, f = tensor.T
103     l1, l2, l3 = principal.T
104
105     # prepare the array to store the 3x3 transformation matrix composed of
106     # rows of principal axes
107     T = np.zeros((principal.shape[0], 3, 3), dtype=tensor.dtype)
108
109     # prepare a mask where True is if the stress tensor is 0
110     mask_zero = np.isclose(a ** 2 + b ** 2 + c ** 2, 0.)

```

```

107
108     # prepare a mask where True is if the prinicpal stresses are equal
109     mask_equal = np.equal(l1, l2, l3)
110
111     # other values - a general case
112     mask_nonz = np.logical_not(mask_zero) & np.logical_not(mask_equal)
113
114     # a zero tensor
115     if np.any(mask_zero):
116         T[mask_zero] = np.array([[1., 0., 0.], [0., 1., 0.], [0., 0., 1.]],
117             ↪ dtype=tensor.dtype)
118
119     # triaxial tensrion/compression
120     if np.any(mask_equal):
121         T[mask_equal] = np.array([[1., 0., 0.], [0., 1., 0.], [0., 0.,
122             ↪ 1.]], dtype=tensor.dtype)
123
124     # process each eigenvalue (principal stress) individually (only first
125     ↪ two needed)
126     for i, l in enumerate([l1, l2]): # , l3]:
127         # get rows of the tensor (homogeneous equation)
128         v1 = np.array([a - l, d, f], dtype=tensor.dtype).T
129         v2 = np.array([d, b - l, e], dtype=tensor.dtype).T
130         v3 = np.array([f, e, c - l], dtype=tensor.dtype).T
131
132         # get the vector in the tensor nullspace for all combinations of
133         ↪ rows
134         v0 = np.array([np.cross(v1, v2),
135             ↪ np.cross(v1, v3),
136             ↪ np.cross(v2, v3)],
137             ↪ dtype=tensor.dtype).transpose(1,0,2)
138
139         # get the vector length
140         len_v0 = np.array([
141             ↪ np.linalg.norm(v0[:,0], axis=1),
142             ↪ np.linalg.norm(v0[:,1], axis=1),
143             ↪ np.linalg.norm(v0[:,2], axis=1)], dtype=tensor.dtype).T
144
145         # find the index of largest column value for each row
146         idx_v0 = np.argmax(len_v0, axis=1)

```

```

143     # select the longest vector in the nullspace of the homogeneous
144     ↪ equation
145     v0 = np.take_along_axis(v0, idx_v0[:,None,None],
146     ↪ axis=1).reshape(-1,3)
147
148     # and its respective length
149     len_v0 = np.take_along_axis(len_v0, idx_v0[:,None],
150     ↪ axis=1).flatten()
151
152     # prepare an array masking vectors with 0 or close to 0 length
153     mask_zero_len = np.isclose(len_v0, 0.)
154     mask_some_len = np.logical_not(mask_zero_len)
155
156     # norm the vectors that have nonzero length
157     v0[mask_some_len] /= len_v0[mask_some_len,None]
158
159     # now to deal with vectors that have zero length
160     # 1. case where we are dealing with 1st eigenvalue
161     # should not happen due to the presorting of eigenvalues by
162     # magniture from largest to smallest. If the largest (by magnitude)
163     # eigenvalue has close to 0 length, it is a degenerative case where
164     # there are infinite possibilities for axis direction, so we can
165     # choose one aligned with the global x axis
166     if i == 0:
167         # fallback axis value - global x axis because it does not
168         ↪ matter
169         v0[mask_zero_len] = np.array([1., 0., 0.], dtype=float)
170
171     # 2. case where we are dealing with 2nd eigenvalue
172     elif i == 1:
173         # get first principal axis
174         v = T[mask_zero_len,0,:]
175         # check whether it is the fallback axis
176         mask_occupied = np.all(np.isclose(v, np.array([1., 0., 0.])),
177         ↪ axis=1)
178         # where it is a fallback axis, use global z axis
179         v[mask_occupied] = np.array([0., 0., 1.])
180         # obtain an arbitrary axis that is perpendicular to 1st and
181         ↪ fallback
182         v = np.cross(v, np.array([1., 0., 0.]))
183         # norm it

```

```

178         v /= np.linalg.norm(v, axis=1)[:None]
179         # store it
180         v0[mask_zero_len] = v
181
182         # 3. case where we are dealing with 3rd eigenvalue
183         else:
184             # create an axis perpendicular to the first 2
185             v = np.cross(T[mask_zero_len,0,:], T[mask_zero_len,1,:])
186             # norm it
187             v /= np.linalg.norm(v, axis=1)[:None]
188             # store it
189             v0[mask_zero_len] = v
190
191         # store the axis
192         T[mask_nonz,i,:] = v0[mask_nonz,:]
193
194         # create the 3rd axis using a cross product of the two previous ones
195         T[:,2,:] = np.cross(T[:,0,:], T[:,1,:])
196         T[:,2,:] /= np.linalg.norm(T[:,2,:], axis=1)[:None]
197
198
199         # reorder the principal values and axes back to  $s_1 > s_2 > s_3$ 
200         principal = np.take_along_axis(principal, amount_idx, axis=1) # for
        ↪ checking
201         T = np.take_along_axis(T, amount_idx[:,None], axis=1)
202
203         return orthonormalise(T)

```

1.4 Moment of Inertia

From: Parallel axis theorem

Parallel axis theorem, also known as *Huygens-Steiner's theorem* or just *Steiner's theorem* can be used to determine the **moment of inertia** or **second moment of area** of a rigid body about any axis, given the body's moment of inertia about a parallel axis through the objects center of gravity and the perpendicular distance between the axes.

Moment of Inertia = **tensor**.

- Steiner's share:

$$\mathbf{I}_{ss} = m \times \begin{bmatrix} y^2 + z^2 & -xy & -xz \\ -yz & x^2 + z^2 & -yz \\ -zx & -zy & x^2 + y^2 \end{bmatrix}. \quad (1.102)$$

Identities for a skew-symmetric matrix

In order to compare formulations of the parallel axis theorem using skew-symmetric matrices and the tensor formulation, the following identities are useful.

Let $[\mathbf{R}]$ be the skew-symmetric matrix associated with the position vector $\mathbf{R} = (x, y, z)$, then the product in the inertia matrix becomes:

$$-[\mathbf{R}][\mathbf{R}] = \begin{bmatrix} 0 & -z & y \\ z & 0 & -x \\ -y & x & 0 \end{bmatrix}^2 = \begin{bmatrix} y^2 + z^2 & -xy & -xz \\ -yz & x^2 + z^2 & -yz \\ -zx & -zy & x^2 + y^2 \end{bmatrix}. \quad (1.103)$$

This product can be computed using the matrix formed by the outer product $[\mathbf{R}\mathbf{R}^T]$ using the identity

$$\begin{aligned} -[\mathbf{R}]^2 &= |\mathbf{R}|^2 |\mathbf{E}_3| - [\mathbf{R}\mathbf{R}^T] \\ &= \begin{bmatrix} x^2 + y^2 + z^2 & 0 & 0 \\ 0 & x^2 + y^2 + z^2 & 0 \\ 0 & 0 & x^2 + y^2 + z^2 \end{bmatrix} - \begin{bmatrix} x^2 & xy & xz \\ yx & y^2 & yz \\ zx & zy & z^2 \end{bmatrix}, \end{aligned} \quad (1.104)$$

where $[\mathbf{E}_3]$ is the $n \times n$ identity matrix.

Also notice, that

$$|\mathbf{R}|^2 = \mathbf{R} \cdot \mathbf{R} = \text{tr}[\mathbf{R}\mathbf{R}^T]. \quad (1.105)$$

where tr denotes the sum of the diagonal elements of the outer product matrix, known as its *trace*.

To obtain matrix of moments of inertia by rotating about any random point A while having the moment of inertia in the center of gravity of the object $\mathbf{I}_{COG}$ and the coordinate distance $d = [x, y, z]$ between the point A and **COG** we:

1. Compute the moment of inertia relative to point A

$$\mathbf{I}_A = \mathbf{I}_{COG} + m \times \mathbf{I}_{SS}^{A \rightarrow COG}. \quad (1.106)$$

2. Transform the tensor $\mathbf{I}_A$ to new rotated coordinate system using transformation matrix $\mathbf{T}$:

$$\mathbf{I}'_A = \mathbf{T} \times \mathbf{I}_A \times \mathbf{T}^T. \quad (1.107)$$

3. Subtract the **Steiner's share** relative to new **COG**

$$\mathbf{I}'_{COG} = \mathbf{I}'_A - \mathbf{I}_{SS}^{A' \rightarrow COG'}. \quad (1.108)$$

1.5 Quaternions

In mathematics, the quaternion number system extends the complex numbers. Quaternions were first described by the Irish mathematician William Rowan Hamilton in 1843 and applied to mechanics in three-dimensional space. The algebra of quaternions is often denoted by $\mathbf{H}$ or $\mathbb{H}$ (for Hamilton).

Although multiplication of quaternions is noncommutative, it gives a definition of the quotient of two vectors in a three-dimensional space. Quaternions are generally represented in the form:

$$a + b\mathbf{i} + c\mathbf{j} + d\mathbf{k} \quad (1.109)$$

where the coefficients a, b, c, d are *real numbers* and $\mathbf{1}, \mathbf{i}, \mathbf{j}, \mathbf{k}$ are *basis vectors* or *basis elements*.

The Quaternion multiplication table (left column shows pre-multiplier, top row shows post-multiplier):

$r \times c$	$\mathbf{1}$	$\mathbf{i}$	$\mathbf{j}$	$\mathbf{k}$
$\mathbf{1}$	$\mathbf{1}$	$\mathbf{i}$	$\mathbf{j}$	$\mathbf{k}$
$\mathbf{i}$	$\mathbf{i}$	$-\mathbf{1}$	$\mathbf{k}$	$-\mathbf{j}$
$\mathbf{j}$	$\mathbf{j}$	$-\mathbf{k}$	$-\mathbf{1}$	$\mathbf{i}$
$\mathbf{k}$	$\mathbf{k}$	$\mathbf{j}$	$-\mathbf{i}$	$-\mathbf{1}$

Table 1.1: Quaternion multiplication table

also, $a\mathbf{b} = \mathbf{b}a$ and $-\mathbf{b} = (-1)\mathbf{b}$ for $a \in \mathbb{R}$, $b = \mathbf{i}, \mathbf{j}, \mathbf{k}$.

1.5.1 Quaternion properties

The following properties apply:

1. Addition:

$$\begin{aligned}
 (a_1 + b_1\mathbf{i} + c_1\mathbf{j} + d_1\mathbf{k}) + (a_2 + b_2\mathbf{i} + c_2\mathbf{j} + d_2\mathbf{k}) &= \\
 = (a_1 + a_2) + (b_1 + b_2)\mathbf{i} + (c_1 + c_2)\mathbf{j} + (d_1 + d_2)\mathbf{k}
 \end{aligned} \tag{1.110}$$

2. Component-wise scalar multiplication:

$$\lambda(a + b\mathbf{i} + c\mathbf{j} + d\mathbf{k}) = \lambda a + (\lambda b)\mathbf{i} + (\lambda c)\mathbf{j} + (\lambda d)\mathbf{k} \tag{1.111}$$

3. Multiplication of basis elements (the revelation of Hamilton in 1943):

$$\begin{aligned}
 \mathbf{i}^2 = \mathbf{j}^2 = \mathbf{k}^2 &= -1 \\
 \mathbf{ijk} &= -1
 \end{aligned} \tag{1.112}$$

4. Multiplicative identity:

$$\begin{aligned}
 \mathbf{i}1 &= 1\mathbf{i} = \mathbf{i} \\
 \mathbf{j}1 &= 1\mathbf{j} = \mathbf{j} \\
 \mathbf{k}1 &= 1\mathbf{k} = \mathbf{k}
 \end{aligned} \tag{1.113}$$

5. Products of basis elements:

$$\begin{aligned}
 \mathbf{ij} &= -\mathbf{ji} = \mathbf{k} \\
 \mathbf{jk} &= -\mathbf{kj} = \mathbf{i} \\
 \mathbf{ki} &= -\mathbf{ik} = \mathbf{j}
 \end{aligned} \tag{1.114}$$

6. Inverse:

$$(a + b\mathbf{i} + c\mathbf{j} + d\mathbf{k})^{-1} = \frac{1}{a^2 + b^2 + c^2 + d^2}(a - b\mathbf{i} - c\mathbf{j} - d\mathbf{k}) \tag{1.115}$$

7. Product of two **quaternions** (also called the **Hamilton product**) is determined by the products of the basis elements and the **distributive** law:

$$(a_1 + b_1\mathbf{i} + c_1\mathbf{j} + d_1\mathbf{k}) \times (a_2 + b_2\mathbf{i} + c_2\mathbf{j} + d_2\mathbf{k}) = \quad (1.116)$$

$$\begin{aligned} &= a_1a_2 & +a_1b_2\mathbf{i} & +a_1c_2\mathbf{j} & +a_1d_2\mathbf{k} \\ &+b_1a_2\mathbf{i} & +b_1b_2\mathbf{i}^2 & +b_1c_2\mathbf{ij} & +b_1d_2\mathbf{ik} \\ &+c_1a_2\mathbf{j} & +c_1b_2\mathbf{ij} & +c_1c_2\mathbf{j}^2 & +c_1d_2\mathbf{jk} \\ &+d_1a_2\mathbf{k} & +d_1b_2\mathbf{ik} & +d_1c_2\mathbf{jk} & +d_1d_2\mathbf{k}^2 \end{aligned} \quad (1.117)$$

$$\begin{aligned} &= a_1a_2 & -b_1b_2 & -c_1c_2 & -d_1d_2 \\ &+(a_1b_2 & +b_1a_2 & +c_1d_2 & -d_1c_2) \times \mathbf{i} \\ &+(a_1c_2 & -b_1d_2 & +c_1a_2 & +d_1b_2) \times \mathbf{j} \\ &+(a_1d_2 & +b_1c_2 & -c_1b_2 & +d_1a_2) \times \mathbf{k} \end{aligned} \quad (1.118)$$

1.5.2 Scalar and vector parts

A quaternion of the form $a + 0\mathbf{i} + 0\mathbf{j} + 0\mathbf{k}$, where a is a real number, is called **scalar**, and a quaternion of the form $0 + b\mathbf{i} + c\mathbf{j} + d\mathbf{k}$ where b, c and d are real numbers, and at least one of b, c or d is nonzero, is called a **vector quaternion**. If $a + b\mathbf{i} + c\mathbf{j} + d\mathbf{k}$ is any quaternion, then a is called its **scalar part** and $b\mathbf{i} + c\mathbf{j} + d\mathbf{k}$ is called its **vector part**. Although every quaternion can be viewed as a vector in a four-dimensional vector space, it is common to refer to the **vector** part as vectors in three-dimensional space.

Hamilton also called vector quaternions **right quaternions** and real numbers (considered as quaternions with zero vector part) **scalar quaternions**.

If a quaternion is divided up into a scalar and a vector part, that is,

$$\mathbf{q} = (r, \vec{v}), \mathbf{q} \in \mathbb{H}, r \in \mathbb{R}, \vec{v} \in \mathbb{R}^3 \quad (1.119)$$

then the formulas for **addition**, **multiplication** and **multiplicative inverse** are:

$$(r_1, \vec{v}_1) + (r_2, \vec{v}_2) = (r_1 + r_2, \vec{v}_1 + \vec{v}_2) \quad (1.120)$$

$$(r_1, \vec{v}_1) \times (r_2, \vec{v}_2) = (r_1 r_2 - \vec{v}_1 \cdot \vec{v}_2, r_1 \vec{v}_2 + r_2 \vec{v}_1 + \vec{v}_1 \times \vec{v}_2) \quad (1.121)$$

$$(r, \vec{v})^{-1} = \left(\frac{r}{r^2 + \vec{v} \cdot \vec{v}}, \frac{-\vec{v}}{r^2 + \vec{v} \cdot \vec{v}} \right) \quad (1.122)$$

1.5.3 Conjugation, the norm and reciprocal

Conjugation of quaternions is analogous to conjugation of complex numbers. To define it, let $\mathbf{q} = a + b\mathbf{i} + c\mathbf{j} + d\mathbf{k}$ be a quaternion. The **conjugate** of $\mathbf{q}$ is the quaternion $\mathbf{q}^* = a - b\mathbf{i} - c\mathbf{j} - d\mathbf{k}$. It is denoted by $\mathbf{q}^*$, $\mathbf{q}^t$ or $\bar{\mathbf{q}}$.

Conjugation is **involution**, meaning that it is its own inverse, so **conjugating** an element **twice** returns the original element. The **conjugate** of a **product** of two quaternions is the product of the conjugates in the **reverse order**. That is, if $\mathbf{p}$ and $\mathbf{q}$ are quaternions, then:

$$(\mathbf{pq})^* = \mathbf{q}^* \mathbf{p}^* \quad (1.123)$$

The conjugation of a quaternion, in stark contrast to the complex setting, can be expressed with multiplication and addition of quaternions:

$$\mathbf{q}^* = -\frac{1}{2}(\mathbf{q} + \mathbf{iqi} + \mathbf{jqj} + \mathbf{kqi}) \quad (1.124)$$

Conjugation can be used to extract the **scalar** and **vector** part of a quaternion. The **scalar** part of $\mathbf{q}$ is:

$$r_q = \frac{1}{2}(\mathbf{q} + \mathbf{q}^*) \quad (1.125)$$

The **vector** part of $\mathbf{q}$ is:

$$\vec{v}_q = \frac{1}{2}(\mathbf{q} - \mathbf{q}^*) \quad (1.126)$$

The **square root** of the product of a quaternion with its conjugate is called its **norm** and is denoted $\|q\|$ (Hamilton called this quantity the **tensor** of $\mathbf{q}$).

$$\|\mathbf{q}\| = \sqrt{\mathbf{q}^* \mathbf{q}} = \sqrt{\mathbf{q} \mathbf{q}^*} = \sqrt{a^2 + b^2 + c^2 + d^2} \in \mathbb{R} \geq 0 \quad (1.127)$$

This is always a non-negative, real number and is the same as the Euclidean norm on $\mathbb{H}$ considered as the vector space $\mathbb{R}^4$. Multiplying a quaternion by a real number scales its norm by the absolute value of the number. That is, if $\alpha \in \mathbb{R}$ then:

$$\|\alpha \mathbf{q}\| = |\alpha| \|\mathbf{q}\| \quad (1.128)$$

The norm is **multiplicative**, meaning that:

$$\|\mathbf{p} \mathbf{q}\| = \|\mathbf{p}\| \|\mathbf{q}\| \quad (1.129)$$

for any two quaternion $\mathbf{p}$ and $\mathbf{q}$. Multiplicativity is a consequence of the formula for the conjugate of a product. Alternatively it follows from the identity

$$\det \begin{pmatrix} a + ib & id + c \\ id - c & a - ib \end{pmatrix} = a^2 + b^2 + c^2 + d^2 \quad (1.130)$$

(where i denotes the usual imaginary unit) and hence from the multiplicative property of determinants of square matrices.

This norm makes it possible to define the **distance** $d(\mathbf{p}, \mathbf{q})$ between $\mathbf{p}$ and $\mathbf{q}$ as the norm of their difference:

$$d(\mathbf{p}, \mathbf{q}) = \|\mathbf{p} - \mathbf{q}\| \quad (1.131)$$

This makes $\mathbb{H}$ a **metric space**. Addition and multiplication are **continuous** in regard to the associated metric topology. This follows exactly the same proof as for the real numbers $\mathbb{R}$ from the fact that $\mathbb{H}$ is a **normed algebra**.

1.5.4 Unit quaternion (or versor)

A **unit quaternion** is a quaternion of norm one. Dividing a nonzero quaternion $\mathbf{q}$ by its norm produces a unit quaternion $\mathbf{Uq}$ called the **versor** of $\mathbf{q}$:

$$\mathbf{Uq} = \frac{\mathbf{q}}{\|\mathbf{q}\|} \quad (1.132)$$

Every nonzero quaternion has a unique **polar decomposition** $\mathbf{q} = \|\mathbf{q}\| \cdot \mathbf{Uq}$, while the zero quaternion can be formed from any unit quaternion.

Using conjugation and the norm makes it possible to define the **reciprocal** of a nonzero quaternion. The product of a quaternion with its reciprocal should be equal 1, and the considerations above imply that the product of $\mathbf{q}$ and $\mathbf{q}^*/\|\mathbf{q}\|^2$ is 1 (for any order of multiplication). So the **reciprocal** of $\mathbf{q}$ is defined to be:

$$\mathbf{q}^{-1} = \frac{\mathbf{q}^*}{\|\mathbf{q}\|^2} \quad (1.133)$$

Since the multiplication is non-commutative, the quotient quantities $\mathbf{pq}^{-1}$ or $\mathbf{q}^{-1}\mathbf{p}$ are different (except if $\mathbf{p}$ and $\mathbf{q}$ are scalar multiples of each other or if one is a scalar). The notation $\frac{\mathbf{p}}{\mathbf{q}}$ is ambiguous and **should not be used**.

1.5.5 Quaternions and three-dimensional geometry

The vector part of a quaternion can be interpreted as a coordinate vector in $\mathbb{R}^3$, therefore the algebraic operations for the quaternions reflect the geometry of $\mathbb{R}^3$. Operations such as the vector dot and cross products can be defined in terms of quaternions and this makes it possible to apply quaternion techniques wherever spatial vectors arise. A useful application of quaternions has been to interpolate the orientations of key-frames in computer graphics.

For the remainder of this section, $\mathbf{i}$, $\mathbf{j}$ and $\mathbf{k}$ will denote both the three imaginary basis vectors of $\mathbb{H}$ and a basis for $\mathbb{R}^3$. Replacing $\mathbf{i}$ by $-\mathbf{i}$, $\mathbf{j}$ by $-\mathbf{j}$ and $\mathbf{k}$ by $-\mathbf{k}$ sends a vector to its **additive inverse**, so the additive inverse of a vector is the

same as its conjugate as a quaternion. For this reason, conjugation is sometimes called the **spatial inverse**.

For two vector quaternions $\mathbf{p} = b_1\mathbf{i} + c_1\mathbf{j} + d_1\mathbf{k}$ and $\mathbf{q} = b_2\mathbf{i} + c_2\mathbf{j} + d_2\mathbf{k}$ their **dot product**, by analogy to vectors in $\mathbb{R}^3$, is:

$$\mathbf{p} \cdot \mathbf{q} = b_1b_2 + c_1c_2 + d_1d_2 \quad (1.134)$$

It can also be expressed in a component-free manner as:

$$\mathbf{p} \cdot \mathbf{q} = \frac{1}{2}(\mathbf{p}^*\mathbf{q} + \mathbf{q}^*\mathbf{p}) = \frac{1}{2}(\mathbf{pq}^* + \mathbf{qp}^*) \quad (1.135)$$

This is equal to the scalar parts of the products $\mathbf{pq}^*$, $\mathbf{qp}^*$, $\mathbf{p}^*\mathbf{q}$ and $\mathbf{q}^*\mathbf{p}$. Note that their vector parts are different.

The **cross product** of $\mathbf{p}$ and $\mathbf{q}$ relative to the orientation determined by the ordered basis $\mathbf{i}$, $\mathbf{j}$ and $\mathbf{k}$ is:

$$\mathbf{p} \times \mathbf{q} = (c_1d_2 - d_1c_2)\mathbf{i} + (d_1b_2 - b_1d_2)\mathbf{j} + (b_1c_2 - c_1b_2)\mathbf{k} \quad (1.136)$$

(The orientation is necessary to determine the sign.) This is equal to the vector part of the product $\mathbf{pq}$ (as quaternions), as well as the vector part of $-\mathbf{q}^*\mathbf{p}^*$. It also has the formula:

$$\mathbf{p} \times \mathbf{q} = \frac{1}{2}(\mathbf{pq} - \mathbf{qp}) \quad (1.137)$$

For the **commutator**, $[\mathbf{p}, \mathbf{q}] = \mathbf{pq} - \mathbf{qp}$ of two vector quaternions one obtains:

$$[\mathbf{p}, \mathbf{q}] = 2\mathbf{p} \times \mathbf{q} \quad (1.138)$$

In general, let $\mathbf{p}$ and $\mathbf{q}$ be quaternions and write:

$$\begin{aligned}\mathbf{p} &= \mathbf{p}_s + \mathbf{p}_v \\ \mathbf{q} &= \mathbf{q}_s + \mathbf{q}_v\end{aligned}\tag{1.139}$$

where $\mathbf{p}_s$ and $\mathbf{q}_s$ are the scalar parts, and $\mathbf{p}_v$ and $\mathbf{q}_v$. Then we have the formula:

$$\mathbf{pq} = (\mathbf{pq})_s + (\mathbf{pq})_v = (\mathbf{p}_s\mathbf{q}_s - \mathbf{p}_v \cdot \mathbf{q}_v) + (\mathbf{p}_s\mathbf{q}_v + \mathbf{q}_s\mathbf{p}_v + \mathbf{p}_v \times \mathbf{q}_v) \tag{1.140}$$

This shows that the noncommutativity of quaternion multiplication comes from the multiplication of vector quaternions. It also shows that two quaternions commute if and only if their vector parts are collinear. **Hamilton** showed that this product computes the third vertex of spherical triangle from two given vertices and their associated arc-lengths, which is also an algebra of points in **Elliptic geometry**.

Unit quaternions can be identified with rotations in $\mathbb{R}^3$ and were called **versors** by **Hamilton**.

1.5.6 Matrix representations

Just as complex numbers can be **represented as matrices**, so can be quaternions. There are *at least* two ways to represent quaternions as matrices in such a way that quaternion **addition** and **multiplication** correspond to the matrix addition and multiplication.

The quaternion $a + b\mathbf{i} + c\mathbf{j} + d\mathbf{k}$ can be represented:

1. as a 2×2 complex matrix:

$$\begin{bmatrix} a + bi & c + di \\ -c + di & a - bi \end{bmatrix} \tag{1.141}$$

This representation has the following properties:

- Constraining any two of b , c and d to zero produces a representation for complex numbers. For example, setting $c = d = 0$ produces a diagonal complex matrix representation of complex numbers, and setting $b = d = 0$ produces a real matrix representation.
- The **norm** of a quaternion (the square root of the product with its conjugate, as with complex numbers) is the square root of the **determinant** of the corresponding matrix.
- The **conjugate** of a quaternion corresponds to the **conjugate transpose** of the matrix.
- There is a strong relation between quaternion units and **Pauli matrices** (these are needed to describe the *spin in quantum mechanics*). Obtain the eight quaternion unit matrices by taking a , b , c and d , set three of them to zero and the fourth to 1 or -1 . Multiplying any two Pauli matrices always yields a quaternion unit matrix, all of them except for -1 . One obtains -1 via the last equality:

$$\begin{aligned} \mathbf{i}^2 = \mathbf{j}^2 = \mathbf{k}^2 = \mathbf{ijk} &= -1 \\ \mathbf{ijk} &= \sigma_1 \sigma_2 \sigma_3 \sigma_1 \sigma_2 \sigma_3 = -1 \end{aligned} \tag{1.142}$$

2. Using 4×4 real matrix the same quaternion can be written as:

$$\begin{aligned} \begin{bmatrix} a & -b & -c & -d \\ b & a & -d & c \\ c & d & a & -b \\ d & -c & b & a \end{bmatrix} &= a \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\ &+ b \begin{bmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \\ &+ c \begin{bmatrix} 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \end{bmatrix} \\ &+ d \begin{bmatrix} 0 & 0 & 0 & -1 \\ 0 & 0 & -1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \end{aligned} \tag{1.143}$$

However the representation above is not unique. For example, the same quaternion can be also represented as:

$$\begin{aligned}
 \begin{bmatrix} a & d & -b & -c \\ -d & a & c & -b \\ b & -c & a & -d \\ c & b & d & a \end{bmatrix} &= a \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
 &+ b \begin{bmatrix} 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \\
 &+ c \begin{bmatrix} 0 & 0 & 0 & -1 \\ 0 & 0 & 1 & 0 \\ 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \\
 &+ d \begin{bmatrix} 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 \\ 0 & 0 & 1 & 0 \end{bmatrix}
 \end{aligned} \tag{1.144}$$

There exist 48 distinct matrix representations of thei form in which one of the matrices represents the scalar part and the other three are all skew-symmetric.

1.5.7 Exponential, logarithm and power functions

Given a quaternion:

$$\mathbf{q} = a + b\mathbf{i} + c\mathbf{j} + d\mathbf{k} = r + \vec{\mathbf{v}} \tag{1.145}$$

1. the exponential is computed as

$$\exp(\mathbf{q}) = \sum_{n=0}^{\infty} \frac{\mathbf{q}^n}{n!} = e^a \left(\cos \|\vec{\mathbf{v}}\| + \frac{\vec{\mathbf{v}}}{\|\vec{\mathbf{v}}\|} \sin \|\vec{\mathbf{v}}\| \right) \tag{1.146}$$

2. the logarithm is:

$$\ln(\mathbf{q}) = \ln \|\mathbf{q}\| + \frac{\vec{\mathbf{v}}}{\|\vec{\mathbf{v}}\|} \arccos \frac{r}{\|\mathbf{q}\|} \quad (1.147)$$

3. it follows that the polar decomposition of a quaternion may be written

$$\mathbf{q} = \|\mathbf{q}\| e^{\hat{\mathbf{n}}\varphi} = \|\mathbf{q}\| (\cos(\varphi) + \hat{\mathbf{n}} \sin(\varphi)) \quad (1.148)$$

where the angle φ is defined as

$$r = \|\mathbf{q}\| \cos(\varphi) \quad (1.149)$$

and the unit vector $\hat{\mathbf{n}}$ is defined by

$$\vec{\mathbf{v}} = \hat{\mathbf{n}} \|\vec{\mathbf{v}}\| = \hat{\mathbf{n}} \|\mathbf{q}\| \sin(\varphi) \quad (1.150)$$

Any unit quaternion may be expressed in polar form as:

$$\mathbf{q} = \exp(\hat{\mathbf{n}}\varphi) \quad (1.151)$$

4. The power of a quaternion raised to an arbitrary (real) exponent x is given by:

$$\mathbf{q}^x = \|\mathbf{q}\|^x e^{\hat{\mathbf{n}}x\varphi} = \|\mathbf{q}\|^x (\cos(x\varphi) + \hat{\mathbf{n}} \sin(x\varphi)) \quad (1.152)$$

1.5.8 Quaternions used for coordinate transformation

One can think of **quaternions** as a stored information about coordinate mapping from one space to another. We can split arbitrary **quaternion**

$\mathbf{q} = a + b\mathbf{i} + c\mathbf{j} + d\mathbf{k}$ into **two** parts:

1. a **real** part $r = a$ and
2. a **vector** part $\mathbf{v} = [b, c, d]$.

Next, if we **impose** restrictions on such **quaternions** that:

1. the **real** part must satisfy: $r \in \langle -\pi; \pi \rangle$
2. the **vector** part must have **unit** length: $|\mathbf{v}| = \sqrt{b^2 + c^2 + d^2} = 0$

Then, if one thinks of the complex numbers **i**, **j** and **k** as of basis vectors of cartesian system **x**, **y** and **z**, where

$$\begin{aligned}\mathbf{x} &= [1.0, 0.0, 0.0] \\ \mathbf{y} &= [0.0, 1.0, 0.0] \\ \mathbf{z} &= [0.0, 0.0, 1.0]\end{aligned}\tag{1.153}$$

the quaternion multiplication table 1.1 makes sense in relation to **vector cross product**:

r × c	1	x	y	z
1	1	x	y	z
x	x	-1	z	-y
y	y	-z	-1	x
z	z	y	-x	-1

Table 1.2: Quaternion cartesian base vectors multiplication table

In 3-dimensional space, according to the **Euler's rotation theorem**, any rotation or a sequence of rotations of a rigid body or coordinate system about a fixed point is equivalent to a single rotation by a given angle θ about a fixed axis (called the *Euler axis*) that runs through the fixed point. The Euler axis is typically represented by a **unit vector** $\vec{\mathbf{u}}$. Therefore, any rotation in three dimensions can be represented as via a **vector** $\vec{\mathbf{u}}$ and and angle θ .

Quaternions give a simple way to encode this **axis-angle** representation using four real numbers and can be used to apply (calculate) the corresponding rotation to a **position vector** $[x, y, z]$, representing a point relative to the origin in $\mathbb{R}^3$.

Euclidean vectors such as $[a_x, a_y, a_z]$ can be rewritten as $a_x \mathbf{i} + a_y \mathbf{j} + a_z \mathbf{k}$, where $\mathbf{i}$, $\mathbf{j}$ and $\mathbf{k}$ are unit vectors representing the three **Cartesian axes** (traditionally $\mathbf{x}$, $\mathbf{y}$ and $\mathbf{z}$) and also obey the **multiplication** rules of the fundamental quaternion units by interpreting the Euclidean vector $[a_x, a_y, a_z]$ as the vector part of the pure quaternion $(0, a_x, a_y, a_z)$.

A rotation angle θ around the axis defined by the unit vector

$$\vec{\mathbf{u}} = (u_x, u_y, u_z) = u_x \mathbf{i} + u_y \mathbf{j} + u_z \mathbf{k} \quad (1.154)$$

can be represented by **conjugation** by a unit quaternion $\mathbf{q}$.

Since the quaternion product

$$(0 + u_x \mathbf{i} + u_y \mathbf{j} + u_z \mathbf{k})(0 - u_x \mathbf{i} - u_y \mathbf{j} - u_z \mathbf{k}) = -1 \quad (1.155)$$

using **Taylor series** of the exponential function, the extension of the Euler's formula results in:

$$\begin{aligned} \mathbf{q} &= e^{\frac{\theta}{2}(u_x \mathbf{i} + u_y \mathbf{j} + u_z \mathbf{k})} \\ &= \cos \frac{\theta}{2} + (u_x \mathbf{i} + u_y \mathbf{j} + u_z \mathbf{k}) \sin \frac{\theta}{2} \\ &= \cos \frac{\theta}{2} + \vec{\mathbf{u}} \sin \frac{\theta}{2} \end{aligned} \quad (1.156)$$

It can be shown that the desired rotation can be applied to an ordinary vector

$$\mathbf{p} = (p_x, p_y, p_z) = p_x \mathbf{i} + p_y \mathbf{j} + p_z \mathbf{k} \quad (1.157)$$

in 3-dimensional space, considered as the vector part of the pure quaternion $\mathbf{p}'$, by evaluating the **conjugation** of $\mathbf{p}'$ by $\mathbf{q}$, given by:

$$\begin{aligned} L(\mathbf{p}') &:= \mathbf{q} \mathbf{p}' \mathbf{q}^{-1} \\ r &= \left(\cos^2 \frac{\theta}{2} - \|\mathbf{u}\|^2 \right) + 2(\mathbf{u} \cdot \mathbf{p})\mathbf{u} + 2 \cos \frac{\theta}{2} (\mathbf{u} \times \mathbf{p}) \end{aligned} \quad (1.158)$$

using the **Hamilton product** where the vector part of the pure quaternion $L(\mathbf{p}') = (0, r_x, r_y, r_z)$ is the new position vector after the rotation. In a programmatic implementation **the conjugation** is achieved by constructing a pure quaternion whose vector part is $\mathbf{p}$, and then performing the quaternion conjugation. The vector part of the resulting pure quaternion is the desired vector $\mathbf{r}$. Clearly L provides a linear transformation of the quaternion space to itself, also, since $\mathbf{q}$ is unitary, the transformation is an isometry. Also $L(\mathbf{q}) = \mathbf{q}$ and so L leaves vectors parallel to $\mathbf{q}$ invariant. So, by decomposing $\mathbf{p}$ as a vector parallel to the vector part $(u_x, u_y, u_z) \sin \frac{\theta}{2}$ of $\mathbf{q}$ and a vector normal to the vector part of $\mathbf{q}$ and showing that the application of L to the normal component of $\mathbf{p}$ rotates it, the claim is shown.

So let $\mathbf{n}$ be the component of $\mathbf{p}$ orthogonal to the vector part of $\mathbf{q}$ and let $\mathbf{n}_T = \mathbf{n} \times \mathbf{u}$. It turns out that the vector part of $L(0, \mathbf{n})$ is given by:

$$\left(\cos^2 \frac{\theta}{2} - \sin^2 \frac{\theta}{2} \right) \mathbf{n} + 2 \left(\cos^2 \frac{\theta}{2} \sin^2 \frac{\theta}{2} \right) \mathbf{n}_T = \cos \theta \mathbf{n} + \sin \theta \mathbf{n}_T \quad (1.159)$$

A geometric fact independent of quaternion is the existence of **two-to-one** mapping from physical rotations to rotational transformation matrices.

If $0 \leq \theta \leq 2\pi$, a physical rotation about $\bar{\mathbf{u}}$ by θ and a physical rotation about $-\bar{\mathbf{u}}$ by $2\pi - \theta$ both achieve the **same final orientation** by disjoint paths through intermediate orientations.

By inserting those vectors and angles into the formula for $\mathbf{q}$ above, one finds that if $\mathbf{q}$ represents the first rotation, $-\mathbf{q}$ represents the second rotation. This is a geometric proof that conjugation by $\mathbf{q}$ and by $-\mathbf{q}$ must produce the same rotational transformation matrix. That fact is confirmed algebraically by noting that the conjugation is quadratic in $\mathbf{q}$, so the sign of $\mathbf{q}$ cancels, and does not affect the result.

If both rotations are a half-turn ($\theta = \pi$), both $\mathbf{q}$ and $-\mathbf{q}$ will have real coordinate equal to zero. Otherwise, one will have a positive real part, representing a rotation by an angle less than π , and the other will have a negative real part, representing a rotation by an angle greater than π .

Mathematically, this operation carries the set of all "pure" quaternions $\mathbf{p}$ (those with real part equal to zero) - which constitute a 3-dimensional space among the

quaternions - into itself, by the desired rotation about the axis $\mathbf{u}$, by the angle θ .

The rotation is **clockwise** if our line of sight points in the same direction as $\bar{\mathbf{u}}$.

In the instance that $\mathbf{q}$ is a **unit quaternion** and

$$\mathbf{q}^{-1} = e^{-\frac{\theta}{2}(u_x\mathbf{i}+u_y\mathbf{j}+u_z\mathbf{k})} = \cos \frac{\theta}{2} - (u_x\mathbf{i} + u_y\mathbf{j} + u_z\mathbf{k}) \sin \frac{\theta}{2} \quad (1.160)$$

It follows that conjugation by the product of two quaternions is the composition of conjugations by these quaternions. If $\mathbf{p}$ and $\mathbf{q}$ are unit quaternions, then conjugation (**rotation**) by $\mathbf{pq}$ is:

$$\mathbf{pq}\bar{\mathbf{v}}(\mathbf{pq})^{-1} = \mathbf{pq}\bar{\mathbf{v}}\mathbf{q}^{-1}\mathbf{p}^{-1} = \mathbf{p}(\mathbf{q}\bar{\mathbf{v}}\mathbf{q}^{-1})\mathbf{p}^{-1} \quad (1.161)$$

which is **the same** as conjugating (**rotating**) by $\mathbf{q}$ and then by $\mathbf{p}$. The scalar component of the result is **necessarily zero**.

The quaternion **inverse** of a rotation is the opposite rotation, since

$$\mathbf{q}^{-1}(\mathbf{q}\bar{\mathbf{v}}\mathbf{q}^{-1})\mathbf{q} = \bar{\mathbf{v}} \quad (1.162)$$

The square of a quaternion rotation is a rotation by twice the angle around the same axis.

Chapter 2

Boundary conditions not in GCS

2.1 Skew boundary conditions

Skew boundary conditions, also known as **Rotated BCs** are implemented after assembling the global stiffness matrix $\mathbf{K}$.

Basic equation:

$$\mathbf{M}\ddot{\mathbf{u}} + \mathbf{C}\dot{\mathbf{u}} + \mathbf{K}\mathbf{u} = \mathbf{f} \quad (2.1)$$

First we need to identify all prescribed BCs that do not correspond to the defined assemblage DOFs and transform the finite equilibrium equations to correspond to the prescribed BCs. Thus we write:

$$\mathbf{u} = \mathbf{T}\bar{\mathbf{u}} \quad (2.2)$$

where $\bar{\mathbf{u}}$ is the vector of nodal point DOFs in the required directions. The transformation matrix $\mathbf{T}$ is an identity matrix that has been altered by the directional

sines and cosines of the components $\bar{\mathbf{u}}$ measured in the original displacement directions. Using (2.2), we obtain:

$$\bar{\mathbf{M}}\bar{\mathbf{u}} + \bar{\mathbf{C}}\bar{\mathbf{u}} + \bar{\mathbf{K}}\bar{\mathbf{u}} = \bar{\mathbf{f}} \quad (2.3)$$

where:

$$\begin{aligned} \bar{\mathbf{M}} &= \mathbf{T}^T \mathbf{M} \mathbf{T} \\ \bar{\mathbf{C}} &= \mathbf{T}^T \mathbf{C} \mathbf{T} \\ \bar{\mathbf{K}} &= \mathbf{T}^T \mathbf{K} \mathbf{T} \\ \bar{\mathbf{f}} &= \mathbf{T}^T \mathbf{f} \end{aligned} \quad (2.4)$$

We should note that the matrix multiplications in (2.4) involve changes only in those columns and rows of $\mathbf{M}$, $\mathbf{C}$, $\mathbf{K}$ and $\mathbf{f}$ that are actually affected. In practice, the transformation is carried out effectively on the element level just prior to adding the element matrices to the global matrices.

2.2 Cartesian CSYS

When a node or an SPC is defined as a **rotated/skewed** or in a different type of Coordinate system (e.g. cylindrical) a transformation matrix needs to be applied to such nodes.

Cartesian Coordinate system definition by angle (2D):

$$\mathbf{u}_l = \mathbf{T}\mathbf{u}_g \quad (2.5)$$

Unpacked:

$$\begin{bmatrix} u_l \\ w_l \\ \psi_l \end{bmatrix} = \begin{bmatrix} \cos \alpha & \sin \alpha & 0 \\ -\sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_g \\ w_g \\ \psi_g \end{bmatrix} \quad (2.6)$$

Cartesian Coordinate system definition by vectors (3D):

when a **CSYS** is defined by an **origin** and **two vectors** $\mathbf{x} = (x_1 \ x_2 \ x_3)$ and $\mathbf{y} = (y_1 \ y_2 \ y_3)$, then for a Cartesian CSYS it suffices to:

$$\begin{aligned} \bar{\mathbf{x}} &= \frac{\mathbf{x}}{||\mathbf{x}||} \\ \bar{\mathbf{y}} &= \frac{\mathbf{y}}{||\mathbf{y}||} \\ \bar{\mathbf{z}} &= \bar{\mathbf{x}} \times \bar{\mathbf{y}} \end{aligned} \quad (2.7)$$

The transformation relation:

$$\mathbf{u}_l = \mathbf{T}\mathbf{u}_g \quad (2.8)$$

when unpacked:

$$\begin{bmatrix} u_l \\ v_l \\ w_l \\ \phi_l \\ \psi_l \\ \rho_l \end{bmatrix} = \begin{bmatrix} \mathbf{x} & \mathbf{0} \\ \mathbf{y} & \mathbf{0} \\ \mathbf{z} & \mathbf{0} \\ \mathbf{0} & \mathbf{x} \\ \mathbf{0} & \mathbf{y} \\ \mathbf{0} & \mathbf{z} \end{bmatrix} \begin{bmatrix} u_g \\ v_g \\ w_g \\ \phi_g \\ \psi_g \\ \rho_g \end{bmatrix} \quad (2.9)$$

Where $\mathbf{0}$ is a 3×1 zero vector.

For a **2D** case just **one vector** is enough, the $\bar{\mathbf{z}} = (0 \ 0 \ 1)$ and following relations apply:

$$\begin{aligned}\bar{\mathbf{x}} &= \bar{\mathbf{y}} \times \bar{\mathbf{z}} \\ \bar{\mathbf{y}} &= \bar{\mathbf{z}} \times \bar{\mathbf{x}} \\ \bar{\mathbf{z}} &= \bar{\mathbf{x}} \times \bar{\mathbf{y}}\end{aligned}\tag{2.10}$$

This means that a **CSYS** defined by an **origin** point $\mathbf{O} = (x_O \ y_O \ z_O)$, point laying on **x axis** $\mathbf{P}_x = (x_{P_x} \ y_{P_x} \ z_{P_x})$ and a point laying in **xy plane** $\mathbf{P}_{xy} = (x_{P_{xy}} \ y_{P_{xy}} \ z_{P_{xy}})$ define a CSYS followingly:

$$\begin{aligned}\mathbf{x} &= (x_{P_x} - x_O \ y_{P_x} - y_O \ z_{P_x} - z_O) \\ \bar{\mathbf{x}} &= \frac{\mathbf{x}}{||\mathbf{x}||} \\ \hat{\mathbf{y}} &= (x_{P_{xy}} - x_O \ y_{P_{xy}} - y_O \ z_{P_{xy}} - z_O) \\ \bar{\hat{\mathbf{y}}} &= \frac{\hat{\mathbf{y}}}{||\hat{\mathbf{y}}||} \\ \bar{\mathbf{z}} &= \bar{\mathbf{x}} \times \bar{\hat{\mathbf{y}}} \\ \bar{\mathbf{y}} &= \bar{\mathbf{z}} \times \bar{\mathbf{x}}\end{aligned}\tag{2.11}$$

2.3 Cylindrical CSYS

The basic relation between a point in **cartesian** and **cylindrical** CSYS is (when the cartesian CSYS has the same origin as the cylindrical one and both CSYS z axes overlay):

$$\begin{aligned}x &= r \cos \varphi \\y &= r \sin \varphi \\z &= z\end{aligned}\tag{2.12}$$

in one direction and:

$$\begin{aligned}r &= \sqrt{x^2 + y^2} \\ \varphi &= \begin{cases} \text{indeterminate} & \text{if } x = 0 \text{ and } y = 0 \\ \arcsin \frac{y}{r} & \text{if } x \geq 0 \\ -\arcsin \frac{y}{r} + \pi & \text{if } x < 0 \text{ and } y \geq 0 \\ -\arcsin \frac{y}{r} + \pi & \text{if } x < 0 \text{ and } y < 0 \end{cases}\end{aligned}\tag{2.13}$$

in the other. One can also use the arctan function to compute φ :

$$\varphi = \begin{cases} \text{indeterminate} & \text{if } x = 0 \text{ and } y = 0 \\ \frac{\pi}{2} \frac{y}{|y|} & \text{if } x = 0 \text{ and } y \neq 0 \\ \arctan \frac{y}{x} & \text{if } x > 0 \\ \arctan \frac{y}{x} + \pi & \text{if } x < 0 \text{ and } y \geq 0 \\ \arctan \frac{y}{x} - \pi & \text{if } x < 0 \text{ and } y < 0 \end{cases}\tag{2.14}$$

This is also called the $\text{atan2}(y, x)$ function!

To transform node coordinates from **cartesian** to **cylindrical**, three transforms are in order:

1. translate the coordinates so that **origin** of the new CSYS conforms to the **origin** of the **cylindrical** CSYS.

$$\mathbf{x}_1 = \mathbf{T}_T \mathbf{x}_0\tag{2.15}$$

$$\begin{bmatrix} x_1 \\ y_1 \\ z_1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & -x_O^c \\ 0 & 1 & 0 & -y_O^c \\ 0 & 0 & 1 & -z_O^c \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_0 \\ y_0 \\ z_0 \\ 1 \end{bmatrix} \quad (2.16)$$

where $\mathbf{x}_O^c$ denotes the **cylindrical** CSYS origin and $\mathbf{T}_T$ denotes the translation matrix.

2. **rotate** the node to a new **cartesian** CSYS so that the new CSYS **x-axis** conforms to the cylindrical CSYS **r-axis** and the new CSYS **z-axis** conforms to the cylindrical **z-axis**.

$$\mathbf{x}_2 = \mathbf{T}_R \mathbf{x}_1 \quad (2.17)$$

When the cylindrical CSYS is defined by three points - **origin** $\mathbf{x}_O^c = [x_O^c \ y_O^c \ z_O^c]^T$, point on **r-axis** $\mathbf{x}_R^c$ and a point in **rz-plane** $\mathbf{x}_{RZ}^c$, then:

$$\vec{\mathbf{r}} = \frac{\mathbf{x}_R^c - \mathbf{x}_O^c}{\|\mathbf{x}_R^c - \mathbf{x}_O^c\|} \quad (2.18)$$

$$\hat{\mathbf{z}} = \frac{\mathbf{x}_{RZ}^c - \mathbf{x}_O^c}{\|\mathbf{x}_{RZ}^c - \mathbf{x}_O^c\|} \quad (2.19)$$

$$\vec{\mathbf{y}} = \hat{\mathbf{z}} \times \vec{\mathbf{r}} \quad (2.20)$$

$$\vec{\mathbf{z}} = \vec{\mathbf{r}} \times \vec{\mathbf{y}} \quad (2.21)$$

then the transformation can be written:

$$\begin{bmatrix} x_2 \\ y_2 \\ z_2 \\ 1 \end{bmatrix} = \begin{bmatrix} \vec{\mathbf{r}}^T & 0 \\ \vec{\mathbf{y}}^T & 0 \\ \vec{\mathbf{z}}^T & 0 \\ \vec{\mathbf{0}}^T & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \\ z_1 \\ 1 \end{bmatrix} \quad (2.22)$$

where $\vec{\mathbf{0}} = [0 \ 0 \ 0]^T$

Combining $\mathbf{T}_R$ and $\mathbf{T}_T$ together:

$$\mathbf{T} = \mathbf{T}_R \mathbf{T}_T \quad (2.23)$$

A **cylindrical** coordinate system in FEM in the range of **small displacements** is basically a Cartesian one, just skewed for each node separately so that its **x** axis coincides with **r** axis.

Then the cylindrical to global coordinate transformation is such that:

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix}_{global} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix} \begin{bmatrix} r \cos \varphi \\ r \sin \varphi \\ z \end{bmatrix}_{local} + \begin{bmatrix} x \\ y \\ z \end{bmatrix}_{origin} \quad (2.27)$$

Therefore the Cartesian CSYS transformation matrix for definition of boundary conditions is:

1. first subtract the cylindrical coordinate origin from the global coordinates:

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix}_{local} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}_{global} - \begin{bmatrix} x \\ y \\ z \end{bmatrix}_{origin} \quad (2.28)$$

2. then create a transformation matrix that maps the global csys to rotated one such that the **r** axis coincides with the **x** axis, **φ** axis coincides with the new **y** axis and **z** is the same:

$$\mathbf{r}' = \frac{\mathbf{x}_{local}}{\|\mathbf{x}_{local}\|} \quad (2.29)$$

$$\mathbf{z} = \frac{\mathbf{z}_{cyl}}{\|\mathbf{z}_{cyl}\|} \quad (2.30)$$

$$\boldsymbol{\varphi} = \mathbf{z} \times \mathbf{r}' \quad (2.31)$$

$$\mathbf{r} = \boldsymbol{\varphi} \times \mathbf{z} \quad (2.32)$$

the transformation matrix is then:

$$\mathbf{T} = \begin{bmatrix} r_1 & r_2 & r_3 \\ \varphi_1 & \varphi_2 & \varphi_3 \\ z_1 & z_2 & z_3 \end{bmatrix} \quad (2.33)$$

3. This transformation matrix **T** is finally applied and **BCs** are imposed.

$$\mathbf{T}^T \mathbf{K} \mathbf{T} \bar{\mathbf{u}} = \mathbf{T}^T \mathbf{f} \quad (2.34)$$

If there already is a transformation matrix for the cylindrical system prepared, such that:

$$\mathbf{x}_{local} = \mathbf{T}_{cyl} (\mathbf{x}_{global} - \mathbf{x}_{origin}) \quad (2.35)$$

or:

$$\mathbf{x}_{global} = \mathbf{T}_{cyl}^T \mathbf{x}_{local} + \mathbf{x}_{origin} \quad (2.36)$$

then:

$$r = \sqrt{x_{local}^2 + y_{local}^2} \quad (2.37)$$

$$\varphi = \text{atan2} \frac{y_{local}}{x_{local}} \quad (2.38)$$

$$z = z_{local} \quad (2.39)$$

because $\cos \varphi = x/r$ and $\sin \varphi = y/r$, then the new transformation matrix is composed:

$$\mathbf{T} = \begin{bmatrix} \cos & -\sin & 0 \\ \sin & \cos & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} x/r & -y/r & 0 \\ y/r & x/r & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (2.40)$$

and finally displacements are transformed from global to local:

$$\mathbf{u}_{local} = \mathbf{T} \mathbf{T}_{cyl} \mathbf{u}_{global} \quad (2.41)$$

Chapter 3

Multifreedom Constraints

The *multifreedom equality constraints*, or *multifreedom constraints* for short (**MFCs**) are functional equations that connect *two or more* displacement components:

$$F(\text{nodal displacement components}) = \text{prescribed value} \quad (3.1)$$

where function F vanishes if all its nodal displacement arguments do. Equation (3.1) is called the *canonical form* of the constraint.

An **MFC** of this form is called a *multipoint* or *multinode* if it involves displacement components at different nodes. The constraint is called *linear* if all displacement components appear linearly on the **LHS**, and *nonlinear* otherwise.

The constraint is called *homogeneous* if, upon transferring all terms that depend on displacement components to the **LHS**, the **RHS** - the "prescribed value" in (3.1) is **zero**. It is called *nonhomogeneous* otherwise.

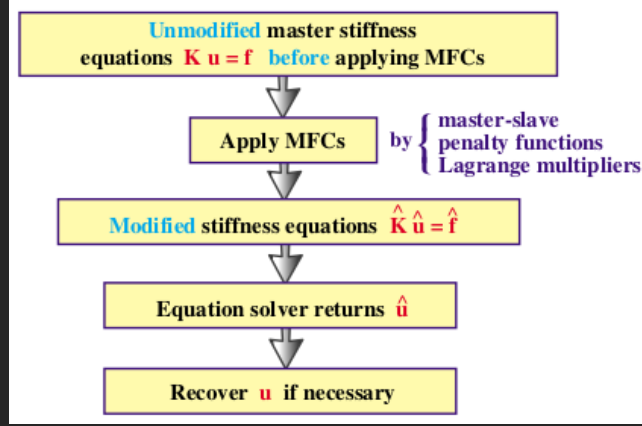


Figure 3.1: Schematics of MFC application.

3.1 Methods for Imposing MFCs

Accounting for MFCs is done, at least conceptually, by changing the assembled master stiffness equations to produce a *modified* system of equations:

$$\mathbf{K} \mathbf{u} = \mathbf{f} \xrightarrow{MFC} \hat{\mathbf{K}} \hat{\mathbf{u}} = \hat{\mathbf{f}} \quad (3.2)$$

The modification process (3.2) is also called *constraint application* or *constraint imposition*. The modified system is that submitted to the equation solver, which returns $\hat{\mathbf{u}}$.

Three methods for applying MFCs are written below:

1. *Master-Slave Elimination*

The DOFs involved in each MFC are separated into master and slave freedoms. The slave freedoms are then explicitly eliminated. The modified equations do not contain the slave freedoms.

2. *Penalty Augmentation*

Also called the *penalty function method*. Each **MFC** is viewed as the presence of a fictitious elastic structural element called the *penalty element* that enforces it approximately. This element is parametrized by a numerical *weight*. The exact constraint is recovered if the weight goes to **infinity**. The MFCs are imposed by augmenting the finite element model with the penalty elements.

3. *Lagrange Multiplier Adjunction*

For each MFC an additional unknown is adjoined to the master stiffness equations. Physically this set of unknowns represent *constraint forces* that would enforce the constraints exactly should they be applied to the unconstrained system.

The master stiffness equations are assembled ignoring all constraints. Then the MFCs are imposed by appropriate modification of those equations. There are, however, two important practical differences:

1. The modification process is not unique because there are alternative constraint imposition methods. These methods offer tradeoffs in generality, programming implementation complexity, computational effort, numerical accuracy and stability.
2. In the implementation of some of these methods - notably penalty augmentation - constraint imposition and assembly are carried out simultaneously. In that case the framework "first assemble, then modify", is not strictly respected in the actual implementation.

3.1.1 MFC Matrix Forms

Matrix forms of *linear* **MFCs** are often convenient for compact notation. An individual constraint may be written:

$$\begin{bmatrix} 1 & -2 & 1 \end{bmatrix} \begin{bmatrix} u_{x2} \\ u_{x4} \\ u_{x6} \end{bmatrix} = 0.25 \quad (3.3)$$

In direct matrix notation:

$$\bar{\mathbf{a}}_i \bar{\mathbf{u}}_i = g_i, \quad (\text{no sum on } i) \quad (3.4)$$

in which index i ($i = 1, 2, \dots$) identifies constraint, $\bar{\mathbf{a}}_i$ is a row vector, $\bar{\mathbf{u}}_i$ collects the set of **DOFs** that participate in the constraint, and g_i is the **RHS** scalar. The bars over $\mathbf{a}$ and $\mathbf{u}$ distinguishes (3.4) from the expanded form below.

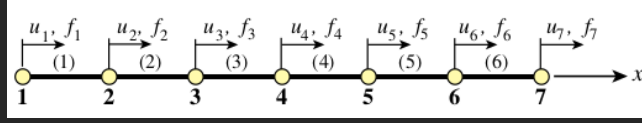


Figure 3.2: 1D problem discretized with six bar finite elements.

For method description and general proof it is often convenient to *expand* matrix forms so that they embody *all* DOFs. For example, if (3.3) is a part of a two-dimensional FE model with 12 DOFs:

$u_{x1}, u_{y1}, \dots, u_{y6}$, the LHS row vector may be expanded with 9 zeros as follows:

$$\begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 & -2 & 0 & 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} u_{x1} \\ u_{y1} \\ u_{x2} \\ \vdots \\ u_{y6} \end{bmatrix} = 0.25 \quad (3.5)$$

In which case the matrix notation:

$$\mathbf{a}_i \mathbf{u}_i = g_i \quad (3.6)$$

is used. Finally, all **MFCs** expressed as (3.6) may be collected into a single matrix relation:

$$\mathbf{A} \mathbf{u} = \mathbf{g} \quad (3.7)$$

in which the rectangular matrix $\mathbf{A}$ is formed by stacking the $\mathbf{a}_i$'s as rows and column vector $\mathbf{g}$ is formed by stacking the g_i s as entries. If there are 12 DOFs in $\mathbf{u}$ and 5 MFCs, then $\mathbf{A}$ will be 5×12 .

3.1.2 The Example Structure

The one-dimensional FE discretization shown in Figure 3.2 will be used to illustrate the three MFC application methods. The structure consists of six bar elements connected by seven nodes that can only displace in the x direction.

Before imposing various **multifreedom** constraints discussed below, the master stiffness equations for this problem are assumed to be:

$$\begin{bmatrix} K_{11} & K_{12} & 0 & 0 & 0 & 0 & 0 \\ K_{12} & K_{22} & K_{23} & 0 & 0 & 0 & 0 \\ 0 & K_{23} & K_{33} & K_{34} & 0 & 0 & 0 \\ 0 & 0 & K_{34} & K_{44} & K_{45} & 0 & 0 \\ 0 & 0 & 0 & K_{45} & K_{55} & K_{56} & 0 \\ 0 & 0 & 0 & 0 & K_{56} & K_{66} & K_{67} \\ 0 & 0 & 0 & 0 & 0 & K_{67} & K_{77} \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \\ u_7 \end{bmatrix} = \begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \\ f_5 \\ f_6 \\ f_7 \end{bmatrix} \quad (3.8)$$

or

$$\mathbf{K}\mathbf{u} = \mathbf{f} \quad (3.9)$$

The nonzero stiffness coefficients K_{ij} in (3.8) depend on the bar rigidity properties. For example, if $E^e A^e / L^e = 100$ for each element $e = 1, \dots, 6$, then $K_{11} = K_{77} = 100$, $K_{22} = \dots = K_{66} = 200$, $K_{12} = K_{23} = \dots = K_{67} = -100$. However, for the purposes of the following treatment the coefficient may be kept arbitrary. The component index x in the nodal displacements u and nodal forces f has been omitted for brevity.

Now let us specify a MFC that states that nodes **2** and **6** must move by the same amount:

$$u_2 = u_6 \quad (3.10)$$

Passing all node displacements to the RHS gives the canonical form:

$$u_2 - u_6 = 0 \quad (3.11)$$

Constraint conditions of this type are sometimes called **rigid links** because they can be mechanically interpreted as forcing node points 2 and 6 to move together as if they were tied together by a rigid member.

We now study the imposition of constraint (3.11) on the master equations (3.8) by the methods mentioned above. First the master-slave method is treated

3.2 The Master-Slave Method

To apply this method by *hand*, the MFCs are taken one at a time. For each constraint a **slave** DOF is chosen. The freedoms remaining in that constraint are labeled **master**. A new set of DOFs $\hat{\mathbf{u}}$ is established by removing all slave freedoms from $\mathbf{u}$. This new vector contains master freedoms as well as those that do not appear in the MFCs. A matrix transformation equation that relates $\mathbf{u}$ to $\hat{\mathbf{u}}$ is generated. This equation is used to apply a congruent transformation to the master stiffness equations. This procedure yields a set of modified stiffness equations that are expressed in terms of the new freedom set $\hat{\mathbf{u}}$. Because the modified system does not contain the slave freedoms, these have been effectively eliminated.

3.2.1 A One-Constraint Example

The mechanics of the process is best seen by going through an example. To impose (3.11) pick u_6 as slave and u_2 as master. Relate the original unknowns $u_1, \dots, u_7$ to the new set in which u_6 is missing:

$$\begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \\ u_7 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_7 \end{bmatrix} \quad (3.12)$$

This is the required transformation relation. In compact form:

$$\mathbf{u} = \mathbf{T}\hat{\mathbf{u}} \quad (3.13)$$

Replacing (3.12) into (3.9) and premultiplying by $\mathbf{T}^T$ yields the modified system:

$$\hat{\mathbf{K}}\hat{\mathbf{u}} = \hat{\mathbf{f}}, \quad \text{in which} \quad \hat{\mathbf{K}} = \mathbf{T}^T \mathbf{K} \mathbf{T}, \quad \hat{\mathbf{f}} = \mathbf{T}^T \mathbf{f}. \quad (3.14)$$

The form of modified system (3.14) can be remembered by a simple mnemonic rule: premultiply both sides of $\mathbf{T}\hat{\mathbf{u}} = \mathbf{u}$ by $\mathbf{T}^T \mathbf{K}$ and replace $\mathbf{K}\mathbf{u}$ by $\mathbf{f}$ on the right hand side.

Carrying out the indicated matrix multiplication yields:

$$\begin{bmatrix} K_{11} & K_{12} & 0 & 0 & 0 & 0 \\ K_{12} & K_{22} + K_{66} & K_{23} & 0 & K_{56} & K_{67} \\ 0 & K_{23} & K_{33} & K_{34} & 0 & 0 \\ 0 & 0 & K_{34} & K_{44} & K_{45} & 0 \\ 0 & K_{56} & 0 & K_{45} & K_{55} & 0 \\ 0 & K_{67} & 0 & 0 & 0 & K_{77} \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_7 \end{bmatrix} = \begin{bmatrix} f_1 \\ f_2 + f_6 \\ f_3 \\ f_4 \\ f_5 \\ f_7 \end{bmatrix} \quad (3.15)$$

Equation (3.15) is a new linear system containing 6 equations in the remaining 6 unknowns u_1, u_2, u_3, u_4, u_5 and u_7 . Upon solving it, u_6 is recovered from the constraint (3.10).

For a simple freedom constraint such as $u_4 = 0$ the only possible choice of slave is of course u_4 and there is no master. The congruent transformation is then nothing more than the elimination of u_4 by striking out rows and columns from the master stiffness equations.

3.2.2 Multiple Homogeneous MFCs

The matrix equation (3.14) in fact holds for the general case of multiple homogeneous linear constraints. Direct establishment of the transformation equation, however, is more complicated if slave freedoms in one constraint appear as masters in another. To illustrate this point, suppose that for the example system we have three homogeneous MFCs:

$$\begin{aligned} u_2 - u_6 &= 0 \\ u_1 + 4u_4 &= 0 \\ 2u_3 + u_4 + u_5 &= 0 \end{aligned} \tag{3.16}$$

Picking as slave freedoms u_6 , u_4 and u_3 from the first, second and third constraint, respectively, we can solve for them as:

$$\begin{aligned} u_6 &= u_2 \\ u_4 &= -\frac{1}{4}u_1 \\ u_3 &= -\frac{1}{2}(u_4 + u_5) = \frac{1}{8}u_1 - \frac{1}{2}u_5 \end{aligned} \tag{3.17}$$

Observe that solving for u_3 from the third constraint brings u_4 to the RHS. But because u_4 is also slave freedom (it was chosen as such for the second constraint) it is replaced in favor of u_1 using $u_4 = -\frac{1}{4}u_1$. The matrix form transformation is:

$$\begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \\ u_7 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ \frac{1}{8} & 0 & -\frac{1}{2} & 0 \\ -\frac{1}{4} & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_5 \\ u_7 \end{bmatrix} \quad (3.18)$$

The modified master system is now formed through the congruent transformation (3.14). Note that the slave freedoms selected from each constraint must be distinct. For example the choice u_6 , u_4 and u_4 would be inadmissible as long the constraints are independent. This rule is easy to enforce when slave freedoms are chosen by hand, but can lead to implementation and numerical difficulties when it is programmed as an automated procedure, as further discussed later.

The 3 MFCs (3.16) with u_6 , u_4 and u_2 chosen as slaves and u_1 , u_2 and u_5 chosen as masters, may be presented in the partitioned matrix form:

$$\begin{bmatrix} 0 & 0 & 1 \\ 0 & 4 & 0 \\ 2 & 1 & 0 \end{bmatrix} \begin{bmatrix} u_3 \\ u_4 \\ u_6 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & -1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_5 \end{bmatrix} \quad (3.19)$$

This may be compactly written as:

$$\mathbf{A}_s \mathbf{u}_s + \mathbf{A}_m \mathbf{u}_m = \mathbf{0} \quad (3.20)$$

Solving for the slave freedoms gives

$$\mathbf{u}_s = -\mathbf{A}_s^{-1} \mathbf{A}_m \mathbf{u}_m \quad (3.21)$$

Expanding with zeros to fill out $\mathbf{u}$ and $\hat{\mathbf{u}}$ produces (3.18). Note that non-singularity of $\mathbf{A}_s$ is essential for this method to work.

3.2.3 Nonhomogeneous MFCs

Extension to nonhomogeneous constraints is immediate. In this case the transformation equation becomes nonhomogeneous. For example suppose that (3.11) has non-zero prescribed value:

$$u_2 - u_6 = 0.2 \quad (3.22)$$

Nonzero RHS values such as 0.2 in (3.22) may often be interpreted physically as "gaps" (thus the use of the symbol $\mathbf{g}$ in the matrix form). Chose u_6 again as slave: $u_6 = u_2 - 0.2$ and build the transformation:

$$\begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \\ u_7 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \\ u_7 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ -0.2 \\ 0 \end{bmatrix} \quad (3.23)$$

In the compact matrix form:

$$\mathbf{u} = \mathbf{T}\hat{\mathbf{u}} + \mathbf{g} \quad (3.24)$$

Here the constraint gap vector $\mathbf{g}$ is nonzero and $\mathbf{T}$ is the same as before. To get the modified system applying the shortcut rule, premultiply both sides of (3.24) by $\mathbf{T}^T \mathbf{K}$, replace $\mathbf{K}\mathbf{u}$ by $\mathbf{f}$ and pass the data to the RHS:

$$\hat{\mathbf{K}}\hat{\mathbf{u}} = \hat{\mathbf{f}}, \quad \text{in which} \quad \hat{\mathbf{K}} = \mathbf{T}^T \mathbf{K} \mathbf{T}, \quad \hat{\mathbf{f}} = \mathbf{T}^T (\mathbf{f} - \mathbf{K}\mathbf{g}). \quad (3.25)$$

Upon solving (3.25) for $\hat{\mathbf{u}}$, the complete displacement vector is recovered from (3.24). For the MFC (3.22) this technique gives the system:

$$\begin{bmatrix} K_{11} & K_{12} & 0 & 0 & 0 & 0 \\ K_{12} & K_{22} + K_{66} & K_{23} & 0 & K_{56} & K_{67} \\ 0 & K_{23} & K_{33} & K_{34} & 0 & 0 \\ 0 & 0 & K_{34} & K_{44} & K_{45} & 0 \\ 0 & K_{56} & 0 & K_{45} & K_{55} & 0 \\ 0 & K_{67} & 0 & 0 & 0 & K_{77} \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_7 \end{bmatrix} = \begin{bmatrix} f_1 \\ f_2 + f_6 - 0.2K_{66} \\ f_3 \\ f_4 \\ f_5 - 0.2K_{56} \\ f_7 - 0.2K_{67} \end{bmatrix} \quad (3.26)$$

3.2.4 The General Case

For implementation in general-purpose programs the **master-slave method** can be described as follows. The **DOFs** in $\mathbf{u}$ are classified into three types: **independent** or unconstrained, **masters** and **slaves**. The unconstrained DOFs are those that do not appear in any MFC. Label these sets as $\mathbf{u}_u$, $\mathbf{u}_m$ and $\mathbf{u}_s$, respectively, and partition the stiffness equations accordingly:

$$\begin{bmatrix} \mathbf{K}_{uu} & \mathbf{K}_{um} & \mathbf{K}_{us} \\ \mathbf{K}_{um}^T & \mathbf{K}_{mm} & \mathbf{K}_{ms} \\ \mathbf{K}_{us}^T & \mathbf{K}_{ms}^T & \mathbf{K}_{ss} \end{bmatrix} \begin{bmatrix} \mathbf{u}_u \\ \mathbf{u}_m \\ \mathbf{u}_s \end{bmatrix} = \begin{bmatrix} \mathbf{f}_u \\ \mathbf{f}_m \\ \mathbf{f}_s \end{bmatrix} \quad (3.27)$$

The MFCs may be written in matrix form as:

$$\mathbf{A}_m \mathbf{u}_m + \mathbf{A}_s \mathbf{u}_s = \mathbf{g}_A \quad (3.28)$$

where $\mathbf{A}_s$ is assumed square and nonsingular. If so we can solve for the slave freedoms:

$$\mathbf{u}_s = \mathbf{A}_s^{-1} \mathbf{A}_m \mathbf{u}_m + \mathbf{A}_s^{-1} \mathbf{g}_A \stackrel{\triangle}{=} \mathbf{T} \mathbf{u}_m + \mathbf{g} \quad (3.29)$$

where $\stackrel{\triangle}{=}$ means *equal by definition*. Inserting into the partitioned stiffness equation (3.27) and symmetrizing yields:

$$\begin{bmatrix} \mathbf{K}_{uu} & \hat{\mathbf{K}}_{um} \\ \hat{\mathbf{K}}_{um}^T & \hat{\mathbf{K}}_{mm} \end{bmatrix} \begin{bmatrix} \mathbf{u}_u \\ \mathbf{u}_m \end{bmatrix} = \begin{bmatrix} \mathbf{f}_u - \mathbf{K}_{us}\mathbf{g} \\ \mathbf{f}_m - \mathbf{K}_{ms}\mathbf{g} \end{bmatrix} \quad (3.30)$$

where $\hat{\mathbf{K}}_{um} = \mathbf{K}_{um} + \mathbf{K}_{us}\mathbf{T}$
and $\hat{\mathbf{K}}_{mm} = \mathbf{K}_{mm} + \mathbf{T}^T\mathbf{K}_{ms}^T + \mathbf{K}_{ms}\mathbf{T} + \mathbf{T}^T\mathbf{K}_{ss}\mathbf{T}$.

It is seen that the misleading simplicity of the handworked example is gone.

3.2.5 Retaining the Original Freedoms

A potential disadvantage of the master-slave method in computer work is that it requires a rearrangement of the original stiffness equations because $\hat{\mathbf{u}}$ is a subset of $\mathbf{u}$. The disadvantage can be annoying when sparse matrix storage schemes are used for the stiffness matrix, and becomes intolerable if secondary storage is used for that purpose.

With a bit of trickery it is possible to maintain the original freedom ordering. Let us display it for the example problem under (3.11). Instead of (3.12), use the *square* transformation:

$$\begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \\ u_7 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ \bar{u}_6 \\ u_7 \end{bmatrix} \quad (3.31)$$

in which $\bar{u}_6$ is a *placeholder* for the slave freedom u_6 . The modified equations are:

$$\begin{bmatrix}
K_{11} & K_{12} & 0 & 0 & 0 & 0 & 0 & 0 \\
K_{12} & K_{22} + K_{66} & K_{23} & 0 & 0 & K_{56} & 0 & K_{67} \\
0 & K_{23} & K_{33} & K_{34} & 0 & 0 & 0 & 0 \\
0 & 0 & K_{34} & K_{44} & K_{45} & 0 & 0 & 0 \\
0 & K_{56} & 0 & K_{45} & K_{55} & 0 & 0 & 0 \\
0 & K_{67} & 0 & 0 & 0 & 0 & 0 & K_{77}
\end{bmatrix}
\begin{bmatrix}
u_1 \\
u_2 \\
u_3 \\
u_4 \\
u_5 \\
\bar{u}_6 \\
u_7
\end{bmatrix}
=
\begin{bmatrix}
f_1 \\
f_2 + f_6 \\
f_3 \\
f_4 \\
f_5 \\
0 \\
f_7
\end{bmatrix}
\tag{3.32}$$

which are submitted to the equation solver. If the solver is not trained to skip zero rows and columns, a one should be placed in the diagonal entry for the $\bar{u}_6$ equation. The solver will return $\bar{u}_6 = 0$, and this placeholder value is replaced by u_2 .

3.2.6 Model Reduction by Kinematic Constraints

The congruent transformation equations (3.14) and (3.25) have additional applications beyond the master-slave method. An important one is *model reduction by kinematic constraints*. Through this procedure the number of **DOFs** of a static or dynamic FEM model is reduced by a significant number, typically to 1% – 10% of the original number. This is done by taking a lot of slaves and a few masters. Only the masters are left after the transformation. The reduced model is commonly used in subsequent calculations as a component of a larger system, particularly during design or in parameter identification.

Example:

Consider the bar assembly of Figure 3.3. Assume that the only masters are the end motions u_1 and u_7 , as illustrated in Figure 3.3, and interpolate all freedoms linearly:

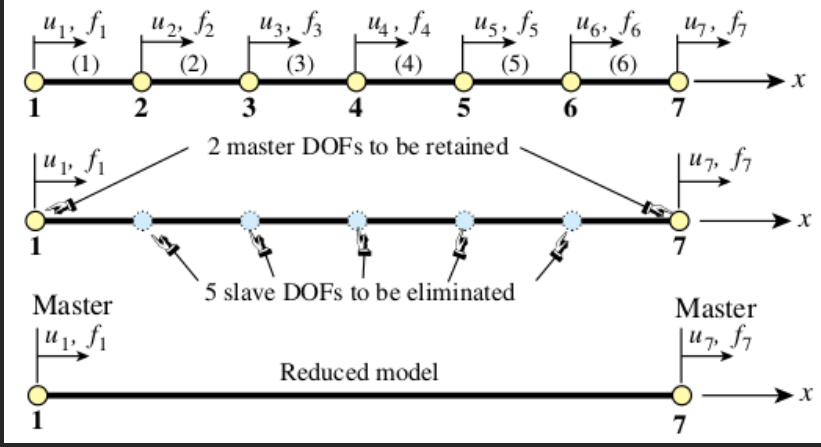


Figure 3.3: Model reduction of the example structure to the end freedoms

$$\begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \\ u_7 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 5 & 1 \\ 6 & 4 \\ 6 & 3 \\ 6 & 3 \\ 6 & 4 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_7 \end{bmatrix}, \quad \text{or} \quad \mathbf{u} = \mathbf{T}\hat{\mathbf{u}} \quad (3.33)$$

The reduced-order-model (**ROM**) equations are:

$$\hat{\mathbf{k}}\hat{\mathbf{u}} = \mathbf{T}^T \mathbf{K} \mathbf{T} \hat{\mathbf{u}} = \mathbf{T}^T \mathbf{f} = \hat{\mathbf{f}} \quad (3.34)$$

or in detail

$$\begin{bmatrix} \hat{K}_{11} & \hat{K}_{17} \\ \hat{K}_{17}^T & \hat{K}_{77} \end{bmatrix} \begin{bmatrix} u_1 \\ u_7 \end{bmatrix} = \begin{bmatrix} \hat{f}_1 \\ \hat{f}_7 \end{bmatrix} \quad (3.35)$$

in which:

$$\begin{aligned}
\hat{K}_{11} &= \frac{1}{36} (36K_{11} + 60K_{12} + 25K_{22} + 40K_{23} + 16K_{33} + 24K_{34} \\
&\quad + 9K_{44} + 12K_{45} + 4K_{55} + 4K_{56} + K_{66}) \\
\hat{K}_{17} &= \frac{1}{36} (6K_{12} + 5K_{22} + 14K_{23} + 8K_{33} + 18K_{34} + 9K_{44} \\
&\quad + 18K_{45} + 8K_{55} + 14K_{56} + 5K_{66} + 6K_{67}) \\
\hat{K}_{77} &= \frac{1}{36} (K_{22} + 4K_{23} + 4K_{33} + 12K_{34} + 9K_{44} + 24K_{45} \\
&\quad + 16K_{55} + 40K_{56} + 25K_{66} + 60K_{67} + 36K_{77}) \\
\hat{f}_1 &= \frac{1}{6} (6f_1 + 5f_2 + 4f_3 + 3f_4 + 2f_5 + f_6) \\
\hat{f}_7 &= \frac{1}{6} (f_2 + 2f_3 + 3f_4 + 4f_5 + 5f_6 + 6f_7)
\end{aligned} \tag{3.36}$$

This reduces the order of the FEM model from 7 to 2. The **key feature** is that the masters are picked **a priori**, as the freedoms to be retained in the model for further use.

Model reduction can be also done by the **static condensation** (Guyan) method. As its name indicates, condensation is restricted to static analysis. On the other hand, for such problems it is exact whereas model reduction by kinematic constraints generally introduces approximations.

3.2.7 Assesment of the Master-Slave Method

The **Master-Slave Method** enjoys the advantage of being **exact** (except for inevitable solution errors from finite number precision) and of reducing the nuber of unknowns. The concept is also easy to explain and learn. The main implementation drawback is the complexity of the general case. The complexity is due to three factors:

1. The equations may have to be rearranged because of the disappearance

of the slave freedoms. This drawback can be alleviated, however, by the **placeholder** trick.

2. An auxiliary linear system has to be assembled and solved to produce the transformation matrix $\mathbf{T}$ and vector $\mathbf{g}$.
3. The transformation process may generate many additional matrix terms. If a sparse matrix storage scheme is used for $\mathbf{K}$, the logic for allocating memory and storing these entries can be difficult and expensive.

The level of complexity depends on the generality allowed as well as on programming decisions. If $\mathbf{K}$ is stored as full matrix and slave freedom coupling in the MFCs is disallowed the logic is simple (this is the case in model reduction, since each slave freedom appears in one and only one MFC). On the other hand, if arbitrary couplings are permitted and $\mathbf{K}$ is placed in secondary (disk) storage according to some sparse scheme, the complexity can become overwhelming.

Another, more subtle, drawback of this method is that it requires decisions as to which DOFs are to be treated as slaves. This can lead to implementation and numerical stability problems. Although for disjointed constraints the process can be programmed in reliable form, in more general cases of coupled constraint equations it can lead to incorrect decisions. For example, suppose that in the example problem you have the following two MFCs:

$$\begin{aligned}\frac{1}{6}u_2 + \frac{1}{2}u_4 &= u_6 \\ u_3 + 6u_6 &= u_7\end{aligned}\tag{3.37}$$

For numerical stability reasons it is usually better to pick as slaves the freedoms with largest coefficients. If this is done, the program would select u_6 as slave freedoms from both constraints. This leads to a contradiction because having two constraints we must eliminate two slave DOFs, not just one. The resulting modified system would in fact be inconsistent. Although this defect can be easily fixed by the program logic in this case, one can imagine the complexity burden if faced with hundreds of thousands of MFCs.

Serious numerical problems can arise if the MFCs are not independent. For example:

$$\begin{aligned}
\frac{1}{6}u_2 &= u_6 \\
\frac{1}{3}u_3 + 6u_6 &= u_7 \\
u_2 + u_3 - u_7 &= 0
\end{aligned} \tag{3.38}$$

The last constraint is an exact linear combination of the first two. If the program blindly choses u_2 , u_3 and u_7 as slaves, the modified system is incorrect because we eliminate three equations when in fact there are only two independent constraints. Exact linear dependence, as before, can be recognised by a rank analysis of the $\mathbf{A}_s$ matrix defined. In the floating-point arithmetic, however, such detection may fail because that kind of computation is inexact by nature (The safest technique to identify dependencies is to do a singular value decomposition (SVD) of $\mathbf{A}_s$. This can be, however, prohibitively expensive if one is dealing with hundreds of thousands of constraints.).

The complexity of slave selection is in fact equivalent to that of automatically selecting kinematic redundancies in the Force Method of structural analysis. **It has led implementators of programs that use this method to require masters and slaves to be prescribed in the input data, thus transferring the burden to users.**

The method is not generally extensible to nonlinear constraints without case by case programming.

In conclusion, the master-slave method is useful when a few simple linear constraints are imposed by hand. As a general purpose technique for FEM it suffers from complexity and lack of robustness. It is worth learning, however, because of the great importance of congruent transformations in *model reduction* for static and dynamic problems.

Bibliography

Felippa - IFEM (Chapter 8)

Zienkiewicz and Taylor - The Finite Element Method *4th ed.* 1988

3.3 The Penalty Method

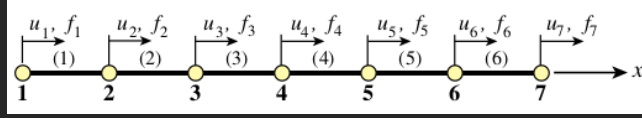


Figure 3.4: 1D problem discretized with six bar finite elements.

The penalty method will be first presented using a physical interpretation, leaving the mathematical formulation to a subsequent section. Consider again the bar example structure. To impose $u_2 = u_6$ imagine that nodes 2 and 6 are connected with a "fat" bar of axial stiffness w , labeled with element number 7, as shown in Figure 3.5. This bar is called **penalty element** and w is its **penalty weight**.

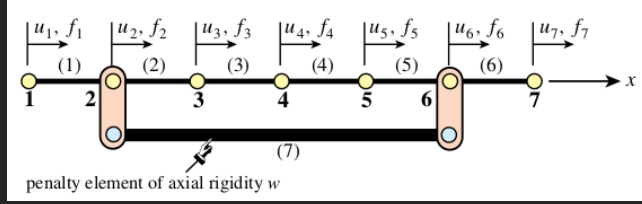


Figure 3.5: Adjunction of a fictitious penalty bar of axial stiffness w , identified as element 7, to enforce $u_2 = u_6$.

Such an element, albeit fictitious, can be treated exactly like another bar element insofar as continuing the assembly of the master stiffness equations. The penalty element stiffness equations $\mathbf{K}^{(7)}\mathbf{u}^{(7)} = \mathbf{f}^{(7)}$, are:

$$w \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} u_2 \\ u_6 \end{bmatrix} = \begin{bmatrix} f_2^{(7)} \\ f_6^{(7)} \end{bmatrix} \quad (3.39)$$

Because there is one freedom per node, the two local element freedoms map into global freedoms 2 and 6, respectively. Using the assembly rules we obtain the following modified master stiffness equations: $\hat{\mathbf{K}}\hat{\mathbf{u}} = \hat{\mathbf{f}}$, which shown in detail are:

$$\begin{bmatrix} K_{11} & K_{12} & 0 & 0 & 0 & 0 & 0 \\ K_{12} & K_{22} + w & K_{23} & 0 & 0 & -w & 0 \\ 0 & K_{23} & K_{33} & K_{34} & 0 & 0 & 0 \\ 0 & 0 & K_{34} & K_{44} & K_{45} & 0 & 0 \\ 0 & 0 & 0 & K_{45} & K_{55} & K_{56} & 0 \\ 0 & -w & 0 & 0 & K_{56} & K_{66} + w & K_{67} \\ 0 & 0 & 0 & 0 & 0 & K_{67} & K_{77} \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \\ u_7 \end{bmatrix} = \begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \\ f_5 \\ f_6 \\ f_7 \end{bmatrix} \quad (3.40)$$

This system can now be submitted to the equation solver. Note that $\hat{\mathbf{u}} \equiv \mathbf{u}$ and only $\mathbf{K}$ has changed.

3.3.1 Choosing the Penalty Weight

What happens when (3.40) is solved numerically? If a *finite* weight w is chosen the constraint $u_2 = u_6$ is approximately satisfied in the sense that one gets $u_2 - u_6 = e_g$, where $e_g \neq 0$. The "gap error" e_g is called the *constraint violation*. The magnitude $|e_g|$ of this violation depends on the weight: the larger w , the smaller the violation. More precisely, it can be shown that $|e_g|$ becomes proportional to $\frac{1}{w}$ as w gets to be sufficiently large. For example, raising w from, say, 10^6 to 10^7 can be expected to cut the constraint violation roughly by 10 if the physical stiffnesses are small compared to w .

Therefore it seems as if the proper strategy should be: try to make w as large as possible while respecting the computer overflow limits. However, this is misleading. As the penalty weight w tends to ∞ the modified stiffness matrix becomes more and more *ill-conditioned with respect to inversion*.

To make this point clear, suppose for definiteness that rigidities $E^e A^e / L^e$ of the actual bars $e = 1, \dots, 6$ are unity, that $w \gg 1$, and that the computer solving the stiffness equations has a floating-point precision of 16 decimal places. Numerical analysts characterize such precision by saying that $\epsilon_f = \mathcal{O}(10^{-16})$, where $|\epsilon_f|$ is the smallest power of 10 that perceptibly adds to 1 in floating-point arithmetic (this is usually done not for decimal numbers but base-2 binary numbers). The modified stiffness matrix of (3.40) becomes:

$$\hat{\mathbf{K}} = \begin{bmatrix} 1 & -1 & 0 & 0 & 0 & 0 & 0 \\ -1 & 2+w & -1 & 0 & 0 & -w & 0 \\ 0 & -1 & 2 & -1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 2 & -1 & 0 & 0 \\ 0 & 0 & 0 & -1 & 2 & -1 & 0 \\ 0 & -w & 0 & 0 & -1 & 2+w & -1 \\ 0 & 0 & 0 & 0 & 0 & -1 & 1 \end{bmatrix} \quad (3.41)$$

Clearly as $w \rightarrow \infty$ rows 2 and 6, as well as columns 2 and 6, tend to become linearly dependent, in fact the negative of each other. But **linear dependency means singularity!** Thus $\hat{\mathbf{K}}$ approaches singularity as $w \rightarrow \infty$. In fact, if w exceeds $1/\epsilon_f = 10^{16}$ the computer will not be able to distinguish $\hat{\mathbf{K}}$ from an exactly singular matrix. If $w \ll 10^{16}$, the effect will be seen in increasing solution errors affecting the computed displacements $\hat{\mathbf{u}}$ returned by the equation solver. These errors, however, tend to be more random in nature than the constraint violation error.

3.3.2 The Square Root Rule

Obviously we have two effects at odds with each other. Making w larger reduces the constraint violation error but increases the solution error. The best w is that which makes both errors roughly equal in absolute value. This tradeoff value is difficult to find aside of systematically running numerical experiments. In practice *square root rule* is often followed.

This rule can be stated as follows:

Suppose that the largest stiffness coefficient, before adding penalty elements, is of the order of 10^k and that the working machine precision is p digits (such order-of magnitude estimates can be easily found by scanning just the diagonal of $\mathbf{K}$). Then choose penalty weights to be of order $10^{k+p/2}$ with the proviso that such a choice would not cause arithmetic overflow (if overflow occurs, the master stiffness matrix should be scaled throughout or a better choice of physical units made).

For the above example in which $k \approx 0$ and $p \approx 16$, the optimal w given by this rule would be $w \approx 10^8$. This w would yield a constraint violation and a solution

error of order 10^{-8} . Note that there is no simple way to do better than this accuracy aside from using **extended floating-point precision**. This is not easy to do when using standard low-level programming languages.

The name "square root@" arises because the recommended w is in fact $10^k \sqrt{10^p}$. It is seen that picking the weight by this rule requires knowledge of both stiffness magnitudes and floating-point hardware properties of the computer used, as well as the precision selected by the program.

3.3.3 Penalty Elements for General MFCs

For the constraint $u_2 = u_6$ the physical interpretation of the penalty element is clear. Points 2 and 6 must move in lockstep along x , which can be approximately enforced by the heavy bar device. But how about $3u_3 + u_5 - 4u_6 = 1$ or just $u_2 = -u_6$?

The treatment of more general constraints is linked to the theory of *Courant penalty functions* from the *variational calculus*. Consider the homogenous constraint:

$$4u_3 + u_5 - 4u_6 = 0 \quad (3.42)$$

Rewrite this equation in matrix form:

$$\begin{bmatrix} 3 & 1 & -4 \end{bmatrix} \begin{bmatrix} u_3 \\ u_5 \\ u_6 \end{bmatrix} = 0 \quad (3.43)$$

and premultiply both sides by the transpose of the coefficient matrix:

$$\begin{bmatrix} 3 \\ 1 \\ -4 \end{bmatrix} \begin{bmatrix} 3 & 1 & -4 \end{bmatrix} \begin{bmatrix} u_3 \\ u_5 \\ u_6 \end{bmatrix} = \begin{bmatrix} 9 & 3 & -12 \\ 3 & 1 & -4 \\ -12 & -4 & 16 \end{bmatrix} \begin{bmatrix} u_3 \\ u_5 \\ u_6 \end{bmatrix} = \bar{\mathbf{K}}^e \mathbf{u} = 0 \quad (3.44)$$

Here $\bar{\mathbf{K}}^e$ is the *unscaled* stiffness matrix of the penalty element. This is now multiplied by the penalty weight w and assembled into the master stiffness

matrix following the usual rules. For the example problem, augmentint the master stiffness matrix with the w -scaled penalty element yields:

$$\begin{bmatrix} K_{11} & K_{12} & 0 & 0 & 0 & 0 & 0 \\ K_{12} & K_{22} & K_{23} & 0 & 0 & 0 & 0 \\ 0 & K_{23} & K_{33} + 9w & K_{34} & 3w & -12w & 0 \\ 0 & 0 & K_{34} & K_{44} & K_{45} & 0 & 0 \\ 0 & 0 & 3w & K_{45} & K_{55} + w & K_{56} - 4w & 0 \\ 0 & 0 & -12w & 0 & K_{56} - 4w & K_{66} + 16w & K_{67} \\ 0 & 0 & 0 & 0 & 0 & K_{67} & K_{77} \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \\ u_7 \end{bmatrix} = \begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \\ f_5 \\ f_6 \\ f_7 \end{bmatrix} \quad (3.45)$$

If the constraint is nonhomogeneous the force vector is also modified. To illustrate this effect, consider the MFC: $3u_3 + u_5 - 4u_6 = 1$. Rewrite this in matrix form as:

$$\begin{bmatrix} 3 & 1 & -4 \end{bmatrix} \begin{bmatrix} u_3 \\ u_5 \\ u_6 \end{bmatrix} = 1 \quad (3.46)$$

Premultiply both sides by the transpose of the coefficient matrix:

$$\begin{bmatrix} 9 & 3 & -12 \\ 3 & 1 & -4 \\ -12 & -4 & 16 \end{bmatrix} \begin{bmatrix} u_3 \\ u_5 \\ u_6 \end{bmatrix} = \begin{bmatrix} 3 \\ 1 \\ -4 \end{bmatrix} \quad (3.47)$$

Scaling by w and assembling yields:

$$\begin{bmatrix} K_{11} & K_{12} & 0 & 0 & 0 & 0 & 0 \\ K_{12} & K_{22} & K_{23} & 0 & 0 & 0 & 0 \\ 0 & K_{23} & K_{33} + 9w & K_{34} & 3w & -12w & 0 \\ 0 & 0 & K_{34} & K_{44} & K_{45} & 0 & 0 \\ 0 & 0 & 3w & K_{45} & K_{55} + w & K_{56} - 4w & 0 \\ 0 & 0 & -12w & 0 & K_{56} - 4w & K_{66} + 16w & K_{67} \\ 0 & 0 & 0 & 0 & 0 & K_{67} & K_{77} \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \\ u_7 \end{bmatrix} = \begin{bmatrix} f_1 \\ f_2 \\ f_3 + 3w \\ f_4 \\ f_5 + w \\ f_6 - 4w \\ f_7 \end{bmatrix} \quad (3.48)$$

3.3.4 The Theory Behind the Penalty Method Coefficient Matrix

The rule of *Courant penalty functions* comes from the following mathematical theory. Suppose we have a set of m linear MFCs. Using the matrix notation introduced, these will be stated as:

$$\mathbf{a}_p \mathbf{u} = b_p, \quad p = 1, \dots, m \quad (3.49)$$

where $\mathbf{u}$ contains all DOFs and each $\mathbf{a}_p$ is a row vector with the same length as $\mathbf{u}$. To incorporate the MFCs into the FEM model one selects a weight $w_p > 0$ for each constraint and constructs the so-called **Courant quadratic penalty function** or "penalty energy":

$$P = \sum_{p=1}^m P_p, \text{ with } P_p = \mathbf{u}^T \left(\frac{1}{2} \mathbf{a}_p^T \mathbf{a}_p \mathbf{u} - w_p \mathbf{a}_p^T b_p \right) = \frac{1}{2} \mathbf{u}^T \mathbf{K}^{(p)} \mathbf{u} - \mathbf{u}^T \mathbf{f}^{(p)} \quad (3.50)$$

where we have called $\mathbf{K}^{(p)} = w_p \mathbf{a}_p^T \mathbf{a}_p$ and $\mathbf{f}^{(p)} = w_p \mathbf{a}_p^T b_p$. P is added to the potential energy function $\Pi = \frac{1}{2} \mathbf{u}^T \mathbf{K} \mathbf{u} - \mathbf{u}^T \mathbf{f}$ to form the *augmented potential energy* $\Pi_a = \Pi + P$. Minimization of Π_a with respect to $\mathbf{u}$ yields:

$$\left(\mathbf{K} \mathbf{u} + \sum_{p=1}^m \mathbf{K}^{(p)} \right) \mathbf{u} = \mathbf{f} + \sum_{p=1}^m \mathbf{f}^{(p)} \quad (3.51)$$

Each term of the sum on p , which derives from term P_p may be viewed as contributed by a penalty element with globalised stiffness matrix $\mathbf{K}^{(p)} = w_p \mathbf{a}_p^T \mathbf{a}_p$ and globalised added force term $\mathbf{f}^{(p)} = w_p \mathbf{a}_p^T b_p$.

To use a even more compact form we may write the set of MFCs as $\mathbf{A} \mathbf{u} = \mathbf{b}$. Then the penalty augmented system can be written compactly as:

$$(\mathbf{K} + \mathbf{A}^T \mathbf{W} \mathbf{A}) \mathbf{u} = \mathbf{f} + \mathbf{W} \mathbf{A}^T \mathbf{b} \quad (3.52)$$

where $\mathbf{W}$ is a diagonal matrix of penalty weights. This compact form, however, conceals the configuration of the penalty elements.

3.3.5 Assessment of the Penalty Method

The main advantage of the penalty function method is its straightforward computer implementation. Looking at the modified systems it is obvious that master equations need not be rearranged. That is, $\mathbf{u}$ and $\hat{\mathbf{u}}$ are the same. Constraints may be programmed as "penalty elements", and stiffness and force contributions for these elements merged through the standard assembler. In fact using this method there is no need to distinguish between unconstrained and constrained equations! Once all elements - regular and penalty - are assembled, the system can be passed to the equation solver. **Single Freedom Constraints**, are usually processed separately for efficiency.

An important advantage with respect to the master-slave (elimination) method is its lack of sensitivity with respect to whether constraints are linearly dependent. To give a simplistic example, suppose that the constraint $u_2 = u_6$ appears twice. Then two penalty elements connecting 2 and 6 will be inserted, doubling the intended weight but not otherwise causing undue harm.

An advantage with respect to the Lagrange multiplier method is that positive definiteness is not lost. Such loss can affect the performance of certain numerical processes (for example solving the master stiffness equations by Cholesky factorisation or conjugate-gradients). Finally, it is worth noting that the penalty method is easily extensible to nonlinear constraints.

The main disadvantage, however, is a serious one: the choice of weight values that balance solution accuracy with the violation of constraint conditions. For simple cases the square root rule previously described often works, although its effective use calls for knowledge of the magnitude of stiffness coefficients. Such knowledge may be difficult to extract from general purpose "black box" program. For difficult cases selection of appropriate weights may require extensive numerical experimentation, wasting the user time with numerical games that have no bearing on the actual objective, which is getting a solution.

The deterioration of the condition number for the penalty-augmented stiffness matrix can have serious side effects in some solution procedures such as eigenvalue extraction or iterative solver.

Finally, even if optimal weights are selected, the combined solution error cannot be lowered beyond a threshold value.

From this assessment it is evident that penalty augmentation, although superior to the master-slave method from the standpoint of generality and ease of implementation, is no panacea.

3.4 Lagrange Multiplier Adjunction

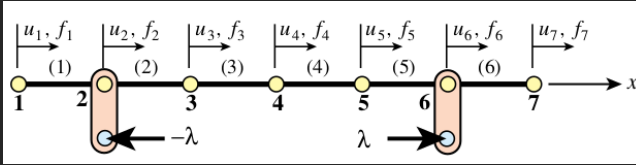


Figure 3.6: Physical interpretation of Lagrange multiplier adjunction to enforce the MFC $u_2 = u_6$

3.4.1 Physical Interpretation

As in the case of the penalty function method, the method Lagrange multiplier can be given a rigorous justification within the framework of variational calculus. But in the same spirit it will be introduced for the example structure from a physical standpoint that is particularly illuminating.

Consider again the constraint $u_2 = u_6$. Borrowing some ideas from the penalty method, imagine that nodes 2 and 6 are connected now by a *rigid* link rather than a flexible one. Thus the constraint is imposed exactly. But of course the penalty method with an infinite weight would "blow up".

We may remove the link if it is replaced by an appropriate reaction force pair $(-\lambda, +\lambda)$, as illustrated in Figure 3.6. These are called the *constraint forces*. Incorporating these forces into the original stiffness equations we get:

$$\begin{bmatrix} K_{11} & K_{12} & 0 & 0 & 0 & 0 & 0 & 0 \\ K_{12} & K_{22} & K_{23} & 0 & 0 & 0 & 0 & 0 \\ 0 & K_{23} & K_{33} & K_{34} & 0 & 0 & 0 & 0 \\ 0 & 0 & K_{34} & K_{44} & K_{45} & 0 & 0 & 0 \\ 0 & 0 & 0 & K_{45} & K_{55} & K_{56} & 0 & 0 \\ 0 & 0 & 0 & 0 & K_{56} & K_{66} & K_{67} & 0 \\ 0 & 0 & 0 & 0 & 0 & K_{67} & K_{77} & 0 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \\ u_7 \end{bmatrix} = \begin{bmatrix} f_1 \\ f_2 - \lambda \\ f_3 \\ f_4 \\ f_5 \\ f_6 + \lambda \\ f_7 \end{bmatrix} \quad (3.53)$$

This λ is called **Lagrange multiplier**. Because λ is an unknown, let us transfer it to the **LHS** by **appending** it to the vector of unknowns:

$$\begin{bmatrix} K_{11} & K_{12} & 0 & 0 & 0 & 0 & 0 & 0 \\ K_{12} & K_{22} & K_{23} & 0 & 0 & 0 & 0 & 1 \\ 0 & K_{23} & K_{33} & K_{34} & 0 & 0 & 0 & 0 \\ 0 & 0 & K_{34} & K_{44} & K_{45} & 0 & 0 & 0 \\ 0 & 0 & 0 & K_{45} & K_{55} & K_{56} & 0 & 0 \\ 0 & 0 & 0 & 0 & K_{56} & K_{66} & K_{67} & -1 \\ 0 & 0 & 0 & 0 & 0 & K_{67} & K_{77} & 0 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \\ u_7 \\ \lambda \end{bmatrix} = \begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \\ f_5 \\ f_6 \\ f_7 \end{bmatrix} \quad (3.54)$$

But now we have 7 equations for 8 unknowns. To render the system determinate, the constraint condition $u_2 - u_6 = 0$ is appended as eight equation:

$$\begin{bmatrix} K_{11} & K_{12} & 0 & 0 & 0 & 0 & 0 & 0 \\ K_{12} & K_{22} & K_{23} & 0 & 0 & 0 & 0 & 1 \\ 0 & K_{23} & K_{33} & K_{34} & 0 & 0 & 0 & 0 \\ 0 & 0 & K_{34} & K_{44} & K_{45} & 0 & 0 & 0 \\ 0 & 0 & 0 & K_{45} & K_{55} & K_{56} & 0 & 0 \\ 0 & 0 & 0 & 0 & K_{56} & K_{66} & K_{67} & -1 \\ 0 & 0 & 0 & 0 & 0 & K_{67} & K_{77} & 0 \\ 0 & 1 & 0 & 0 & 0 - 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \\ u_7 \\ \lambda \end{bmatrix} = \begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \\ f_5 \\ f_6 \\ f_7 \\ 0 \end{bmatrix} \quad (3.55)$$

This is called the **multiplier-augmented** system. Its coefficient matrix, which is symmetric, is called the **bordered stiffness matrix**. The process by which λ is appended to the vector of original unknowns is called **adjunction**. Solving this system provides the desired solution for the DOFs while also characterising the constraint forces through λ .

3.4.2 Lagrange Multipliers fro General MFCs

The general procedure will be stated first as a recipe. Suppose that we want to solve the example structure subjected to three MFCs:

$$u_2 - u_6 = 0, \quad 5u_2 - 8u_7 = 3, \quad 3u_3 + u_5 - 4u_6 = 1 \quad (3.56)$$

Adjoin these MFCs as the eighth, ninth, and tenth equations:

$$\begin{bmatrix} K_{11} & K_{12} & 0 & 0 & 0 & 0 & 0 \\ K_{12} & K_{22} & K_{23} & 0 & 0 & 0 & 0 \\ 0 & K_{23} & K_{33} & K_{34} & 0 & 0 & 0 \\ 0 & 0 & K_{34} & K_{44} & K_{45} & 0 & 0 \\ 0 & 0 & 0 & K_{45} & K_{55} & K_{56} & 0 \\ 0 & 0 & 0 & 0 & K_{56} & K_{66} & K_{67} \\ 0 & 0 & 0 & 0 & 0 & K_{67} & K_{77} \\ 0 & 1 & 0 & 0 & 0 & -1 & 0 \\ 0 & 5 & 0 & 0 & 0 & 0 & -8 \\ 0 & 0 & 3 & 0 & 1 & -4 & 0 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \\ u_7 \end{bmatrix} = \begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \\ f_5 \\ f_6 \\ f_7 \\ 0 \\ 3 \\ 1 \end{bmatrix} \quad (3.57)$$

Three Lagrange multipliers: λ_1, λ_2 and λ_3 are required to care of three MFCs. Adjoin those unknowns to the nodal displacement vector. Symmetrize the coefficient matrix by appending 3 columns that are the transpose of the 3 last rows and filling the bottom right-hand corner with a 3×3 zero matrix:

$$\begin{bmatrix} K_{11} & K_{12} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ K_{12} & K_{22} & K_{23} & 0 & 0 & 0 & 0 & 1 & 5 & 0 \\ 0 & K_{23} & K_{33} & K_{34} & 0 & 0 & 0 & 0 & 0 & 3 \\ 0 & 0 & K_{34} & K_{44} & K_{45} & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & K_{45} & K_{55} & K_{56} & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & K_{56} & K_{66} & K_{67} & -1 & 0 & -4 \\ 0 & 0 & 0 & 0 & 0 & K_{67} & K_{77} & 0 & -8 & 0 \\ 0 & 1 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 \\ 0 & 5 & 0 & 0 & 0 & 0 & -8 & 0 & 0 & 0 \\ 0 & 0 & 3 & 0 & 1 & -4 & 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \\ u_7 \\ \lambda_1 \\ \lambda_2 \\ \lambda_3 \end{bmatrix} = \begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \\ f_5 \\ f_6 \\ f_7 \\ 0 \\ 3 \\ 1 \end{bmatrix} \quad (3.58)$$

3.4.3 The Theory Behind

The recipe illustrated above comes from a well known technique of variational calculus. Using the matrix notation introduced, compactly denote the set of m MFCs by $\mathbf{A}\mathbf{u} = \mathbf{b}$, where $\mathbf{A}$ is $m \times n$. The potential energy of the unconstrained finite element model is $\Pi = \frac{1}{2}\mathbf{u}^T \mathbf{K}\mathbf{u} - \mathbf{u}^T \mathbf{f}$. To impose the constraint, adjoin m Lagrange multipliers collected in vector λ and form the Lagrangian

$$\mathcal{L}(\mathbf{u}, \lambda) = \Pi + \lambda(\mathbf{A}\mathbf{u} - \mathbf{b}) = \frac{1}{2}\mathbf{u}^T \mathbf{K}\mathbf{u} - \mathbf{u}^T \mathbf{f} + \lambda^T (\mathbf{A}\mathbf{u} - \mathbf{b}) \quad (3.59)$$

Extremizing $\mathcal{L}$ with respect to $\mathbf{u}$ and λ yields the multiplier-augmented form

$$\begin{bmatrix} \mathbf{K} & \mathbf{A}^T \\ \mathbf{A} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ \lambda \end{bmatrix} = \begin{bmatrix} \mathbf{f} \\ \mathbf{b} \end{bmatrix} \quad (3.60)$$

The master stiffness matrix $\mathbf{K}$ is said to be **bordered** with $\mathbf{A}$ and $\mathbf{A}^T$. Solving this system provides $\mathbf{u}$ and λ . The latter can be interpreted as forces of constraint in the following sense: a removed constraint can be replaced by a system of forces characterised by λ multiplied by the constraint coefficients. More precisely, the constraint forces are $-\mathbf{A}^T \lambda$.

3.4.4 Assessment of the Lagrange Multiplier Method

In contrast to the penalty method, the method of Lagrange multipliers has the advantage of being exact (aside from computational errors due to finite precision arithmetic). It provides directly the constraint forces, which are of interest in many applications. It does not require guesses as regards to weights. As the penalty method, it can be extended without difficulty to nonlinear constraints.

It is not free of disadvantages. It introduces additional unknowns, requiring expansion of the original stiffness method, and more complicated storage allocation procedures. It renders the augmented stiffness matrix indefinite, an effect that may cause grief with some linear equation solving methods that rely on positive definiteness. Finally, as the master-slave method, it is sensitive to

the degree of linear independence of the constraints: if the constraint $u_2 = u_6$ is specified twice, the bordered stiffness is obviously singular.

On the whole this method appears to be the most elegant one for a general-purpose finite element program that is supposed to work as a "black box" by minimizing guesses and choices from its users. Its implementation, however, is not simple. Special care must be exercised to detect singularities due to constraint dependency and to account for the effect of loss of positive definiteness of the bordered stiffness on equation solver.

3.4.5 The Augmented Lagrangian Method

The general matrix of the penalty function and Lagrangian multiplier methods are given by expressions (3.51) and (3.60), respectively. A useful connection between these methods can be established as follows.

Because the lower diagonal block of the bordered stiffness matrix in (3.60) is null, it is not possible to directly eliminate λ . To make this possible, replace this block by $\epsilon \mathbf{S}^{-1}$, where $\mathbf{S}$ is a constraint-scaling diagonal matrix of appropriate order and ϵ is a small number. The reciprocal of ϵ is a large number called $w = 1/\epsilon$. To maintain exactness of the second equation, $\epsilon \mathbf{S}^{-1} \lambda$ is added to the RHS:

$$\begin{bmatrix} \mathbf{K} & \mathbf{A}^T \\ \mathbf{A} & \epsilon \mathbf{S}^{-1} \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ \lambda \end{bmatrix} = \begin{bmatrix} \mathbf{f} \\ \epsilon \mathbf{S}^{-1} \lambda^P \end{bmatrix} \quad (3.61)$$

Here superscript P (for "predicted" value) is attached to the λ on the RHS as a "tracer". We can now formally solve for λ and subsequently for $\mathbf{u}$. The results may be presented as:

$$\begin{aligned} (\mathbf{K} + w \mathbf{A}^T \mathbf{S} \mathbf{A}) \mathbf{u} &= \mathbf{f} + w \mathbf{A}^T \mathbf{S} \mathbf{b} - \mathbf{A}^T \lambda^P \\ \lambda &= \lambda^P + w \mathbf{S} (\mathbf{b} - \mathbf{A} \mathbf{u}) \end{aligned} \quad (3.62)$$

Setting $\lambda^P = \mathbf{0}$ in the first matrix equation yields:

$$(\mathbf{K} + w \mathbf{A}^T \mathbf{S} \mathbf{A}) \mathbf{u} = \mathbf{f} + w \mathbf{A}^T \mathbf{S} \mathbf{b} \quad (3.63)$$

on taking $\mathbf{W} = w\mathbf{S}$, the general matrix equation (3.51) of the penalty method is recovered.

This relation suggests the construction of *iterative procedures* in which one tries to *improve the accuracy of the penalty function while w is kept constant*. This strategy circumvents the aforementioned ill-conditioning problems when the weight w is gradually increased. One such method is easily constructed by inspecting (3.62). Using superscript k as an iteration index and keeping w fixed, solve equations (3.62) in tandem as follows:

$$\begin{aligned} (\mathbf{K} + \mathbf{A}^T \mathbf{W} \mathbf{A}) \mathbf{u}^k &= \mathbf{f} + \mathbf{A}^T \mathbf{W} \mathbf{b} - \mathbf{A}^T \boldsymbol{\lambda}^k \\ \boldsymbol{\lambda}^{k+1} &= \boldsymbol{\lambda}^k + \mathbf{W} (\mathbf{b} - \mathbf{A} \mathbf{u}^k) \end{aligned} \tag{3.64}$$

for $k = 0, 1, \dots$, beginning with $\boldsymbol{\lambda}^0 = 0$. Then $\mathbf{u}^0$ is the penalty solution. If the process converges one recovers the exact Lagrangian solution without having to solve the Lagrangian system (3.61) directly.

The family of iterative procedures that may be precipitated from (3.62) collectively pertains to the class of *augmented Lagrangian methods*.

3.5 Summary

The treatment of linear MFCs in finite element systems can be carried out by several methods. Three of these: **master-slave elimination**, **penalty augmentation** and **Lagrange multiplier** adjunction have been discussed.

It is emphasised that no method is uniformly satisfactory in terms of generality, robustness, numerical behavior and simplicity of implementation.

For a general purpose program that tries to attain "black box" behavior (that is minimal decisions on the part of users) the Lagrange multipliers has the edge. This edge is unfortunately blunted by a fairly complex computer implementation and by the loss of positive definiteness in the bordered stiffness matrix.

Chapter 4

Rigid Body Elements

4.1 RBE2

The rigid freebody relations between **slave** and **master** node are:

$$\begin{aligned}\mathbf{T}_s &= \mathbf{T}_m + \mathbf{R}_s \times \bar{\mathbf{x}} \\ \mathbf{R}_s &= \mathbf{R}_m\end{aligned}\tag{4.1}$$

Where $\bar{\mathbf{x}}$ is a vector from **slave** to **master**:

$$\bar{\mathbf{x}} = \begin{bmatrix} \bar{x} \\ \bar{y} \\ \bar{z} \end{bmatrix} = \begin{bmatrix} x_s - x_m \\ y_s - y_m \\ z_s - z_m \end{bmatrix}\tag{4.2}$$

Then *vector multiplication* (cross product) is defined as:

$$\begin{aligned}\mathbf{R}_s \times \mathbf{x}_s &= \mathbf{R}_m \times \mathbf{x} = \\ &= \begin{bmatrix} \varphi_m \\ \psi_m \\ \theta_m \end{bmatrix} \times \begin{bmatrix} x_s - x_m \\ y_s - y_m \\ z_s - z_m \end{bmatrix} = \begin{bmatrix} \varphi_m \\ \psi_m \\ \theta_m \end{bmatrix} \times \begin{bmatrix} \bar{x} \\ \bar{y} \\ \bar{z} \end{bmatrix} = \\ &= \begin{bmatrix} \psi_m(z_s - z_m) - \theta_m(y_s - y_m) \\ \theta_m(x_s - x_m) - \varphi_m(z_s - z_m) \\ \varphi_m(y_s - y_m) - \psi_m(x_s - x_m) \end{bmatrix} = \begin{bmatrix} \psi_m\bar{z} - \theta_m\bar{y} \\ \theta_m\bar{x} - \varphi_m\bar{z} \\ \varphi_m\bar{y} - \psi_m\bar{x} \end{bmatrix} = \\ &= \begin{bmatrix} 0 & \bar{z} & -\bar{y} \\ -\bar{z} & 0 & \bar{x} \\ \bar{y} & -\bar{x} & 0 \end{bmatrix} \begin{bmatrix} \varphi_m \\ \psi_m \\ \theta_m \end{bmatrix}\end{aligned}\tag{4.3}$$

The **slave** equations can be written as:

$$\begin{bmatrix} u_s \\ v_s \\ w_s \\ \varphi_s \\ \psi_s \\ \theta_s \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & z_s - z_m & -(y_s - y_m) \\ 0 & 1 & 0 & -(z_s - z_m) & 0 & x_s - x_m \\ 0 & 0 & 1 & y_s - y_m & -(x_s - x_m) & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_m \\ v_m \\ w_m \\ \varphi_m \\ \psi_m \\ \theta_m \end{bmatrix} \quad (4.4)$$

or:

$$\begin{bmatrix} u_s \\ v_s \\ w_s \\ \varphi_s \\ \psi_s \\ \theta_s \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & \bar{z} & -\bar{y} \\ 0 & 1 & 0 & -\bar{z} & 0 & \bar{x} \\ 0 & 0 & 1 & \bar{y} & -\bar{x} & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_m \\ v_m \\ w_m \\ \varphi_m \\ \psi_m \\ \theta_m \end{bmatrix} \quad (4.5)$$

4.1.1 RBE2 Coefficients Example:

Master Node:

$$\mathbf{X}_m = [1 \quad 1 \quad 1]^T \quad (4.6)$$

Slave Nodes:

$$\begin{aligned} \mathbf{X}_{s,1} &= [0 \quad 0 \quad 0]^T \\ \mathbf{X}_{s,1} &= [2 \quad 0 \quad 0]^T \\ \mathbf{X}_{s,1} &= [2 \quad 2 \quad 0]^T \\ \mathbf{X}_{s,1} &= [0 \quad 2 \quad 0]^T \end{aligned} \quad (4.7)$$

Then Slave Node 1 equations are:

$$\mathbf{x}_1 = [x_{s,1} - x_m \quad y_{s,1} - y_m \quad z_{s,1} - z_m]^T = [-1 \quad -1 \quad -1]^T \quad (4.8)$$

Other Slave Nodes:

$$\begin{aligned} \mathbf{x}_2 &= [2 - 1 \quad 0 - 1 \quad 0 - 1]^T = [1 \quad -1 \quad -1]^T \\ \mathbf{x}_3 &= [2 - 1 \quad 2 - 1 \quad 0 - 1]^T = [1 \quad 1 \quad -1]^T \\ \mathbf{x}_4 &= [0 - 1 \quad 2 - 1 \quad 0 - 1]^T = [-1 \quad 1 \quad -1]^T \end{aligned} \quad (4.9)$$

Therefore, after filling in the coefficients:

$$\begin{aligned}
 \begin{bmatrix} u_{s,1} \\ v_{s,1} \\ w_{s,1} \\ \varphi_{s,1} \\ \psi_{s,1} \\ \theta_{s,1} \end{bmatrix} &= \begin{bmatrix} 1 & 0 & 0 & 0 & -1 & 1 \\ 0 & 1 & 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & -1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_m \\ v_m \\ w_m \\ \varphi_m \\ \psi_m \\ \theta_m \end{bmatrix} \\
 \begin{bmatrix} u_{s,2} \\ v_{s,2} \\ w_{s,2} \\ \varphi_{s,2} \\ \psi_{s,2} \\ \theta_{s,2} \end{bmatrix} &= \begin{bmatrix} 1 & 0 & 0 & 0 & -1 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & -1 & -1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_m \\ v_m \\ w_m \\ \varphi_m \\ \psi_m \\ \theta_m \end{bmatrix} \\
 \begin{bmatrix} u_{s,3} \\ v_{s,3} \\ w_{s,3} \\ \varphi_{s,3} \\ \psi_{s,3} \\ \theta_{s,3} \end{bmatrix} &= \begin{bmatrix} 1 & 0 & 0 & 0 & -1 & -1 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & -1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_m \\ v_m \\ w_m \\ \varphi_m \\ \psi_m \\ \theta_m \end{bmatrix} \\
 \begin{bmatrix} u_{s,4} \\ v_{s,4} \\ w_{s,4} \\ \varphi_{s,4} \\ \psi_{s,4} \\ \theta_{s,4} \end{bmatrix} &= \begin{bmatrix} 1 & 0 & 0 & 0 & -1 & -1 \\ 0 & 1 & 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_m \\ v_m \\ w_m \\ \varphi_m \\ \psi_m \\ \theta_m \end{bmatrix}
 \end{aligned} \tag{4.10}$$

When all put together:

$$\begin{bmatrix} u_{s,1} \\ v_{s,1} \\ w_{s,1} \\ \varphi_{s,1} \\ \psi_{s,1} \\ \theta_{s,1} \\ u_{s,2} \\ v_{s,2} \\ w_{s,2} \\ \varphi_{s,2} \\ \psi_{s,2} \\ \theta_{s,2} \\ u_{s,3} \\ v_{s,3} \\ w_{s,3} \\ \varphi_{s,3} \\ \psi_{s,3} \\ \theta_{s,3} \\ u_{s,4} \\ v_{s,4} \\ w_{s,4} \\ \varphi_{s,4} \\ \psi_{s,4} \\ \theta_{s,4} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & -1 & 1 \\ 0 & 1 & 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & -1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & -1 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & -1 & -1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & -1 & -1 \\ 0 & 1 & 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & -1 & -1 \\ 0 & 1 & 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_m \\ v_m \\ w_m \\ \varphi_m \\ \psi_m \\ \theta_m \end{bmatrix} \quad (4.11)$$

It can be read as:

$$\begin{aligned} u_{s,1} &= u_m - \psi_m + \theta_m \\ v_{s,1} &= v_m \varphi_m - \theta_m \\ &\vdots \\ \theta_{s,4} &= \theta_m \end{aligned} \quad (4.12)$$

Or:

$$\mathbf{u}_s = \bar{\mathbf{T}} \mathbf{u}_m \quad (4.13)$$

4.1.2 Editing the Master Stiffness Matrix

This equation should be then expanded to all DOFs, where in $\mathbf{T}$ the slave DOFs columns are missing and the DOFs that do not take part in RBEs have 1 on the diagonal. The resulting matrix should have the same number of rows as the master stiffness matrix.

Then the master stiffness equation is modified:

$$\begin{aligned}\hat{\mathbf{K}} &= \mathbf{T}^T \mathbf{K} \mathbf{T} \\ \hat{\mathbf{f}} &= \mathbf{T}^T \mathbf{f} \\ \hat{\mathbf{K}} \hat{\mathbf{u}} &= \hat{\mathbf{f}}\end{aligned}\tag{4.14}$$

If we'd like to make the procedure more general, partition the master stiffness equations such that:

$$\begin{bmatrix} \mathbf{K}_{uu} & \mathbf{K}_{um} & \mathbf{K}_{us} \\ \mathbf{K}_{um}^T & \mathbf{K}_{mm} & \mathbf{K}_{ms} \\ \mathbf{K}_{us}^T & \mathbf{K}_{ms}^T & \mathbf{K}_{ss} \end{bmatrix} \begin{bmatrix} \mathbf{u}_u \\ \mathbf{u}_m \\ \mathbf{u}_s \end{bmatrix} = \begin{bmatrix} \mathbf{f}_u \\ \mathbf{f}_m \\ \mathbf{f}_s \end{bmatrix}\tag{4.15}$$

Based on (4.14) we could probably partition the matrices as:

$$\begin{array}{c|c} & \begin{array}{cc} u & m \end{array} \\ \begin{array}{c} u \\ m \\ s \end{array} & \overline{\begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{0} & \mathbf{I} \\ \mathbf{0} & \overline{\mathbf{T}} \end{bmatrix}} \end{array}\tag{4.16}$$

If we expand (4.13) and partition the matrices based on (4.16) into independent or **unconstrained**, **master** and **slave** nodes, then the equation:

$$\mathbf{u} = \mathbf{T} \hat{\mathbf{u}}\tag{4.17}$$

becomes:

$$\begin{bmatrix} \mathbf{u}_u \\ \mathbf{u}_m \\ \mathbf{u}_s \end{bmatrix} = \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{0} & \mathbf{I} \\ \mathbf{0} & \overline{\mathbf{T}} \end{bmatrix} \begin{bmatrix} \mathbf{u}_u \\ \mathbf{u}_m \end{bmatrix}\tag{4.18}$$

Finally the equation (4.14) can be rewritten as:

$$\begin{bmatrix} \mathbf{I} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{I} & \bar{\mathbf{T}}^T \end{bmatrix} \begin{bmatrix} \mathbf{K}_{uu} & \mathbf{K}_{um} & \mathbf{K}_{us} \\ \mathbf{K}_{mu} & \mathbf{K}_{mm} & \mathbf{K}_{ms} \\ \mathbf{K}_{su} & \mathbf{K}_{sm} & \mathbf{K}_{ss} \end{bmatrix} \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{0} & \mathbf{I} \\ \mathbf{0} & \bar{\mathbf{T}} \end{bmatrix} \begin{bmatrix} \mathbf{u}_u \\ \mathbf{u}_m \end{bmatrix} = \begin{bmatrix} \mathbf{I} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{I} & \bar{\mathbf{T}}^T \end{bmatrix} \begin{bmatrix} \mathbf{f}_u \\ \mathbf{f}_m \\ \mathbf{f}_s \end{bmatrix} \quad (4.19)$$

Expanding and multiplying:

$$\begin{bmatrix} \mathbf{K}_{uu} & \mathbf{K}_{um} & \mathbf{K}_{us} \\ \mathbf{K}_{mu} + \bar{\mathbf{T}}^T \mathbf{K}_{su} & \mathbf{K}_{mm} + \bar{\mathbf{T}}^T \mathbf{K}_{sm} & \mathbf{K}_{ms} + \bar{\mathbf{T}}^T \mathbf{K}_{ss} \end{bmatrix} \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{0} & \mathbf{I} \\ \mathbf{0} & \bar{\mathbf{T}} \end{bmatrix} \begin{bmatrix} \mathbf{u}_u \\ \mathbf{u}_m \end{bmatrix} = \begin{bmatrix} \mathbf{f}_u \\ \mathbf{f}_m + \bar{\mathbf{T}}^T \mathbf{f}_s \end{bmatrix} \quad (4.20)$$

$$\begin{bmatrix} \mathbf{K}_{uu} & \mathbf{K}_{um} + \mathbf{K}_{us} \bar{\mathbf{T}} \\ \mathbf{K}_{mu} + \bar{\mathbf{T}}^T \mathbf{K}_{su} & \mathbf{K}_{mm} + \bar{\mathbf{T}}^T \mathbf{K}_{sm} + \mathbf{K}_{ms} \bar{\mathbf{T}} + \bar{\mathbf{T}}^T \mathbf{K}_{ss} \bar{\mathbf{T}} \end{bmatrix} \begin{bmatrix} \mathbf{u}_u \\ \mathbf{u}_m \end{bmatrix} = \begin{bmatrix} \mathbf{f}_u \\ \mathbf{f}_m + \bar{\mathbf{T}}^T \mathbf{f}_s \end{bmatrix} \quad (4.21)$$

which is equal to:

$$\begin{bmatrix} \mathbf{K}_{uu} & \mathbf{K}_{um} + \mathbf{K}_{us} \bar{\mathbf{T}} \\ (\mathbf{K}_{um} + \mathbf{K}_{us} \bar{\mathbf{T}})^T & \mathbf{K}_{mm} + \bar{\mathbf{T}}^T \mathbf{K}_{ms}^T + \mathbf{K}_{ms} \bar{\mathbf{T}} + \bar{\mathbf{T}}^T \mathbf{K}_{ss} \bar{\mathbf{T}} \end{bmatrix} \begin{bmatrix} \mathbf{u}_u \\ \mathbf{u}_m \end{bmatrix} = \begin{bmatrix} \mathbf{f}_u \\ \mathbf{f}_m + \bar{\mathbf{T}}^T \mathbf{f}_s \end{bmatrix} \quad (4.22)$$

considering that $\mathbf{f}_s = \mathbf{0}$ the equations reduce to:

$$\begin{bmatrix} \mathbf{K}_{uu} & \hat{\mathbf{K}}_{um} \\ \hat{\mathbf{K}}_{um}^T & \hat{\mathbf{K}}_{mm} \end{bmatrix} \begin{bmatrix} \mathbf{u}_u \\ \mathbf{u}_m \end{bmatrix} = \begin{bmatrix} \mathbf{f}_u \\ \mathbf{f}_m \end{bmatrix} \quad (4.23)$$

where:

$$\begin{aligned} \hat{\mathbf{K}}_{um} &= \mathbf{K}_{um} + \mathbf{K}_{us} \bar{\mathbf{T}} \\ \hat{\mathbf{K}}_{mm} &= \mathbf{K}_{mm} + \bar{\mathbf{T}}^T \mathbf{K}_{ms}^T + \mathbf{K}_{ms} \bar{\mathbf{T}} + \bar{\mathbf{T}}^T \mathbf{K}_{ss} \bar{\mathbf{T}} \\ &= \mathbf{K}_{mm} + \mathbf{K}_{ms} \bar{\mathbf{T}} + (\mathbf{K}_{ms} \bar{\mathbf{T}})^T + \bar{\mathbf{T}}^T \mathbf{K}_{ss} \bar{\mathbf{T}} \end{aligned} \quad (4.24)$$

Then if **master-slave** relations are written as (*general case*):

$$\mathbf{u}_s = \bar{\mathbf{T}}\mathbf{u}_m + \mathbf{g} \quad (4.25)$$

Inserting into the partitioned master stiffness equations:

$$\begin{bmatrix} \mathbf{K}_{uu} & \hat{\mathbf{K}}_{um} \\ \hat{\mathbf{K}}_{um}^T & \hat{\mathbf{K}}_{mm} \end{bmatrix} \begin{bmatrix} \mathbf{u}_u \\ \mathbf{u}_m \end{bmatrix} = \begin{bmatrix} \mathbf{f}_u - \mathbf{K}_{us}\mathbf{g} \\ \mathbf{f}_m - \mathbf{K}_{ms}\mathbf{g} \end{bmatrix} \quad (4.26)$$

where:

$$\begin{aligned} \hat{\mathbf{K}}_{um} &= \mathbf{K}_{um} + \mathbf{K}_{us}\bar{\mathbf{T}} \\ \hat{\mathbf{K}}_{mm} &= \mathbf{K}_{mm} + \bar{\mathbf{T}}^T \mathbf{K}_{ms}^T + \mathbf{K}_{ms}\bar{\mathbf{T}} + \bar{\mathbf{T}}^T \mathbf{K}_{ss} \bar{\mathbf{T}} \end{aligned} \quad (4.27)$$

MPC Application

Afterwards the **MPCs** are applied:

$$\mathbf{u}_s = \bar{\mathbf{T}}\mathbf{u}_m \quad (4.28)$$

This, when applied to the whole model:

$$\begin{bmatrix} \mathbf{u}_u \\ \mathbf{u}_m \\ \mathbf{u}_s \end{bmatrix} = \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{0} & \mathbf{I} \\ \mathbf{0} & \bar{\mathbf{T}} \end{bmatrix} \begin{bmatrix} \mathbf{u}_u \\ \mathbf{u}_m \end{bmatrix} \quad (4.29)$$

with **transformation matrix** $\mathbf{T}$ being:

$$\mathbf{T} = \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{0} & \mathbf{I} \\ \mathbf{0} & \bar{\mathbf{T}} \end{bmatrix} \quad (4.30)$$

Applying the transformation matrix to get **modified master equation system**:

$$\begin{aligned} \mathbf{T}^T \mathbf{K}_g \mathbf{T} \hat{\mathbf{u}}_g &= \mathbf{T}^T \mathbf{f}_g \\ \hat{\mathbf{K}}_g \hat{\mathbf{u}}_g &= \hat{\mathbf{f}}_g \end{aligned} \quad (4.31)$$

Where $\hat{\mathbf{u}}_g = [\mathbf{u}_{g,u} \quad \mathbf{u}_{g,m}]^T$.

From now on, the subscript $_g$ (*as global*) is implied.

Rotated Nodal Basis

When a node or an SPC is defined as a **rotated/skewed** or in a different type of Coordinate system (e.g. cylindrical) a transformation matrix needs to be applied to such nodes.

Cartesian Coordinate system definition by angle (2D):

$$\mathbf{u}_l = \mathbf{T} \mathbf{u}_g \quad (4.32)$$

SPC Application

Then to solve the **master stiffness equations** for prescribed **BCs** partitioned to **unconstrained** and **master** DOFs such as:

$$\begin{bmatrix} \mathbf{K}_{uu} & \hat{\mathbf{K}}_{um} \\ \hat{\mathbf{K}}_{um}^T & \hat{\mathbf{K}}_{mm} \end{bmatrix} \begin{bmatrix} \mathbf{u}_u \\ \mathbf{u}_m \end{bmatrix} = \begin{bmatrix} \mathbf{f}_u \\ \mathbf{f}_m \end{bmatrix} \quad (4.33)$$

where:

$$\begin{aligned} \hat{\mathbf{K}}_{um} &= \mathbf{K}_{um} + \mathbf{K}_{us} \bar{\mathbf{T}} \\ \hat{\mathbf{K}}_{mm} &= \mathbf{K}_{mm} + \mathbf{K}_{ms} \bar{\mathbf{T}} + (\mathbf{K}_{ms} \bar{\mathbf{T}})^T + \bar{\mathbf{T}}^T \mathbf{K}_{ss} \bar{\mathbf{T}} \end{aligned} \quad (4.34)$$

Any of these **DOFs** can be constrained, so this partitioning scheme loses all purposes. Therefore the matrix is repartitioned again into **independent** and **constrained** partitions:

$$\begin{bmatrix} \mathbf{K}_{ii} & \mathbf{K}_{ic} \\ \mathbf{K}_{ic}^T & \mathbf{K}_{cc} \end{bmatrix} \begin{bmatrix} \mathbf{u}_i \\ \mathbf{u}_c \end{bmatrix} = \begin{bmatrix} \mathbf{f}_i \\ \mathbf{f}_c \end{bmatrix} \quad (4.35)$$

Solving Linear Equations

Where the **unknowns** are $\mathbf{u}_i$, $\mathbf{f}_c$:

$$\begin{bmatrix} \mathbf{K}_{ii} & \mathbf{K}_{ic} \\ \mathbf{K}_{ic}^T & \mathbf{K}_{cc} \end{bmatrix} \begin{bmatrix} \boxed{\mathbf{u}_i} \\ \boxed{\mathbf{u}_c} \end{bmatrix} = \begin{bmatrix} \boxed{\mathbf{f}_i} \\ \boxed{\mathbf{f}_c} \end{bmatrix} \quad (4.36)$$

Unpacking the equations:

$$\begin{aligned} \mathbf{K}_{ii} \boxed{\mathbf{u}_i} + \mathbf{K}_{ic} \mathbf{u}_c &= \mathbf{f}_i \\ \mathbf{K}_{ic}^T \boxed{\mathbf{u}_i} + \mathbf{K}_{cc} \mathbf{u}_c &= \boxed{\mathbf{f}_c} \end{aligned} \quad (4.37)$$

Then first solve for the **unknown displacements**:

$$\begin{aligned} \mathbf{K}_{ii} \boxed{\mathbf{u}_i} &= \mathbf{f}_i - \mathbf{K}_{ic} \mathbf{u}_c \\ \boxed{\mathbf{u}_i} &= \mathbf{K}_{ii}^{-1} (\mathbf{f}_i - \mathbf{K}_{ic} \mathbf{u}_c) \end{aligned} \quad (4.38)$$

Lastly solve for the **unknown reaction forces**:

$$\boxed{\mathbf{f}_c} = \mathbf{K}_{ii} \mathbf{u}_i + \mathbf{K}_{ic} \mathbf{u}_c \quad (4.39)$$

Recovering Full Displacement Vector

The full displacement vector is then recovered:

$$\begin{aligned} \mathbf{T} &= \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{0} & \mathbf{I} \\ \mathbf{0} & \overline{\mathbf{T}} \end{bmatrix} \\ \hat{\mathbf{u}} &= \begin{bmatrix} \mathbf{u}_i \\ \mathbf{u}_c \end{bmatrix} = \begin{bmatrix} \mathbf{u}_u \\ \mathbf{u}_m \end{bmatrix} \\ \mathbf{u}_g &= \mathbf{T} \hat{\mathbf{u}} \\ \begin{bmatrix} \mathbf{u}_u \\ \mathbf{u}_m \\ \mathbf{u}_s \end{bmatrix} &= \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{0} & \mathbf{I} \\ \mathbf{0} & \overline{\mathbf{T}} \end{bmatrix} \begin{bmatrix} \mathbf{u}_u \\ \mathbf{u}_m \end{bmatrix} \end{aligned} \quad (4.40)$$

4.2 RBE3

The first step is to calculate the **weighted COG** of the element, where

$$\mathbf{x}_{COG} = \frac{\sum w_i \mathbf{x}_i}{\sum w_i} \quad (4.41)$$

where w_i is a weighting factor of DOF i and $\mathbf{x}_i$ is the position of the node related with DOF i

Once the **COG** is found, the loading is translocated from the reference point $\mathbf{x}_{app}$ where it is applied to the **COG** by performing a equilibrium equivalency.

$$\mathbf{F}_{COG} = \mathbf{F}_{app} \quad (4.42)$$

$$\mathbf{M}_{COG} = \mathbf{M}_{app} + \mathbf{r} \times \mathbf{F}_{app} \quad (4.43)$$

Then the $\mathbf{F}_{COG}$ is reacted (shared) between all master nodes relative to their weight.

$$\mathbf{F}_i = \mathbf{F}_{COG} \frac{w_i}{\sum w_i} \quad (4.44)$$

The moment $\mathbf{M}_{COG}$ is assumed to be reacted (shared) by the mater DOFs according to the relative weight of each one of them and the relative position with regards to the *COG*. The reaction force that compensates the moment is in the direction perpendicular to the vector joining the *COG* with the position of the node where the master DOFs are:

$$\mathbf{P}_i = -w_i \frac{\mathbf{M}_{COG} \times \mathbf{r}_i}{\sum w_i \cdot \mathbf{r}_i \cdot \mathbf{r}_i} \quad (4.45)$$

where $\mathbf{r}_i$ is the vector from the *COG* position to the i -th node $\mathbf{r}_i = \mathbf{x}_i - \mathbf{x}_{COG}$.

The total reaction force on the master DOFs is the sum $\mathbf{F}_i + \mathbf{P}_i$ for each master DOF.

This force is not dependent on the stiffness of the model. It is only dependent on the weighting factors and the relative position of the master nodes.

To get the **slave** node displacements:

1. **COG** position
2. $\mathbf{r}_i$ vectors
3. compute the weighted average of the master displacements
4. an unknown rotation is assumed and the displacements on the master nodes are written as if they were calculated with a rigid body movement with the weighted average displacements and the unknown rotations for each node i :

$$\mathbf{U}_s = \frac{\sum w_i \mathbf{U}_i}{\sum w_i} + \mathbf{R}_s \times \mathbf{r}_i \quad (4.46)$$

where $\mathbf{U}$ are translational displacements and $\mathbf{R}$ are rotational displacements.

Chapter 5

Governing Equations

5.1 Material Matrices

Problem	Displacement components	Strain vector ϵ^T	Stress vector τ^T
Bar	u	$[\epsilon_{xx}]$	$[\sigma_{xx}]$
Beam	w	$[\kappa_{xx}]$	$[m_{xx}]$
Plane stress	u, v	$[\epsilon_{xx}, \epsilon_{yy}, \gamma_{xy}]$	$[\sigma_{xx}, \sigma_{yy}, \tau_{xy}]$
Plane strain	u, v	$[\epsilon_{xx}, \epsilon_{yy}, \gamma_{xy}]$	$[\sigma_{xx}, \sigma_{yy}, \tau_{xy}]$
Axisymmetric	u, v	$[\epsilon_{xx}, \epsilon_{yy}, \gamma_{xy}, \epsilon_{zz}]$	$[\sigma_{xx}, \sigma_{yy}, \tau_{xy}, \sigma_{zz}]$
3D	u, v, w	$[\epsilon_{xx}, \epsilon_{yy}, \epsilon_{zz}, \gamma_{xy}, \gamma_{yz}, \gamma_{zx}]$	$[\sigma_{xx}, \sigma_{yy}, \sigma_{zz}, \tau_{xy}, \tau_{yz}, \tau_{zx}]$
Plate bending	w	$[\kappa_{xx}, \kappa_{yy}, \kappa_{xy}]$	$[m_{xx}, m_{yy}, m_{xy}]$

Notation: $\epsilon_{xx} = \frac{\partial u}{\partial x}$, $\epsilon_{yy} = \frac{\partial v}{\partial y}$, $\gamma_{xy} = \frac{\partial u}{\partial y} + \frac{\partial v}{\partial x}$, $\dots$, $\kappa_{xx} = \frac{\partial^2 w}{\partial x^2}$, $\kappa_{yy} = \frac{\partial^2 w}{\partial y^2}$, $\kappa_{xy} = \frac{\partial^2 w}{\partial x \partial y}$.

Table 5.1: Corresponding kinematic and static variables in various problems.

Problem	Material matrix C
Bar	E
Beam	EI
Plane stress	$\frac{E}{1-\nu^2} \begin{bmatrix} 1 & \nu & 0 \\ \nu & 1 & 0 \\ 0 & 0 & \frac{1-\nu}{2} \end{bmatrix}$
Plane strain	$\frac{E(1-\nu)}{(1+\nu)(1-2\nu)} \begin{bmatrix} 1 & \frac{\nu}{1-\nu} & 0 \\ \frac{\nu}{1-\nu} & 1 & 0 \\ 0 & 0 & \frac{1-2\nu}{2(1-\nu)} \end{bmatrix}$
Axisymmetric	$\frac{E(1-\nu)}{(1+\nu)(1-2\nu)} \begin{bmatrix} 1 & \frac{\nu}{1-\nu} & 0 & \frac{\nu}{1-\nu} \\ \frac{\nu}{1-\nu} & 1 & 0 & \frac{\nu}{1-\nu} \\ 0 & 0 & \frac{1-2\nu}{2(1-\nu)} & 0 \\ \frac{\nu}{1-\nu} & \frac{\nu}{1-\nu} & 0 & 1 \end{bmatrix}$
3D	$\frac{E(1-\nu)}{(1+\nu)(1-2\nu)} \begin{bmatrix} 1 & \frac{\nu}{1-\nu} & \frac{\nu}{1-\nu} & 0 & 0 & 0 \\ \frac{\nu}{1-\nu} & 1 & \frac{\nu}{1-\nu} & 0 & 0 & 0 \\ \frac{\nu}{1-\nu} & \frac{\nu}{1-\nu} & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & \frac{1-2\nu}{2(1-\nu)} & 0 & 0 \\ 0 & 0 & 0 & 0 & \frac{1-2\nu}{2(1-\nu)} & 0 \\ 0 & 0 & 0 & 0 & 0 & \frac{1-2\nu}{2(1-\nu)} \end{bmatrix}$
Plate bending	$\frac{Eh^3}{12(1-\nu^2)} \begin{bmatrix} 1 & \nu & 0 \\ \nu & 1 & 0 \\ 0 & 0 & \frac{1-\nu}{2} \end{bmatrix}$

Notation: E = Young's modulus, ν = Poisson's ratio, h = thickness of plate, I = moment of inertia

Table 5.2: Generalised stress-strain matrices for isotropic materials

Chapter 6

Dynamics

6.1 Function properties

Sine and Cosine functions

Let a function $f(t)$ be in the form:

$$f(t) = A_1 \cos \omega t + B_1 \sin \omega t \quad (6.1)$$

then the following relation applies:

$$\begin{aligned} f(t) &= A \cos(\omega t - \phi) \\ A &= \sqrt{A_1^2 + B_1^2} \\ \phi &= \tan^{-1} \left(\frac{B_1}{A_1} \right) \end{aligned} \quad (6.2)$$

also:

$$\begin{aligned} \cos(\omega t - \phi) &= \cos \omega t \cos \phi + \sin \omega t \sin \phi \\ \sin(\omega t - \phi) &= \sin \omega t \cos \phi - \cos \omega t \sin \phi \end{aligned} \quad (6.3)$$

6.2 SDOF system

6.2.1 Definition

Let:

$$m\ddot{u}(t) + c\dot{u}(t) + ku(t) = f(t) \quad (6.4)$$

or:

$$m\ddot{a}(t) + c\dot{v}(t) + ku(t) = f(t) \quad (6.5)$$

be the **SDOF equation of motion**, then the following applies:

$$\begin{aligned} \omega^2 &= \frac{k}{m} \\ \omega &= \sqrt{\frac{k}{m}} \\ f &= \frac{\omega}{2\pi} \\ T &= \frac{1}{f} \end{aligned} \quad (6.6)$$

where:

ω is radial eigenfrequency [rad/s],

f is eigenfrequency [Hz] and

T is period [s]

The simplest vibratory system can be described by a single mass connected to a spring (and possibly a dashpot). The mass is allowed to travel only along the spring elongation direction. Such systems are called *Single Degree-of-Freedom* (**SDOF**) systems and are shown in the figure.

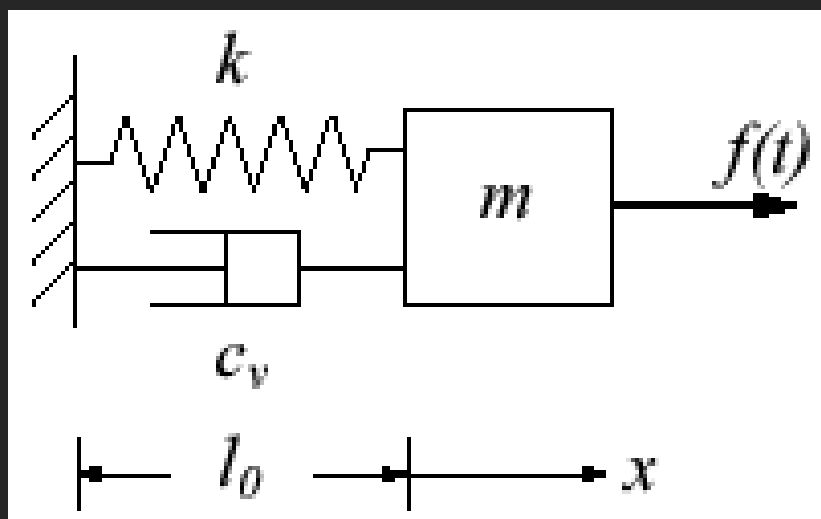


Figure 6.1: SDOF system with a linear spring and dashpot

6.2.2 Equation of Motion for SDOF Systems

SDOF vibration can be analyzed by **Newton's second law of motion**, $F = ma$. The analysis can be easily visualised with the aid of a **free body diagram**,

The resulting equation of motion is a **second order, non-homogeneous, ordinary differential equation**:

$$m\ddot{u} + c\dot{u} + ku = f(t) \begin{cases} u(t=0) = u_0 \\ \dot{u}(t=0) = \dot{u}_0 \end{cases} \quad (6.7)$$

with the initial conditions u_0 and $\dot{u}_0$.

6.2.3 Time Solution for Unforced Undamped SDOF Systems

<https://www.efunda.com/formulae/vibrations/sdof-free-undamped.cfm>

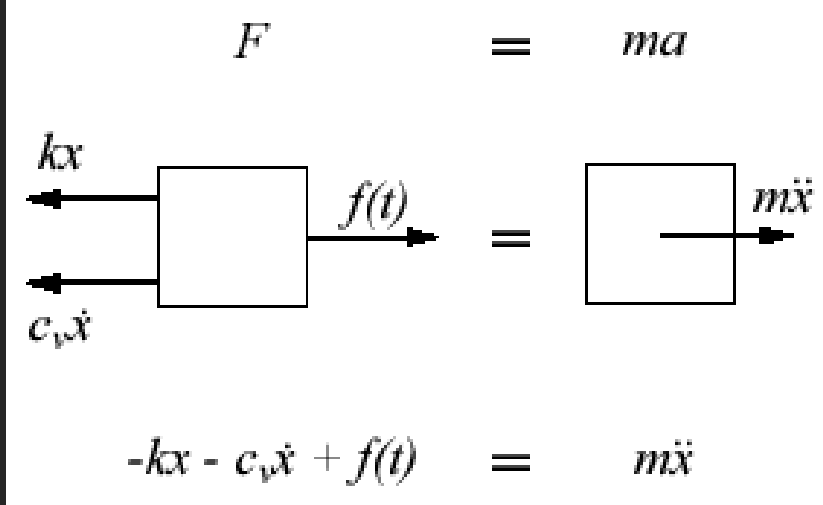


Figure 6.2: SDOF Free Body Diagram

The equation of motion derived in the definition can be simplified to:

$$m\ddot{u} + ku = 0 \begin{cases} u(t=0) = u_0 \\ \dot{u}(t=0) = \dot{u}_0 \end{cases} \quad (6.8)$$

This equation of motion is a **second order, homogeneous, ordinary differential equation** (ODE). If the mass and spring stiffness are constants, the ODE becomes a **linear, homogeneous ODE with constant coefficients** and can be solved by the Characteristic Equation method. The characteristic equation for this problem is:

$$ms^2 + k = 0 \quad (6.9)$$

which determines the 2 independent roots for the undamped vibration problem. The final solution (that contains the 2 independent roots from the characteristic equation and satisfies the initial conditions) is:

$$\begin{aligned}
u(t) &= c_1 e^{i\omega_n t} + c_2 e^{-i\omega_n t} \\
&= d_1 \cos \omega_n t + d_2 \sin \omega_n t \\
\implies u(t) &= u_0 \cos \omega_n t + \frac{\dot{u}_0}{\omega_n} \sin \omega_n t
\end{aligned} \tag{6.10}$$

The natural frequency ω_n is defined by:

$$\omega_n = \sqrt{\frac{k}{m}} \tag{6.11}$$

and depends only on the system mass and the spring stiffness (i.e. any damping will not change the natural frequency of a system).

Alternatively, the solution may be expressed by the equivalent form,

$$u(t) = A_0 \cos(\omega_n t - \phi_0) \tag{6.12}$$

where the amplitude A_0 and the initial phase ϕ_0 are given by:

$$\begin{aligned}
A_0 &= \sqrt{u_0^2 + \left(\frac{\dot{u}_0}{\omega_n}\right)^2} \\
\phi_0 &= \tan^{-1} \frac{\dot{u}_0}{u_0 \omega_n}
\end{aligned} \tag{6.13}$$

Note that an assumption of zero damping is typically not accurate. In reality, there almost always exists some resistance in vibratory systems. This resistance will damp the vibration and dissipate energy, the oscillatory motion caused by the initial disturbance will eventually be reduced to zero.

6.2.4 Time Solution for Unforced Damped SDOF Systems

<https://www.efunda.com/formulae/vibrations/sdof-free-damped.cfm>

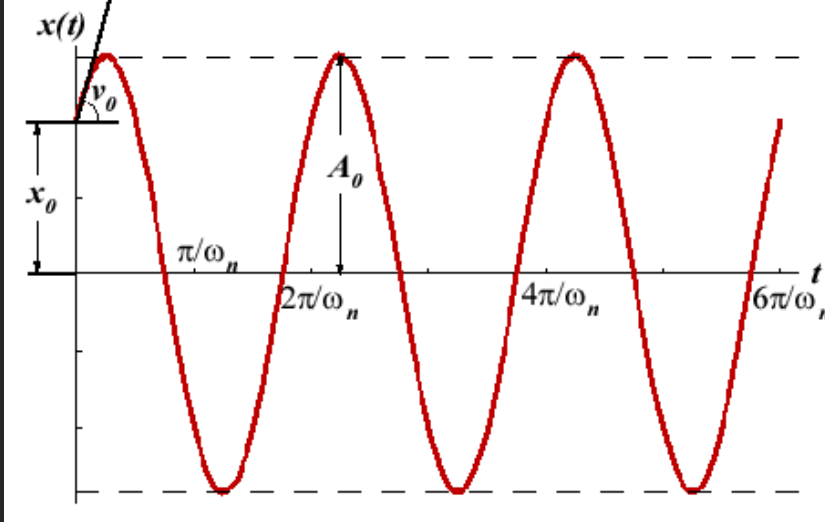


Figure 6.3: SDOF Undamped sample time behavior

Damping that produces a damping force proportional to the mass's velocity is commonly referred to as *viscous damping*, and is denoted graphically by a dashpot.

For an unforced damped **SDOF** system, the general equation of motion becomes:

$$m\ddot{u} + c\dot{u} + ku = 0 \quad \begin{cases} u(t=0) = u_0 \\ \dot{u}(t=0) = \dot{u}_0 \end{cases} \quad (6.14)$$

This equation of motion is a **second order, homogeneous, ODE**. If all parameters (mass, stiffness and viscous damping) are constants, the **ODE** becomes a **linear ODE with constant coefficients** and can be solved by the *Characteristic Equation method*. The characteristic equation for this problem is:

$$ms^2 + s_v s + k = 0 \quad (6.15)$$

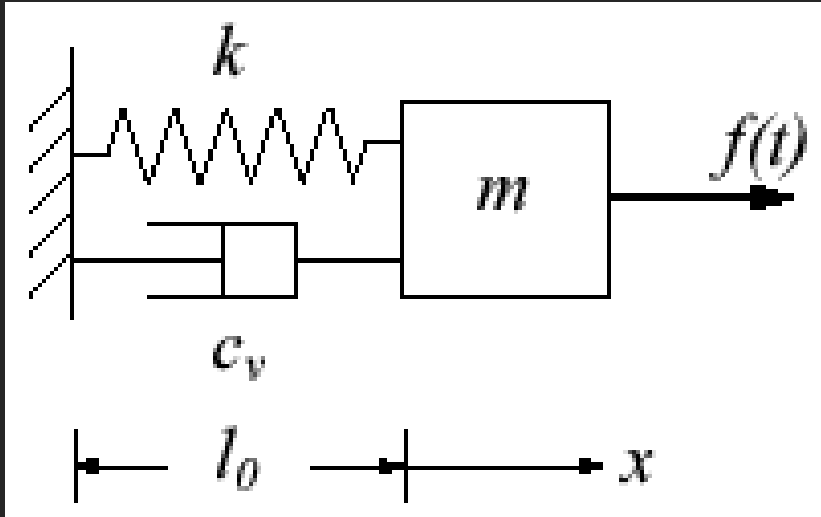


Figure 6.4: SDOF system with a linear spring and dashpot

which determines the 2 independent roots for the damped vibration problem. The roots to the characteristic equation fall into one of the following 3 cases:

- If $c_v^2 - 4mk < 0$, the system is termed **underdamped**. The roots of the characteristic equation are complex conjugates, corresponding to *oscillatory motion* with an *exponential decay* in amplitude.
- If $c_v^2 - 4mk = 0$, the system is termed **critically-damped**. The roots of the characteristic equation are repeated, corresponding to *simple decay motion* with at most *one overshoot* of the systems resting position.
- If $c_v^2 - 4mk > 0$, the system is termed **overdamped**. The roots of the characteristic equation are purely real and distinct, corresponding to *simple exponentially decaying motion*.

To simplify the solutions coming up, we define the critical damping c_c , the damping ratio ζ , and the damped vibration frequency ω_d as:

$$\begin{aligned}
c_c &= 2m\sqrt{\frac{k}{m}} = 2m\omega_n \\
\zeta &= \frac{c_v}{c_c} \\
\omega_d &= \sqrt{1 - \zeta^2}\omega_n
\end{aligned} \tag{6.16}$$

where the natural frequency of the system ω_n is given by:

$$\omega_n = \sqrt{\frac{k}{m}} \tag{6.17}$$

Note that ω_d will equal ω_n when the damping of the system is zero (i.e. undamped). The time solutions for the free **SDOF** system is presented below for each of the three case scenarios.

Time Solution of Unforced Underdamped SDOF Systems

When $c_v^2 - 4mk < 0$ (equivalent to $\zeta < 1$ or $c_v < c_c$), the characteristic equation has a pair of complex conjugate roots. The displacement solution for this kind of system is:

$$\begin{aligned}
u(t) &= c_1 e^{-\zeta + i\sqrt{1-\zeta^2}\omega_n t} + c_2 e^{-\zeta - i\sqrt{1-\zeta^2}\omega_n t} \\
&= e^{-\zeta\omega_n t} [d_1 \cos(\omega_d t) + d_2 \sin(\omega_d t)] \\
\Rightarrow u(t) &= \underbrace{e^{-\zeta\omega_n t}}_{\text{exponential decay}} \underbrace{\left[u_0 \cos(\omega_d t) + \frac{\dot{u}_0 + \zeta\omega_n u_0}{\omega_d} \sin(\omega_d t) \right]}_{\text{periodic motion}}
\end{aligned} \tag{6.18}$$

An alternate but equivalent solution is given by:

$$u(t) = A_0 \underbrace{e^{-\zeta\omega_n t}}_{\text{exponential decay}} \underbrace{\cos(\omega_d t - \phi_0)}_{\text{periodic motion}} \tag{6.19}$$

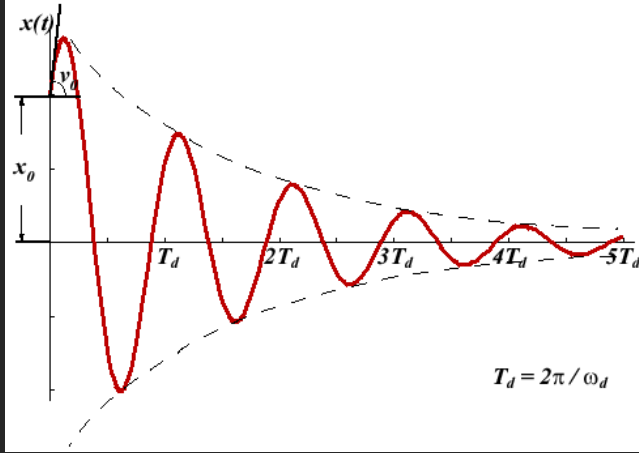


Figure 6.5: The displacement plot of an underdamped system

Note that the displacement amplitude decays exponentially (i.e. the natural logarithm of the amplitude ratio for any two displacements separated in time by a constant ratio is a constant):

$$\begin{aligned}
 \frac{A_k}{A_{k+1}} &= \frac{A_0 e^{-\zeta \omega_n (kT_d)} \cos(\phi_0)}{A_0 e^{-\zeta \omega_n [(k+1)T_d]} \cos(\phi_0)} \\
 &= \frac{e^{-\zeta \omega_n (kT_d)}}{e^{-\zeta \omega_n [(k+1)T_d]}} \\
 &= e^{\zeta \omega_n T_d} \\
 \Rightarrow \ln \left(\frac{A_k}{A_{k+1}} \right) &= \zeta \omega_n T_d = \zeta \omega_n \frac{2\pi}{\omega_d} = \frac{2\pi\zeta}{\sqrt{1-\zeta^2}}
 \end{aligned} \tag{6.20}$$

where $T_d = \frac{1}{f_d} = \frac{2\pi}{\omega_d}$ is the period of the damped vibration.

Time Solution of Unforced Critically-Damped Systems

When $c_v^2 - 4mk = 0$ (equivalent to $\zeta = 1$ or $c_v = c_c$), the characteristic equation has repeated real roots. The displacement solution for this kind of system is:

$$\begin{aligned}
 u(t) &= (c_1 + c_2 t) e^{-\omega_n t} \\
 \implies u(t) &= e^{-\omega_n t} [u_0 + (v_0 + \omega_n u_0) t]
 \end{aligned} \tag{6.21}$$

The critical damping factor c_c can be interpreted as the **minimum damping** that results in non-periodic motion (i.e. simple decay).

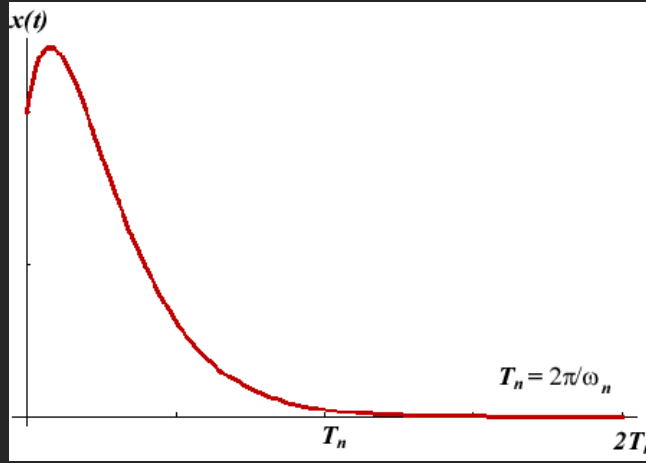


Figure 6.6: The displacement plot of a critically-damped system with positive initial displacement and velocity.

The displacement decays to a negligible level after one natural period T_n . Note that if the initial velocity v_0 is negative while the initial displacement u_0 is positive, there will exist one overshoot of the resting position in the displacement plot.

Time Solution of Unforced Overdamped SDOF Systems

When $c_v^2 - 4mk > 0$ (equivalent to $\zeta > 1$ or $c_v > c_c$), the characteristic equation has two distinct real roots. The displacement solution for this kind of system is:

$$\begin{aligned}
u(t) &= c_1 e^{(-\zeta + \sqrt{\zeta^2 - 1})\omega_n t} + c_2 e^{(-\zeta - \sqrt{\zeta^2 - 1})\omega_n t} \\
\Rightarrow u(t) &= \frac{u_0 \omega_n (\zeta + \sqrt{\zeta^2 - 1}) + v_0}{2\omega_n \sqrt{\zeta^2 - 1}} e^{(-\zeta + \sqrt{\zeta^2 - 1})\omega_n t} + \\
&\quad + \frac{-u_0 \omega_n (\zeta - \sqrt{\zeta^2 - 1}) - v_0}{2\omega_n \sqrt{\zeta^2 - 1}} e^{(-\zeta - \sqrt{\zeta^2 - 1})\omega_n t}
\end{aligned} \tag{6.22}$$

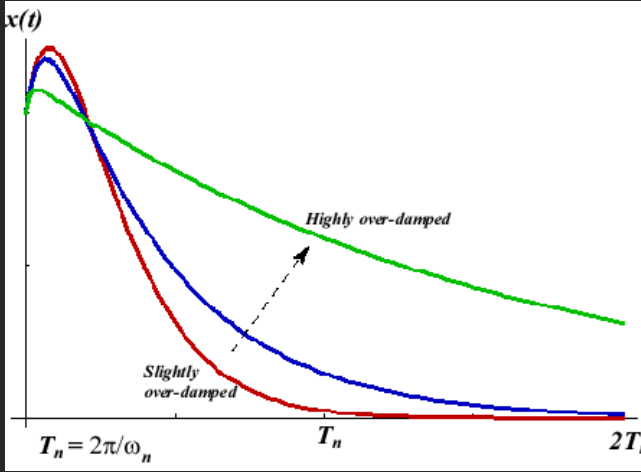


Figure 6.7: The displacement plot of an overdamped system response

The motion of an overdamped system is non-periodic, regardless of the initial conditions. The larger the damping, the longer time to decay from an initial disturbance.

If the system is heavily damped, $\zeta \gg 1$, the displacement solution takes the approximate form:

$$u(t) \approx u_0 + \frac{v_0}{2\zeta\omega_n} (1 - e^{-2\zeta\omega_n t}) \tag{6.23}$$

6.2.5 SDOF Systems under Harmonic Excitation

https://www.efunda.com/formulae/vibrations/sdof_harmo.cfm

When a **SDOF** System is forced by $f(t)$, the solution for the displacement $x(t)$ consists of two parts: the *complimentary solution*, and the *particular solution*. The complimentary solution for the problem is given by the **free vibration of an unforced SDOF System**. The **particular solution** depends on the nature of the forcing function.

When the forcing function is **harmonic** (i.e. it consists of at most a **sine** and **cosine** at the same **frequency**, a quantity that can be expressed by the complex exponential $e^{i\omega t}$), the method of **Undetermined Coefficients** can be used to find the particular solution. Non-harmonic forcing functions are handled by other techniques.

Consider the SDOF system forced by the harmonic function $f(t)$.

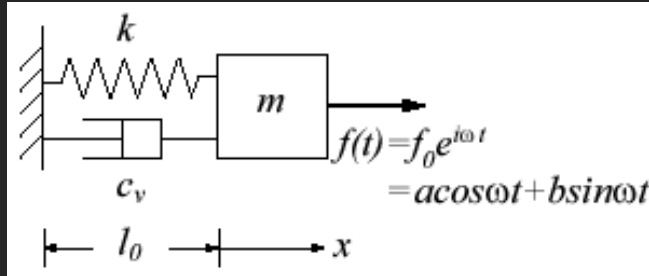


Figure 6.8: SDOF System forced by the harmonic function $f(t)$

The particular solution for this problem is found to be:

$$u_p(t) = \frac{f_0}{(k - m\omega^2) + ic\omega} e^{i\omega t} \quad (6.24)$$

The general solution is given by the sum of the complimentary and particular solutions multiplied by two weighting constants c_1 and c_2 ,

$$u(t) = c_1 u_c(t) + c_2 u_p(t) \quad (6.25)$$

The values of c_1 and c_2 are found by matching $u(t = 0)$ to the initial conditions.

Undamped SDOF Systems under Harmonic Excitation

For an undamped system ($c_v = 0$) the total displacement solution is:

$$\begin{aligned}
 u(t) &= d_1 \cos \omega_n t + d_2 \sin \omega_n t + \frac{f_0}{k - m\omega^2} e^{i\omega t} \\
 \implies u(t) &= \left(u_0 - \frac{f_0}{k - m\omega^2} \right) \cos \omega_n t \\
 &\quad + \left(\frac{v_0 - \frac{i\omega f_0}{k - m\omega^2}}{\omega_n} \right) \sin \omega_n t \\
 &\quad + \frac{f_0}{k - m\omega^2} e^{i\omega t}
 \end{aligned} \tag{6.26}$$

If the forcing frequency is close to the natural frequency, $\omega \approx \omega_n$, the system will exhibit **resonance** (very large displacements) due to near-zeros in the denominators of $u(t)$.

When the forcing frequency is equal to the natural frequency, we cannot use the $u(t)$ given above as it would give divide-by-zero. Instead we must use **L'Hôpital's Rule** to derive a solution free of zeros in the denominators.

$$\begin{aligned}
 u(t) &= u_0 \cos \omega_n t + \frac{v_0}{\omega_n} \sin \omega_n t \\
 &\quad + \lim_{\omega \rightarrow \omega_n} \left\{ \frac{f_0}{k - m\omega^2} (e^{i\omega t} - \cos \omega_n t - i\omega \sin \omega_n t) \right\} \\
 &= u_0 \cos \omega_n t + \frac{v_0}{\omega_n} \sin \omega_n t - \frac{f_0 \omega_n}{2k} (ite^{i\omega} nt - i \sin \omega_n t)
 \end{aligned} \tag{6.27}$$

To simplify $u(t)$, let's assume that the driving force consists only of the cosine function $f(t) = f_0 \cos \omega t$.

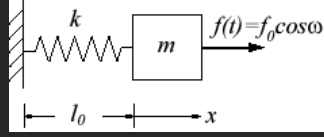


Figure 6.9: SDOF System forced by the harmonic function $f(t) = f_0 \cos \omega t$

The displacement solution reduces to:

$$\begin{aligned}
 u(t) &= u_0 \cos \omega_n t + \frac{v_0}{\omega_n} \sin \omega_n t \\
 &+ \lim_{\omega \rightarrow \omega_n} \left\{ \frac{f_0}{k - m\omega^2} (\cos \omega t - \cos \omega_n t) \right\} \\
 &= \underbrace{u_0 \cos \omega_n t + \frac{v_0}{\omega_n} \sin \omega_n t}_{\text{free vibration (complementary)}} + \underbrace{\frac{f_0 \omega_n t}{2k} \sin \omega_n t}_{\substack{\text{amplitude} \\ \text{linearly} \\ \text{increased}}} \sin \omega_n t
 \end{aligned} \tag{6.28}$$

This solution contains one term multiplied by t . This term will cause the displacement amplitude to increase linearly with time as the forcing function pumps energy into the system.

The maximum displacement of an undamped system forced at its resonant frequency will increase unbounded according to the solution for $u(t)$ above. However, real systems will inject additional physics once displacements become large enough. These additional physics (nonlinear plastic deformation, heat transfer, buckling etc.) will serve to limit the maximum displacement exhibited by the system, and allow one to escape the "sudden death" impression that such systems will immediately fail.

Damped SDOF Systems under Harmonic Excitation

$$m\ddot{u} + c\dot{u} + ku = F_0 \cos(\omega t) \tag{6.29}$$

then **steady state** solution is in the form:

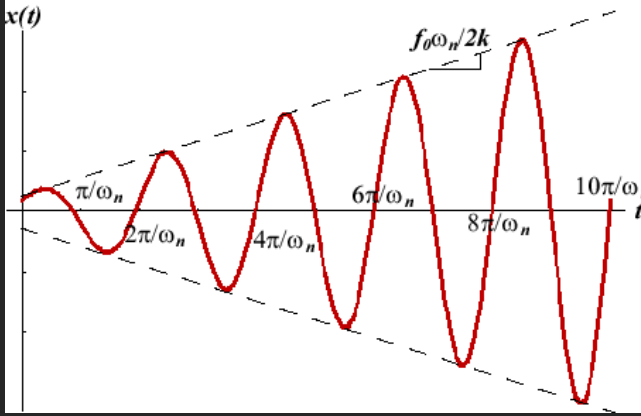


Figure 6.10: Resonance response of undamped SDOF with harmonic excitation

$$u_{s.s.} = u_0 \cos(\omega t - \phi) \quad (6.30)$$

If we plug the **steady state** solution to the **equation of motion**, we get:

$$\begin{aligned} u_o \left[(k - m\omega^2) \cos \omega t - \phi - c\omega \sin(\omega t - \phi) \right] &= F_0 \cos(\omega t) \quad | \times \frac{1}{k} \\ u_o \left[\left(1 - \frac{\omega^2}{\omega_n^2} \right) \cos \omega t - \phi - 2\zeta \frac{\omega}{\omega_n} \sin(\omega t - \phi) \right] &= \frac{F_0}{k} \cos(\omega t) \end{aligned} \quad (6.31)$$

because:

$$\frac{k}{m} = \omega_n^2 \quad (6.32)$$

$$\zeta = \frac{c}{c_c} \quad (6.33)$$

$$c = 2m\sqrt{\frac{k}{m}} = 2m\omega_n \quad (6.34)$$

$$\frac{c}{k} = \frac{\zeta c_c}{k} = \frac{\zeta 2m\omega_n}{k} = \frac{2\zeta\omega_n^2}{\omega_n^2} = \frac{2\zeta}{\omega_n} \quad (6.35)$$

after utilising the sine and cosine relations:

$$\begin{aligned} \cos(\omega t - \phi) &= \cos \omega t \cos \phi + \sin \omega t \sin \phi \\ \sin(\omega t - \phi) &= \sin \omega t \cos \phi + \cos \omega t \sin \phi \end{aligned} \quad (6.36)$$

we get a set of two equatons:

$$\begin{aligned} u_o \left[\left(1 - \frac{\omega^2}{\omega_n^2} \right) \cos \phi + 2\zeta \frac{\omega}{\omega_n} \sin \phi \right] \cos \omega t &= \frac{F_0}{k} \cos(\omega t) \\ u_o \left[\left(1 - \frac{\omega^2}{\omega_n^2} \right) \sin \phi - 2\zeta \frac{\omega}{\omega_n} \cos \phi \right] \sin \omega t &= 0 \end{aligned} \quad (6.37)$$

and when solving just for amplitudes this reduces to a set of two algebraic equations which is then solved for u_0 and ϕ :

$$\begin{aligned} u_o \left[\left(1 - \frac{\omega^2}{\omega_n^2} \right) \cos \phi + 2\zeta \frac{\omega}{\omega_n} \sin \phi \right] &= \frac{F_0}{k} \\ u_o \left[\left(1 - \frac{\omega^2}{\omega_n^2} \right) \sin \phi - 2\zeta \frac{\omega}{\omega_n} \cos \phi \right] &= 0 \end{aligned} \quad (6.38)$$

The solution (**transfer function**) is finally:

$$\begin{aligned} u_0 &= \frac{\frac{F_0}{k}}{\left[\left(1 - \frac{\omega^2}{\omega_n^2} \right)^2 + \left(2\zeta \frac{\omega}{\omega_n} \right)^2 \right]^{\frac{1}{2}}} \\ \phi &= \tan^{-1} \left(\frac{2\zeta \frac{\omega}{\omega_n}}{1 - \frac{\omega^2}{\omega_n^2}} \right) \end{aligned} \quad (6.39)$$

where:

u_0 is displacement amplitude

$\frac{F_0}{k}$ is displacement from static simulation.

6.3 Damping

6.3.1 Rayleigh Damping

<https://www.simscale.com/knowledge-base/rayleigh-damping-coefficients/>

Let **Rayleigh Damping** be defined as:

$$c = \alpha k + \beta m \quad (6.41)$$

where:

c is damping value [-],

α is stiffness dependent damping coefficient,

β is inertia (mass) dependent damping coefficient,

m is mass and

k is stiffness.

Thus, substituting this relation to the equation of motion:

$$\begin{aligned} m\ddot{u} + (\alpha k + \beta m) \dot{u} + ku &= f(t) & | \times \frac{1}{m} \\ \ddot{u} + \left(\alpha \frac{k}{m} + \beta \frac{m}{m} \right) \dot{u} + \frac{k}{m} u &= \frac{f(t)}{m} & | \frac{k}{m} = \omega_n^2 \\ \ddot{u} + (\alpha \omega_n^2 + \beta) \dot{u} + \omega_n^2 u &= \frac{f(t)}{m} \end{aligned} \quad (6.42)$$

Damping Ratio:

$$\begin{aligned} \zeta &= \frac{c}{2m\omega_n} \\ \zeta &= \frac{1}{2m\omega_n} (\alpha k + \beta m) \\ \zeta &= \frac{1}{2} \left(\alpha \omega_n + \frac{\beta}{\omega_n} \right) \end{aligned} \quad (6.43)$$

where:

ζ is damping ratio,

c is damping value [-],

m is mass and

ω_n is natural frequency [rad/s]

Substituting back:

$$\ddot{u} + 2\zeta\omega_n\dot{u} + \omega_n^2 ku = \frac{f(t)}{m} \quad (6.44)$$

When **Damping is proportional to inertia**:

In this case, the stiffness coefficient $\alpha = 0$ and thus:

$$\zeta = \frac{\beta}{2\omega_n} \quad (6.45)$$

For a given constant value of β , it is seen that the damping is inversely proportional to the natural frequency, as shown in the illustration:

Moreover, if one computes β from the damping ration ζ_1 at a given natural frequency ω_1 , all the natural frequencies below it will be amplified and the frequencies above it will be attenuated. The effect is more dramatic the farther the frequencies are from the reference value.

When **Damping is proportional to stiffness**:

In this case, the mass coefficient $\beta = 0$ and thus:

$$\zeta = \frac{1}{2}\alpha\omega_n \quad (6.46)$$

It is seen that, contrary to the first case, here the damping is directly proportional to the natural frequency:

If one computes α from the damping ratio ζ_1 at a natural frequency ω_1 , then the natural frequencies below will be attenuated and the frequencies above will be amplified.

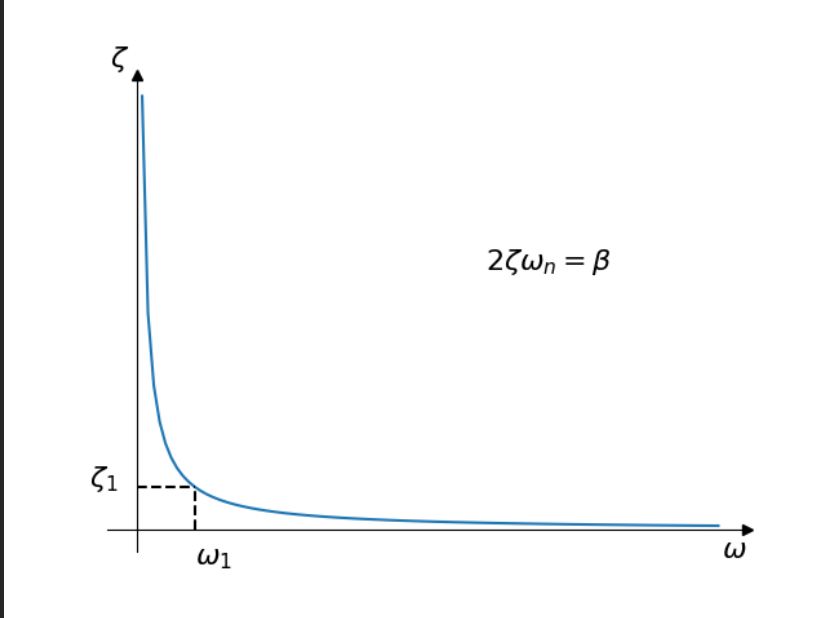


Figure 6.11: Schematic of damping proportional to inertia

General Case:

In the case of using the model with two parameters, the proportionality of damping against frequency is convex:

In this case one needs two damping ratios and two natural frequencies to create a pair of equations and solve for α and β . The model gives some flexibility on where to place the natural frequencies, but in general, frequencies too far away from the ones used in the computation will be amplified.

In the particular case of using equal damping ratios for the two frequencies, it is important to note that the damping ratio **will not be constant inside the range** defined by the sample points, but the inner frequencies will be attenuated. That is, the inner frequencies will have a lower damping ratio.

Computing the Rayleigh Damping Coefficients

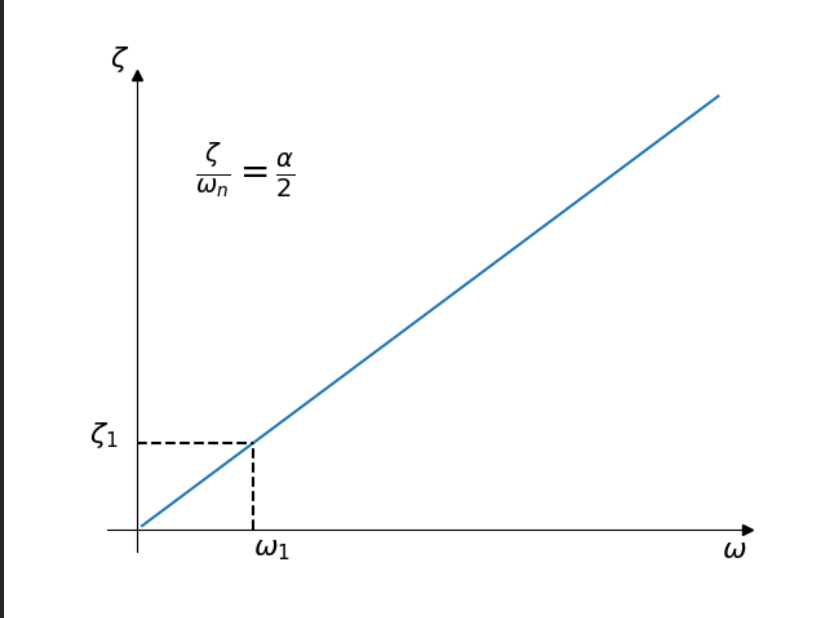


Figure 6.12: Schematic of damping proportional to stiffness

In the most common case, a transient response curve from the system is obtained and the damping ratio ζ_1 is determined for the lowest natural frequency ω_1 by measuring the (logarithmic) attenuation of successive peaks.

$$\begin{aligned}
 \zeta &= \frac{\delta}{\sqrt{\delta^2 + (2\pi)^2}} \\
 \delta &= \ln \frac{x_0}{x_1} \\
 f &= \frac{1}{T} = \frac{1}{t_1 - t_0} \\
 \omega &= 2\pi f
 \end{aligned} \tag{6.47}$$

It is then most common to assume the case of damping proportional to the stiffness, that is, $\beta = 0$, and the α stiffness coefficient is computed from:

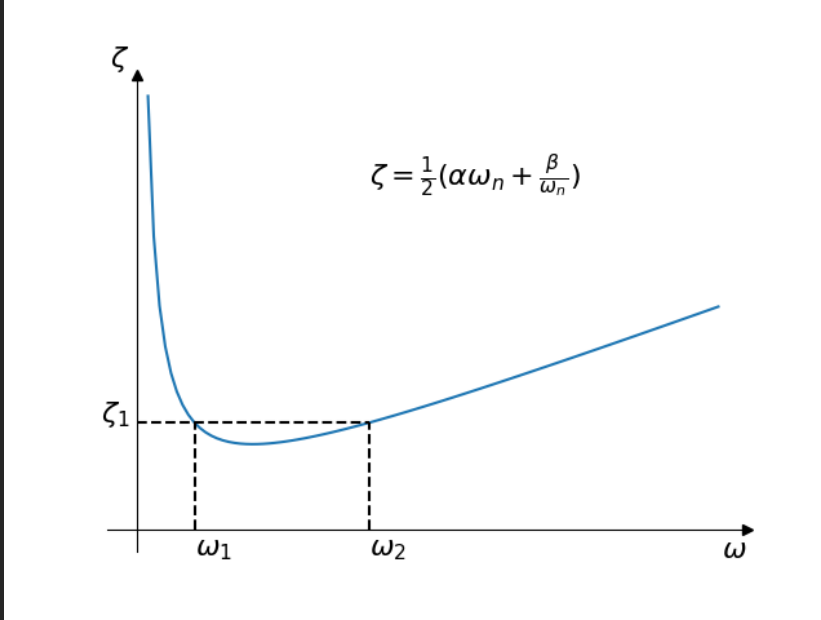


Figure 6.13: Schematic of full damping model

$$\alpha = \frac{2\zeta_1}{\omega_1} = \frac{\zeta + 1}{\pi f_1} \quad (6.48)$$

If the knowledge on the system indicates the case of damping decreasing with the frequency, then one can assume the case of damping proportional to the inertia, where $\alpha = 0$ and determine the mass coefficient β :

$$\beta = 2\zeta_1\omega_1 = 4\pi\zeta_1f_1 \quad (6.49)$$

If there is not such test data or knowledge of the system, or if one wishes to apply an approximate damping ration over a range of frequencies, then we can use the general case and build a system of two equations:

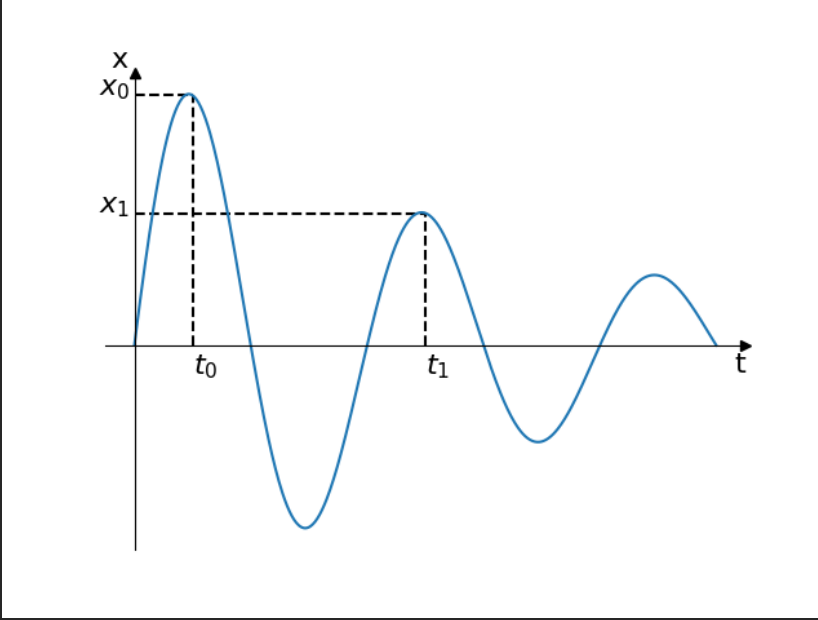


Figure 6.14: Determination of the damping ration from the logarithmic decay

$$\begin{aligned}\zeta_1 &= \frac{1}{2} \left(\alpha \omega_1 + \frac{\beta}{\omega_1} \right) \\ \zeta_2 &= \frac{1}{2} \left(\alpha \omega_2 + \frac{\beta}{\omega_2} \right)\end{aligned}\tag{6.50}$$

Then solve for the unknown coefficients, keeping in mind the considerations given above for the general case and the influence of the model on natural frequencies inside and outside the range of interest. That is, perhaps one wants to achieve a mean damping ratio over the range, then compensate the attenuation by modifying the input damping ratios, or by performing some least-squares approximation from more than two frequency points.

6.4 Modal decomposition

6.4.1 Definition

Let:

$$\mathbf{M}\ddot{\mathbf{u}} + \mathbf{C}\dot{\mathbf{u}} + \mathbf{K}\mathbf{u} = \mathbf{F}(t) \quad (6.51)$$

be the **equation of motion**, then:

$\mathbf{\Lambda}$ is a **spectral matrix**

$\mathbf{\Psi}$ is a **mode shape matrix**,

such that:

$$\mathbf{\Lambda} = \begin{bmatrix} \lambda_1^2 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & \lambda_n^2 \end{bmatrix} \quad (6.52)$$

where λ_n^2 is the n-th eigenvalue of the problem $(\mathbf{K} - \lambda^2\mathbf{M})\mathbf{X} = 0$

and:

$$\mathbf{\Psi} = [\psi_1 \quad \dots \quad \psi_n] \quad (6.53)$$

where $\psi_n = \{\psi_{n,1}, \dots, \psi_{n,m}\}^T$ is the eigenvector of the n-th eigenvalue λ_n^2 .

The **displacement** $\mathbf{u}$ can be expressed as a linear combination of eigenvectors such that:

$$\begin{aligned} \mathbf{u}(t) &= \mathbf{\Psi}\mathbf{q}(t) \\ &= [\psi_1 \quad \dots \quad \psi_n] \mathbf{q}(t) \\ &= \begin{bmatrix} \psi_{1,1} & \dots & \psi_{n,1} \\ \vdots & \ddots & \vdots \\ \psi_{1,m} & \dots & \psi_{n,m} \end{bmatrix} \begin{bmatrix} q_1(t) \\ \vdots \\ q_n(t) \end{bmatrix} \end{aligned} \quad (6.54)$$

where $\mathbf{q}(t)$ is a vector of modal coefficients.

Then:

$$\begin{aligned}\dot{\mathbf{u}}(t) &= \Psi \dot{\mathbf{q}}(t) \\ \ddot{\mathbf{u}}(t) &= \Psi \ddot{\mathbf{q}}(t)\end{aligned}\tag{6.55}$$

First we substitute $\mathbf{u} = \Psi \mathbf{q}$ to the **equation of motion**:

$$\mathbf{M}\Psi\ddot{\mathbf{q}} + \mathbf{C}\Psi\dot{\mathbf{q}} + \mathbf{K}\Psi\mathbf{q} = \mathbf{F}(t)\tag{6.56}$$

By **premultiplying** with Ψ^T we get:

$$\Psi^T \mathbf{M} \Psi \ddot{\mathbf{q}} + \Psi^T \mathbf{C} \Psi \dot{\mathbf{q}} + \Psi^T \mathbf{K} \Psi \mathbf{q} = \Psi^T \mathbf{F}(t) = \mathbf{Q}(t)\tag{6.57}$$

where $\mathbf{Q}(t) = \Psi^T \mathbf{F}(t)$ is called a **modal load**.

6.4.2 Orthonormalised Modal Base

Then one can exploit the properties of **orthonormalised eigenvectors**, where:

$$\psi_r^T \psi_s = 0 \Leftrightarrow r \neq s\tag{6.58}$$

The **mass**, **damping** and **stiffness** matrices are therefore reduced to:

$$\begin{aligned}
\Psi^T \mathbf{M} \Psi &= \begin{bmatrix} m_1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & m_n \end{bmatrix} \\
\Psi^T \mathbf{C} \Psi &= \begin{bmatrix} c_1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & c_n \end{bmatrix} \\
\Psi^T \mathbf{K} \Psi &= \begin{bmatrix} k_1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & k_n \end{bmatrix}
\end{aligned} \tag{6.59}$$

where:

m_i is the **modal mass** of the i-th shape

c_i is the **modal damping** of the i-th shape

k_i is the **modal stiffness** of the i-th shape

Note:

When are the eigenvectors **orthonormalised** to **mass**, the above equations reduce to:

$$\begin{aligned}
 \Psi^T \mathbf{M} \Psi &= \begin{bmatrix} 1.0 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & 1.0 \end{bmatrix} = \mathbf{I} \\
 \Psi^T \mathbf{C} \Psi &= \begin{bmatrix} c_1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & c_n \end{bmatrix} \\
 \Psi^T \mathbf{K} \Psi &= \begin{bmatrix} \lambda_1^2 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & \lambda_n^2 \end{bmatrix} = \Lambda^2
 \end{aligned} \tag{6.60}$$

Mass normalisation:

$$\psi_{i, mass} = \frac{\psi_i}{\psi_i^T \mathbf{m}_i \psi_i} \tag{6.61}$$

where:

$\mathbf{m}_i$ is the i-th row of the **mass** matrix $\mathbf{M}$.

6.4.3 MDOF to SDOF

The MDOF **equation of motion** can be then rewritten as a set of **SDOF** equations:

$$\begin{aligned}
 \Psi^T \mathbf{M} \Psi \ddot{\mathbf{q}}(t) + \Psi^T \mathbf{C} \Psi \dot{\mathbf{q}}(t) + \Psi^T \mathbf{K} \Psi \mathbf{q}(t) &= \Psi^T \mathbf{F}(t) \\
 \mathbf{M}_\Psi \ddot{\mathbf{q}}(t) + \mathbf{C}_\Psi \dot{\mathbf{q}}(t) + \mathbf{K}_\Psi \mathbf{q}(t) &= \mathbf{Q}(t)
 \end{aligned} \tag{6.62}$$

Written explicitly:

$$\begin{aligned}
& \begin{bmatrix} m_1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & m_n \end{bmatrix} \begin{bmatrix} \ddot{q}_1(t) \\ \vdots \\ \ddot{q}_n(t) \end{bmatrix} + \\
& \begin{bmatrix} c_1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & c_n \end{bmatrix} \begin{bmatrix} \dot{q}_1(t) \\ \vdots \\ \dot{q}_n(t) \end{bmatrix} + \\
& \begin{bmatrix} k_1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & k_n \end{bmatrix} \begin{bmatrix} q_1(t) \\ \vdots \\ q_n(t) \end{bmatrix} = \begin{bmatrix} Q_1(t) \\ \vdots \\ Q_n(t) \end{bmatrix}
\end{aligned} \tag{6.63}$$

It is important to note that all modal matrices (mass, damping and stiffness) are purely diagonal (with simple enough damping), so the **MDOF** problem reduces to a series of **SDOF** problems:

$$\begin{aligned}
m_1 \ddot{q}_1(t) + c_1 \dot{q}_1(t) + k_1 q_1(t) &= Q_1(t) \\
m_2 \ddot{q}_2(t) + c_2 \dot{q}_2(t) + k_2 q_2(t) &= Q_2(t) \\
&\vdots \\
m_n \ddot{q}_n(t) + c_n \dot{q}_n(t) + k_n q_n(t) &= Q_n(t)
\end{aligned} \tag{6.64}$$

which are solved separately and the results are **linearly** combined:

$$\begin{aligned}
\mathbf{u} &= \sum_{i=1}^n \psi_i q_i \\
\dot{\mathbf{u}} &= \sum_{i=1}^n \psi_i \dot{q}_i \\
\ddot{\mathbf{u}} &= \sum_{i=1}^n \psi_i \ddot{q}_i
\end{aligned} \tag{6.65}$$

Note: Above equation can be also written as:

$$\begin{aligned}\mathbf{u} &= \sum_{i=1}^n \psi_i q_{u,i} \\ \mathbf{v} &= \sum_{i=1}^n \psi_i q_{v,i} \\ \mathbf{a} &= \sum_{i=1}^n \psi_i q_{a,i}\end{aligned}\tag{6.66}$$

where:

$\mathbf{u}$ is displacement

$\mathbf{v}$ is velocity

$\mathbf{a}$ is acceleration

and

q_u is modal displacement

q_v is modal velocity

q_a is modal acceleration

6.4.4 Initial Conditions

To get the initial modal conditions, one simply:

$$\begin{aligned}\mathbf{q}_0 &= \Psi^{-1} \mathbf{u}_0 \\ \dot{\mathbf{q}}_0 &= \Psi^{-1} \dot{\mathbf{u}}_0 \\ \ddot{\mathbf{q}}_0 &= \Psi^{-1} \ddot{\mathbf{u}}_0\end{aligned}\tag{6.67}$$

where:

Ψ^{-1} is an inverse of eigenshape matrix $\Leftrightarrow$ all eigenvalues and eigenshapes are used.

Note: This is not usually so. Normally one would use **reduced modal base**, where only the first $\mathbf{n}$ eigenmodes are used. Then Ψ^{-1} is written as Ψ^+ and called the **pseudoinverse of mode shape matrix** defined (at least in **numpy** - `numpy.linalg.pinv()`) as:

The pseudo-inverse of a matrix $\mathbf{A}$, denoted $\mathbf{A}^+$, is defined as: “the matrix that ‘solves’ [the least-squares problem] $\mathbf{A}\mathbf{x} = \mathbf{b}$,” i.e., if $\bar{\mathbf{x}}$ is said solution, then $\mathbf{A}^+$ is that matrix such that $\bar{\mathbf{x}} = \mathbf{A}^+\mathbf{b}$.

It can be shown that if $\mathbf{Q}_1\mathbf{\Sigma}\mathbf{Q}_2^T = \mathbf{A}$ is the singular value decomposition of $\mathbf{A}$, then $\mathbf{A}^+ = \mathbf{Q}_2\mathbf{\Sigma}^+\mathbf{Q}_1^T$, where $\mathbf{Q}_{1,2}$ are orthogonal matrices, $\mathbf{\Sigma}$ is a diagonal matrix consisting of $\mathbf{A}$ ’s so-called singular values, (followed, typically, by zeros), and then $\mathbf{\Sigma}^+$ is simply the diagonal matrix consisting of the reciprocals of $\mathbf{A}$ ’s singular values (again, followed by zeros).

Chapter 7

Kinematics

7.1 Rigid Body Motion

A rigid freebody relation between **slave** (dependent) and **master** (independent) DOFs is calculated following the equation:

$$\begin{aligned}\mathbf{T}_s &= \mathbf{T}_m + \mathbf{R}_s \times \mathbf{x} \\ \mathbf{R}_s &= \mathbf{R}_m\end{aligned}\tag{7.1}$$

where $\mathbf{T}$ is a translations vector = $\{\mathbf{T}_1, \mathbf{T}_2, \mathbf{T}_3\}$, $\mathbf{R}$ is a rotations vector = $\{\mathbf{R}_1, \mathbf{R}_2, \mathbf{R}_3\}$, $\mathbf{x}$ is a vector from slave (base) to the master node (tip) = $\{\mathbf{x}_s - \mathbf{x}_m, \mathbf{y}_s - \mathbf{y}_m, \mathbf{z}_s - \mathbf{z}_m\}$ and $\times$ is a vector (cross) product.

Chapter 8

Iterative Methods

8.1 Root finding problems

8.1.1 Incremental Search Method

The approximate locations of the roots are best determined by plotting the function. Often a very rough plot, based on a few points, is sufficient to provide reasonable starting values.

The basic thought behind the incremental search method is simple: If $f(x_1)$ and $f(x_2)$ have opposite signs, then there is at least one root in the interval (x_1, x_2) . If the interval is small enough, it is likely to contain a single root. Thus the zeros of $f(x)$ can be detected by evaluating the function $f(x)$ at intervals Δx and looking for a change in sign.

There are several potential problems with the incremental search method:

- It is possible to miss two closely spaced roots if the search increment Δx is larger than the spacing of the roots
- A double root (two roots that coincide) will not be detected.

- Certain singularities (poles) of $f(x)$ can be mistaken for roots. For example, $f(x) = \tan(x)$ changes sign at $x \pm \frac{1}{2}n\pi$, $n = 1, 3, 5, \dots$. However, these locations are not true zeroes, because the function does not cross the x-axis.

Example of incremental function in python:

```

1 import math
2
3 def rootsearch(f: object, a: float, b: float, dx: float) -> float:
4     """function using incremental search on the interval <a, b>, in increments
5     of dx, to find the bounds <x1, x2> of the smallest root of f(x).
6
7     Args:
8         f (object): function to find the smallest root of
9         a (float): lower bound of the search region
10        b (float): upper bound of the search region
11        dx (float): search step
12
13    Returns:
14        x1 = x2 = None if no roots were detected.
15        x1 < x2 = the bounds specified by dx (x2 = x1 + dx)
16    """
17    x1, f1 = a, f(a)
18    x2, f2 = b, f(b)
19    while math.sign(f1) == math.sign(f2):
20        if x1 >= b:
21            return None, None
22        x1, f1 = x2, f2
23        x2, f2 = x2 + dx, f(x2 + dx)
24    else:
25        return x1, x3
26

```

8.1.2 Bisection Method

After a root $f(x) = 0$ has been bracketed in the interval $< x_1, x_2 >$, several methods can be used to close in on it. The method of **bisection** accomplishes it by *halving* the interval until the error is sufficiently small:

$$\begin{aligned}\varepsilon_{err} &= |x_2 - x_1| \\ \varepsilon_{err} &\leq \varepsilon_{tol}\end{aligned}\tag{8.1}$$

The method of **bisection** uses the same principle as the **incremental** search method. If there is a root $f(x) = 0$ in the interval $x \in < x_1, x_2 >$ then:

$$\text{sign}(f(x_1)) \neq \text{sign}(f(x_2))\tag{8.2}$$

To halve the interval, we compute $f(x_3)$, where $x_3 = \frac{1}{2}(x_1 + x_2)$ is the midpoint of the interval. If $\text{sign}(f(x_3)) \neq \text{sign}(f(x_2))$, then the root must be in the interval $< x_3, x_2 >$ and we replace the x_1 by x_3 . Otherwise the root must be in the interval $< x_1, x_3 >$ and we replace the x_2 by x_3 . In either case the interval is **halved**. The bisection is repeated until the interval has been reduced such that $\varepsilon_{err} \leq \varepsilon_{tol}$.

It is easy to compute the number of bisections needed to reach the prescribed ε_{tol} . The original interval Δx is reduced to $\Delta x/2$ after one bisection, $\Delta x/2^2$ after two bisections, and after n bisections it is $\Delta x/2^n$. Setting $\Delta x/2^n = \varepsilon$ and solving for n , we get:

$$n = \frac{\ln(\Delta x/\varepsilon)}{\ln 2}\tag{8.3}$$

Since n must be an integer, the ceiling of n is used (smallest integer greater n).

Python implementation:

```

1 import math
2
3 def bisection(f: callable, x1: float, x2: float,
4             deny_increase: bool = True, tol: float = 1.E-09) -> float:
5     """Root finding algorithm using a bisection method. The root must be
6     ↪ bracketed in
7     (x1, x2).
8
9     Args:
10         f (callable):          reference to the function to evaluate
11         x1 (float):            lower bounds of the root bracket
12         x2 (float):            upper bounds of the root bracket
13         deny_increase (bool): returns None if |f(x)| increases upon bisection
14                                (default = False)
15         tol (float):           allowed tolerance for root finding
16                                (default = 1.0E-09)
17
18     Returns:
19         (float): if root has been found
20         None:    otherwise
21
22     Raises:
23         ValueError: The root has not been bracketed.
24     """
25     f1 = f(x1)
26     if f1 == 0.: # jackpot -> no more searching
27         return x1
28
29     f2 = f(x2)
30     if f2 == 0.: # jackpot -> no more searching
31         return x2
32
33     if math.sign(f1) == math.sign(f2):
34         raise ValueError(f"The root has not been bracketed in <{x1:f}, {x2:f}>.")
35
36     n = int(math.ceil(math.log(abs(x2 - x1) / tol) / math.log(2.)))

```

```

37     # the bisection itself
38     for i in range(n):
39         x3 = 0.5 * (x1 + x2)
40         f3 = f(x3)
41         # abs value of f(x) is larger than before
42         if deny_increase and ((abs(f3) > abs(f1)) or (abs(f3) > abs(f2))):
43             return None
44         if f3 == 0.: # jackpot -> no more searching
45             return x3
46         elif math.sign(f3) != math.sign(f2):
47             x1, f1 = x3, f3
48         else:
49             x2, f2 = x3, f3
50
51     # return the midpoint of the resulting span
52     return 0.5 * (x1 + x2)
53

```

8.1.3 Methods Based on Linear Interpolation

Secant and False Position Methods

The **secant** and **false position** methods are closely related. Both methods require two starting estimates of the root, say x_1 and x_2 . The function $f(x)$ is assumed to be approximately linear near the root, so that the improved value x_3 of the root can be estimated by linear interpolation between x_1 and x_2 .

This yields the relationship:

$$\frac{f_2}{x_3 - x_2} = \frac{f_1 - f_2}{x_2 - x_1} \quad (8.4)$$

where the notation $f_i = f(x_i)$ is used. Thus the improved estimate of the root is:

$$x_3 = x_2 - f_2 \frac{x_2 - x_1}{f_2 - f_1} \quad (8.5)$$

The difference between **false position** and **secant** method is in the manner the new estimate is dealt with.

- The **false position** method is similar to the **bisection** method where it is necessary that the root is **bracketed** by $x \in \langle x_1, x_2 \rangle$. When a new estimate x_3 is obtained, then if $\text{sign}(f_3) = \text{sign}(f_1)$, we let $x_1 \leftarrow x_3$, otherwise we let $x_2 \leftarrow x_3$. In this manner, the root is always bracketed in $\langle x_1, x_2 \rangle$. Then we run a new iteration until convergence criteria are met.
- The **secant** method is different in two ways. It does not require prior bracketing of the root, and it discards the oldest prior estimate of the root (i.e. after x_3 is computed, we let $x_1 \leftarrow x_2 \leftarrow x_3$).

The convergence of the **secant** method is **superlinear**, with the error behaving as $E_{k+1} = cE_k^{1.618}$, where the exponent 1.618... is called the "golden ratio".

Python implementation of the secant method:

```

1 import math
2
3 def secant(f: callable, x1: float, x2: float,
4           tol: float = 1.E-09, maxiter: int = 1000) -> float:
5     """Root finding algorithm using a secant method.
6
7     Args:
8         f (callable): reference to the function to evaluate
9         x1 (float):    lower bounds of the root bracket
10        x2 (float):    upper bounds of the root bracket
11        tol (float):   allowed tolerance for root finding
12                      (default = 1.0E-09)
13        maxiter (int): maximum number of iterations allowed (default = 1000)
14
15    Returns:
16        (float): if root has been found
17
18    Raises:
19        ValueError: No convergence is achievable as (f1 == f2, function is
20        ↪ constant)
21        ValueError: Maximum number of iterations reached.
22    """
23    f1 = f(x1)
24    if f1 == 0.: # jackpot -> no more searching
25        return x1
26
27    f2 = f(x2)
28    if f2 == 0.: # jackpot -> no more searching
29        return x2
30
31    # the secant method itself
32    for i in range(maxiter):
33        if f1 == f2:
34            raise ValueError(f"The function is constant in interval ({x1:f},
35            ↪ {x2:f}).")
36        x3 = x2 - f2 * (x2 - x1) / (f2 - f1)
37        f3 = f(x3)

```


Ridder's Method

Ridder's method is a clever modification of the **false position** method. Assuming the root is bracketed in $< x_1, x_2 >$, we first compute $f_3 = f(x_3)$, where x_3 is the midpoint of the bracket.

It can be noted:

$$\begin{aligned} x_3 &= \frac{1}{2}(x_1 + x_2) \\ h &= \frac{1}{2}(x_2 - x_1) \end{aligned} \tag{8.6}$$

Next we introduce a new function $g(x)$ such, that:

$$g(x) = f(x)e^{(x-x_1)Q} \tag{8.7}$$

where the constant Q is determined by requiring the points (x_1, g_1) , (x_2, g_2) and (x_3, g_3) to lie on a straight line. As before, the notation is $g_i = g(x_i)$. The improved value x_4 is then obtained by linear interpolation of $g(x)$ rather than $f(x)$.

From 8.7 we obtain:

$$\begin{aligned} g_1 &= f_1 \\ g_2 &= f_2 e^{2hQ} \\ g_3 &= f_3 e^{hQ} \end{aligned} \tag{8.8}$$

where h is obtained from 8.6.

Using the requirement that the three points g_1 , g_2 and g_3 lie on a straight line or:

$$g_3 = \frac{g_1 + g_2}{2} \quad (8.9)$$

or:

$$f_3 e^{hQ} = \frac{1}{2} (f_1 + f_2 e^{2hQ}) \quad (8.10)$$

which is a quadratic equation in e^{hQ} . The solution is

$$e^{hQ} = \frac{f_3 \pm \sqrt{f_3^2 - f_1 f_2}}{f_2} \quad (8.11)$$

Linear interpolation based on points (x_1, g_1) and (x_3, g_3) now yields for the improved root:

$$\begin{aligned} x_4 &= x_3 - g_3 \frac{x_3 - x_1}{g_3 - g_1} \\ x_4 &= x_3 - f_3 e^{hQ} \frac{x_3 - x_1}{f_3 e^{hQ} - f_1} \end{aligned} \quad (8.12)$$

where in the last step the 8.8 is used. As the final step, a substitution for e^{hQ} is used from 8.11 and after some algebra the following solution is obtained:

$$x_4 = x_3 \pm (x_3 - x_1) \frac{f_3}{\sqrt{f_3^2 - f_1 f_2}} \quad (8.13)$$

it can be shown that the correct result is obtained by choosing the + sign if $f_1 - f_2 > 0$ and the minus sign if $f_1 - f_2 < 0$.

The 8.13 can be then adjusted as:

$$x_4 = x_3 + \text{sign}(f_1 - f_2) \times (x_3 - x_1) \frac{f_3}{\sqrt{f_3^2 - f_1 f_2}} \quad (8.14)$$

After the computation of x_4 , new brackets are determined for the root and 8.14 is applied again. The procedure is repeated until the difference between two successive values of x_4 becomes negligible.

The **Ridder's** iterative formula in 8.14 has a very useful property: If x_1 and x_2 bracket the root, then $x_4 \in \langle x_1, x_2 \rangle$ in all cases. In other words, once the root is bracketed, it stays bracketed.

Ridder's method can be shown to converge **quadratically**, making it faster than either the **secant** or the **false position** method.

It is the method to use if the derivative of $f(x)$ is impossible or difficult to compute.

Python implementation:

```

1 import math
2
3 def ridder(f: callable, x1: float, x2: float,
4           rtol: float = 1.E-09, maxiter: int = 1000) -> float:
5     """Root finding algorithm using a Ridder's algorithm. The root must be
6     bracketed in (x1, x2).
7
8     Args:
9         f (callable): reference to the function to evaluate
10        x1 (float):    lower bounds of the root bracket
11        x2 (float):    upper bounds of the root bracket
12        tol (float):   allowed tolerance for root finding
13                      absolute for found root, relative for
14                      change between root iterations
15                      (default = 1.0E-09)
16        maxiter (int): maximum number of iterations
17                      (default = 1000)
18
19     Returns:
20         (float): if root has been found
21         None:    if root is not bracketed
22
23     Raises:
24         ValueError: Maximum number of iterations reached.
25     """
26     f1 = f(x1)
27     if abs(f1) <= tol: # jackpot -> no more searching (absolute error)
28         return x1
29
30     f2 = f(x2)
31     if abs(f2) <= tol: # jackpot -> no more searching (absolute error)
32         return x2
33
34     if math.sign(f1) == math.sign(f2):
35         return None
36
37     x3 = 0.5 * (x1 + x2)

```

```

38
39     # the Ridder's method itself
40     xold = x1
41     for i in range(maxiter):
42         x3 = (x1 + x2) / 2
43         f3 = f(x3)
44         if abs(f3) <= tol: # jackpot -> no more searching (absolute error)
45             return x3
46         s = math.sqrt(f_3 ** 2 - f_1 * f_2)
47         if s == 0.:
48             return None
49         dx = (x3 - x1) * f3 / s
50         x4 = x3 + math.sign(f1 - f2) * dx
51         f4 = f(x4)
52         if abs(f4) <= tol: # jackpot -> no more searching (absolute error)
53             return x4
54
55         # test for convergence -> relative error
56         if abs(x4 - xold) <= tol * abs(x4):
57             return x4
58         xold = x4
59
60         # Re-bracket the root as tightly as possible
61         if math.sign(f3) == math.sign(f4):
62             if math.sign(f1) != math.sign(f4):
63                 # f1 0 f4 f3 f2
64                 x2, f2 = x4, f4
65             else:
66                 # f1 f3 f4 0 f2
67                 x1, f1 = x4, f4
68         else:
69             # f1 f3 0 f4 f2
70             x1, f1, x2, f2 = x3, f3, x4, f4
71
72     else:
73         raise ValueError(f"Maximum number of iterations ({maxiter:d}) reached!")
74

```

Newton-Raphson Method

The **Newton-Raphson** algorithm is the best known method of finding roots for a good reason: it is **simple** and **fast**. The only drawback of the method is that it uses the **derivative** $f'(x)$ of the function as well as the function $f(x)$ itself. Therefore, the **Newton-Raphson** method is usable only in problems where $f'(x)$ can be readily computed.

The **Newton-Raphson** formula can be derived from the **Taylor series** expansion of $f(x)$ about x :

$$f(x_{i+1}) = f(x_i) + f'(x_i)(x_{i+1} - x_i) + \mathcal{O}(x_{i+1} - x_i)^2 \quad (8.15)$$

where $\mathcal{O}(z)$ is to be read as "of the order of z ". If x_{i+1} is a root of $f(x) = 0$. The formula 8.15 becomes:

$$0 = f(x_i) + f'(x_i)(x_{i+1} - x_i) + \mathcal{O}(x_{i+1} - x_i)^2 \quad (8.16)$$

Assuming that $x_i \rightarrow x_{i+1}$, we can drop the last term in 8.16 and solve for x_{i+1} .

Then the **Newton-Raphson** formula is:

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)} \quad (8.17)$$

The formula 8.17 approximates $f(x)$ by the straight line that is tangent to the curve at x_i . This x_{i+1} is at the intersection of the x-axis and the tangent line.

The formula including $f'(x_i)$ can be also derived using the following discrete approach (using Δ as a finite neighborhood of x_i):

$$\begin{aligned} f'(x_1) &= \frac{f(x_i + \Delta/2) - f(x_i - \Delta/2)}{(x_i + \Delta/2) - (x_i - \Delta/2)} \\ f'(x_1) &= \frac{f(x_i + \Delta/2) - f(x_i - \Delta/2)}{\Delta} \end{aligned} \quad (8.18)$$

then setting $dx = \Delta$ and $dy = f(x_i + \Delta/2) - f(x_i - \Delta/2)$:

$$f'(x_1) = \frac{dy}{dx} \quad (8.19)$$

we can write:

$$x_{i+1} = x_i - dx \frac{f(x_i)}{dy} \quad (8.20)$$

The **algorithm** for the **Newton-Raphson** method is simple:

It repeatedly applies 8.17, starting with an initial value x_0 , until convergence criterion $|x_{i+1} - x_i| \leq \varepsilon_{tol}$ is reached, the ε_{tol} being the error tolerance. Only the **latest** value of x has to be stored.

The **algorithm** is here:

1. Let x be an estimate of the root of $f(x) = 0$.
2. Do until $|\Delta x| \leq \varepsilon_{tol}$:
 - (a) Compute $\Delta x = -f(x)/f'(x)$
 - (b) Let $x \leftarrow x + \Delta x$

The truncation error E in the **Newton-Raphson** formula can be shown to behave as

$$E_{i+1} = -\frac{f''(x)}{2f'(x)}E_i^2 \quad (8.21)$$

where x is the **root**. This indicates that the method converges *quadratically* (the error is the square of the error in the previous step). Consequently the number of significant figures is roughly doubled in every iteration.

Althou the **Newton-Raphson** method converges fast near the root, its global convergence characteristics are poor. The reason is that the tangent line is not always an acceptable approximation of the function. However, the method can be made nearly fail-safe by **combining** it with **bisection**.

The following *safe version* of the **Newton-Raphson** method assumes that the root to be computed is initially bracketed in $< x_1, x_2 >$. The midpoint of the bracket is used as the initial guess of the root. The brackets are updated after each iteration. If a **Newton-Raphson** iteration does not stay within the brackets, it is disregarded and replaced with bisection. Because the **Newton-Raphson** method uses the function $f(x)$ as well as its derivative, function routines for both (denoted as f and df) must be provided.

Python implementation:

```

1 import math
2
3 def newtonRaphson(f: callable, df: callable, x: float,
4                 tol: float = 1.E-09, maxiter: int = 1000) -> float:
5     for i in range(maxiter):
6         try:
7             dx = - f(x) / df(x)
8             except ZeroDivisionError as e:
9                 raise ValueError(f"Function derivation f'(x = {x:f}) = 0")
10        x += dx
11        if abs(dx) <= tol * abs(x):
12            return x
13
```

```

14     else:
15         raise ValueError(f"Maximum iteration number reached ({maxiter:d}).")
16
17 def newtonRaphsonImproved(f: callable, df: callable, x1: float, x2: float,
18                           tol: float = 1.E-09, maxiter: int = 1000) -> float:
19     f1 = f(x1)
20     if abs(f1) <= tol:
21         return x1
22
23     f2 = f(x2)
24     if abs(f2) <= tol:
25         return x2
26
27     if math.sign(f1) == math.sign(f2):
28         raise ValueError(f"Root is not bracketed by ({x1:f}, {x2:f}).")
29
30     x3 = (x1 + x2) / 2
31     for i in range(maxiter):
32         f3 = f(x3)
33         if abs(f3) <= tol:
34             return x3
35
36         # tighten the brackets on the root
37         if math.sign(f1) != math.sign(f3):
38             x2 = x3
39         else:
40             x1 = x3
41
42         # try a Newton-Raphson step
43         df3 = df(x3)
44
45         # if division by zero, push x3 out of bounds
46         try:
47             dx = -f3 / df3
48         except ZeroDivisionError as e:
49             dx = x2 - x1
50
51         x3 += dx
52
53         # if the result is out of brackets, use bisection
54         if (x2 - x3) * (x3 - x1) < 0.:

```


8.2 Minimizing function of Multiple variables

Downhill Simplex Method

Downhill Simplex method, also known as **Nelder-Mead** method is best used when optimizing (finding of **minimum**) a function of *multiple* variables, especially when derivatives (the **Jacobian**) are hard or impossible to obtain.

The method is **robust** but **slow**. Special consideration must be taken when preparing the function and when supplying constraints.

The **target function** to optimise usually takes the following form:

$$S(\mathbf{x}) = F(\mathbf{x}) + \sum_{i=1}^k \lambda_i (T_i - C_i(\mathbf{x}))^2 \quad (8.22)$$

where:

$S(\mathbf{x})$ is the optimised function

$\mathbf{x}$ is a vector of variables $\mathbf{x} = \{x_1, x_2, \dots, x_n\}$

$F(\mathbf{x})$ is the function to optimise

T_i is the i-th constraint target value

$C_i(\mathbf{x})$ is the i-th constraint function, and

λ_i is the i-th constraint scaling weight factor, usually $\lambda \gg 1.0$.

The i-th constraint is $\lambda_i (T_i - C_i(\mathbf{x}))^2$ which, using the power of 2, is made to have higher weight the more the actual $C_i(\mathbf{x})$ value differs from the constraint target T_i to both sides of the spectra. The λ_i parameter is used to **scale** the not yet met convergence criterion even further. It is especially effective for values close to 0.

A **maximization** of a function $F(\mathbf{x})$ is actually equal to **minimization** of $-F(\mathbf{x})$.

The idea behind the **downhill simplex** method is to employ a moving **simplex** in the design space to surround the optimal point and then shrink the simplex

until its dimensions reach the specified error of tolerance. In n -dimensional space a **simplex** is a figure of $n + 1$ vertices connected by straight lines and bounded by polygonal faces. If $n = 1$, a **simplex** is a line. If $n = 2$, a **simplex** is a triangle, **tetrahedron** if $n = 3$.

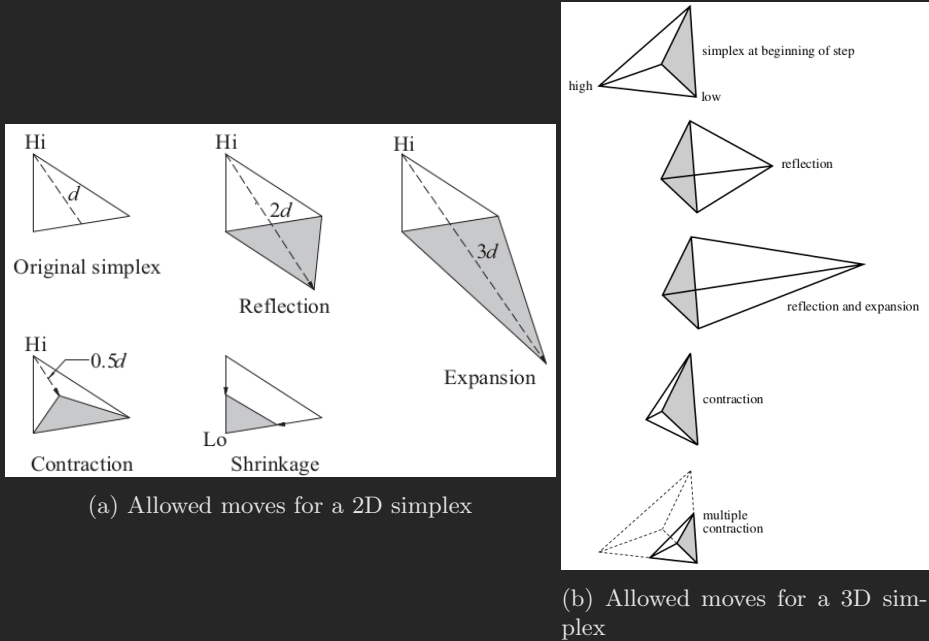


Figure 8.1: Allowed simplex moves for $n = 2$ and $n = 3$

The allowed moves of a **two-dimensional simplex** ($n = 2$) and **three-dimensional simplex** ($n = 3$) are illustrated on pictures 8.1.

By applying these moves in a suitable sequence, the **simplex** can **always** hunt down the minimum point, enclose it, and then shrink around it. The direction of a move is determined by the values of $S(\mathbf{x})$ (the function to be minimised, including constraints) at the **vertices**. The vertex with the **highest** value of S is labeled Hi , and Lo denotes the vertex with the **lowest** value.

The magnitude of a move is controlled by the distance d measured from the Hi vertex to the **centroid** of the opposing vertices.

The **centroid** is simply an arithmetic average of the other vertices.

$$\begin{aligned}
 x_{i_{Hi}} &= H_i \\
 x_c &= \sum_{i \neq i_{Hi}}^n x_i / (n - 1) \\
 d &= x_c - x_{i_{Hi}}
 \end{aligned} \tag{8.23}$$

The outline of the algorithm is as follows:

- Choose a starting simplex (using side parameter)
- Cycle until $d \leq \epsilon_{tol}$
 - find **vertices** v_{Hi} , v_{Lo} with the highest and lowest value of $S(\mathbf{x})$, respectively
 - compute $d = |v_{Hi} - \sum (v \neq v_{Hi}) / (n - 1)|$
 - Try **reflection** $v_n = v_{Hi} - 2d$
 - if $S(v_n) \leq S(v_{Lo})$: accept **reflection** $v_{Hi} = v_n$
 - Try **expansion**: $v_n = v_n - d$
 - if $S(v_n) \leq S(v_{Lo})$: accept **expansion** $v_{Hi} = v_n$
 - else:
 - if $S(v_n) \leq S(v_{Hi})$:
 - Try **contraction**: $v_n = v_{Hi} - d/2$
 - if $S(v_n) \leq S(v_{Hi})$: accept **contraction** $v_{Hi} = v_n$
 - else: use **shrinkage**: $v_i = (v_i + v_{Hi}) / 2 \quad \forall \quad v_i \neq v_{Hi}$

The **downhill simplex method** is much slower than other methods in most cases, but makes up for it in robustness. It often works for problems where other methods hang up.

The starting simplex for a three parameter optimization (with δ side parameter):

$$\mathbf{x}_0 = \{x_1, x_2, x_3\}$$

$$\mathbf{x} = \begin{bmatrix} x_1 & x_2 & x_3 \\ x_1 + \delta & x_2 & x_3 \\ x_1 & x_2 + \delta & x_3 \\ x_1 & x_2 & x_3 + \delta \end{bmatrix} \quad (8.24)$$

Python implementation:

```

1 import numpy as np
2 import math
3
4 def downhill(F: callable, x0: np.ndarray,
5             side: float = 0.1, tol: float = 1.0e-6, maxiter: int = 500,
6             ro: float = 1.0, epsilon: float = 2.,
7             gamma: float = 0.5, sigma: float = 0.5) -> np.ndarray:
8     '''Downhill simplex method for minimization (Nelder-Mead method)
9
10     Args:
11         F (callable):    reference to scalar function that should be minimized
12         x0 (np.ndarray): starting vector x = initial estimate
13         side (float):    side length of the starting simplex (default = 0.1)
14         tol (float):     convergence toelarence (default = 1.0e-06)
15         maxiter (int):   maximum number of iterations (default = 500)
16         ro (float):      reflection coefficient (default ro = 1.0)
17         epsilon (float): expansion coefficient (default epsilon = 2.0)
18         gamma (float):   contraction coefficient (default gamma = 0.5)
19         sigma (float):   shrink coefficient (default sigma = 0.5)
20
21     Returns:
22         (np.ndarray): the optimized values
23
24     '''
25     n = len(x0) # Number of variables
26

```



```

27     # Generate starting simplex
28     x = np.repeat(x0.reshape(1,-1), repeats=n+1, axis=0)
29     # move each parameter by 'side'
30     x[1:,:] += np.eye(n, n) * side
31
32     # Compute values of F at the vertices of the simplex
33     f = np.apply_along_axis(F, axis=1, arr=x)
34
35     # Main loop
36     for k in range(maxiter):
37         # Sort vertices from lowest to highest (indexes: lowest = 0, highest = n)
38         idx = np.argsort(f)
39         f, x = f[idx], x[idx]
40
41         # The centroid of all vertices except the highest  $x_{cog} = \sum x(i < n) / n$ 
42         # Compute the move vector d (from highest vertex to centroid of rest)
43         #  $d = x_{cog} - x_n$ 
44         d = np.sum(x[:-1], axis=0) / n - x[-1]
45
46         # Check for convergence
47         if math.sqrt(np.dot(d, d) / n) < tol:
48             print(f"[+] Convergence reached at iteration: {k:d}")
49             return x[0]
50
51         # Try reflection:  $x_r = x_{cog} + ro * (x_{cog} - x_n)$ 
52         xNew = x[-1] + (1.0 + ro) * d
53         fNew = F(xNew)
54         if fNew <= f[0]: # Accept reflection
55             x[-1], f[-1] = xNew, fNew
56
57         # Try expanding the reflection:  $x_e = x_{cog} + epsilon * (x_r - x_{cog})$ 
58         xNew = x[-1] + (epsilon - ro) * d
59         fNew = F(xNew)
60
61         if fNew <= f[0]: # Accept expansion
62             x[-1], f[-1] = xNew, fNew
63
64         # Try reflection again
65         elif fNew <= f[-1]: # Accept reflection
66             x[-1], f[-1] = xNew, fNew
67

```

```
68         else:
69             # Try contraction:  $w_c = w_{cog} + \gamma * (w_r - w_{cog})$ 
70             xNew = x[-1] + gamma * d
71             fNew = F(xNew)
72
73             if fNew <= f[-1]: # Accept contraction
74                 x[-1], f[-1] = xNew, fNew
75
76         else:
77             # Use shrinkage:  $w_i = w_i + \sigma * (w_i - w_0)$ 
78             x[1:] += sigma * (x[1:] - x[0])
79             f[1:] = np.apply_along_axis(F, axis=1, arr=x[1:])
80
81     print(f"[-] Iteration limit exceeded: {maxiter:d}")
82     return x[0]
83
```

8.3 second order ODE's

Most of FEM problems are **second order Ordinary Differential Equations**, e.g:

$$\begin{aligned} m \frac{u''(t)}{d^2t} + c \frac{u'(t)}{dt} + ku(t) &= 0 \\ m\ddot{u} + c\dot{u} + ku &= 0 \\ ma + cv + ku &= 0 \end{aligned} \quad (8.25)$$

while the iterative methods are for **first order ODE's** only. A **substitution** technique is therefore used to transform the **second order ODE's** to a set of **first order ODE's** and these are then solved sequentially.

First we begin with a **second order ODE** in the form of:

$$a \frac{y''(x)}{d^2x} + b \frac{y'(x)}{dx} + cy = d, \quad \text{with } y(0) = y_0 \quad \text{and} \quad \frac{y'(0)}{dx} = y_1 \quad (8.26)$$

By stating:

$$\frac{dy}{dx} = z \quad (8.27)$$

and substituting back:

$$a \frac{dz}{dx} + bz + cy = d \quad (8.28)$$

now we can write the following set of equations:

$$\begin{aligned} z &= \frac{dy}{dx} \\ \frac{dz}{dx} &= \frac{d - cy - bz}{a} \end{aligned} \quad \begin{cases} y(0) = y_0 \\ z(0) = y_1 \end{cases} \quad (8.29)$$

8.4 Explicit

8.4.1 Euler's method

The **Euler's Method** is an iterative method to solve ODEs. This method's huge plus is it is fast. The minus is it gradually diverges from the ideal solution.

The **Euler's Method** uses the following formula:

$$y(t+h) = y(t) + hf(x, y) \quad (8.30)$$

to construct the tangent at point x and obtain the value of $y(x+h)$, whose slope is:

$$f(x, y) \quad \text{or} \quad \frac{dy}{dx} \quad (8.31)$$

In Euler's method, you can approximate the curve of the solution by the tangent in each interval (that is, by a sequence of short line segments), at steps of h .

In general, if you use small step size, the accuracy of the approximation increases.

General Formula

$$y_{i+1} = y_i + hf(x_i, y_i) \quad (8.32)$$

where:

y_{i+1} is the next estimated solution value,

y_i is the current value,

h is the interval between steps and

$f(x_i, y_i)$ is the value of the derivative at the current (x_i, y_i) point.

Pseudocode:

- define: $f(x, y)$

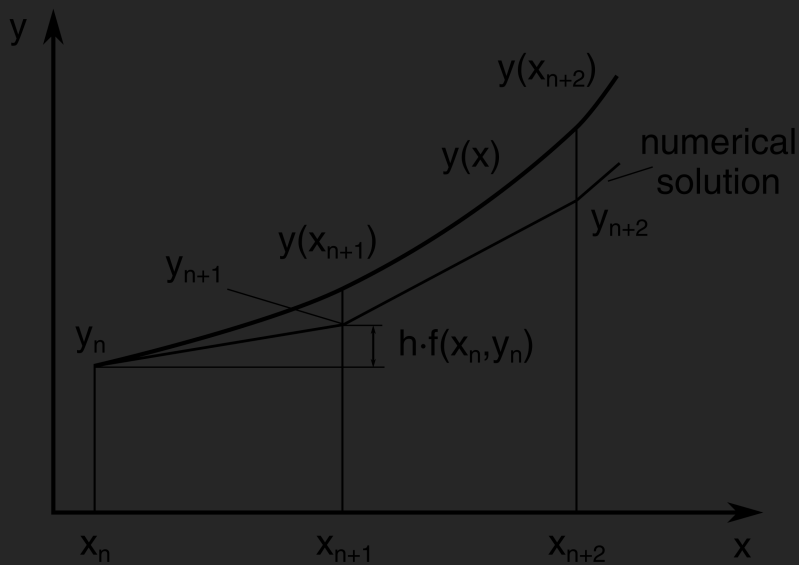


Figure 8.2: Euler's method schematic

- input: x_0, y_0
- input: h, n
- for j from 0 to $(n - 1)$ do
 - $y_{j+1} = y_j + hf(x_j, y_j)$
 - $x_{j+1} = x_j + h$
 - Print x_{j+1} and y_{j+1}
- End.

Note:

If thinking about a problem in time domain:

- define: $f(t, x)$
- input: t_0, x_0
- input: dt, n
- for j from 0 to $(n - 1)$ do
 - $x_{j+1} = x_j + dt f(t_j, x_j)$
 - $t_{j+1} = t_j + dt$
 - Print t_{j+1} and x_{j+1}
- end.

Python Code for vibration problem:

```

1  #!/usr/bin/python3
2  import numpy as np
3  import matplotlib.pyplot as plt
4
5
6  def iterate(h, y0, func, rhs):
7      num_of_odes = y0.shape[0]
8      y1 = np.zeros(num_of_odes, dtype = float)
9      for i in range(num_of_odes-1):
10         y1[i] = y0[i] + y0[i+1] * h
11     y1[-1] = func(y1, rhs)
12     return y1
13
14
15 def euler(t, y, func, rhs):
16     N = t.shape[0]
17     for j in range(N-1):
18         dt = t[j+1] - t[j]
19         y[j+1,:] = iterate(dt, y[j,:], func, rhs[j])
20         # print(y[j+1,:])
21     return y
22
23
24 def euler_vibration():
25     m = 0.55 # tonnes
26     c = 3. # Ns/mm
27     k = 1000. # N/mm
28
29     freq = np.sqrt(k / m) / (2 * np.pi)
30
31     # equation to solve
32     # m * a + c * v + k * u = f
33     # a = (f - c * v - k * u) / m
34     acceleration = lambda u, f: (f - c * u[1] - k * u[0]) / m
35
36     T = 2.0 # seconds
37     dt = 0.0001 # timestep s

```

```

38     u_0 = 0.      # mm of initial displacement
39     v_0 = 0.      # mm/s of initial velocity
40
41     # times at which to solve
42     t = np.linspace(0, T, int(T/dt) + 1)
43
44     # force vector
45     F = 5000. # N of max impulse
46     t0 = 0.1 # impulse start time
47     t1 = 0.2 # impuls end time
48     f = np.zeros(t.shape[0], dtype=float)
49     # create a half sine impulse of force
50     for i in range(t.shape[0]):
51         if t[i] >= t0 and t[i] <= t1:
52             f[i] = F * np.sin((t[i] - t0) / (t1 - t0) * np.pi)
53         else:
54             f[i] = 0.
55
56     uva = np.zeros((t.shape[0],3), dtype=float)
57     uva[0,0] = u_0
58     uva[0,1] = v_0
59
60     uva = euler(t, uva, acceleration, f)
61
62     fig, axf = plt.subplots()
63     fig.subplots_adjust(right=0.60)
64
65     p1, = axf.plot(t, f, label='force', color='violet')
66     axu = axf.twinx()
67     p2, = axu.plot(t, uva[:,0], label='displacement', color='red')
68     axv = axf.twinx()
69     axv.spines.right.set_position(('axes', 1.20))
70     p3, = axv.plot(t, uva[:,1], label='velocity', color='blue')
71     axa = axf.twinx()
72     axa.spines.right.set_position(('axes', 1.40))
73     p4, = axa.plot(t, uva[:,2], label='acceleration', color='green')
74
75     axf.set_xlabel('Time [s]')
76     axf.set_ylabel('Force [N]')
77     axu.set_ylabel('Displacement [mm]')
78     axv.set_ylabel('Velocity [mm/s]')

```



```

79     axa.set_ylabel('Acceleration [mm/s2]')
80
81     axf.legend(handles=[p1, p2, p3, p4])
82
83     plt.show()
84
85 if __name__ == '__main__':
86     euler_vibration()
87

```

Which results to:

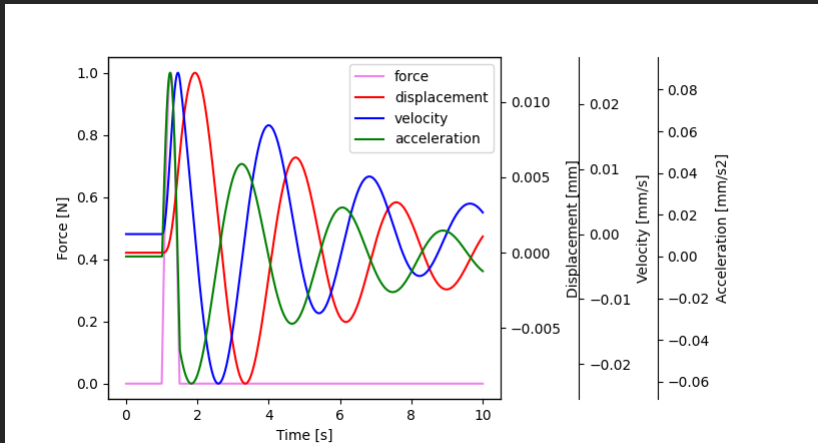


Figure 8.3: Euler's implementation for vibration problem example

The `iterate()` function:

```

1 def iterate(h, y0, func, rhs):
2     num_of_odes = y0.shape[0]
3     y1 = np.zeros(num_of_odes, dtype = float)
4     for i in range(num_of_odes-1):
5         y1[i] = y0[i] + y0[i+1] * h

```

```

6     y1[-1] = func(y1, rhs)
7     return y1
8

```

is written for general case of a set of ODEs. When considering the problem of vibration (set of 2 ODEs):

```

1 def iterate(h, u0, v0, a0, f, m, c, k):
2     u1 = u0 + h * v0
3     v1 = v0 + h * a0
4     # m * a + c * v + k * u = f
5     a1 = (f - c * v1 - k * u1) / m
6     return np.array([u1, v1, a1], dtype=float)

```

which is the iteration of a set of **first order ODEs**:

$$\begin{aligned}
 v &= \dot{u} \\
 \dot{v} &= \frac{f - c * v - k * u}{m}
 \end{aligned}
 \tag{8.33}$$

8.4.2 Runge-Kutta 4th order method

Runge-Kutta 4th order method is another explicit method for solving **first order ODEs**. The basic equation is:

$$\frac{dy}{dx} = f(x, y) \quad , y(0) = y_0 \quad (8.34)$$

The formula for the next value y_{i+1} after a step size equal to h is given by:

$$\begin{aligned} k_1 &= hf(x_i, y_i) \\ k_2 &= hf\left(x_i + \frac{h}{2}, y_i + \frac{k_1}{2}\right) \\ k_3 &= hf\left(x_i + \frac{h}{2}, y_i + \frac{k_2}{2}\right) \\ k_4 &= hf(x_i + h, y_i + k_3) \\ y_{i+1} &= y_i + \frac{k_1}{6} + \frac{k_2}{3} + \frac{k_3}{3} + \frac{k_4}{6} + O(h^5) \end{aligned} \quad (8.35)$$

The formula basically computes next value y_{i+1} using current y_i plus **weighted average of four increments**:

- k_1 is the increment based on the slope at the beginning of the interval, using y
- k_2 is the increment based on the slope at the midpoint of the interval, using $y + hk_1/2$
- k_3 is the increment based on the slope at the midpoint, using $y + hk_2/2$
- k_4 is the increment based on the slope at the end of the interval, using $y + hk_3/2$

The method is a fourth order method, meaning that the local truncation error is on the order of $O(h^5)$, while the total accumulated error is of order $O(h^4)$.

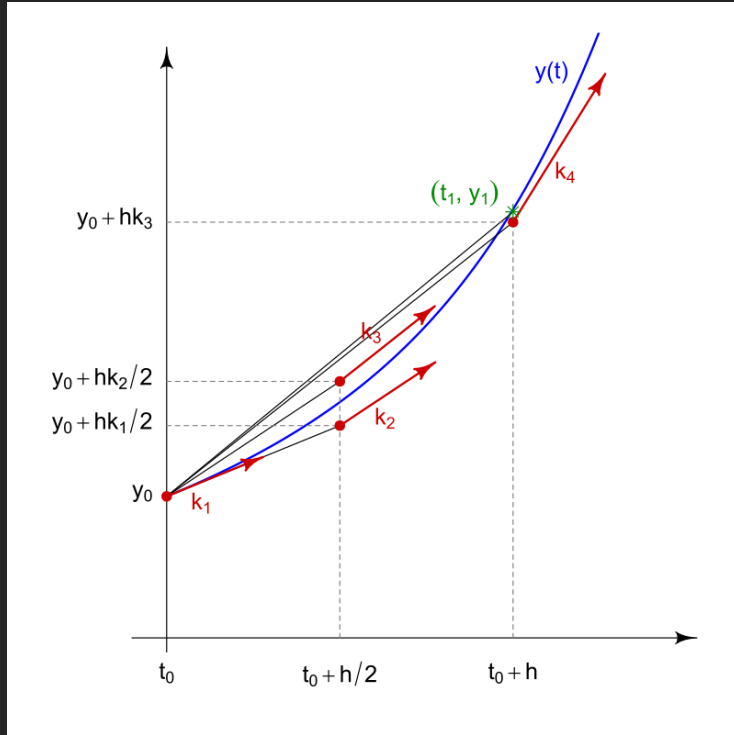


Figure 8.4: 4th Order Runge-Kutta's method schematic

Note:

The formula for the next value y_{i+1} can be also written as:

$$\begin{aligned}
 k_1 &= f(t_i, y_i) \\
 k_2 &= f\left(t_i + \frac{h}{2}, y_i + \frac{h}{2} k_1\right) \\
 k_3 &= f\left(t_i + \frac{h}{2}, y_i + \frac{h}{2} k_2\right) \\
 k_4 &= f(t_i + h, y_i + h k_3) \\
 y_{i+1} &= y_i + h \left(\frac{1}{6} k_1 + \frac{4}{6} k_2 + \frac{2}{6} k_3 + \frac{1}{6} k_4 \right) + O(h^5)
 \end{aligned} \tag{8.36}$$

This notation is more convenient to use when solving sets of ODEs for vibration problem, because it translates to:

$$\begin{aligned}
 u_1 &= u_0 \\
 v_1 &= v_0 \\
 a_1 &= (f - cv_0 - ku_0)/m \\
 u_2 &= u_0 + \frac{h}{2}v_1 \\
 v_2 &= v_0 + \frac{h}{2}a_1 \\
 a_2 &= (f - cv_1 - ku_1)/m \\
 u_3 &= u_0 + \frac{h}{2}v_2 \\
 v_3 &= v_0 + \frac{h}{2}a_2 \\
 a_3 &= (f - cv_2 - ku_2)/m \\
 u_4 &= u_0 + hv_3 \\
 v_4 &= v_0 + ha_3 \\
 a_4 &= (f - cv_3 - ku_3)/m \\
 u_{i+1} &= u_0 + \frac{h}{6}(v_1 + 2v_2 + 2v_3 + v_4) \\
 v_{i+1} &= v_0 + \frac{h}{6}(a_1 + 2a_2 + 2a_3 + a_4) \\
 a_{i+1} &= (f - cv_{i+1} - ku_{i+1})/m
 \end{aligned} \tag{8.37}$$

where each of the **ODEs** are solved sequentially from the values already known.

Python implementation for vibration problem:

```

1  #!/usr/bin/python3
2  import numpy as np
3  import matplotlib.pyplot as plt
4
5
6  def iterate(h, y0, y, func, rhs):
7      num_of_odes = y0.shape[0]
8      y1 = np.zeros(num_of_odes, dtype = float)
9      for i in range(num_of_odes-1):
10         y1[i] = y0[i] + y[i+1] * h
11     y1[-1] = func(y1, rhs)
12     return y1
13
14
15 def runge_kutta_4(t, y, func, rhs):
16     N = t.shape[0]
17     for j in range(N-1):
18         dt = t[j+1] - t[j]
19         y1 = iterate(0., y[j,:], y[j,:], func, rhs[j])
20         y2 = iterate(dt/2, y[j,:], y1, func, 0.5 * (rhs[j+1] + rhs[j]))
21         y3 = iterate(dt/2, y[j,:], y2, func, 0.5 * (rhs[j+1] + rhs[j]))
22         y4 = iterate(dt, y[j,:], y3, func, rhs[j+1])
23
24         # next step solution
25         for i in range(y.shape[1]-1):
26             y[j+1,i] = y[j,i] + dt/6 * (y1[i+1] + 2 * y2[i+1] + 2 * y3[i+1] +
27                                     ↪ y4[i+1])
28         y[j+1,-1] = func(y[j+1,:), rhs[j+1])
29     return y
30
31 def rk4_vibration():
32     m = 10.    # tonnes
33     c = 5.     # Ns/mm
34     k = 50.    # N/mm
35
36     freq = np.sqrt(k / m) / (2 * np.pi)

```

```

37     print(f'f = {freq}')
38
39     # equation to solve
40     #  $m * a + c * v + k * u = f$ 
41     #  $a = (f - c * v - k * u) / m$ 
42     acceleration = lambda u, f: (f - c * u[1] - k * u[0]) / m
43
44     T = 10.      # seconds
45     dt = 0.01   # timestep s
46     u_0 = 0.    # mm of initial displacement
47     v_0 = 0.    # mm/s of initial velocity
48
49     # times at which to solve
50     t = np.linspace(0, T, int(T/dt) + 1)
51
52     # force vector
53     F = 1.      # N of max impulse
54     t0 = 1.     # impulse start time
55     t1 = 1.5    # impuls end time
56     f = np.zeros(t.shape[0], dtype=float)
57     # create a half sine impulse of force
58     for i in range(t.shape[0]):
59         if t[i] >= t0 and t[i] <= t1:
60             f[i] = F * np.sin((t[i] - t0) / (t1 - t0) * np.pi)
61         else:
62             f[i] = 0.
63
64     uva = np.zeros((t.shape[0],3), dtype=float)
65     uva[0,0] = u_0
66     uva[0,1] = v_0
67
68     uva = runge_kutta_4(t, uva, acceleration, f)
69
70     fig, axf = plt.subplots()
71     fig.subplots_adjust(right=0.60)
72
73     p1, = axf.plot(t, f, label='force', color='violet')
74     axu = axf.twinx()
75     p2, = axu.plot(t, uva[:,0], label='displacement', color='red')
76     axv = axf.twinx()
77     axv.spines.right.set_position(('axes', 1.20))

```

```

78     p3, = axv.plot(t, uva[:,1], label='velocity', color='blue')
79     axa = axf.twinx()
80     axa.spines.right.set_position(('axes', 1.40))
81     p4, = axa.plot(t, uva[:,2], label='acceleration', color='green')
82
83     axf.set_xlabel('Time [s]')
84     axf.set_ylabel('Force [N]')
85     axu.set_ylabel('Displacement [mm]')
86     axv.set_ylabel('Velocity [mm/s]')
87     axa.set_ylabel('Acceleration [mm/s2]')
88
89     axf.legend(handles=[p1, p2, p3, p4])
90
91     plt.show()
92
93 if __name__ == '__main__':
94     rk4_vibration()
95

```

Which results to:

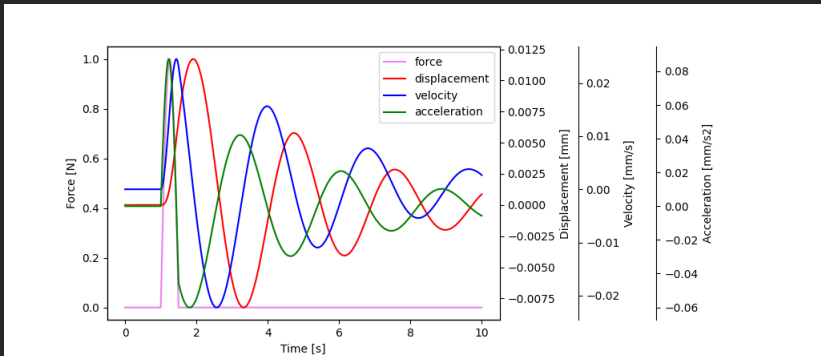


Figure 8.5: 4th order Runge-Kutta's implementation for vibration problem example

8.5 Implicit

The basic approach is an incremental step-by-step solution with an assumption that the solution for a discrete time t is known and that the solution for the discrete time $t + \Delta t$ is required, where Δt is a suitably chosen time increment. Hence, considering time $t + \Delta t$:

$$\mathbf{F}^{t+\Delta t} - \mathbf{N}^{t+\Delta t} = \mathbf{0} \quad (8.38)$$

Where $\mathbf{F}$ is the vector of **external forces** (loads) and $\mathbf{N}$ is the vector of **inner forces** (stresses).

Assume that $\mathbf{F}^{t+\Delta t}$ is independent of the deformations. Since the solution is known at time t , we can write:

$$\mathbf{N}^{t+\Delta t} = \mathbf{N}^t + \Delta \mathbf{N} \quad (8.39)$$

where $\Delta \mathbf{N}$ is the increment in nodal point forces corresponding to the increment in element displacements and stresses from time t to time $t + \Delta t$. This vector can be approximated using a **tangent stiffness matrix** $\mathbf{K}^t$ which corresponds to the geometric and material conditions at time t ,

$$\Delta \mathbf{N} \doteq \mathbf{K}^t \Delta \mathbf{u} \quad (8.40)$$

where $\Delta \mathbf{u}$ is a vector of incremental nodal point displacements and

$$\mathbf{K}^t = \frac{\partial \mathbf{N}^t}{\partial \mathbf{u}^t} \quad (8.41)$$

Hence, the **tangent stiffness matrix** corresponds to the derivative of the internal element nodal point forces $\mathbf{N}^t$ with respect to the nodal point displacements $\mathbf{u}^t$.

Substituting (8.40) and (8.41) into (8.38), we obtain

$$\mathbf{K}^t \Delta \mathbf{u} = \mathbf{F}^{t+\Delta t} - \mathbf{N}^t \quad (8.42)$$

and solving for $\mathbf{u}^{\Delta t}$, we can calculate an approximation to the displacements at time $t + \Delta t$,

$$\mathbf{u}^{t+\Delta t} \doteq \mathbf{u}^t + \Delta \mathbf{u} \quad (8.43)$$

The exact displacements at time $t + \Delta t$ are those that correspond to the applied loads $\mathbf{F}^{t+\Delta t}$. We calculate in (8.43) only an approximation to these displacements because (8.40) was used.

Having evaluated an approximation to the displacements corresponding to time $t + \Delta t$, we could now solve for an approximation to the stresses and corresponding nodal point forces at time $t + \Delta t$, and then proceed to the next time increment calculations. However, because of the assumption (8.40), such a solution may be subject to very significant errors and, depending on the time or load step sizes used, may indeed be unstable. In practice, it is therefore necessary to iterate until the solution of (8.38) is obtained to sufficient accuracy.

8.5.1 Newton-Raphson

Having calculated an *increment* in nodal point displacements, which defines a *new total* displacement vector, we can repeat the incremental solution using the currently known total displacements at time t .

The equations used in **Newton-Raphson** iteration are, *for* $i = 1, 2, 3, \dots$,

$$\begin{aligned} \mathbf{K}_{i-1}^{t+\Delta t} \Delta \mathbf{u}_i &= \mathbf{F}^{t+\Delta t} - \mathbf{N}_{i-1}^{t+\Delta t} \\ \mathbf{u}_i^{t+\Delta t} &= \mathbf{u}_i^{t+\Delta t} + \mathbf{u}_{i-1}^{t+\Delta t} + \Delta \mathbf{u}_i \end{aligned} \quad (8.44)$$

with the initial conditions:

$$\begin{aligned} \mathbf{u}_0^{t+\Delta t} &= \mathbf{u}^t \\ \mathbf{K}_0^{t+\Delta t} &= \mathbf{K}^t \\ \mathbf{N}_0^{t+\Delta t} &= \mathbf{N}^t \end{aligned} \tag{8.45}$$

Note that in the first iteration, the relations in (8.44) reduce to the equations (8.42) and (8.43). Then, in subsequent iterations, the latest estimates for the nodal point displacements are used to evaluate the corresponding element stresses and nodal point forces $\mathbf{N}_{i-1}^{t+\Delta t}$ and **tangent stiffness matrix** $\mathbf{K}_{i-1}^{t+\Delta t}$.

The **tangent stiffness matrix** is achieved in the same way as the **global stiffness matrix**, but of the **deformed** geometry, incorporating the model nonlinearities (if applicable), such as:

- **geometric** non-linearities as large rotations, function-dependent springs, etc.
- **configuration** non-linearities as tension only or compression only elements, contact definitions (usually contacts should be realised on the **RHS** of the $\mathbf{Ku} = \mathbf{f}$ equation)
- **material** non-linearities as plasticity etc.

The out-of-balance load vector $\mathbf{F}^{t+\Delta t} - \mathbf{N}_{i-1}^{t+\Delta t}$ corresponds to a load vector that is not yet balanced by element stress, and hence an increment in the nodal point displacement is required. This updating of the nodal point displacement in the iteration is continued until out-of-balance loads and incremental displacements are small enough.

An important point is that the calculation of $\mathbf{N}_{i-1}^{t+\Delta t}$ from $\mathbf{u}_{i-1}^{t+\Delta t}$ is **crucial**. Any errors in this calculation will, in general, result in an incorrect response prediction.

The correct evaluation of the **tangent stiffness matrix** $\mathbf{K}_{i-1}^{t+\Delta t}$ is also important. The use of proper **tangent stiffness matrix** may be necessary for

convergence and, in general, will result in fewer iterations until convergence is reached.

However, because the expense involved in evaluating and factoring a new **tangent stiffness matrix**, in practice, it can be more efficient, depending on the nonlinearities present in the analysis, to evaluate a new **tangent stiffness matrix** only at certain times. Specifically, in the **modified Newton-Raphson method** a new **tangent stiffness matrix** is established only at the beginning of each load step, and in **quasi-Newton** methods **secant stiffness matrices** are used instead of the tangent stiffness matrix.

The use of the iterative solution requires appropriate convergence criteria. If inappropriate criteria are used, the iteration may be terminated before the necessary solution accuracy is reached or be continued after the required accuracy has been reached.

Full Newton-Raphson Procedure derivation

The finite element equilibrium requirements amount to finding the solution of the equations:

$$f(\mathbf{u}^*) = 0 \quad (8.46)$$

Where:

$$f(\mathbf{u}^*) = \mathbf{F}^{t+\Delta t}(\mathbf{u}^*) - \mathbf{N}^{t+\Delta t}(\mathbf{u}^*) \quad (8.47)$$

We denote here and in the following the complete array of the solution as $\mathbf{u}^*$ but realize that this vector may also contain variables other than displacements, for example, pressure variables and rotations.

Assume that in the iterative solution we have evaluated $\mathbf{u}_{i-1}^{t+\Delta t}$; then a Taylor series expansion gives:

$$f(\mathbf{u}^*) = f(\mathbf{u}_{i-1}^{t+\Delta t}) + \left[\frac{\partial N}{\partial \mathbf{u}} \right]_{\mathbf{u}_{i-1}^{t+\Delta t}} (\mathbf{u}^* - \mathbf{u}_{i-1}^{t+\Delta t}) + \text{higher order terms} \quad (8.48)$$

Substituting from (8.47) into (8.48) and using (8.46), we obtain:

$$\begin{aligned} & \left[\frac{\partial \mathbf{N}}{\partial \mathbf{u}} \right]_{\mathbf{u}_{i-1}^{t+\Delta t}} (\mathbf{u}^* - \mathbf{u}_{i-1}^{t+\Delta t}) + \text{higher order terms} \\ &= \mathbf{F}^{t+\Delta t} - \mathbf{N}_{i-1}^{t+\Delta t} \end{aligned} \quad (8.49)$$

where we assumed that the externally applied loads are deformation independent.

Neglecting the higher-order terms in (8.49), we can calculate an increment in the displacements,

$$\mathbf{K}_{i-1}^{t+\Delta t} \Delta \mathbf{u}_i = \mathbf{F}^{t+\Delta t} - \mathbf{N}_{i-1}^{t+\Delta t} \quad (8.50)$$

where $\mathbf{K}_{i-1}^{t+\Delta t}$ is the current **tangent stiffness matrix**

$$\mathbf{K}_{i-1}^{t+\Delta t} = \left[\frac{\partial \mathbf{N}}{\partial \mathbf{u}} \right]_{\mathbf{u}_{i-1}^{t+\Delta t}} \quad (8.51)$$

and the improved displacement solution is:

$$\mathbf{u}_i^{t+\Delta t} = \mathbf{u}_{i-1}^{t+\Delta t} + \Delta \mathbf{u}_i \quad (8.52)$$

The relations in (8.50) and (8.52) constitute the Newton-Raphson solution of (8.38). Since an incremental analysis is performed with time (or load) steps of size Δt , the initial conditions in this iteration are $\mathbf{K}_0^{t+\Delta t} = \mathbf{K}^t$, $\mathbf{N}_0^{t+\Delta t} = \mathbf{N}^t$,

and $\mathbf{u}_0^{t+\Delta t} = \mathbf{u}^t$. The iteration is continued **until appropriate convergence criteria are satisfied**.

A characteristic of this iteration is that a new tangent stiffness matrix is calculated in **each** iteration, which is why this method is also referred to as the **full Newton-Raphson method**.

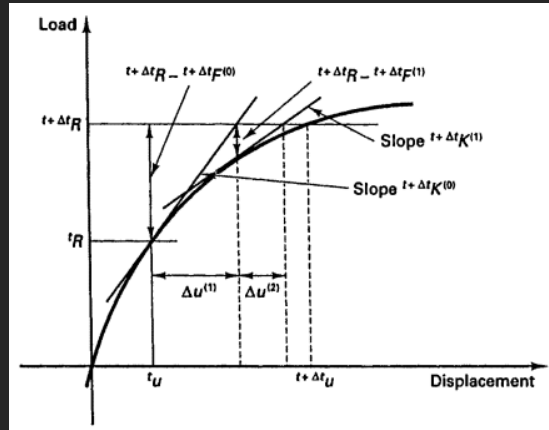


Figure 8.6: Newton-Raphson iteration in solution of SDOF system. $\mathbf{R}$ is load increment, $\mathbf{F}$ are internal forces, $\mathbf{u}$ are displacements.

Chapter 9

Model reduction

9.1 Guyan reduction

Guyan reduction, also known as **static condensation**, is a *dimensionality reduction* method which reduces the number of **DOFs** by ignoring the internal terms of the equilibrium equations and expressing the unloaded **DOFs** in terms of the loaded **DOFs**.

The static equilibrium equation:

$$\begin{bmatrix} \mathbf{K}_{mm} & \mathbf{K}_{ms} \\ \mathbf{K}_{sm} & \mathbf{K}_{ss} \end{bmatrix} \begin{bmatrix} \mathbf{u}_m \\ \mathbf{u}_s \end{bmatrix} = \begin{bmatrix} \mathbf{f}_m \\ \mathbf{f}_s \end{bmatrix} \quad (9.1)$$
$$\begin{aligned} \mathbf{K}_{mm} \mathbf{u}_m + \mathbf{K}_{ms} \mathbf{u}_s &= \mathbf{f}_m \\ \mathbf{K}_{sm} \mathbf{u}_m + \mathbf{K}_{ss} \mathbf{u}_s &= \mathbf{f}_s \end{aligned}$$

Where m denotes the **master** DOFs and s the **slave** DOFs. Stating that s section of the problem DOFs is unloaded, we can write that $\mathbf{f}_s = \mathbf{0}$. Then the slave DOFs are expressed by the following equation:

$$\mathbf{K}_{sm} \mathbf{u}_m + \mathbf{K}_{ss} \mathbf{u}_s = \mathbf{0} \quad (9.2)$$

Solving the above equation in terms of the independent (master) DOFs leads to the following dependency relations:

$$\mathbf{u}_s = -\mathbf{K}_{ss}^{-1}\mathbf{K}_{sm}\mathbf{u}_m \quad (9.3)$$

Substituting the dependency relations to the upper (master) partition of the static equilibrium problem condenses away the slave DOFs, leading to the following reduced system of linear equations:

$$(\mathbf{K}_{mm} - \mathbf{K}_{ms}\mathbf{K}_{ss}^{-1}\mathbf{K}_{sm})\mathbf{u}_m = \mathbf{f}_m \quad (9.4)$$

The above system of linear equations is equivalent to the original problem, but expressed in terms of **master** DOFs alone. Thus, the **Guyan** reduction method results in a reduced system by condensing away the **slave** DOFs.

The **Guyan reduction** can be also expressed as a *change of basis* which produces a low-dimensional representation of the original space, represented by the **master** DOFs. The linear transformation that maps the reduced space onto the full space is expressed as:

$$\begin{bmatrix} \mathbf{u}_m \\ \mathbf{u}_s \end{bmatrix} = \begin{bmatrix} \mathbf{I} \\ -\mathbf{K}_{ss}^{-1}\mathbf{K}_{sm} \end{bmatrix} \mathbf{u}_m = \mathbf{T}_G \mathbf{u}_m \quad (9.5)$$

where $\mathbf{T}_G$ represents the **Guyan** reduction *transformation matrix*. Thus, the reduced problem is represented as:

$$\mathbf{K}_G \mathbf{u}_m = \mathbf{f}_m \quad (9.6)$$

In the above equation, $\mathbf{K}_G$ represents the reduced system of linear equations that's obtained by applying the **Guyan reduction** transformation on the full system, which is expressed as:

$$\mathbf{K}_G = \mathbf{T}_G^T \mathbf{K} \mathbf{T}_G \quad (9.7)$$

where $\mathbf{T}_G$ has the dimension of $n_{DOF} \times n_m$.

9.2 Craig-Bampton reduction

The **Craig-Bampton reduction**, also known as the **fixed-interface component mode synthesis**, extends the **Guyan reduction** by supplementing the static deformation shapes with a truncated set of dynamic deformation shapes. It therefore retains the exact static relation between the interface and internal DOFs, while approximating the internal dynamic deformation by a modal base.

The dynamic equilibrium equation is partitioned into the **boundary** DOFs b and the **internal** DOFs i :

$$\begin{bmatrix} \mathbf{M}_{bb} & \mathbf{M}_{bi} \\ \mathbf{M}_{ib} & \mathbf{M}_{ii} \end{bmatrix} \begin{bmatrix} \ddot{\mathbf{u}}_b \\ \ddot{\mathbf{u}}_i \end{bmatrix} + \begin{bmatrix} \mathbf{K}_{bb} & \mathbf{K}_{bi} \\ \mathbf{K}_{ib} & \mathbf{K}_{ii} \end{bmatrix} \begin{bmatrix} \mathbf{u}_b \\ \mathbf{u}_i \end{bmatrix} = \begin{bmatrix} \mathbf{f}_b \\ \mathbf{f}_i \end{bmatrix} \quad (9.8)$$

where the boundary DOFs are retained in physical coordinates and the internal DOFs are reduced. In the terminology used for the **Guyan reduction**, the boundary DOFs correspond to the master DOFs and the internal DOFs to the slave DOFs.

9.2.1 Constraint modes

The **constraint modes**, also called the **static constraint modes**, describe the static deformation of the internal DOFs caused by a unit displacement of each individual boundary DOF. The inertia terms and the externally applied forces on the internal DOFs are set to zero:

$$\mathbf{K}_{ib}\mathbf{u}_b + \mathbf{K}_{ii}\mathbf{u}_i = \mathbf{0} \quad (9.9)$$

The internal displacement is then expressed as:

$$\mathbf{u}_i = -\mathbf{K}_{ii}^{-1}\mathbf{K}_{ib}\mathbf{u}_b = \Psi_c\mathbf{u}_b \quad (9.10)$$

where:

$$\Psi_c = -\mathbf{K}_{ii}^{-1}\mathbf{K}_{ib} \quad (9.11)$$

is the **constraint mode matrix**. Each column of Ψ_c is the internal static displacement field corresponding to a unit displacement of one boundary DOF, while all remaining boundary DOFs are held fixed.

The inverse $\mathbf{K}_{ii}^{-1}$ is only a mathematical notation. Numerically, the constraint modes are obtained by solving the multiple right-hand-side problem:

$$\mathbf{K}_{ii}\Psi_c = -\mathbf{K}_{ib} \quad (9.12)$$

The matrix $\mathbf{K}_{ii}$ is factorised only once and the factorisation is reused for all boundary DOFs.

9.2.2 Fixed-interface normal modes

The dynamic deformation that cannot be represented by the constraint modes is approximated using the **fixed-interface normal modes**. These modes are calculated by fixing all boundary DOFs:

$$\mathbf{u}_b = \mathbf{0} \quad (9.13)$$

and solving the internal generalised eigenvalue problem:

$$(\mathbf{K}_{ii} - \omega_j^2 \mathbf{M}_{ii}) \boldsymbol{\varphi}_j = \mathbf{0} \quad (9.14)$$

The retained fixed-interface modes are assembled column-wise into the matrix:

$$\Phi_i = [\boldsymbol{\varphi}_1 \quad \boldsymbol{\varphi}_2 \quad \dots \quad \boldsymbol{\varphi}_{n_\varphi}] \quad (9.15)$$

where n_φ is the number of retained fixed-interface modes. When the modes are mass-normalised:

$$\begin{aligned}\Phi_i^T \mathbf{M}_{ii} \Phi_i &= \mathbf{I} \\ \Phi_i^T \mathbf{K}_{ii} \Phi_i &= \Omega_i^2\end{aligned}\tag{9.16}$$

where:

$$\Omega_i^2 = \begin{bmatrix} \omega_1^2 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & \omega_{n_\varphi}^2 \end{bmatrix}\tag{9.17}$$

is the diagonal matrix of the retained fixed-interface eigenvalues.

9.2.3 Craig-Bampton transformation

The internal displacement is represented as a superposition of the static constraint modes and the fixed-interface normal modes:

$$\mathbf{u}_i = \Psi_c \mathbf{u}_b + \Phi_i \boldsymbol{\eta}\tag{9.18}$$

where $\boldsymbol{\eta}$ is the vector of the fixed-interface modal coordinates. The full physical displacement vector is therefore expressed as:

$$\begin{aligned}\begin{bmatrix} \mathbf{u}_b \\ \mathbf{u}_i \end{bmatrix} &= \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \Psi_c & \Phi_i \end{bmatrix} \begin{bmatrix} \mathbf{u}_b \\ \boldsymbol{\eta} \end{bmatrix} \\ \mathbf{u} &= \mathbf{T}_{CB} \mathbf{q}_{CB}\end{aligned}\tag{9.19}$$

where:

$$\begin{aligned}\mathbf{T}_{CB} &= \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \Psi_c & \Phi_i \end{bmatrix} \\ \mathbf{q}_{CB} &= \begin{bmatrix} \mathbf{u}_b \\ \boldsymbol{\eta} \end{bmatrix}\end{aligned}\tag{9.20}$$

The Craig-Bampton coordinates therefore consist of:

- the physical displacements of all retained boundary DOFs,
- the modal amplitudes of the retained fixed-interface normal modes.

Substituting $\mathbf{u} = \mathbf{T}_{CB}\mathbf{q}_{CB}$ to the full equation of motion and premultiplying it with $\mathbf{T}_{CB}^T$ gives the reduced equation:

$$\mathbf{M}_{CB}\ddot{\mathbf{q}}_{CB} + \mathbf{C}_{CB}\dot{\mathbf{q}}_{CB} + \mathbf{K}_{CB}\mathbf{q}_{CB} = \mathbf{f}_{CB} \quad (9.21)$$

where:

$$\begin{aligned} \mathbf{M}_{CB} &= \mathbf{T}_{CB}^T \mathbf{M} \mathbf{T}_{CB} \\ \mathbf{C}_{CB} &= \mathbf{T}_{CB}^T \mathbf{C} \mathbf{T}_{CB} \\ \mathbf{K}_{CB} &= \mathbf{T}_{CB}^T \mathbf{K} \mathbf{T}_{CB} \\ \mathbf{f}_{CB} &= \mathbf{T}_{CB}^T \mathbf{f} \end{aligned} \quad (9.22)$$

For exact constraint modes and a symmetric stiffness matrix, the reduced stiffness matrix has the following block-diagonal form:

$$\mathbf{K}_{CB} = \begin{bmatrix} \mathbf{K}_G & \mathbf{0} \\ \mathbf{0} & \mathbf{\Omega}_i^2 \end{bmatrix} \quad (9.23)$$

where:

$$\mathbf{K}_G = \mathbf{K}_{bb} - \mathbf{K}_{bi}\mathbf{K}_{ii}^{-1}\mathbf{K}_{ib} \quad (9.24)$$

is the **Guyan-condensed stiffness matrix**. The corresponding mass matrix is generally not block-diagonal:

$$\mathbf{M}_{CB} = \begin{bmatrix} \mathbf{M}_{bb}^{CB} & \mathbf{M}_{b\varphi}^{CB} \\ \mathbf{M}_{\varphi b}^{CB} & \mathbf{I} \end{bmatrix} \quad (9.25)$$

with:

$$\begin{aligned} \mathbf{M}_{bb}^{CB} &= \mathbf{M}_{bb} + \mathbf{M}_{bi}\Psi_c + \Psi_c^T \mathbf{M}_{ib} + \Psi_c^T \mathbf{M}_{ii} \Psi_c \\ \mathbf{M}_{b\varphi}^{CB} &= (\mathbf{M}_{bi} + \Psi_c^T \mathbf{M}_{ii}) \Phi_i \\ \mathbf{M}_{\varphi b}^{CB} &= (\mathbf{M}_{b\varphi}^{CB})^T \end{aligned} \quad (9.26)$$

The coupling in the reduced mass matrix is the reason why the Craig-Bampton coordinates are not normal modes of the unconstrained component. The boundary coordinates remain physical and are dynamically coupled with the fixed-interface modal coordinates.

Note:

The Craig-Bampton transformation does not reduce the boundary DOFs. If a component has n_b boundary DOFs and n_φ retained fixed-interface modes, then the reduced model has:

$$n_{CB} = n_b + n_\varphi \quad (9.27)$$

generalised coordinates. A large number of interface nodes can therefore still result in a relatively large Craig-Bampton model.

9.3 Flexible body modal reduction

A **flexible body** used in a multibody or reduced-order simulation is often represented by a small set of free-interface normal modes instead of keeping all Craig-Bampton boundary coordinates as independent generalised coordinates. This can be obtained by performing a second modal reduction in the Craig-Bampton space.

The Craig-Bampton equation without damping and external forces is:

$$\mathbf{M}_{CB}\ddot{\mathbf{q}}_{CB} + \mathbf{K}_{CB}\mathbf{q}_{CB} = \mathbf{0} \quad (9.28)$$

No boundary coordinates are constrained in the second reduction pass. The free-interface generalised eigenvalue problem is therefore:

$$(\mathbf{K}_{CB} - \omega_j^2 \mathbf{M}_{CB}) \boldsymbol{\xi}_j = \mathbf{0} \quad (9.29)$$

The eigenvectors are assembled into the reduced free-interface mode matrix:

$$\boldsymbol{\Xi} = [\boldsymbol{\xi}_1 \quad \boldsymbol{\xi}_2 \quad \dots \quad \boldsymbol{\xi}_{n_f}] \quad (9.30)$$

where n_f is the number of retained free-interface modes. The Craig-Bampton coordinates are then represented by:

$$\mathbf{q}_{CB} = \boldsymbol{\Xi} \mathbf{q}_f \quad (9.31)$$

and the full physical displacement is recovered as:

$$\begin{aligned} \mathbf{u} &= \mathbf{T}_{CB} \mathbf{q}_{CB} \\ &= \mathbf{T}_{CB} \boldsymbol{\Xi} \mathbf{q}_f \\ &= \mathbf{T}_f \mathbf{q}_f \end{aligned} \quad (9.32)$$

where:

$$\mathbf{T}_f = \mathbf{T}_{CB} \boldsymbol{\Xi} \quad (9.33)$$

is the final **flexible body transformation matrix**. The columns of $\mathbf{T}_f$ are free-interface mode shapes expressed at the original FE DOFs. The interface

displacement is no longer an independent coordinate. It is approximated by the retained flexible body modes in the same way as the internal displacement.

The final reduced matrices are calculated as:

$$\begin{aligned}
 \mathbf{M}_f &= \mathbf{\Xi}^T \mathbf{M}_{CB} \mathbf{\Xi} = \mathbf{T}_f^T \mathbf{M} \mathbf{T}_f \\
 \mathbf{K}_f &= \mathbf{\Xi}^T \mathbf{K}_{CB} \mathbf{\Xi} = \mathbf{T}_f^T \mathbf{K} \mathbf{T}_f \\
 \mathbf{C}_f &= \mathbf{\Xi}^T \mathbf{C}_{CB} \mathbf{\Xi} = \mathbf{T}_f^T \mathbf{C} \mathbf{T}_f \\
 \mathbf{f}_f &= \mathbf{\Xi}^T \mathbf{f}_{CB} = \mathbf{T}_f^T \mathbf{f}
 \end{aligned} \tag{9.34}$$

When the free-interface eigenvectors are mass-normalised:

$$\begin{aligned}
 \mathbf{\Xi}^T \mathbf{M}_{CB} \mathbf{\Xi} &= \mathbf{I} \\
 \mathbf{\Xi}^T \mathbf{K}_{CB} \mathbf{\Xi} &= \mathbf{\Omega}_f^2
 \end{aligned} \tag{9.35}$$

The flexible body equation of motion then becomes:

$$\ddot{\mathbf{q}}_f + \mathbf{C}_f \dot{\mathbf{q}}_f + \mathbf{\Omega}_f^2 \mathbf{q}_f = \mathbf{f}_f \tag{9.36}$$

For classical modal damping, the damping matrix can be represented by a diagonal matrix:

$$\mathbf{C}_f = \begin{bmatrix} 2\zeta_1\omega_1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & 2\zeta_{n_f}\omega_{n_f} \end{bmatrix} \tag{9.37}$$

9.3.1 Rigid body modes

A free three-dimensional component has six **rigid body modes**: three translations and three rotations. Their eigenvalues are theoretically equal to zero because a rigid body displacement produces no elastic strain energy:

$$\mathbf{K}_{CB}\boldsymbol{\xi}_r = \mathbf{0} \quad (9.38)$$

In finite-precision arithmetic, the rigid body eigenvalues are usually small but not exactly zero. They must therefore be identified using a tolerance relative to the characteristic stiffness or to the first elastic eigenvalue, rather than by checking whether the calculated frequency is exactly zero.

The exact treatment of the rigid body modes depends on the formulation of the surrounding simulation:

- when the flexible component is described only by modal coordinates, the rigid body modes must be retained to allow free translation and rotation,
- when the multibody formulation describes the rigid body position and orientation separately, the six rigid body modes are not used as elastic coordinates; the retained elastic modes must nevertheless remain mass-orthogonal to the rigid body motion.

The second case is commonly used by floating-frame flexible body formulations. The large rigid motion is described by the reference frame of the body and the modal coordinates describe only the small elastic deformation relative to this frame.

9.3.2 Mode truncation

If all n_{CB} eigenvectors of the reduced free-interface eigenproblem are kept, the transformation Ξ represents only another basis of the same Craig-Bampton space. The second reduction becomes a dimensionality reduction only when the mode matrix is truncated:

$$n_f < n_{CB} \quad (9.39)$$

This provides the distinction between the two reduced representations:

- the Craig-Bampton model retains all boundary DOFs and therefore preserves an independent physical coordinate at every interface DOF,
- the flexible body modal model retains only selected free-interface modes and therefore approximates both the boundary and internal deformation by a smaller number of modal coordinates.

The accuracy of the final flexible body base is limited by both truncation stages. The fixed-interface base must first be rich enough to represent the required internal dynamics. The free-interface base is then selected according to the frequency range and deformation patterns required by the final simulation. Increasing the number of free-interface modes cannot recover deformation that was already excluded from the Craig-Bampton base.

9.3.3 Numerical procedure

A numerically efficient implementation of the complete reduction can be summarised as follows:

1. Reorder the FE matrices into boundary and internal DOFs.
2. Factorise $\mathbf{K}_{ii}$ and solve $\mathbf{K}_{ii}\boldsymbol{\Psi}_c = -\mathbf{K}_{ib}$ for all constraint modes.
3. Solve the lowest required eigenpairs of $\mathbf{K}_{ii}\boldsymbol{\varphi} = \omega^2\mathbf{M}_{ii}\boldsymbol{\varphi}$ with the boundary fixed.
4. Mass-normalise the fixed-interface modes and assemble $\mathbf{T}_{CB}$.
5. Calculate $\mathbf{M}_{CB}$, $\mathbf{K}_{CB}$ and, when required, $\mathbf{C}_{CB}$ by congruent transformation.
6. Solve the free-interface eigenproblem $\mathbf{K}_{CB}\boldsymbol{\xi} = \omega^2\mathbf{M}_{CB}\boldsymbol{\xi}$.
7. Identify the rigid body modes and select the required elastic modes.
8. Mass-normalise the retained eigenvectors and assemble $\mathbf{T}_f = \mathbf{T}_{CB}\boldsymbol{\Xi}$.
9. Calculate the final modal matrices and the load and output projection operators.

The following numerical considerations are important:

- The matrix inverse must not be formed explicitly. A sparse factorisation and multiple right-hand-side solve should be used for the constraint modes.
- The fixed-interface stiffness matrix $\mathbf{K}_{ii}$ must be nonsingular. A singular matrix indicates an internal rigid body motion or a mechanism that is not restrained by the selected boundary DOFs.
- Symmetric generalised eigensolvers preserve the mass orthogonality of distinct modes. An ordinary Euclidean Gram-Schmidt process is therefore not normally applied to the eigenvectors.
- Repeated or tightly clustered eigenvalues can require re-orthogonalisation of the corresponding mode block. This must use the mass inner product $\boldsymbol{\xi}_r^T \mathbf{M}_{CB} \boldsymbol{\xi}_s$.
- The Craig-Bampton matrices should be symmetrised only to remove small round-off asymmetry, not to hide an incorrectly assembled transformation.
- Mode selection should be based on the required frequency range and on the deformation and loading patterns, rather than on the mode count alone.

For a retained mode $\boldsymbol{\xi}_j$, the mass normalisation is performed as:

$$\boldsymbol{\xi}_{j, mass} = \frac{\boldsymbol{\xi}_j}{\sqrt{\boldsymbol{\xi}_j^T \mathbf{M}_{CB} \boldsymbol{\xi}_j}} \quad (9.40)$$

The orthogonality and the eigenpair residual can be checked using:

$$\begin{aligned} \mathbf{E}_M &= \boldsymbol{\Xi}^T \mathbf{M}_{CB} \boldsymbol{\Xi} - \mathbf{I} \\ \mathbf{E}_K &= \boldsymbol{\Xi}^T \mathbf{K}_{CB} \boldsymbol{\Xi} - \boldsymbol{\Omega}_f^2 \\ \mathbf{r}_j &= \mathbf{K}_{CB} \boldsymbol{\xi}_j - \omega_j^2 \mathbf{M}_{CB} \boldsymbol{\xi}_j \end{aligned} \quad (9.41)$$

A valid reduced base should have small off-diagonal entries in $\mathbf{E}_M$ and $\mathbf{E}_K$, and a small residual norm $\|\mathbf{r}_j\|$ relative to the norms of the stiffness and inertia terms.

Note:

The free-interface modes calculated in the Craig-Bampton space are not the original full FE free-free modes. They are approximations restricted to the subspace spanned by $\mathbf{T}_{CB}$. After mapping with $\mathbf{T}_f = \mathbf{T}_{CB}\mathbf{\Xi}$, they have the same physical meaning and converge towards the full FE free-free modes as the Craig-Bampton base is enriched.

Scope of the classical formulation:

The above derivation assumes a linear time-invariant system with symmetric mass and stiffness matrices. For a component linearised about a prestressed operating point, the tangent stiffness matrix can be used in the same reduction procedure.

When a velocity-dependent gyroscopic matrix $\mathbf{G}$ is retained, it must be transformed together with the other system matrices:

$$\mathbf{G}_{CB} = \mathbf{T}_{CB}^T \mathbf{G} \mathbf{T}_{CB} \quad (9.42)$$

The free-interface problem then contains the term $i\omega\mathbf{G}_{CB}$ and is no longer the symmetric real-valued eigenproblem described above. Its modes can be complex and the simple mass-orthogonality relations do not generally apply.

Chapter 10

Element Atlas

10.1 Matrices

The inversion formula for 3x3 matrix:

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$
$$\mathbf{A}^{-1} = \frac{1}{|\mathbf{A}|} \begin{bmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \\ A_{31} & A_{32} & A_{33} \end{bmatrix} \quad (10.1)$$

where:

$$\begin{aligned}
A_{11} &= a_{22}a_{33} - a_{23}a_{32} \\
A_{22} &= a_{33}a_{11} - a_{31}a_{13} \\
A_{33} &= a_{11}a_{22} - a_{12}a_{21} \\
A_{12} &= a_{23}a_{31} - a_{21}a_{33} \\
A_{23} &= a_{31}a_{12} - a_{32}a_{11} \\
A_{31} &= a_{12}a_{23} - a_{13}a_{22} \\
A_{21} &= a_{32}a_{13} - a_{12}a_{33} \\
A_{32} &= a_{13}a_{21} - a_{23}a_{11} \\
A_{13} &= a_{21}a_{22} - a_{31}a_{22} \\
|A_{13}| &= a_{11}A_{11} + a_{12}A_{21} + a_{13}A_{31}
\end{aligned} \tag{10.2}$$

10.2 Numerical Integration

Note:

Quadrature is another term for **Numerical Integration**.

10.2.1 Newton-Cotes quadrature

In the most obvious procedure, points at which the function is to be formed are determined *a priori* - usually at equal intervals - and a polynomial passed through the values of the function at these points and exactly integrated.

As n values define a polynomial of degree $n - 1$, the errors will be of the order $O(h^n)$ where h is the element size. This leads to the well-known **Newton-Cotes** 'quadrature formulae'. The integrals can be written as:

$$I = \int_{-1}^1 f(\xi) d\xi = \sum_1^n H_i f(\xi_i) \tag{10.3}$$

for the range of integration between -1 and $+1$. For example, if $n = 2$, we have the well-known trapezoidal rule:

$$I = f(-1) + f(1) \quad (10.4)$$

for $n = 3$, the well-known **Simpson** one-third rule:

$$I = \frac{1}{3} [f(-1) + 4f(0) + f(1)] \quad (10.5)$$

and for $n = 4$:

$$I = \frac{1}{4} \left[f(-1) + 3f\left(-\frac{1}{3}\right) + 3f\left(\frac{1}{3}\right) + f(1) \right] \quad (10.6)$$

10.2.2 Gauss quadrature

If in place of specifying the position of sampling points *a priori* we allow these to be located at points to be determined so as to aim for best accuracy, then for a given number of sampling points increased accuracy can be obtained. Indeed, if we again consider:

$$I = \int_{-1}^1 f(\xi) d\xi = \sum_1^n H_i f(\xi_i) \quad (10.7)$$

and again assume a polynomial expression, it is easy to see that for n sampling points we have $2n$ unknowns (H_i and ξ_i) and hence a polynomial of degree $2n-1$ could be constructed and exactly integrated. The error is thus of order $O(h^{2n})$.

The simultaneous equations involved are difficult to solve, but some mathematical manipulation will show that the solution can be obtained explicitly in terms of **Legendre polynomials**. Thus this particular process is frequently known as **Gauss-Legendre** quadrature.

For purposes of finite elements analysis complex calculations are involved in determining the values of f , the function to be integrated. Thus the Gauss-type processes, requiring the least number of such evaluations, are ideally suited and are mostly used exclusively.

Other expressions for integration of functions of the type

$$I = \int_{-1}^1 w(\xi) f(\xi) d\xi = \sum_1^n H_i f(\xi_i) \quad (10.8)$$

can be derived for prescribed forms of $w(\xi)$, again integrating up to a certain order of accuracy a polynomial expansion of $f(\xi)$.

10.2.3 Linear and Quadrilateral Elements

For 1D:

$$I = \int_{-1}^1 f(\xi) d\xi = \sum_1^n H_i f(\xi_i) \quad (10.9)$$

For 2D:

The most obvious way of obtaining the integral:

$$I = \int_{-1}^1 \int_{-1}^1 f(\xi, \eta) d\xi d\eta \quad (10.10)$$

Is to first evaluate the inner integral keeping η constant, i.e.:

$$\int_{-1}^1 f(\xi, \eta) d\xi = \sum_{j=1}^n H_j f(\xi_j, \eta) = \psi(\eta) \quad (10.11)$$

Evaluating the outer integral in a similar manner, we have:

$$\begin{aligned}
 I &= \int_{-1}^1 \psi(\eta) d\eta = \sum_{i=1}^n H_i \psi(\eta_i) \\
 &= \sum_{i=1}^n H_i \sum_{j=1}^n H_j f(\xi_j, \eta_i) \\
 &= \sum_{i=1}^n \sum_{j=1}^n H_i H_j f(\xi_j, \eta_i)
 \end{aligned} \tag{10.12}$$

For 3D:

For a 3D element we have similarly:

$$\begin{aligned}
 I &= \int_{-1}^1 \int_{-1}^1 \int_{-1}^1 f(\xi, \eta, \mu) d\xi d\eta d\mu \\
 &= \sum_{i=1}^n \sum_{j=1}^n \sum_{k=1}^n H_i H_j H_k f(\xi_i, \eta_j, \mu_k)
 \end{aligned} \tag{10.13}$$

In the above, the number of integrating points in each direction was assumed to be the same. Clearly this is not necessary and on occasion it may be an advantage to use different numbers in each direction of integration.

To get the **Gauss-Legendre** points and weights in python use:

```

1 import numpy as np
2
3 # number of integration points
4 N = 3
5
6 points, weights = np.polynomial.legendre.leggauss(N)

```

integration point	polynomial order	weight
0	n = 1	2.0
$-1/\sqrt{3}$	n = 2	1.0
$1/\sqrt{3}$		1.0
$-\sqrt{0.6}$	n = 3	5/9
0.0		8/9
$\sqrt{0.6}$		5/9
-0.861 136 311 594 953	n = 4	0.347 854 845 137 454
-0.339 981 043 584 856		0.652 145 154 862 546
0.339 981 043 584 856		0.652 145 154 862 546
0.861 136 311 594 953		0.347 854 845 137 454
-0.906 179 845 938 664	n = 5	0.236 926 885 056 189
-0.538 469 310 105 683		0.478 628 670 499 366
0.000 000 000 000 000		0.568 888 888 888 889
0.538 469 310 105 683		0.478 628 670 499 366
0.906 179 845 938 664		0.236 926 885 056 189
-0.932 469 514 203 152	n = 6	0.171 324 492 379 170
-0.661 209 386 466 265		0.360 761 573 048 139
-0.238 619 186 083 197		0.467 913 934 572 691
0.238 619 186 083 197		0.467 913 934 572 691
0.661 209 386 466 265		0.360 761 573 048 139
0.932 469 514 203 152		0.171 324 492 379 170
-0.949 107 912 342 759	n = 7	0.129 484 966 168 870
-0.741 531 185 599 394		0.279 705 391 489 277
-0.405 845 151 377 397		0.381 830 050 505 119
0.000 000 000 000 000		0.471 959 183 673 469
0.405 845 151 377 397		0.381 830 050 505 119
0.741 531 185 599 394		0.279 705 391 489 277
0.949 107 912 342 759		0.129 484 966 168 870

Table 10.1: Gauss-Legendre Integration Points and Weights

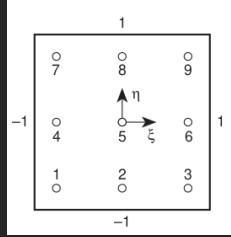


Figure 10.1: Integration points for $n = 3$ in a square region. Exact for a polynomial of fifth order in each direction

10.2.4 Triangular and Tetrahedral elements:

2D

For a triangle, in terms of the **area coordinates** the integrals are of the form:

$$I = \int_0^1 \int_0^{1-\xi} f(\xi, \eta, \mu) d\eta d\xi, \quad \mu = 1 - \xi - \eta \quad (10.14)$$

Once again we could use n Gauss points and arrive at a summation expression of the type used in the previous section. However, the limits of integration now involve the variable itself and it is convenient to use alternative sampling points for the second integration by use of a special Gauss expression for integrals of the type given by equation (10.8) in which w is a linear function. These have been devised by *Radau* and used successfully in finite element context. It is, however, much more desirable (and aesthetically pleasing) to use special formulae in which no bias is given to any of the natural coordinates (ξ, η, μ) . Such formulae were first derived by *Hammer et al.* and *Felippa* and a series of necessary sampling points and weights.

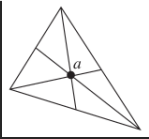
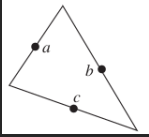
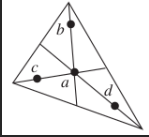
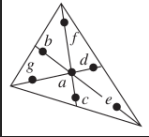
Order	Figure	Error	Points	Triangular Coordinates	Weights
Linear		$R = O(h^2)$	a	$\frac{1}{3}, \frac{1}{3}, \frac{1}{3}$	1
Quadratic		$R = O(h^3)$	a b c	$\frac{1}{2}, \frac{1}{2}, 0$ $0, \frac{1}{2}, \frac{1}{2}$ $\frac{1}{2}, 0, \frac{1}{2}$	$\frac{1}{3}$ $\frac{1}{3}$ $\frac{1}{3}$
Cubic		$R = O(h^4)$	a b c d	$\frac{1}{3}, \frac{1}{3}, \frac{1}{3}$ 0.6, 0.2, 0.2 0.2, 0.6, 0.2 0.2, 0.2, 0.6	$-\frac{27}{48}$ $\frac{25}{48}$ $\frac{25}{48}$ $\frac{25}{48}$
Quintic		$R = O(h^6)$	a b c d e f g with: $\alpha_1 = 0.059\ 715\ 871\ 7$ $\beta_1 = 0.470\ 142\ 064\ 1$ $\alpha_2 = 0.797\ 426\ 985\ 3$ $\beta_2 = 0.101\ 286\ 507\ 3$	$\frac{1}{3}, \frac{1}{3}, \frac{1}{3}$ $\alpha_1, \beta_1, \beta_1$ $\beta_1, \alpha_1, \beta_1$ $\beta_1, \beta_1, \alpha_1$ $\alpha_2, \beta_2, \beta_2$ $\beta_2, \alpha_2, \beta_2$ $\beta_2, \beta_2, \alpha_2$	0.2250000000 0.1323941527 0.1323941527 0.1323941527 0.1259391805 0.1259391805 0.1259391805

Table 10.2: Triangular Element integration points and weights

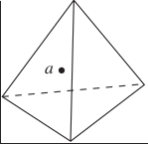
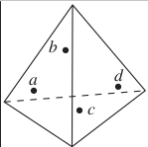
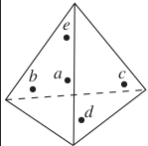
Order	Figure	Error	Points	Tetrahedral Coordinates	Weights
Linear		$R = O(h^2)$	a	$\frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4}$	1
Quadratic		$R = O(h^6)$	a b c d	$\alpha, \beta, \beta, \beta$ $\beta, \alpha, \beta, \beta$ $\beta, \beta, \alpha, \beta$ $\beta, \beta, \beta, \beta$	$\frac{1}{4}$ $\frac{1}{4}$ $\frac{1}{4}$ $\frac{1}{4}$
			with: $\alpha = 0.585 \ 410 \ 20$ $\beta = 0.138 \ 196 \ 60$		
Cubic		$R = O(h^4)$	a b c d e	$\frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4}$ $\frac{1}{2}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}$ $\frac{1}{6}, \frac{1}{2}, \frac{1}{6}, \frac{1}{6}$ $\frac{1}{6}, \frac{1}{6}, \frac{1}{2}, \frac{1}{6}$ $\frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{2}$	$-\frac{4}{5}$ $\frac{9}{20}$ $\frac{9}{20}$ $\frac{9}{20}$ $\frac{9}{20}$

Table 10.3: Tetrahedral Element integration points and weights

10.3 General Element Matrices Procedure

The procedure to generate any element stiffness, mass and load matrices can be generalised to the following steps:

1. Create Element domain in natural coordinates:
 - -1 to 1 for quadrilateral elements
 - 0 to 1 for triangular and linear elements
2. select the number of Gauss points based on the order of the element
 - 1 or 2 for linear elements
 - 3 for quadratic elements

Example:

```
1 import numpy as np
2 gp, gw = np.polynomial.legendre.leggauss(3)
```

3. Generate the shape functions $\mathbf{N}_n^e$ in natural coordinates based on the number of Gauss points, dimension of the element and its domain.

The $\mathbf{N}_n^e$ matrix has the dimensions of:

- number of integration points = number of rows
- number of shape functions = number of columns

Therefore:

$$\mathbf{N}_n^e = \begin{bmatrix} N_{1,1}^e & \cdots & N_{n,1}^e \\ \vdots & \vdots & \vdots \\ N_{1,m}^e & \cdots & N_{n,m}^e \end{bmatrix} \quad (10.15)$$

where $1 \dots m$ are the integration points and $1 \dots n$ are the shape functions.

4. Transform the shape functions from natural coordinates to global coordinates

$$\mathbf{N}_g^e = \mathbf{N}_n^e \mathbf{x}^e$$

$$\mathbf{N}_g^e = \begin{bmatrix} N_{1,1}^e(\xi) & \dots & N_{n,1}^e(\xi) \\ \vdots & \vdots & \vdots \\ N_{1,m}^e(\xi) & \dots & N_{n,m}^e(\xi) \end{bmatrix} \begin{bmatrix} x_1 & y_1 & z_1 \\ \vdots & \vdots & \vdots \\ x_n & y_n & z_n \end{bmatrix} \quad (10.16)$$

The $\mathbf{x}^e$ matrix is a matrix of global coordinates of the element nodes.

The resulting matrix has the dimension of:

- number of integration points = number of rows
- number of coordinates = number of columns (3D = 3, 2D = 2)

5. Generate the shape functions derivatives $\mathbf{B}_n^e$ in natural coordinates

The shape functions derivatives matrix has 3 dimensions being *number of integration points* $\times$ *number of natural coordinates* $\times$ *number of shape functions*

6. Create the Matrix of Jacobians from the shape functions derivatives:

$$\mathbf{J} = \mathbf{B}_n^e \mathbf{x}^e \quad (10.17)$$

The Matrix of Jacobians has the dimension of *number of integration points* $\times$ *number of global coordinates* $\times$ *number of global coordinates*, where number of global coordinates means 2 for 2D and 3 for 3D problem.

7. Get the Matrix of Jacobian determinants and an inverse Jacobian:

$$\mathbf{J}^{-1} \mathbf{J}_d = \det \mathbf{J} \quad (10.18)$$

Example:

```
1 d_jacobi = np.linalg.det(jacobi)
2 i_jacobi = np.linalg.inv(jacobi)
```

The Jacobian determinant is computed for each integration point and has a dimension of *number of integration points* $\times 1$

The inverse Jacobian has the same dimension as the Jacobian (**must have**)

8. Transform the shape function derviatives from natural coordinates to global coordinates

$$\mathbf{B}_g^e = \mathbf{J}^{-1} \mathbf{B}_n^e \quad (10.19)$$

9. Create the Material Stiffness matrix $\mathbf{C}$

10. Finally for each integration point:

3D:

$$\begin{aligned} \mathbf{B}_{i,x}^e &= \mathbf{B}_g(i, 0) \\ \mathbf{B}_{i,y}^e &= \mathbf{B}_g(i, 1) \\ \mathbf{B}_{i,z}^e &= \mathbf{B}_g(i, 2) \end{aligned} \quad (10.20)$$

where $\mathbf{B}_{i,x}^e$, $\mathbf{B}_{i,y}^e$ and $\mathbf{B}_{i,z}^e$ are vectors of legth = *number of shape functions*

Then matrix $\mathbf{B}$ is:

$$\mathbf{B}_i = \begin{bmatrix} \mathbf{B}_{i,x}^e & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{B}_{i,y}^e & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{B}_{i,z}^e \\ \mathbf{B}_{i,y}^e & \mathbf{B}_{i,x}^e & \mathbf{0} \\ \mathbf{0} & \mathbf{B}_{i,z}^e & \mathbf{B}_{i,y}^e \\ \mathbf{B}_{i,z}^e & \mathbf{0} & \mathbf{B}_{i,x}^e \end{bmatrix} \quad (10.21)$$

and matrix $\mathbf{N}$ is:

$$\mathbf{N}_i = \begin{bmatrix} \mathbf{N}_{n,i}^e & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{N}_{n,i}^e & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{N}_{n,i}^e \end{bmatrix} \quad (10.22)$$

where $\mathbf{0}$ is a zero vector of length = *number of shape functions* and $\mathbf{N}_{n,i}^e$ is a vector of shape functions in natural coordinates pertaining to the i -th integration point.

Then:

$$(a) \quad \mathbf{K}_i^e = \mathbf{B}_i^T \mathbf{C} \mathbf{B}_i \det \mathbf{J}_i w_i \quad (10.23)$$

$$(b) \quad \mathbf{M}_i^e = \rho \mathbf{N}_i^T \mathbf{N}_i \det \mathbf{J}_i w_i \quad (10.24)$$

$$(c) \quad \mathbf{F}_i^e = \mathbf{N}_i^T \mathbf{F} \det \mathbf{J}_i w_i \quad (10.25)$$

$$\text{where } \mathbf{F} = \begin{bmatrix} f_x \\ f_y \\ f_z \end{bmatrix}$$

where w_i is the multiple of gaussian weights for the respective integration point (for 3D case brick where the natural coordinates are ξ, η, μ the $w_i = w_{\xi_i} w_{\eta_i} w_{\mu_i}$

11. Finally:

$$(a) \quad \mathbf{K}^e = \sum_{i=1}^n \mathbf{K}_i^e \quad (10.26)$$

$$(b) \quad \mathbf{M}^e = \sum_{i=1}^n \mathbf{M}_i^e \quad (10.27)$$

$$(c) \quad \mathbf{F}^e = \sum_{i=1}^n \mathbf{F}_i^e \quad (10.28)$$

10.4 ROD

10.4.1 Element Formulation

The **rod** element is a 1-D linear element. It has 2 nodes, and 3 dofs for each node u_x, u_y, u_z (or u, v, w). Rotation **DOFs** are not constrained.

Natural coordinates

The natural coordinate ξ of the **ROD** element goes from 0 to 1. The 2-noded rod element is defined by:

$$\begin{bmatrix} 1 \\ x \\ y \\ z \\ u_x \\ u_y \\ u_z \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ x_1 & x_2 \\ y_1 & y_2 \\ z_1 & z_2 \\ u_{x1} & u_{x2} \\ u_{y1} & u_{y2} \\ u_{z1} & u_{z2} \end{bmatrix} \begin{bmatrix} N_1^e \\ N_2^e \end{bmatrix} \quad (10.29)$$

Shape Functions

The **ROD** natural coordinates are:

node	ξ
1	0
2	1

Table 10.4: ROD corners natural coordinates

The shape functions are:

$$\begin{aligned} N_1^e &= 1 - \xi \\ N_2^e &= \xi \end{aligned} \quad (10.30)$$

Partial Derivatives

The shape function derivatives with respect to the natural coordinate:

$$\begin{aligned}\frac{\partial N_1^e}{\partial \xi} &= -1 \\ \frac{\partial N_2^e}{\partial \xi} &= 1\end{aligned}\tag{10.31}$$

The Jacobian

The derivatives of the shape functions are given by the usual chain rule formula:

$$\begin{aligned}\frac{\partial N_i^e}{\partial x} &= \frac{\partial N_i^e}{\partial \xi} \frac{\partial \xi}{\partial x} \\ \frac{\partial N_i^e}{\partial y} &= \frac{\partial N_i^e}{\partial \xi} \frac{\partial \xi}{\partial y} \\ \frac{\partial N_i^e}{\partial z} &= \frac{\partial N_i^e}{\partial \xi} \frac{\partial \xi}{\partial z}\end{aligned}\tag{10.32}$$

In matrix form:

$$\begin{bmatrix} \frac{\partial N_i^e}{\partial x} \\ \frac{\partial N_i^e}{\partial y} \\ \frac{\partial N_i^e}{\partial z} \end{bmatrix} = \begin{bmatrix} \frac{\partial \xi}{\partial x} \\ \frac{\partial \xi}{\partial y} \\ \frac{\partial \xi}{\partial z} \end{bmatrix} \left[\frac{\partial N_i^e}{\partial \xi} \right]\tag{10.33}$$

The 3×1 matrix above is $\mathbf{J}^{-1}$, the inverse of:

$$\mathbf{J} = \frac{\partial(x, y, z)}{\partial(\xi)} = \begin{bmatrix} \frac{\partial x}{\partial \xi} & \frac{\partial y}{\partial \xi} & \frac{\partial z}{\partial \xi} \end{bmatrix}\tag{10.34}$$

Given the element coordinates the Jacobian can be computed:

$$\mathbf{J}_i = \begin{bmatrix} \frac{\partial N_i^e}{\partial \xi} & \frac{\partial N_i^e}{\partial \xi} & \frac{\partial N_i^e}{\partial \xi} \end{bmatrix} \begin{bmatrix} x_i \\ y_i \\ z_i \end{bmatrix}\tag{10.35}$$

end then numerically inverted for $\mathbf{J}^{-1}$.

If the $\mathbf{J}$ is not a square matrix, then **Moore-Penrose pseudoinverse** applies:

$$\mathbf{J}^{-1} = \mathbf{J}^+ = (\mathbf{J}^T \mathbf{J})^{-1} \mathbf{J}^T \quad (10.36)$$

in python:

```

1      import numpy as np
2
3      J_i = np.linalg.pinv(J)
```

Assembling the **Jacobian** together gives essentially that for a 1D case

$$\mathbf{J} = l \quad (10.37)$$

and therefore:

$$\mathbf{J}^{-1} = \frac{1}{l} \quad (10.38)$$

$$\det J = l \quad (10.39)$$

afterwards for each integration point:

$$\mathbf{N}_g^i = \mathbf{N}^i \mathbf{u} \quad (10.40)$$

$$\mathbf{B}_g^i = \mathbf{J}^{i-1} \frac{\partial N^i}{\partial \xi} \quad (10.41)$$

then:

$$\begin{aligned}
\mathbf{K}^e &\stackrel{+}{=} \mathbf{B}_g^{iT} \mathbf{D} \mathbf{B}_g^i \det \mathbf{J}^i w^i \\
\mathbf{M}^e &\stackrel{+}{=} \rho \mathbf{N}^{iT} \mathbf{N}^i \det \mathbf{J}^i w^i \\
\mathbf{F}^e &\stackrel{+}{=} \mathbf{N}^{iT} \mathbf{F} \det \mathbf{J}^i w^i
\end{aligned} \quad (10.42)$$

10.4.2 Example 1D

If assembling the element in local CSYS, then $u_y = 0$ and $u_z = 0$. Simplifying the equations gives that:

$$\mathbf{N} = [N_1^e \quad N_2^e] = [1 - \xi \quad \xi] \quad (10.43)$$

and

$$u^e(\xi) = \mathbf{N} \begin{bmatrix} u_1^e \\ u_2^e \end{bmatrix} \quad (10.44)$$

shape function derivatives:

$$\mathbf{B} = \frac{\partial \mathbf{N}}{\partial \xi} = [-1 \quad 1] \quad (10.45)$$

the strain-displacement relation is given by:

$$\boldsymbol{\epsilon} = \mathbf{B}\mathbf{u} \quad (10.46)$$

Integrating the element at midpoint $\xi = 0.5$ with a weight of $w = 1.0$ (for interval $\langle 0, 1 \rangle$, for an interval $\langle -1, 1 \rangle$ the weight is $w = 2.0$)

$$\mathbf{N} = [0.5 \quad 0.5] \quad (10.47)$$

Shape Functions in global coordinates:

$$\mathbf{N} = 0.5u_1 + 0.5u_2 = 0.5(u_1 + u_2) \quad (10.48)$$

Shape Function Derivatives in Natural Coordinates:

$$\mathbf{B} = [-1 \quad 1] \quad (10.49)$$

Jacobian:

$$\mathbf{J} = \mathbf{B}\mathbf{u} = \begin{bmatrix} -1 & 1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} = u_2 - u_1 = l \quad (10.50)$$

Jacobian Determinant:

$$\det \mathbf{J} = l \quad (10.51)$$

Jacobian Inverse:

$$\mathbf{J}^{-1} = \frac{1}{l} \quad (10.52)$$

Shape function derivatives in global coordinates:

$$\mathbf{B}_g = \mathbf{J}^{-1}\mathbf{B} = \frac{1}{l} \begin{bmatrix} -1 & 1 \end{bmatrix} \quad (10.53)$$

Material Stiffness Matrix

$$\mathbf{D} = EA \quad (10.54)$$

Then for each integration point (here 1):

$$\begin{aligned} \mathbf{K}^e &\pm \mathbf{B}_g^T \mathbf{D} \mathbf{B}_g \det \mathbf{J} w \\ \mathbf{K}^e &\pm \frac{1}{l} \begin{bmatrix} -1 \\ 1 \end{bmatrix} EA \frac{1}{l} \begin{bmatrix} -1 & 1 \end{bmatrix} l 1 \\ \mathbf{K}^e &\pm \frac{EA}{l} \begin{bmatrix} -1 \\ 1 \end{bmatrix} \begin{bmatrix} -1 & 1 \end{bmatrix} \\ \mathbf{K}^e &\pm \frac{EA}{l} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \end{aligned} \quad (10.55)$$

10.4.3 Example 2D

If assembling the element in 2D, then $u_z = 0$. Simplifying the equations gives that:

$$\mathbf{N} = \begin{bmatrix} N_1^e & 0 & N_2^e & 0 \\ 0 & N_1^e & 0 & N_2^e \end{bmatrix} = \begin{bmatrix} 1 - \xi & 0 & \xi & 0 \\ 0 & 1 - \xi & 0 & \xi \end{bmatrix} \quad (10.56)$$

and

$$\begin{bmatrix} u_x^e(\xi) \\ u_y^e(\xi) \end{bmatrix} = \mathbf{N} \begin{bmatrix} u_{x1}^e \\ u_{y1}^e \\ u_{x2}^e \\ u_{y2}^e \end{bmatrix} \quad (10.57)$$

or if:

$$\mathbf{N} = [N_1^e \quad N_2^e] = [1 - \xi \quad \xi] \quad (10.58)$$

then:

$$\begin{bmatrix} u_x^e(\xi) \\ u_y^e(\xi) \end{bmatrix} = \mathbf{N} \begin{bmatrix} u_{x1}^e & u_{y1}^e \\ u_{x2}^e & u_{y2}^e \end{bmatrix} \quad (10.59)$$

shape function derivatives:

$$\mathbf{B} = \frac{\partial \mathbf{N}}{\partial \xi} = \begin{bmatrix} -1 & 0 & 1 & 0 \\ 0 & -1 & 0 & 1 \end{bmatrix} \quad (10.60)$$

the strain-displacement relation is given by:

$$\boldsymbol{\epsilon} = \mathbf{B} \mathbf{u} \quad (10.61)$$

Integrating the element at midpoint $\xi = 0.5$ with a weight of $w = 1.0$ (for interval $\langle 0, 1 \rangle$, for an interval $\langle -1, 1 \rangle$ the weight $w = 2.0$) then numerically:

$$\mathbf{N} = [0.5 \quad 0.5] \quad (10.62)$$

Shape Functions in global coordinates:

$$\mathbf{N} = \begin{bmatrix} 1-\xi & 0 & \xi & 0 \\ 0 & 1-\xi & 0 & \xi \end{bmatrix}_{\xi=0.5} \begin{bmatrix} u_{x1}^e \\ u_{y1}^e \\ u_{x2}^e \\ u_{y2}^e \end{bmatrix} = \begin{bmatrix} 0.5u_{x1} + 0.5u_{x2} \\ 0.5u_{y1} + 0.5u_{y2} \end{bmatrix} \tag{10.63}$$

Shape Function Derivatives in Natural Coordinates (there is a row for each integration point if the coordinates are specified as matrix, not a column vector. otherwise for each integration point there is a separate **B** matrix):

$$\frac{\partial \mathbf{N}}{\partial \xi} = \mathbf{B} = \begin{bmatrix} -1 & 0 & 1 & 0 \\ 0 & -1 & 0 & 1 \end{bmatrix} \tag{10.64}$$

Jacobian:

$$\mathbf{J} = \mathbf{B}\mathbf{u} = \begin{bmatrix} -1 & 0 & 1 & 0 \\ 0 & -1 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_{x1}^e \\ u_{y1}^e \\ u_{x2}^e \\ u_{y2}^e \end{bmatrix} = \begin{bmatrix} u_{x2}^e - u_{x1}^e \\ u_{y2}^e - u_{y1}^e \end{bmatrix} = \begin{bmatrix} l_x \\ l_y \end{bmatrix} \tag{10.65}$$

??

Jacobian Determinant (when the **Jacobian matrix** is a vector, its determinant is simply its **length**):

ll

$$\det \mathbf{J} = ||\mathbf{J}|| = \sqrt{J_1^2 + J_2^2} = \sqrt{l_x^2 + l_y^2} = l \tag{10.66}$$

??

$$\mathbf{K}^e \stackrel{+}{=} \mathbf{B}_g^T \mathbf{D} \mathbf{B}_g \det \mathbf{J} w \quad (10.71)$$

$$\begin{aligned} \mathbf{K}^e &\stackrel{+}{=} \frac{1}{l} \begin{bmatrix} -c \\ -s \\ c \\ s \end{bmatrix} EA \frac{1}{l} \begin{bmatrix} -c & -s & c & s \end{bmatrix} lw \\ &\stackrel{+}{=} \frac{EA}{l} \begin{bmatrix} c^2 & cs & -c^2 & -cs \\ cs & s^2 & -cs & -s^2 \\ -c^2 & -cs & c^2 & cs \\ -cs & -s^2 & cs & s^2 \end{bmatrix} w \end{aligned} \quad (10.72)$$

which is the same result as when the element stiffness matrix is derived in local coordinate system for a situation where ξ axis coincides with the element x axis and the u_y component is 0. Afterwards, the element **stiffness** and **mass** matrices as well as **forces** vector are transformed to the global CSYS using a transformation matrix $\mathbf{T}$:

$$\mathbf{K}_l^e = \frac{EA}{l} \begin{bmatrix} 1 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 \\ -1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \quad (10.73)$$

$$\mathbf{T} = \begin{bmatrix} c & s & 0 & 0 \\ -s & c & 0 & 0 \\ 0 & 0 & c & s \\ 0 & 0 & -s & c \end{bmatrix} \quad (10.74)$$

then the global element stiffness matrix is defined:

$$\begin{aligned}
\mathbf{K}_g^e &= \mathbf{T}^T \mathbf{K}_l^e \mathbf{T} \\
&= \frac{EA}{l} \begin{bmatrix} c & -s & 0 & 0 \\ s & c & 0 & 0 \\ 0 & 0 & c & -s \\ 0 & 0 & s & c \end{bmatrix} \begin{bmatrix} 1 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 \\ -1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} c & s & 0 & 0 \\ -s & c & 0 & 0 \\ 0 & 0 & c & s \\ 0 & 0 & -s & c \end{bmatrix} \\
&= \frac{EA}{l} \begin{bmatrix} c & -s & 0 & 0 \\ s & c & 0 & 0 \\ 0 & 0 & c & -s \\ 0 & 0 & s & c \end{bmatrix} \begin{bmatrix} c & s & -c & -s \\ 0 & 0 & 0 & 0 \\ -c & -s & c & s \\ 0 & 0 & 0 & 0 \end{bmatrix} \\
&= \frac{EA}{l} \begin{bmatrix} c^2 & cs & -c^2 & -cs \\ cs & s^2 & -cs & -s^2 \\ -c^2 & -cs & c^2 & cs \\ -cs & -s^2 & cs & s^2 \end{bmatrix}
\end{aligned} \tag{10.75}$$

10.4.4 Example 3D

If assembling the element in 3D, then $u_x \neq 0$, $u_y \neq 0$ and $u_z \neq 0$. The full equations give that:

$$\mathbf{N} = \begin{bmatrix} N_1^e & 0 & 0 & N_2^e & 0 & 0 \\ 0 & N_1^e & 0 & 0 & N_2^e & 0 \\ 0 & 0 & N_1^e & 0 & 0 & N_2^e \end{bmatrix} = \begin{bmatrix} 1-\xi & 0 & 0 & \xi & 0 & 0 \\ 0 & 1-\xi & 0 & 0 & \xi & 0 \\ 0 & 0 & 1-\xi & 0 & 0 & \xi \end{bmatrix} \tag{10.76}$$

and

$$\begin{bmatrix} u_x^e(\xi) \\ u_y^e(\xi) \\ u_z^e(\xi) \end{bmatrix} = \mathbf{N}(\xi) \begin{bmatrix} u_{x1}^e \\ u_{y1}^e \\ u_{z1}^e \\ u_{x2}^e \\ u_{y2}^e \\ u_{z2}^e \end{bmatrix} \tag{10.77}$$

or if:

$$\mathbf{N} = [N_1^e \quad N_2^e] = [1-\xi \quad \xi] \tag{10.78}$$

then:

$$\begin{bmatrix} u_x^e(\xi) \\ u_y^e(\xi) \\ u_z^e(\xi) \end{bmatrix} = \mathbf{N}(\xi) \begin{bmatrix} u_{x1}^e & u_{y1}^e & u_{z1}^e \\ u_{x2}^e & u_{y2}^e & u_{z2}^e \end{bmatrix} \quad (10.79)$$

shape function derivatives:

$$\mathbf{B} = \frac{\partial \mathbf{N}}{\partial \xi} = \begin{bmatrix} -1 & 0 & 0 & 1 & 0 & 0 \\ 0 & -1 & 0 & 0 & 1 & 0 \\ 0 & 0 & -1 & 0 & 0 & 1 \end{bmatrix} \quad (10.80)$$

the strain-displacement relation is given by:

$$\boldsymbol{\epsilon} = \mathbf{B}\mathbf{u} \quad (10.81)$$

Integrating the element at midpoint $\xi = 0.5$ with a weight of $w = 1.0$ (for interval $\langle 0, 1 \rangle$, for an interval $\langle -1, 1 \rangle$ the weight $w = 2.0$) then numerically:

$$\mathbf{N} = [0.5 \quad 0.5] \quad (10.82)$$

Shape Functions in global coordinates:

$$\mathbf{N} = \begin{bmatrix} 1 - \xi & 0 & 0 & \xi & 0 & 0 \\ 0 & 1 - \xi & 0 & 0 & \xi & 0 \\ 0 & 0 & 1 - \xi & 0 & 0 & \xi \end{bmatrix}_{\xi=0.5} \begin{bmatrix} u_{x1}^e \\ u_{y1}^e \\ u_{z1}^e \\ u_{x2}^e \\ u_{y2}^e \\ u_{z2}^e \end{bmatrix} = \begin{bmatrix} 0.5u_{x1} + 0.5u_{x2} \\ 0.5u_{y1} + 0.5u_{y2} \\ 0.5u_{z1} + 0.5u_{z2} \end{bmatrix} \quad (10.83)$$

Shape Function Derivatives in Natural Coordinates (there is a row for each integration point if the coordinates are specified as matrix, not a column vector. otherwise for each integration point there is a separate $\mathbf{B}$ matrix):

$$\frac{\partial \mathbf{N}}{\partial \xi} = \mathbf{B} = \begin{bmatrix} -1 & 0 & 0 & 1 & 0 & 0 \\ 0 & -1 & 0 & 0 & 1 & 0 \\ 0 & 0 & -1 & 0 & 0 & 1 \end{bmatrix} \quad (10.84)$$

Jacobian:

$$\mathbf{J} = \mathbf{B}\mathbf{u} = \begin{bmatrix} -1 & 0 & 0 & 1 & 0 & 0 \\ 0 & -1 & 0 & 0 & 1 & 0 \\ 0 & 0 & -1 & 0 & 0 & 1 \end{bmatrix}_{\xi=0.5} \begin{bmatrix} u_{x1}^e \\ u_{y1}^e \\ u_{z1}^e \\ u_{x2}^e \\ u_{y2}^e \\ u_{z2}^e \end{bmatrix} = \begin{bmatrix} u_{x2}^e - u_{x1}^e \\ u_{y2}^e - u_{y1}^e \\ u_{z2}^e - u_{z1}^e \end{bmatrix} = \begin{bmatrix} l_x \\ l_y \\ l_z \end{bmatrix}$$

(10.85)

??

Jacobian Determinant (when the **Jacobian matrix** is a vector, its determinant is simply its **length**):

ll

$$\det \mathbf{J} = ||\mathbf{J}|| = \sqrt{J_1^2 + J_2^2 + J_3^2} = \sqrt{l_x^2 + l_y^2 + l_z^2} = l$$

(10.86)

??

Jacobian Inverse (when the **Jacobian matrix** is a row vector, its inverse is just its **transpose** divided by its determinant squared):

$$\mathbf{K}^e \stackrel{+}{=} \mathbf{B}_g^T \mathbf{D} \mathbf{B}_g \det \mathbf{J} w \quad (10.91)$$

$$\begin{aligned} \mathbf{K}^e \stackrel{+}{=} \frac{1}{l^2} \begin{bmatrix} -l_x \\ -l_y \\ -l_z \\ l_x \\ l_y \\ l_z \end{bmatrix} EA \frac{1}{l^2} \begin{bmatrix} -l_x & -l_y & -l_z & l_x & l_y & l_z \end{bmatrix} l w \\ \stackrel{+}{=} \frac{EA}{l^3} \begin{bmatrix} l_x^2 & l_x l_y & l_x l_z & -l_x^2 & -l_x l_y & -l_x l_z \\ l_x l_y & l_y^2 & l_y l_z & -l_x l_y & -l_y^2 & -l_y l_z \\ l_x l_z & l_y l_z & l_z^2 & -l_x l_z & -l_y l_z & -l_z^2 \\ -l_x^2 & -l_x l_y & -l_x l_z & l_x^2 & l_x l_y & l_x l_z \\ -l_x l_y & -l_y^2 & -l_y l_z & l_x l_y & l_y^2 & l_y l_z \\ -l_x l_z & -l_y l_z & -l_z^2 & l_x l_z & l_y l_z & l_z^2 \end{bmatrix} w \end{aligned} \quad (10.92)$$

10.4.5 Example 2D quadratic

If assembling the element in 2D, then $u_z = 0$.

The node numbering scheme is 1 – 3 – 2 and the natural coordinate is defined $\xi \in \langle -1, 1 \rangle$.

The **quadratic** shape functions are:

$$\begin{aligned} N_1^e &= -\frac{1}{2}\xi(1-\xi) \\ N_2^e &= \frac{1}{2}\xi(1+\xi) \\ N_3^e &= (1+\xi)(1-\xi) \end{aligned} \quad (10.93)$$

$$\mathbf{N} = \begin{bmatrix} N_1^e & 0 & N_2^e & 0 & N_3^e & 0 \\ 0 & N_1^e & 0 & N_2^e & 0 & N_3^e \end{bmatrix} \quad (10.94)$$

$$\mathbf{N} = \begin{bmatrix} -\frac{1}{2}\xi(1-\xi) & 0 & \frac{1}{2}\xi(1+\xi) & 0 & (1+\xi)(1-\xi) & 0 \\ -\frac{1}{2}\xi(1-\xi) & -\frac{1}{2}\xi(1-\xi) & 0 & \frac{1}{2}\xi(1+\xi) & 0 & (1+\xi)(1-\xi) \end{bmatrix} \quad (10.95)$$

$$\mathbf{N} = \begin{bmatrix} \frac{\xi^2}{2} - \frac{\xi}{2} & 0 & \frac{\xi^2}{2} + \frac{\xi}{2} & 0 & 1 - \xi^2 & 0 \\ 0 & \frac{\xi^2}{2} - \frac{\xi}{2} & 0 & \frac{\xi^2}{2} + \frac{\xi}{2} & 0 & 1 - \xi^2 \end{bmatrix} \quad (10.96)$$

and

$$\begin{bmatrix} u_x^e(\xi) \\ u_y^e(\xi) \end{bmatrix} = \mathbf{N}(\xi) \begin{bmatrix} u_{x1}^e \\ u_{y1}^e \\ u_{x2}^e \\ u_{y2}^e \\ u_{x3}^e \\ u_{y3}^e \end{bmatrix} \quad (10.97)$$

or if:

$$\mathbf{N} = \begin{bmatrix} N_1^e & N_2^e & N_3^e \end{bmatrix} = \begin{bmatrix} \frac{\xi^2}{2} - \frac{\xi}{2} & \frac{\xi^2}{2} + \frac{\xi}{2} & 1 - \xi^2 \end{bmatrix} \quad (10.98)$$

then:

$$\begin{bmatrix} u_x^e(\xi) \\ u_y^e(\xi) \end{bmatrix} = \mathbf{N}(\xi) \begin{bmatrix} u_{x1}^e & u_{y1}^e \\ u_{x2}^e & u_{y2}^e \\ u_{x3}^e & u_{y3}^e \end{bmatrix} \quad (10.99)$$

shape function derivatives:

$$\mathbf{B} = \frac{\partial \mathbf{N}}{\partial \xi} = \begin{bmatrix} \xi - \frac{1}{2} & 0 & \xi + \frac{1}{2} & 0 & -2\xi & 0 \\ 0 & \xi - \frac{1}{2} & 0 & \xi + \frac{1}{2} & 0 & -2\xi \end{bmatrix} \quad (10.100)$$

the strain-displacement relation is given by:

$$\boldsymbol{\varepsilon} = \mathbf{B} \mathbf{u} \quad (10.101)$$

Integrating the element at two **Gauss-Legendre** points of the natural coordinate $\xi = [-0.5773502692, 0.5773502692]$ with weights of $w = 1.0$ then numerically:

To get the correct interpolation points and weights, python can be used:

```

1      import numpy as np
2
3      xi, w = np.polynomial.legendre.leggauss(2)
```


for $\xi^1 = -0.5773502692$

$$\mathbf{N}^1 = \begin{bmatrix} 0.45534 & -0.12201 & 0.66667 \end{bmatrix} \quad (10.102)$$

for $\xi^2 = 0.5773502692$

$$\mathbf{N}^2 = \begin{bmatrix} -0.12201 & 0.45534 & 0.66667 \end{bmatrix} \quad (10.103)$$

Shape Functions in global coordinates:

$$\begin{bmatrix} u_{x1}^i \\ u_{y1}^i \end{bmatrix} = \begin{bmatrix} \frac{\xi^2}{2} - \frac{\xi}{2} & 0 & \frac{\xi^2}{2} + \frac{\xi}{2} & 0 & 1 - \xi^2 & 0 \\ 0 & \frac{\xi^2}{2} - \frac{\xi}{2} & 0 & \frac{\xi^2}{2} + \frac{\xi}{2} & 0 & 1 - \xi^2 \end{bmatrix}_{\xi^i} \begin{bmatrix} u_{x1}^e \\ u_{y1}^e \\ u_{x2}^e \\ u_{y2}^e \\ u_{x3}^e \\ u_{y3}^e \end{bmatrix} \quad (10.104)$$

$$\begin{bmatrix} u_x^1 \\ u_y^1 \end{bmatrix} = \begin{bmatrix} 0.45534u_{x1} - 0.12201u_{x2} + 0.66667u_{x3} \\ -0.12201u_{y1} + 0.45534u_{y2} + 0.66667u_{y3} \end{bmatrix} \quad (10.105)$$

$$\begin{bmatrix} u_x^2 \\ u_y^2 \end{bmatrix} = \begin{bmatrix} -0.12201u_{x1} + 0.45534u_{x2} + 0.66667u_{x3} \\ 0.45534u_{y1} - 0.12201u_{y2} + 0.66667u_{y3} \end{bmatrix} \quad (10.106)$$

Shape Function Derivatives in Natural Coordinates (there is a row for each integration point if the coordinates are specified as matrix, not a column vector. otherwise for each integration point there is a separate $\mathbf{B}$ matrix):

$$\frac{\partial \mathbf{N}}{\partial \xi} = \mathbf{B} = \begin{bmatrix} \xi - \frac{1}{2} & 0 & \xi + \frac{1}{2} & 0 & -2\xi & 0 \\ 0 & \xi - \frac{1}{2} & 0 & \xi + \frac{1}{2} & 0 & -2\xi \end{bmatrix} \quad (10.107)$$

Jacobian:

$$\mathbf{J} = \mathbf{B}\mathbf{u} = \begin{bmatrix} \xi - \frac{1}{2} & 0 & \xi + \frac{1}{2} & 0 & -2\xi & 0 \\ 0 & \xi - \frac{1}{2} & 0 & \xi + \frac{1}{2} & 0 & -2\xi \end{bmatrix} \begin{bmatrix} u_{x1}^e \\ u_{y1}^e \\ u_{x2}^e \\ u_{y2}^e \\ u_{x3}^e \\ u_{y3}^e \end{bmatrix} \quad (10.108)$$

for $\xi^1 = -0.5773502692$

$$\mathbf{J}^1 = \begin{bmatrix} -1.07735u_{x1} - 0.07735u_{x2} + 1.15470u_3 \\ -1.07735u_{y1} - 0.07735u_{y2} + 1.15470u_{y3} \end{bmatrix} \quad (10.109)$$

for $\xi^2 = 0.5773502692$

$$\mathbf{J}^2 = \begin{bmatrix} 0.07735u_{x1} + 1.07735u_{x2} - 1.15470u_3 \\ 0.07735u_{y1} + 1.07735u_{y2} - 1.15470u_{y3} \end{bmatrix} \quad (10.110)$$

Jacobian Determinant (when the **Jacobian matrix** is a vector, its determinant is simply its **length**):

$$\det \mathbf{J} = \|\mathbf{J}\| \quad (10.111)$$

for $\xi^1 = -0.5773502692$

$$\det \mathbf{J}^1 = \sqrt{(-1.07735u_{x1} - 0.07735u_{x2} + 1.15470u_3)^2 + (-1.07735u_{y1} - 0.07735u_{y2} + 1.15470u_{y3})^2} \quad (10.112)$$

for $\xi^2 = 0.5773502692$

$$\det \mathbf{J}^2 = \sqrt{(0.07735u_{x1} + 1.07735u_{x2} - 1.15470u_3)^2 + (0.07735u_{y1} + 1.07735u_{y2} - 1.15470u_{y3})^2} \quad (10.113)$$

Jacobian Inverse (when the **Jacobian matrix** is a row vector, its inverse is just its **transpose** divided by its determinant squared):

Moore-Penrose pseudoinverse:

$$\mathbf{J}^{-1} = \mathbf{J}^+ = (\mathbf{J}^T \mathbf{J})^{-1} \mathbf{J}^T \quad (10.114)$$

for $\xi^1 = -0.5773502692$

$$\mathbf{J}^{1-1} = \frac{1}{\det \mathbf{J}^{12}} \mathbf{J}^{1T} \quad (10.115)$$

for $\xi^2 = 0.5773502692$

$$\mathbf{J}^{2-1} = \frac{1}{\det \mathbf{J}^{22}} \mathbf{J}^{2T} \quad (10.116)$$

Shape function derivatives in global coordinates:

$$\mathbf{B}_g = \mathbf{J}^{-1} \mathbf{B} \quad (10.117)$$

for $\xi^1 = -0.5773502692$

$$\mathbf{B}_g^1 = \mathbf{J}^{1-1} \mathbf{B}^1 \quad (10.118)$$

for $\xi^2 = 0.5773502692$

$$\mathbf{B}_g^2 = \mathbf{J}^{2-1} \mathbf{B}^2 \quad (10.119)$$

Material Stiffness Matrix

$$\mathbf{D} = EA \quad (10.120)$$

Then the stiffness matrix is composed as:

$$\mathbf{K}^e = \sum_{i=1}^2 \mathbf{B}_g^{iT} \mathbf{D} \mathbf{B}_g^i \det \mathbf{J}^i w^i \quad (10.121)$$

10.4.6 Python Implementation

```

1  #!/usr/bin/python3
2
3  import numpy as np
4  np.set_printoptions(precision=6, suppress=True)
5
6  def rod2(xyz, E, A, rho, fx, fy, fz, gauss_points=1):
7      print('\nStarting ROD element generation:\n')
8      coors = xyz.reshape(-1, 1)
9      print('----- Input')
10     print('coors =\n{0}'.format(coors))
11     print(f'{A = } mm2\n{rho = } t/mm2\n{fx = } N/mm\n{fy = } N/mm' +
12         f'\n{fz = } N/mm\n{gauss_points = }\n')
13

```

```

14     domain = np.array([[ -1],
15                        [ 1]], dtype=float)
16     print('Domain: {0}\n{1}\n'.format(domain.shape, domain))
17
18     print('----- Gauss-Legendre')
19     # gauss-legendre integration
20     ip, iw = np.polynomial.legendre.leggauss(gauss_points)
21     print('Gauss-Legendre Integration\nip = {0}\n{1}\n'.format(ip, iw))
22
23     full_domain = np.vstack((domain, ip.reshape(-1, 1)))
24
25     print('----- Shape Functions')
26     # shape functions in natural coordinates
27     xi = ip.T
28     print('xi = {0}\n'.format(xi))
29     psi = []
30     for i in range(xi.shape[0]):
31         psi.append(np.array([[1/2 - xi[i]/2, 0, 0, 1/2 + xi[i]/2, 0, 0],
32                             [0, 1/2 - xi[i]/2, 0, 0, 1/2 + xi[i]/2, 0],
33                             [0, 0, 1/2 - xi[i]/2, 0, 0, 1/2 + xi[i]/2]],
34                             dtype=float))
35     psi = np.array(psi)
36     # psi = psi.transpose(0, 2, 1)
37     print('Shape Functions: {0}\n{1}\n'.format(psi.shape, psi))
38
39     # integration points in global coordinates
40     psi_g = psi @ coors
41     print('Integration Points in '
42           'Global Coordinates: {0}\n{1}\n'.format(psi_g.shape, psi_g))
43
44     # shape function derivatives in natural coordinates
45     dpsi = []
46     for i in range(xi.shape[0]):
47         dpsi.append(np.array([[ -1/2, 0, 0, 1/2, 0, 0],
48                               [0, -1/2, 0, 0, 1/2, 0],
49                               [0, 0, -1/2, 0, 0, 1/2]], dtype=float))
50     dpsi = np.array(dpsi)
51     print('Shape Functions Derivatives: {0}\n{1}\n'.format(dpsi.shape, dpsi))
52
53     print('----- Jacobians')
54     # Jacobian Matrix

```

```

55     jacobi = dps_i @ coors
56     print('Jacobian Matrix: {0}\n{1}\n'.format(jacobi.shape, jacobi))
57
58     # Jacobian Determinants
59     d_jacobi = []
60     for i in range(xi.shape[0]):
61         d_jacobi.append(np.linalg.norm(jacobi[i]))
62     d_jacobi = np.array(d_jacobi)
63     print('Determinant of Jacobian: {0}\n{1}\n'.format(d_jacobi.shape, d_jacobi))
64
65     # Inverse Jacobian
66     i_jacobi = np.linalg.pinv(jacobi)
67     print('Inverse Jacobian Matrix: {0}\n{1}\n'.format(i_jacobi.shape, i_jacobi))
68
69     # Shape Function Derivatives in Global Coordinates
70     dps_i_g = i_jacobi @ dps_i
71     print('Shape Function Derivatives in Global Coordinates:
72     ↪ {0}\n{1}\n'.format(dps_i_g.shape, dps_i_g))
73
74     # material stiffness
75     D = np.array([[E * A]], dtype=float)
76     print('Material Stiffness Matrix: {0}\n{1}\n'.format(D.shape, D))
77
78     # create element stiffness and mass matrix
79     Ke = np.zeros((domain.shape[0] * 3, domain.shape[0] * 3), dtype=float)
80     Me = np.zeros((domain.shape[0] * 3, domain.shape[0] * 3), dtype=float)
81     Fe = np.zeros((domain.shape[0] * 3, 1), dtype=float)
82
83     print('----- System Matrices')
84     # iterate over integration points
85     F = np.array([[fx], [fy], [fz]], dtype=float)
86     for i in range(xi.shape[0]):
87         B = dps_i_g[i]
88         N = psi[i]
89
90         Ke += (B.T @ D @ B) * d_jacobi[i] * iw[i]
91         Me += rho * (N.T @ N) * d_jacobi[i] * iw[i]
92         Fe += (N.T @ F) * d_jacobi[i] * iw[i]
93
94     print('Element Stiffness Matrix: {0}\n{1}\n'.format(Ke.shape, Ke))
95     print('Element Mass Matrix: {0}\n{1}\n'.format(Me.shape, Me))

```

```

95     print('Element Volume Force Vector: {0}\n{1}\n'.format(Fe.shape, Fe))
96
97     print('----- Displacements')
98     # xyz displacements at end nodes
99     u = np.array([0.,
100                 0.,
101                 0.,
102                 .001,
103                 .001,
104                 .001], dtype=float).reshape(-1, 1)
105     print('Displacements {0}\n{1}\n'.format(u.shape, u))
106
107     print('----- Strains')
108     eps = []
109     for i in range(xi.shape[0]):
110         eps.append(dpsi_g[i] @ u)
111     eps = np.array(eps)
112     print('Strains {0}\n{1}\n'.format(eps.shape, eps))
113
114     print('----- Stresses')
115     sig = D @ eps
116     print('Stresses {0}\n{1}\n'.format(sig.shape, sig))
117
118     return Ke, Me, Fe
119
120
121
122 if __name__ == '__main__':
123     coor = np.array([[ 0, 0., 0.],
124                     [1000., 0., 0.]], dtype=float)
125
126     E = 210000.0 # MPa steel
127     A = 200. # steel
128     rho = 9.81E-9 # steel t/mm3
129
130     fx = 1. # volumetric continuous load
131     fy = 0. # volumetric continuous load
132     fz = 1. # volumetric continuous load
133
134     rod2(coor, E, A, rho, fx, fy, fz, 1)

```


10.5 HEX8

10.5.1 Element Formulation

The **Hexahedral** element is a 3-D quadrilateral **trilinear** element, otherwise known as a **brick**. Topologically is hexahedron equivalent to a cube. It has eight corners, twelve edges and six faces. Especially the **HEX8** element has 8 interpolation points.

Natural coordinates

The **natural coordinate system** of a hexahedral element are called **isoparametric hexahedral coordinates**, denoted ξ , η and μ . The coordinates go from -1 to 1 and span from each face to the opposite one. Each coordinate is 0 on the plane at midpoint of opposing faces called the **median** face. This choice of limits is to facilitate the use of standard *Gauss integration formulas*.

Corner Numbering Rules

The eight corners of a hexahedron are locally numbered $1, 2, \dots, 8$. The corner numbering rule is:

- select one face and numbers the corners of this face $1 \dots 4$ in a **counter-clockwise** direction.
- number the corners directly opposite to corners $1, 2, 3, 4$ as $5, 6, 7, 8$ respectively.

The purpose of this numbering manner is so that a positive volume (or more precisely, a positive **Jacobian determinant** at every point).

The definition of ξ , η and μ can be now made more precise:

- ξ goes from -1 from (center of) face 1485 to $+1$ on face 2376

- η goes from -1 from (center of) face 1265 to $+1$ on face 3487
- μ goes from -1 from (center of) face 1234 to $+1$ on face 5678

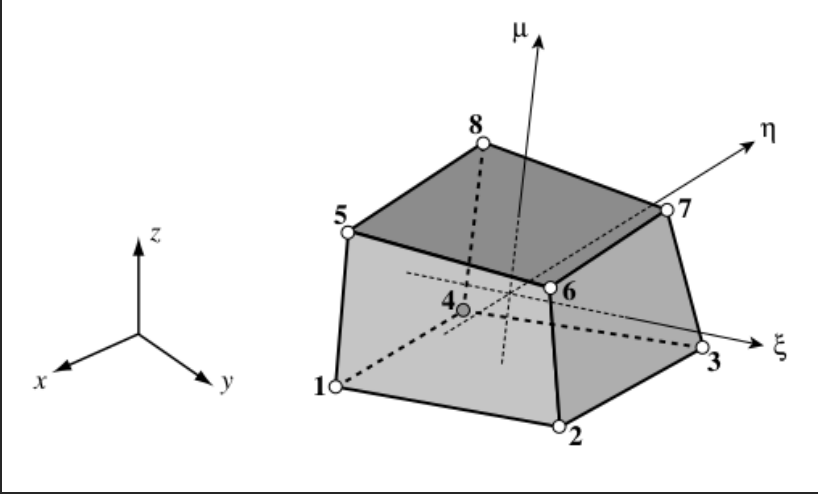


Figure 10.2: The 8-node hexahedron and the natural coordinates η , ξ and μ .

Element Definition

The 8-noded hexahedral element is defined by:

$$\begin{bmatrix} 1 \\ x \\ y \\ z \\ u_x \\ u_y \\ u_z \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ x_1 & x_2 & x_3 & x_4 & x_5 & x_6 & x_7 & x_8 \\ y_1 & y_2 & y_3 & y_4 & y_5 & y_6 & y_7 & y_8 \\ y_1 & y_2 & y_3 & y_4 & y_5 & y_6 & y_7 & y_8 \\ u_{x1} & u_{x2} & u_{x3} & u_{x4} & u_{x5} & u_{x6} & u_{x7} & u_{x8} \\ u_{y1} & u_{y2} & u_{y3} & u_{y4} & u_{y5} & u_{y6} & u_{y7} & u_{y8} \\ u_{z1} & u_{z2} & u_{z3} & u_{z4} & u_{z5} & u_{z6} & u_{z7} & u_{z8} \end{bmatrix} \begin{bmatrix} N_1^e \\ N_2^e \\ N_3^e \\ N_4^e \\ N_5^e \\ N_6^e \\ N_7^e \\ N_8^e \end{bmatrix} \quad (10.122)$$

The hexahedron corners natural coordinates are:

Partial Derivatives

The calculation of the shape functions derivatives with respect to the natural coordinates:

$$\begin{aligned}
 \frac{\partial N_1^e}{\partial \xi} &= -\frac{1}{8} (1 - \eta) (1 - \mu) \\
 \frac{\partial N_2^e}{\partial \xi} &= \frac{1}{8} (1 - \eta) (1 - \mu) \\
 \frac{\partial N_3^e}{\partial \xi} &= \frac{1}{8} (1 + \eta) (1 - \mu) \\
 \frac{\partial N_4^e}{\partial \xi} &= -\frac{1}{8} (1 + \eta) (1 - \mu) \\
 \frac{\partial N_5^e}{\partial \xi} &= -\frac{1}{8} (1 - \eta) (1 + \mu) \\
 \frac{\partial N_6^e}{\partial \xi} &= \frac{1}{8} (1 - \eta) (1 + \mu) \\
 \frac{\partial N_7^e}{\partial \xi} &= \frac{1}{8} (1 + \eta) (1 + \mu) \\
 \frac{\partial N_8^e}{\partial \xi} &= -\frac{1}{8} (1 + \eta) (1 + \mu)
 \end{aligned} \tag{10.125}$$

Note:

The partial derivatives can be also written as:

$$\begin{aligned}
 \frac{\partial N_i^e}{\partial \xi} &= \frac{1}{8} \frac{\xi_i}{|\xi_i|} (1 + \eta \eta_i) (1 + \mu \mu_i) \\
 \frac{\partial N_i^e}{\partial \eta} &= \frac{1}{8} \frac{\eta_i}{|\eta_i|} (1 + \xi \xi_i) (1 + \mu \mu_i) \\
 \frac{\partial N_i^e}{\partial \mu} &= \frac{1}{8} \frac{\mu_i}{|\mu_i|} (1 + \xi \xi_i) (1 + \eta \eta_i)
 \end{aligned} \tag{10.128}$$

The Jacobian:

The derivatives of the shape functions are given by the usual chain rule formulas:

$$\begin{aligned}
 \frac{\partial N_i^e}{\partial x} &= \frac{\partial N_i^e}{\partial \xi} \frac{\partial \xi}{\partial x} + \frac{\partial N_i^e}{\partial \eta} \frac{\partial \eta}{\partial x} + \frac{\partial N_i^e}{\partial \mu} \frac{\partial \mu}{\partial x} \\
 \frac{\partial N_i^e}{\partial y} &= \frac{\partial N_i^e}{\partial \xi} \frac{\partial \xi}{\partial y} + \frac{\partial N_i^e}{\partial \eta} \frac{\partial \eta}{\partial y} + \frac{\partial N_i^e}{\partial \mu} \frac{\partial \mu}{\partial y} \\
 \frac{\partial N_i^e}{\partial z} &= \frac{\partial N_i^e}{\partial \xi} \frac{\partial \xi}{\partial z} + \frac{\partial N_i^e}{\partial \eta} \frac{\partial \eta}{\partial z} + \frac{\partial N_i^e}{\partial \mu} \frac{\partial \mu}{\partial z}
 \end{aligned} \tag{10.129}$$

In matrix form:

$$\begin{bmatrix} \frac{\partial N_i^e}{\partial x} \\ \frac{\partial N_i^e}{\partial y} \\ \frac{\partial N_i^e}{\partial z} \end{bmatrix} = \begin{bmatrix} \frac{\partial \xi}{\partial x} & \frac{\partial \eta}{\partial x} & \frac{\partial \mu}{\partial x} \\ \frac{\partial \xi}{\partial y} & \frac{\partial \eta}{\partial y} & \frac{\partial \mu}{\partial y} \\ \frac{\partial \xi}{\partial z} & \frac{\partial \eta}{\partial z} & \frac{\partial \mu}{\partial z} \end{bmatrix} \begin{bmatrix} \frac{\partial N_i^e}{\partial \xi} \\ \frac{\partial N_i^e}{\partial \eta} \\ \frac{\partial N_i^e}{\partial \mu} \end{bmatrix} \tag{10.130}$$

The 3×3 matrix above is $\mathbf{J}^{-1}$, the inverse of:

$$\mathbf{J} = \frac{\partial (x, y, z)}{\partial (\xi, \eta, \mu)} = \begin{bmatrix} \frac{\partial x}{\partial \xi} & \frac{\partial y}{\partial \xi} & \frac{\partial z}{\partial \xi} \\ \frac{\partial x}{\partial \eta} & \frac{\partial y}{\partial \eta} & \frac{\partial z}{\partial \eta} \\ \frac{\partial x}{\partial \mu} & \frac{\partial y}{\partial \mu} & \frac{\partial z}{\partial \mu} \end{bmatrix} \tag{10.131}$$

Matrix $\mathbf{J}$ is called the *Jacobian matrix* of (x, y, z) with respect to (ξ, η, μ) . In the finite element literature, matrices $\mathbf{J}$ and $\mathbf{J}^{-1}$ are called simply the *Jacobian* and *inverse Jacobian*, respectively, although such a short name is sometimes ambiguous. The notation

$$\begin{aligned}\mathbf{J} &= \frac{\partial (x, y, z)}{\partial (\xi, \eta, \mu)} \\ \mathbf{J}^{-1} &= \frac{\partial (\xi, \eta, \mu)}{\partial (x, y, z)}\end{aligned}\tag{10.132}$$

is standard in multivariable calculus and suggests that the Jacobian may be viewed as a generalisation of the ordinary derivative to which it reduces for a scalar function $x = x(\xi)$.

Computing the Jacobian Matrix

The isoparametric definition of hexahedron element geometry is:

$$\begin{aligned}x &= x_i N_i^e \\ y &= y_i N_i^e \\ z &= z_i N_i^e\end{aligned}\tag{10.133}$$

where the summation convention is understood to apply over $i = 1, 2, \dots, n$, in which n denotes the number of element nodes.

Note: This for a given hexahedral element gives an $n \times 3$ matrix.

Differentiating these relations with respect to the hexahedron coordinates we construct the matrix $\mathbf{J}$ as follows:

$$\mathbf{J} = \begin{bmatrix} x_i \frac{\partial N_i^e}{\partial \xi} & y_i \frac{\partial N_i^e}{\partial \xi} & z_i \frac{\partial N_i^e}{\partial \xi} \\ x_i \frac{\partial N_i^e}{\partial \eta} & y_i \frac{\partial N_i^e}{\partial \eta} & z_i \frac{\partial N_i^e}{\partial \eta} \\ x_i \frac{\partial N_i^e}{\partial \mu} & y_i \frac{\partial N_i^e}{\partial \mu} & z_i \frac{\partial N_i^e}{\partial \mu} \end{bmatrix} \quad (10.134)$$

or:

$$\mathbf{J}_i = \begin{bmatrix} \frac{\partial N_1^e}{\partial \xi} & \frac{\partial N_2^e}{\partial \xi} & \cdots & \frac{\partial N_n^e}{\partial \xi} \\ \frac{\partial N_1^e}{\partial \eta} & \frac{\partial N_2^e}{\partial \eta} & \cdots & \frac{\partial N_n^e}{\partial \eta} \\ \frac{\partial N_1^e}{\partial \mu} & \frac{\partial N_2^e}{\partial \mu} & \cdots & \frac{\partial N_n^e}{\partial \mu} \end{bmatrix} \begin{bmatrix} x_1 & y_1 & z_1 \\ x_2 & y_2 & z_2 \\ \vdots & \vdots & \vdots \\ x_n & y_n & z_n \end{bmatrix} \quad (10.135)$$

Given a point of hexahedron coordinates (ξ, η, μ) the Jacobian $\mathbf{J}$ can be easily formed using the above formula, and numerically inverted to form $\mathbf{J}^{-1}$.

The Strain-Displacement Matrix

Having obtained the shape function derivatives, the matrix $\mathbf{B}$ for hexahedron element displays the usual structure for 3D elements:

$$\mathbf{B} = \mathbf{D}\Phi = \begin{bmatrix} \frac{\partial}{\partial x} & 0 & 0 \\ 0 & \frac{\partial}{\partial y} & 0 \\ 0 & 0 & \frac{\partial}{\partial z} \\ \frac{\partial}{\partial y} & \frac{\partial}{\partial x} & 0 \\ 0 & \frac{\partial}{\partial z} & \frac{\partial}{\partial y} \\ \frac{\partial}{\partial z} & 0 & \frac{\partial}{\partial x} \end{bmatrix} \begin{bmatrix} \mathbf{q} & 0 & 0 \\ 0 & \mathbf{q} & 0 \\ 0 & 0 & \mathbf{q} \end{bmatrix} = \begin{bmatrix} \mathbf{q}_x & 0 & 0 \\ 0 & \mathbf{q}_y & 0 \\ 0 & 0 & \mathbf{q}_z \\ \mathbf{q}_y & \mathbf{q}_x & 0 \\ 0 & \mathbf{q}_z & \mathbf{q}_y \\ \mathbf{q}_z & 0 & \mathbf{q}_x \end{bmatrix} \quad (10.136)$$

where:

$$\begin{aligned} \mathbf{q} &= [N_1^e \quad \cdots \quad N_n^e] \\ \mathbf{q}_x &= \left[\frac{\partial N_1^e}{\partial x} \quad \cdots \quad \frac{\partial N_n^e}{\partial x} \right] \\ \mathbf{q}_y &= \left[\frac{\partial N_1^e}{\partial y} \quad \cdots \quad \frac{\partial N_n^e}{\partial y} \right] \\ \mathbf{q}_z &= \left[\frac{\partial N_1^e}{\partial z} \quad \cdots \quad \frac{\partial N_n^e}{\partial z} \right] \end{aligned} \quad (10.137)$$

are row vectors of length n , n being the number of nodes in the element.

Stiffness Matrix Evaluation

The element stiffness Matrix is given by:

$$\mathbf{K}^e = \int_{V^e} \mathbf{B}^T \mathbf{E} \mathbf{B} dV^e \quad (10.138)$$

As in two-dimensional case, this is replaced by a numerical integration formula which now involves a triple loop over conventional Gauss quadrature rules. Assuming that the stress-strain matrix $\mathbf{E}$ is constant over the element,

$$\mathbf{K}^e = \sum_{i=1}^{p_1} \sum_{j=1}^{p_2} \sum_{k=1}^{p_3} w_i w_j w_k \mathbf{B}_{ijk}^T \mathbf{E} \mathbf{B}_{ijk} \mathbf{J}_{ijk} \quad (10.139)$$

Here p_1 , p_2 and p_3 are the number of Gauss points in the ξ , η and μ direction, respectively, while $\mathbf{B}_{ijk}$ and $\mathbf{J}_{ijk}$ are abbreviations for:

$$\begin{aligned} \mathbf{B}_{ijk} &= \mathbf{B}(\xi_i, \eta_i, \mu_i) \\ \mathbf{J}_{ijk} &= \det \mathbf{J}(\xi_i, \eta_i, \mu_i) \end{aligned} \quad (10.140)$$

Usually the number of integration points is taken the same in all directions: $p = p_1 = p_2 = p_3$. The total number of Gauss points is thus p^3 . Each point adds at most 6 to the stiffness matrix rank. The minimum rank-sufficient rules for the 8-node and 20-node hexahedra are $p = 2$ and $p = 3$, respectively.

10.5.2 Python implementation

```

1 #!/usr/bin/python3
2
3 import numpy as np
4 np.set_printoptions(precision=2) # , suppress=True)
5
```



```

6 def hex8_stiffness_mass_load(coors: np.ndarray,
7                               E: float,
8                               nu: float,
9                               rho: float,
10                              fx: float = 0.,
11                              fy: float = 0.,
12                              fz: float = 0.):
13     """
14     In:
15         coors - 8 x 3 coordinate matrix
16               [[x1,y1,z1], ... , [xn,yn,zn]]
17         E     - Youngs's Modulus
18         nu    - Poisson's Constant
19         rho   - Density
20         fx    - Volumetric Load in x direction
21         fy    - Volumetric Load in y direction
22         fz    - Volumetric Load in z direction
23     """
24     domain = np.array([[-1., -1., -1.],
25                       [1., -1., -1.],
26                       [1., 1., -1.],
27                       [-1., 1., -1.],
28                       [-1., -1., 1.],
29                       [1., -1., 1.],
30                       [1., 1., 1.],
31                       [-1., 1., 1.]], dtype=float)
32     print('Domain: {0}\n{1}'.format(domain.shape, domain))
33
34     # gaussian interpolation
35     gauss_points = 3
36     g_points, g_weights = np.polynomial.legendre.leggauss(gauss_points)
37     integration_points = []
38     interpolation_weights = []
39     for i, xi in enumerate(g_points):
40         for j, eta in enumerate(g_points):
41             for k, mu in enumerate(g_points):
42                 integration_points.append([xi, eta, mu])
43                 interpolation_weights.append(g_weights[i] *
44                                             g_weights[j] *
45                                             g_weights[k])
46     integration_points = np.array(integration_points, dtype=float)

```

```

47 interpolation_weights = np.array(interpolation_weights, dtype=float)
48 print('Gauss Integration '
49       '{0}:\n{1}\nWeights:\n{2}'.format(gauss_points,
50                                           integration_points,
51                                           interpolation_weights))
52
53 full_domain = np.vstack((domain, integration_points))
54
55 # Shape Functions in Natural Coordinates
56 xi = integration_points.T[0]
57 eta = integration_points.T[1]
58 mu = integration_points.T[2]
59 psi = np.zeros((8, integration_points.shape[0]), dtype=float)
60 for i in range(8):
61     psi[i] = 1/8 * (1 + xi * domain[i, 0]) *
62                  (1 + eta * domain[i, 1]) *
63                  (1 + mu * domain[i, 2])
64 psi = psi.T
65 print('Shape Functions: {0}\n{1}'.format(psi.shape, psi))
66
67 # Shape Functions in Global Coordinates
68 psi_g = psi @ coors
69 print('Shape Functions in '
70       'Global Coordinates: {0}\n{1}'.format(psi_g.shape, psi_g))
71
72 # Shape Functions Derivatives in Natural Coordinates
73 dpsi = 1 / 8 * np.array([[ (eta - 1.0) * (1.0 - mu), # 1
74                           (xi - 1) * (1 - mu),
75                           -(1 - xi) * (1 - eta)],
76                          [(1 - eta) * (1 - mu), # 2
77                           (-1 - xi) * (1 - mu),
78                           -(1 + xi) * (1 - eta)],
79                          [(1 + eta) * (1 - mu), # 3
80                           (1 + xi) * (1 - mu),
81                           -(1 + xi) * (1 + eta)],
82                          [(-1.0 - eta) * (1 - mu), # 4
83                           (1 - xi) * (1 - mu),
84                           -(1 - xi) * (1 + eta)],
85                          [(1 - eta) * (-1 - mu), # 5
86                           (-1 - xi) * (1 + mu),
87                           (1 - xi) * (1 - eta)],

```

```

88             [(1 - eta) * (1 + mu),      # 6
89             -(1 + xi) * (1 + mu),
90             (1 + xi) * (1 - eta)],
91             [(1 + eta) * (1 + mu),      # 7
92             (1 + xi) * (1 + mu),
93             (1 + xi) * (1 + eta)],
94             [-(1 + eta) * (1 + mu),     # 8
95             (1 - xi) * (1 + mu),
96             (1 - xi) * (1 + eta)]]
97     dpsi = dpsi.T
98     print('Shape Functions Derivatives: '
99           '{0}\n{1}'.format(dpsi.shape, dpsi))
100
101     # Jacobian Matrix
102     jacobi = dpsi @ coors
103     print('Jacobian Matrix: {0}\n{1}'.format(jacobi.shape, jacobi))
104
105     # Jacobian Determinants
106     d_jacobi = np.linalg.det(jacobi)
107     print('Determinant of Jacobian: '
108           '{0}\n{1}'.format(d_jacobi.shape, d_jacobi))
109
110     # Inverse Jacobian
111     i_jacobi = np.linalg.inv(jacobi)
112     print('Inverse Jacobian Matrix: '
113           '{0}\n{1}'.format(i_jacobi.shape, i_jacobi))
114
115     # Shape Function Derivatives in Global Coordinates
116     dpsi_g = i_jacobi @ dpsi
117     print('Shape Function Derivatives in Global Coordinates: '
118           '{0}\n{1}'.format(dpsi_g.shape, dpsi_g))
119
120     # Material Stiffness Matrix
121     C = E / ((1.0 + nu) * (1.0 - 2.0 * nu)) *
122         np.array([[1.0 - nu, nu, nu, 0.0, 0.0, 0.0],
123                 [nu, 1.0 - nu, nu, 0.0, 0.0, 0.0],
124                 [nu, nu, 1.0 - nu, 0.0, 0.0, 0.0],
125                 [0.0, 0.0, 0.0, (1.0 - 2.0 * nu) / 2.0, 0.0, 0.0],
126                 [0.0, 0.0, 0.0, 0.0, (1.0 - 2.0 * nu) / 2.0, 0.0],
127                 [0.0, 0.0, 0.0, 0.0, 0.0, (1.0 - 2.0 * nu) / 2.0]],
128                 dtype=float)

```

```

129     print('Material Stiffness Matrix: {0}\n{1}'.format(C.shape, C))
130
131     # Create Element Stiffness and Mass Matrix
132     Ke = np.zeros((domain.size, domain.size), dtype=float)
133     Me = np.zeros((domain.size, domain.size), dtype=float)
134     Fe = np.zeros((domain.size, 1), dtype=float)
135     # o = np.zeros(domain.shape[0], dtype=float)
136
137     # iterate over Gauss points of domain, * means unpack values
138     for i in range(integration_points.shape[0]):
139         # this is smart but arranges dofs by component, not by node
140         # component-wise dof ordering (x1, .. , y1, .. y4, z1, .. , z4)
141         # B = np.array([
142         #     [*dpsi_g[i, 0, :], *o, *o],
143         #     [*o, *dpsi_g[i, 1, :], *o],
144         #     [*o, *o, *dpsi_g[i, 2, :]],
145         #     [*dpsi_g[i, 2, :], *o, *dpsi_g[i, 0, :]],
146         #     [*o, *dpsi_g[i, 2, :], *dpsi_g[i, 1, :]],
147         #     [*dpsi_g[i, 1, :], *dpsi_g[i, 0, :], *o]], dtype=float)
148
149         # dofs ordered by node
150         # node wise dof ordering (x1, y1, z1, .. , x4, y4, z4)
151         dpx = dpsi_g[i,0,:]
152         dpy = dpsi_g[i,1,:]
153         dpz = dpsi_g[i,2,:]
154         # B = np.array(
155         #
156         ↪ [[dpx[0],0,0,dpx[1],0,0,dpx[2],0,0,dpx[3],0,0,dpx[4],0,0,dpx[5],0,0,dpx[6],0,0,dpx[7]
157         #
158         ↪ [0,dpy[0],0,0,dpy[1],0,0,dpy[2],0,0,dpy[3],0,0,dpy[4],0,0,dpy[5],0,0,dpy[6],0,0,dpy[7]
159         #
160         ↪ [0,0,dpz[0],0,0,dpz[1],0,0,dpz[2],0,0,dpz[3],0,0,dpz[4],0,0,dpz[5],0,0,dpz[6],0,0,dpz[7]
161         #
162         ↪ [dpy[0],dpx[0],0,dpy[1],dpx[1],0,dpy[2],dpx[2],0,dpy[3],dpx[3],0,dpy[4],dpx[4],0,dpy[5],dpx[5],0,
163         ↪ [0,dpz[0],dpy[0],0,dpz[1],dpy[1],0,dpz[2],dpy[2],0,dpz[3],dpy[3],0,dpz[4],dpy[4],0,dpz[5],dpy[5],0,
164         #
165         ↪ [dpz[0],0,dpx[0],dpz[1],0,dpx[1],dpz[2],0,dpx[2],dpz[3],0,dpx[3],dpz[4],0,dpx[4],dpz[5],0,dpx[5],dpz[6],0,dpx[6],dpz[7],0,dpx[7]]
166         #
167         dtype=_FLOAT)
168     B = np.array([np.column_stack((dpx, o, o)).flatten(),
169                   np.column_stack((o, dpy, o)).flatten(),

```

```

164         np.column_stack(( o,   o, dpz)).flatten(),
165         np.column_stack((dpy, dpx,  o)).flatten(),
166         np.column_stack(( o, dpz, dpy)).flatten(),
167         np.column_stack((dpz,   o, dpx)).flatten()], dtype=float)
168
169     # this is smart but arranges dofs by component, not by node
170     # component-wise dof ordering (x1, .. , y1, .. y4, z1, .. , z4)
171     # N = np.array([
172     #     [*psi[i], *o, *o],
173     #     [*o, *psi[i], *o],
174     #     [*o, *o, *psi[i]]],
175     #     dtype=float)
176
177     # node wise dof ordering (x1, y1, z1, .. , x4, y4, z4)
178     # N =
179     ↪ np.array([[p[0],0,0,p[1],0,0,p[2],0,0,p[3],0,0,p[4],0,0,p[5],0,0,p[6],0,0,p[7],0,0],
180     #
181     ↪ [0,p[0],0,0,p[1],0,0,p[2],0,0,p[3],0,0,p[4],0,0,p[5],0,0,p[6],0,0,p[7],0],
182     #
183     ↪ [0,0,p[0],0,0,p[1],0,0,p[2],0,0,p[3],0,0,p[4],0,0,p[5],0,0,p[6],0,0,p[7]]],
184     #
185     dtype=_FLOAT)
186     N = np.array([np.column_stack((p, o, o)).flatten(),
187         np.column_stack((o, p, o)).flatten(),
188         np.column_stack((p, o, o)).flatten()], dtype=float)
189
190     F = np.array([[fx], [fy], [fz]], dtype=float)
191
192     Ke += (B.T @ C @ B) * d_jacobi[i] * interpolation_weights[i]
193     Me += rho * (N.T @ N) * d_jacobi[i] * interpolation_weights[i]
194     Fe += (N.T @ F) * d_jacobi[i] * interpolation_weights[i]
195
196     print('Element Stiffness Matrix: {0}\n{1}'.format(Ke.shape, Ke))
197     print('Element Mass Matrix: {0}\n{1}'.format(Me.shape, Me))
198     print('Element Volume Force Vector: {0}\n{1}'.format(Fe.shape, Fe))

```

And to get stresses:

```

1 #!/usr/bin/python3

```

```

2
3 import numpy as np
4 np.set_printoptions(precision=2) # , suppress=True)
5
6 def hex8_stresses(coors: np.ndarray,
7                  disp: np.ndarray,
8                  E: float,
9                  nu: float):
10     """
11     In:
12         coors - 8 x 3 coordinate matrix
13              [[x1,y1,z1], ... , [xn,yn,zn]]
14         disp - 8 x 3 coordinate matrix
15              [[u1,v1,w1], ... , [un,vn,wn]]
16         E    - Youngs's Modulus
17         nu    - Poisson's Constant
18     """
19     domain = np.array([[ -1., -1., -1.],
20                       [ 1., -1., -1.],
21                       [ 1., 1., -1.],
22                       [-1., 1., -1.],
23                       [-1., -1., 1.],
24                       [ 1., -1., 1.],
25                       [ 1., 1., 1.],
26                       [-1., 1., 1.]], dtype=float)
27     print('Domain: {0}\n{1}'.format(domain.shape, domain))
28
29     # gaussian interpolation
30     gauss_points = 3
31     g_points, g_weights = np.polynomial.legendre.leggauss(gauss_points)
32     integration_points = []
33     interpolation_weights = []
34     for i, xi in enumerate(g_points):
35         for j, eta in enumerate(g_points):
36             for k, mu in enumerate(g_points):
37                 integration_points.append([xi, eta, mu])
38                 interpolation_weights.append(g_weights[i] *
39                                           g_weights[j] *
40                                           g_weights[k])
41     integration_points = np.array(integration_points, dtype=float)
42     interpolation_weights = np.array(interpolation_weights, dtype=float)

```

```

43 print('Gauss Integration '
44       '{0}:\n{1}\nWeights:\n{2}'.format(gauss_points,
45                                           integration_points,
46                                           interpolation_weights))
47
48 full_domain = np.vstack((domain, integration_points))
49
50 # Shape Functions in Natural Coordinates
51 xi = integration_points.T[0]
52 eta = integration_points.T[1]
53 mu = integration_points.T[2]
54 psi = np.zeros((8, integration_points.shape[0]), dtype=float)
55 for i in range(8):
56     psi[i] = 1/8 * (1 + xi * domain[i, 0]) *
57                  (1 + eta * domain[i, 1]) *
58                  (1 + mu * domain[i, 2])
59
60 psi = psi.T
61 print('Shape Functions: {0}\n{1}'.format(psi.shape, psi))
62
63 # Shape Functions in Global Coordinates
64 psi_g = psi @ coors
65 print('Shape Functions in '
66       'Global Coordinates: {0}\n{1}'.format(psi_g.shape, psi_g))
67
68 # Shape Functions Derivatives in Natural Coordinates
69 dps1 = 1 / 8 * np.array([[ (eta - 1.0) * (1.0 - mu), # 1
70                          (xi - 1) * (1 - mu),
71                          -(1 - xi) * (1 - eta)],
72                        [(1 - eta) * (1 - mu), # 2
73                          (-1 - xi) * (1 - mu),
74                          -(1 + xi) * (1 - eta)],
75                        [(1 + eta) * (1 - mu), # 3
76                          (1 + xi) * (1 - mu),
77                          -(1 + xi) * (1 + eta)],
78                        [(-1.0 - eta) * (1 - mu), # 4
79                          (1 - xi) * (1 - mu),
80                          -(1 - xi) * (1 + eta)],
81                        [(1 - eta) * (-1 - mu), # 5
82                          -(1 - xi) * (1 + mu),
83                          (1 - xi) * (1 - eta)],
84                        [(1 - eta) * (1 + mu), # 6

```

```

84         -(1 + xi) * (1 + mu),
85         (1 + xi) * (1 - eta)],
86     [(1 + eta) * (1 + mu),      # 7
87      (1 + xi) * (1 + mu),
88      (1 + xi) * (1 + eta)],
89     [-(1 + eta) * (1 + mu),    # 8
90      (1 - xi) * (1 + mu),
91      (1 - xi) * (1 + eta)]]
92     dpsi = dpsi.T
93     print('Shape Functions Derivatives: '
94           '{0}\n{1}'.format(dpsi.shape, dpsi))
95
96     # Jacobian Matrix
97     jacobi = dpsi @ coors
98     print('Jacobian Matrix: {0}\n{1}'.format(jacobi.shape, jacobi))
99
100    # Jacobian Determinants
101    d_jacobi = np.linalg.det(jacobi)
102    print('Determinant of Jacobian: '
103          '{0}\n{1}'.format(d_jacobi.shape, d_jacobi))
104
105    # Inverse Jacobian
106    i_jacobi = np.linalg.inv(jacobi)
107    print('Inverse Jacobian Matrix: '
108          '{0}\n{1}'.format(i_jacobi.shape, i_jacobi))
109
110    # Shape Function Derivatives in Global Coordinates
111    dpsi_g = i_jacobi @ dpsi
112    print('Shape Function Derivatives in Global Coordinates: '
113          '{0}\n{1}'.format(dpsi_g.shape, dpsi_g))
114
115    # Material Stiffness Matrix
116    C = E / ((1.0 + nu) * (1.0 - 2.0 * nu)) *
117        np.array([[1.0 - nu, nu, nu, 0.0, 0.0, 0.0],
118                 [nu, 1.0 - nu, nu, 0.0, 0.0, 0.0],
119                 [nu, nu, 1.0 - nu, 0.0, 0.0, 0.0],
120                 [0.0, 0.0, 0.0, (1.0 - 2.0 * nu) / 2.0, 0.0, 0.0],
121                 [0.0, 0.0, 0.0, 0.0, (1.0 - 2.0 * nu) / 2.0, 0.0],
122                 [0.0, 0.0, 0.0, 0.0, 0.0, (1.0 - 2.0 * nu) / 2.0]],
123                 dtype=float)
124    print('Material Stiffness Matrix: {0}\n{1}'.format(C.shape, C))

```



```
125
126     du = disp.T @ np.transpose(dpsi_g, axes=[0, 2, 1])
127
128     exx = du[:, 0, 0]
129     eyy = du[:, 1, 1]
130     ezz = du[:, 2, 2]
131     exy = du[:, 0, 1] + du[:, 1, 0]
132     eyz = du[:, 1, 2] + du[:, 2, 1]
133     exz = du[:, 0, 2] + du[:, 2, 0]
134     epsilons = np.array([exx, eyy, ezz, exy, eyz, exz])
135
136     sigmas = (C @ epsilons).T
137     epsilons = epsilons.T
138     print('Element Strains: {0}\n{1}'.format(epsilons.shape, epsilons))
139     print('Element Stresses: {0}\n{1}'.format(stresses.shape, stresses))
```

10.6 TET4 Element

10.6.1 Element Formulation

For a linear tetrahedron no numerical integration is needed (there can be only one integration point, shape functions derivatives are equal to 1).

The linear tetrahedron is not often used for stress analysis because of its poor performance. Its main value in structural and solid mechanics is educational: it serves as a vehicle to introduce the basic steps of formulation of 3D solid elements, particularly with regards to the use of natural coordinate systems and node numbering conventions. It should be noted that 3D visualisation is notoriously more difficult than 2D, so it is needed to procede more slowly here.

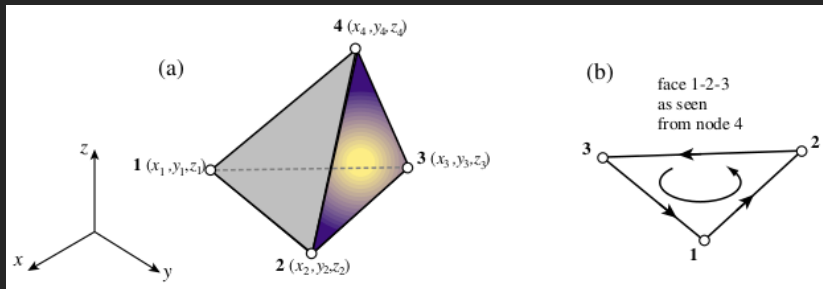


Figure 10.3: (a) The linear tetrahedron element TET4, (b) Node numbering convention.

Tetrahedron Geometry

The tetrahedron geometry is fully defined by giving the location of the four corner nodes with respect to Right-Hand Coordinate Sysetm notation (x, y, z) :


```

38
39     elif gauss_points == 5:
40         integration_points = np.array([[1/4, 1/4, 1/4, 1/4],
41                                       [1/2, 1/6, 1/6, 1/6],
42                                       [1/6, 1/2, 1/6, 1/6],
43                                       [1/6, 1/6, 1/2, 1/6],
44                                       [1/6, 1/6, 1/6, 1/2]], dtype=float)
45         integration_weights = np.array([-4/5, 9/20, 9/20, 9/20, 9/20],
46                                       dtype=float) * 1/6
47
48     print('Gauss Integration '
49           '{0}:\n{1}\nWeights:\n{2}'.format(gauss_points,
50                                             integration_points,
51                                             integration_weights))
52
53     full_domain = np.vstack((domain, integration_points))
54
55     # Shape Functions in Natural Coordinates
56     xi = integration_points.T[0]
57     eta = integration_points.T[1]
58     mu = integration_points.T[2]
59     # zeta is not used to enforce
60     # (1 - xi - eta - mu - zeta) = 0
61     # zeta = integration_points.T[3]
62     psi = np.zeros((4, integration_points.shape[0]), dtype=float)
63     # another way of writing that psi = [xi, eta, mu, zeta]
64     # for i in range(4):
65     #     psi[i] = (1 + xi * domain[i, 0]) * (1 + eta * domain[i, 1]) * (1 + mu * domain[i, 2]) * (1 + zeta * domain[i, 3])
66     psi[0] = xi
67     psi[1] = eta
68     psi[2] = mu
69     psi[3] = 1 - xi - eta - mu # = zeta
70     psi = psi.T
71     print('Shape Functions: {0}\n{1}'.format(psi.shape, psi))
72
73     # Shape Functions in Global Coordinates
74     psi_g = psi @ coors
75     print('Shape Functions in Global Coordinates: '
76           '{0}\n{1}'.format(psi_g.shape, psi_g))
77

```

```

78  # Shape Functions Derivatives in Natural Coordinates
79  dpsi = np.zeros((integration_points.shape[0], 4, 3), dtype=float)
80  dpsi[:,0,0] = 1
81  dpsi[:,1,1] = 1
82  dpsi[:,2,2] = 1
83  dpsi[:,3,:] = -1
84  dpsi = dpsi.transpose((0, 2, 1))
85  print('Shape Functions Derivatives: '
86        '{0}\n{1}'.format(dpsi.shape, dpsi))
87
88  # Jacobian Matrix
89  jacobi = dpsi @ coors
90  print('Jacobian Matrix: {0}\n{1}'.format(jacobi.shape, jacobi))
91
92  # Jacobian Determinants
93  d_jacobi = np.linalg.det(jacobi)
94  print('Determinant of Jacobian: '
95        '{0}\n{1}'.format(d_jacobi.shape, d_jacobi))
96
97  # Inverse Jacobian
98  i_jacobi = np.linalg.inv(jacobi)
99  print('Inverse Jacobian Matrix: '
100        '{0}\n{1}'.format(i_jacobi.shape, i_jacobi))
101
102  # Shape Function Derivatives in Global Coordinates
103  dpsi_g = i_jacobi @ dpsi
104  print('Shape Function Derivatives in Global Coordinates: '
105        '{0}\n{1}'.format(dpsi_g.shape, dpsi_g))
106
107  # Material Stiffness Matrix
108  C = E / ((1.0 + nu) * (1.0 - 2.0 * nu)) *
109      np.array([[1.0 - nu, nu, nu, 0.0, 0.0, 0.0],
110               [nu, 1.0 - nu, nu, 0.0, 0.0, 0.0],
111               [nu, nu, 1.0 - nu, 0.0, 0.0, 0.0],
112               [0.0, 0.0, 0.0, (1.0 - 2.0 * nu) / 2.0, 0.0, 0.0],
113               [0.0, 0.0, 0.0, 0.0, (1.0 - 2.0 * nu) / 2.0, 0.0],
114               [0.0, 0.0, 0.0, 0.0, 0.0, (1.0 - 2.0 * nu) / 2.0]],
115               dtype=float)
116  print('Material Stiffness Matrix: {0}\n{1}'.format(C.shape, C))
117
118  # Create Element Stiffness and Mass Matrix

```

```

119 Ke = np.zeros((domain.shape[0] * 3, domain.shape[0] * 3), dtype=float)
120 Me = np.zeros((domain.shape[0] * 3, domain.shape[0] * 3), dtype=float)
121 Fe = np.zeros((domain.shape[0] * 3, 1), dtype=float)
122 o = np.zeros(domain.shape[0], dtype=float)
123
124 # iterate over Gauss points of domain, * means unpack values
125 for i in range(integration_points.shape[0]):
126     # component-wise dof ordering (x1, .., y1, .. y4, z1, .., z4)
127     # B = np.array([[*dpsi_g[i, 0, :], *o, *o],
128     #               [*o, *dpsi_g[i, 1, :], *o],
129     #               [*o, *o, *dpsi_g[i, 2, :]],
130     #               [*dpsi_g[i, 2, :], *o, *dpsi_g[i, 0, :]],
131     #               [*o, *dpsi_g[i, 2, :], *dpsi_g[i, 1, :]],
132     #               [*dpsi_g[i, 1, :], *dpsi_g[i, 0, :], *o]],
133     #               dtype=float)
134
135     # node wise dof ordering (x1, y1, z1, .., x4, y4, z4)
136     dpx = dpsi_g[i,0,:]
137     dpy = dpsi_g[i,1,:]
138     dpz = dpsi_g[i,2,:]
139     B = np.array(
140         [[dpx[0],0,0,dpx[1],0,0,dpx[2],0,0,dpx[3],0,0],
141          [0,dpy[0],0,0,dpy[1],0,0,dpy[2],0,0,dpy[3],0],
142          [0,0,dpz[0],0,0,dpz[1],0,0,dpz[2],0,0,dpz[3]],
143          [dpy[0],dpx[0],0,dpy[1],dpx[1],0,dpy[2],dpx[2],0,dpy[3],dpx[3],0],
144          [0,dpz[0],dpy[0],0,dpz[1],dpy[1],0,dpz[2],dpy[2],0,dpz[3],dpy[3]],
145          [dpz[0],0,dpx[0],dpz[1],0,dpx[1],dpz[2],0,dpx[2],dpz[3],0,dpx[3]],
146          dtype=float)
147
148     # component-wise dof ordering (x1, .., y1, .. y4, z1, .., z4)
149     # N = np.array([[*psi[i], *o, *o],
150     #               [*o, *psi[i], *o],
151     #               [*o, *o, *psi[i]]], dtype=float)
152
153     # node wise dof ordering (x1, y1, z1, .., x4, y4, z4)
154     N = np.array([[psi[i,0],0,0,psi[i,1],0,0,psi[i,2],0,0,psi[i,3],0,0],
155                  [0,psi[i,0],0,0,psi[i,1],0,0,psi[i,2],0,0,psi[i,3],0],
156                  [0,0,psi[i,0],0,0,psi[i,1],0,0,psi[i,2],0,0,psi[i,3]],
157                  dtype=float)
158
159     F = np.array([[fx], [fy], [fz]], dtype=float)

```

```

160
161     print((B.T @ C @ B).shape)
162     print(Ke.shape)
163     Ke += (B.T @ C @ B) * d_jacobi[i] * integration_weights[i]
164     Me += rho * (N.T @ N) * d_jacobi[i] * integration_weights[i]
165     Fe += (N.T @ F) * d_jacobi[i] * integration_weights[i]
166
167     print('Element Stiffness Matrix: {0}\n{1}'.format(Ke.shape, Ke))
168     print('Element Mass Matrix: {0}\n{1}'.format(Me.shape, Me))
169     print('Element Volume Force Vector: {0}\n{1}'.format(Fe.shape, Fe))

```

And to get stresses:

```

1  #!/usr/bin/python3
2
3  import numpy as np
4  np.set_printoptions(precision=2) # , suppress=True)
5
6  def tet4_stresses(coors: np.ndarray,
7                    disp: np.ndarray,
8                    E: float,
9                    nu: float):
10
11     """
12     In:
13         coors - 4 x 3 coordinate matrix
14                [[x1,y1,z1], ... , [xn,yn,zn]]
15         disp  - 4 x 3 coordinate matrix
16                [[u1,v1,w1], ... , [un,vn,wn]]
17         E     - Youngs's Modulus
18         nu    - Poisson's Constant
19     """
20     domain = np.array([[1., 0., 0., 0.],
21                       [0., 1., 0., 0.],
22                       [0., 0., 1., 0.],
23                       [0., 0., 0., 1.]], dtype=float)
24     print('Domain: {0}\n{1}'.format(domain.shape, domain))
25
26     # gaussian interpolation

```

```

26     integration_points = None
27     integration_weights = None
28     if gauss_points == 1:
29         integration_points = np.array([1/4, 1/4, 1/4, 1/4],
30                                       dtype=float).reshape(1, 4)
31         integration_weights = np.array([1 * 1 * 1 * 1],
32                                       dtype=float) * 1/6
33
34     elif gauss_points == 4:
35         a = 0.58541020
36         b = 0.13819660
37         integration_points = np.array([[a, b, b, b],
38                                       [b, a, b, b],
39                                       [b, b, a, b],
40                                       [b, b, b, a]], dtype=float)
41         integration_weights = np.array([1/4, 1/4, 1/4, 1/4],
42                                       dtype=float) * 1/6
43
44     elif gauss_points == 5:
45         integration_points = np.array([[1/4, 1/4, 1/4, 1/4],
46                                       [1/2, 1/6, 1/6, 1/6],
47                                       [1/6, 1/2, 1/6, 1/6],
48                                       [1/6, 1/6, 1/2, 1/6],
49                                       [1/6, 1/6, 1/6, 1/2]], dtype=float)
50         integration_weights = np.array([-4/5, 9/20, 9/20, 9/20, 9/20],
51                                       dtype=float) * 1/6
52
53     print('Gauss Integration '
54           '{0}:\n{1}\nWeights:\n{2}'.format(gauss_points,
55                                               integration_points,
56                                               integration_weights))
57
58     full_domain = np.vstack((domain, integration_points))
59
60     # Shape Functions in Natural Coordinates
61     xi = integration_points.T[0]
62     eta = integration_points.T[1]
63     mu = integration_points.T[2]
64     # zeta is not used to enforce
65     # (1 - xi - eta - mu - zeta) = 0
66     # zeta = integration_points.T[3]

```



```

67  psi = np.zeros((4, integration_points.shape[0]), dtype=float)
68  # another way of writing that psi = [xi, eta, mu, zeta]
69  # for i in range(4):
70  #     psi[i] = (1 + xi * domain[i, 0]) * (1 + eta * domain[i, 1]) * (1 + mu *
    ↪ domain[i, 2]) * (1 + zeta * domain[i, 3])
71  psi[0] = xi
72  psi[1] = eta
73  psi[2] = mu
74  psi[3] = 1 - xi - eta - mu # = zeta
75  psi = psi.T
76  print('Shape Functions: {0}\n{1}'.format(psi.shape, psi))
77
78  # Shape Functions in Global Coordinates
79  psi_g = psi @ coors
80  print('Shape Functions in Global Coordinates: '
81        '{0}\n{1}'.format(psi_g.shape, psi_g))
82
83  # Shape Functions Derivatives in Natural Coordinates
84  dpsi = np.zeros((integration_points.shape[0], 4, 3), dtype=float)
85  dpsi[:,0,0] = 1
86  dpsi[:,1,1] = 1
87  dpsi[:,2,2] = 1
88  dpsi[:,3,:] = -1
89  dpsi = dpsi.transpose((0, 2, 1))
90  print('Shape Functions Derivatives: '
91        '{0}\n{1}'.format(dpsi.shape, dpsi))
92
93  # Jacobian Matrix
94  jacobi = dpsi @ coors
95  print('Jacobian Matrix: {0}\n{1}'.format(jacobi.shape, jacobi))
96
97  # Jacobian Determinants
98  d_jacobi = np.linalg.det(jacobi)
99  print('Determinant of Jacobian: '
100        '{0}\n{1}'.format(d_jacobi.shape, d_jacobi))
101
102  # Inverse Jacobian
103  i_jacobi = np.linalg.inv(jacobi)
104  print('Inverse Jacobian Matrix: '
105        '{0}\n{1}'.format(i_jacobi.shape, i_jacobi))
106

```

```

107     # Shape Function Derivatives in Global Coordinates
108     dpsi_g = i_jacobi @ dpsi
109     print('Shape Function Derivatives in Global Coordinates: '
110           '{0}\n{1}'.format(dpsi_g.shape, dpsi_g))
111
112     # Material Stiffness Matrix
113     C = E / ((1.0 + nu) * (1.0 - 2.0 * nu)) *
114         np.array([[1.0 - nu, nu, nu, 0.0, 0.0, 0.0],
115                 [nu, 1.0 - nu, nu, 0.0, 0.0, 0.0],
116                 [nu, nu, 1.0 - nu, 0.0, 0.0, 0.0],
117                 [0.0, 0.0, 0.0, (1.0 - 2.0 * nu) / 2.0, 0.0, 0.0],
118                 [0.0, 0.0, 0.0, 0.0, (1.0 - 2.0 * nu) / 2.0, 0.0],
119                 [0.0, 0.0, 0.0, 0.0, 0.0, (1.0 - 2.0 * nu) / 2.0]],
120                 dtype=float)
121     print('Material Stiffness Matrix: {0}\n{1}'.format(C.shape, C))
122     du = disp.T @ np.transpose(dpsi_g, axes=[0, 2, 1])
123
124     exx = du[:, 0, 0]
125     eyy = du[:, 1, 1]
126     ezz = du[:, 2, 2]
127     exy = du[:, 0, 1] + du[:, 1, 0]
128     eyz = du[:, 1, 2] + du[:, 2, 1]
129     exz = du[:, 0, 2] + du[:, 2, 0]
130     epsilons = np.array([exx, eyy, ezz, exy, eyz, exz])
131
132     sigmas = (C @ epsilons).T
133     epsilons = epsilons.T
134     print('Element Strains: {0}\n{1}'.format(epsilons.shape, epsilons))
135     print('Element Stresses: {0}\n{1}'.format(stresses.shape, stresses))

```

Chapter 11

Numerics

11.1 Numerical Spaces

11.1.1 Hilbert Space

11.1.2 Sobolev Space

11.1.3 Banach Space

11.1.4 Krylov Space

11.2 Orthogonalisation

11.2.1 Gram-Schmidt process

In linear algebra the **Gram-Schmidt algorithm** is a way of finding two or more vectors perpendicular to each other. It is a method of constructing an

orthonormal basis form a set of vectors in an inner product space, most commonly the Euclidean space $\mathbb{R}^n$ equipped with a standard inner product.

The Gram-Schmidt process takes a finite, *linearly independent* set of vectors $\mathbf{S} = \{\mathbf{v}_1, \dots, \mathbf{v}_k\}$ for $k \leq n$ and generates an **orthogonal** set $\mathbf{S}' = \{\mathbf{u}_1, \dots, \mathbf{u}_n\}$ that spans the same k -dimensional subspace of $\mathbb{R}^n$ as $\mathbf{S}$.

The application of the Gram-Schmidt process to the column vectors of a full column rank matrix yields the **QR decomposition**.

The vector projection of a vector $\mathbf{v}$ onto $\mathbf{u}$ is defined as:

$$proj_{\mathbf{u}}(\mathbf{v}) = \frac{\langle \mathbf{v}, \mathbf{u} \rangle}{\langle \mathbf{u}, \mathbf{u} \rangle} \mathbf{u} \quad (11.1)$$

where $\langle \mathbf{v}, \mathbf{u} \rangle$ denotes the **dot product** of the vectors $\mathbf{u}$ and $\mathbf{v}$. This means that the $proj_{\mathbf{u}}(\mathbf{v})$ is the orthogonal projection of $\mathbf{v}$ onto the line spanned by $\mathbf{u}$. It can be also thought of as the part of the vector $\mathbf{v}$ that is common with vector $\mathbf{u}$. If the two vectors are perpendicular, then the dot product is equal 0.

By subtracting the $proj_{\mathbf{u}}(\mathbf{v})$ from vector $\mathbf{v}$ we obtain a new vector that is perpendicular to the vector $\mathbf{u}$.

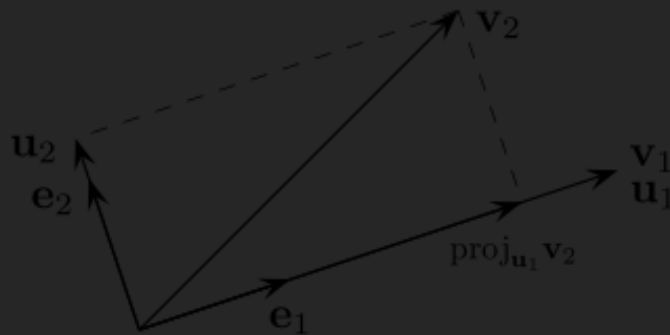


Figure 11.1: first two steps of the Gram-Schmidt process

Given k nonzero linearly independent vectors $\mathbf{v}_1, \dots, \mathbf{v}_k$ the **Gram-Schmidt** process defines the vectors $\mathbf{u}_1, \dots, \mathbf{u}_k$ as follows:

$$\begin{aligned}
\mathbf{u}_1 &= \mathbf{v}_1 & \mathbf{e}_1 &= \frac{\mathbf{u}_1}{\|\mathbf{u}_1\|} \\
\mathbf{u}_2 &= \mathbf{v}_2 - \text{proj}_{\mathbf{u}_1}(\mathbf{v}_2) & \mathbf{e}_2 &= \frac{\mathbf{u}_2}{\|\mathbf{u}_2\|} \\
\mathbf{u}_3 &= \mathbf{v}_3 - \text{proj}_{\mathbf{u}_1}(\mathbf{v}_3) - \text{proj}_{\mathbf{u}_2}(\mathbf{v}_3) & \mathbf{e}_3 &= \frac{\mathbf{u}_3}{\|\mathbf{u}_3\|} \\
&\vdots & & \vdots \\
\mathbf{u}_i &= \mathbf{v}_i - \sum_{j=1}^{i-1} \text{proj}_{\mathbf{u}_j}(\mathbf{v}_i) & \mathbf{e}_i &= \frac{\mathbf{u}_i}{\|\mathbf{u}_i\|}
\end{aligned} \tag{11.2}$$

The sequence $\mathbf{u}_1, \dots, \mathbf{u}_k$ is the required system of orthogonal vectors, and the normalised vectors $\mathbf{e}_1, \dots, \mathbf{e}_k$ form an **orthonormal set**. The calculation of $\mathbf{u}_1, \dots, \mathbf{u}_k$ is known as **Gram-Schmidt orthogonalisation** and the calculation of sequence $\mathbf{e}_1, \dots, \mathbf{e}_k$ is called **Gram-Schmidt orthonormalisation**.

Python implementation:

```

1 import numpy as np
2
3 def GramSchmidtRows(A: np.ndarray, normalise: bool = False):
4     """Perform the Gram-Schmidt algorithm on the rows of matrix A so that all
5     rows are orthogonalised to the preceding rows"""
6
7     # copy the matrix
8     G = A.copy()
9
10    # row to orthogonalise
11    for j in range(1, G.shape[0]):
12        # the predecessors
13        for i in range(j):
14            # unit vector representation of preceding row
15            unit_row = G[i,:] / np.linalg.norm(G[i,:])
16            # projection of current row onto the preceding rows
17            G[j,:] -= unit_row * np.dot(G[j,:], unit_row)
18    if normalise:

```

```

19         G[j,:] /= np.linalg.norm(G[j,:])
20
21     return G
22
23 def GramSchmidtCols(A: np.ndarray, normalise : bool = False):
24     """Perform the Gram-Schmidt algorithm on the columns of matrix A so that all
25     columns are orthogonalised to the preceding rows"""
26
27     # copy the matrix
28     G = A.copy()
29
30     # column to orthogonalise
31     for j in range(1, G.shape[1]):
32         # the predecessors
33         for i in range(j):
34             # unit vector representation of preceding column
35             unit_col = G[:,i] / np.linalg.norm(G[:,i])
36             # projection of current column onto the preceding columns
37             G[:,j] -= unit_col * np.dot(G[:,j], unit_col)
38         if normalise:
39             G[j,:] /= np.linalg.norm(G[j,:])
40
41     return G

```

11.2.2 Modified Gram-Schmidt process

The **Gram-Schmidt orthogonalisation** is prone to **rounding errors** and is therefore considered **numerically unstable**. For the process as described above it is considered particularly bad.

The **Gram-Schmidt process** can be stabilised by a small modification, sometimes referred to as **modified Gram-Schmidt process** or MGS. This approach gives the same result as the original formula in exact arithmetic and introduces smaller errors in finite-precision arithmetic.

Instead of computing the $\mathbf{u}_i$ as:

$$\mathbf{u}_i = \mathbf{v}_i - \sum_{j=1}^{i-1} \text{proj}_{\mathbf{u}_j}(\mathbf{v}_i) \quad (11.3)$$

it is computed as:

$$\begin{aligned} \mathbf{u}_i^{(1)} &= \mathbf{v}_i - \text{proj}_{\mathbf{u}_1}(\mathbf{v}_i) \\ \mathbf{u}_i^{(2)} &= \mathbf{u}_i^{(1)} - \text{proj}_{\mathbf{u}_2}(\mathbf{u}_i^{(1)}) \\ &\vdots \\ \mathbf{u}_i^{(i-1)} &= \mathbf{u}_i^{(i-2)} - \text{proj}_{\mathbf{u}_{i-1}}(\mathbf{u}_i^{(i-2)}) \\ \mathbf{e}_i &= \frac{\mathbf{u}_i^{(i-1)}}{\|\mathbf{u}_i^{(i-1)}\|} \end{aligned} \quad (11.4)$$

Another description would be that the orthogonalisation is performed as a multistep process where in first step all vectors but first are made orthogonal to the first vectors, in the second step all vectors but the first two are made orthogonal to the second vector (already orthogonal to 1st one), and so on until all are processed.

Python implementation:

```

1 import numpy as np
2
3 def ModifiedGramSchmidtRows(A: np.ndarray, normalise: bool = False):
4     """Perform the Modified Gram-Schmidt algorithm on the rows of matrix A
5     so that all rows are orthogonalised to the preceding rows"""
6
7     # copy the matrix
8     G = A.copy()
9
10    # vector to orthogonalise against
11    for i in range(G.shape[0]-1):
12        # simplify dot product

```

```

13     unit_row = G[i,:] / np.linalg.norm(G[i,:])
14     if normalise:
15         G[i,:] = unit_row
16         # all later vectors
17         for j in range(i+1, G.shape[0]):
18             # subtract the projection
19             G[j,:] -= unit_row * np.dot(G[j,:], unit_row)
20
21     return G
22
23 def ModifiedGramSchmidtCols(A: np.ndarray, normalise: bool = False):
24     """Perform the Modified Gram-Schmidt algorithm on the columns of matrix A
25     so that all columns are orthogonalised to the preceding rows"""
26
27     # copy the matrix
28     G = A.copy()
29
30     # vector to orthogonalise against
31     for i in range(G.shape[1]-1):
32         # simplify dot product
33         unit_col = G[:,i] / np.linalg.norm(G[:,i])
34         if normalise:
35             G[:,i] = unit_col
36         # all later vectors
37         for j in range(i+1, G.shape[1]):
38             # subtract the projection
39             G[:,j] -= unit_col * np.dot(G[:,j], unit_col)
40
41     return G

```

11.3 Factorisation

11.3.1 Cholesky Factorisation

Cholesky Factorisation is a matrix operation such that a matrix $\mathbf{A}$ is decomposed to a multiplication of a lower triangular matrix $\mathbf{L}$ and its conjugate transpose $\mathbf{L}^T$.

The **Cholesky Factorisation** is applicable to a **Hermitian, positive definite** matrices.

$$\mathbf{A} = \mathbf{L}\mathbf{L}^* \quad (11.5)$$

Every Hermitian positive definite matrix (and thus every real-valued symmetric positive definite matrix) has a unique Cholesky factorisation.

If $\mathbf{A}$ can be written as $\mathbf{L}\mathbf{L}^*$ for some invertible $\mathbf{L}$, lower triangular or otherwise, then $\mathbf{A}$ is Hermitian and positive definite.

When $\mathbf{A}$ is a real matrix (hence symmetric and positive definite), the factorisation may be written:

$$\mathbf{A} = \mathbf{L}\mathbf{L}^T \quad (11.6)$$

A Python example of *Cholesky-Banachiewicz* factorisation algorithm:

```

1 import numpy as np
2
3 def Cholesky_Banachiewicz(A: np.ndarray) -> np.ndarray:
4     """Perform the Choleski factorisation of matrix A such that:
5
6     A = L @ L.T
7
8     where:
9         L is lower triangular matrix
10
11     Args:
12         A (np.ndarray): a real, symmetric, positive-definite matrix
13
14     Returns:
15         (np.ndarray): the result of the decomposition stored as a lower
16         ↪ triangular
17         matrix.
18     """

```

```

18 L = np.zeros(A.shape, dtype=A.dtype)
19 for i in range(A.shape[0]):
20     for j in range(i+1):
21         sum = 0
22         for k in range(j):
23             sum += L[i,k] * L[j,k]
24
25     if i == j:
26         L[i,j] = np.sqrt(A[i,i] - sum)
27     else:
28         L[i,j] = 1. / L[j,j] * (A[i,j] - sum)
29
30 return L

```

And the solution of matrix equation:

$$\begin{aligned}
 \mathbf{Ax} &= \mathbf{b} \\
 \mathbf{A} &= \mathbf{LL}^T \\
 \mathbf{LL}^T \mathbf{x} &= \mathbf{b} \quad , \text{ substitute } \mathbf{y} = \mathbf{L}^T \mathbf{x} \\
 \mathbf{Ly} &= \mathbf{b} \quad , \text{ solve using forward substitution} \\
 \mathbf{L}^T \mathbf{x} &= \mathbf{y} \quad , \text{ solve using backward substitution}
 \end{aligned}
 \tag{11.7}$$

the $\mathbf{b}$ can be either a vector or a matrix.

Python example:

```

1 import numpy as np
2
3 def Solve_Triangular(A: np.ndarray, b: np.ndarray, kind="lower") -> np.ndarray:
4     """Solve Ax = b using either forward or backward substitution. For lower
5     triangular matrix the forward substitution is used, for upper triangular
6     matrix the backward substitution is used.
7     """
8     b = b.copy()

```

```

9     flatten = False
10    if len(b.shape) == 1:
11        b = b.reshape(-1,1)
12        flatten = True
13
14    m = A.shape[0]
15    n = b.shape[1]
16    x = np.zeros(b.shape, dtype=b.dtype)
17
18    if kind == "lower":
19        for i in range(m):           # rows of A
20            x[i,:] = b[i,:]          # perform operations on whole row of b
21            for j in range(i):       # columns of A from start to diagonal
22                x[i,:] -= A[i,j] * x[j,:] # perform operations on whole row of b
23                x[i,:] /= A[i,i]       # divide by the element on the diagonal
24
25    elif kind == "upper":
26        for i in reversed(range(m)): # rows of A from end
27            x[i,:] = b[i,:]          # perform operations on whole row of b
28            for j in range(i+1, m):  # columns of A from diagonal up to end
29                x[i,:] -= A[i,j] * x[j,:] # perform operations on whole row of b
30                x[i,:] /= A[i,i]       # divide by the element on the diagonal
31
32    else:
33        raise NotImplementedError(f"Substitution for other than 'lower' or
34        ↪ 'upper' triangular not implemented: '{kind:s}'")
35
36    if flatten:
37        return x.flatten()
38    else:
39        return x
40
41 def Cholesky_Solve(L: np.ndarray, b: np.ndarray) -> np.ndarray:
42     """Solve the matrix equation using results of Cholesky decomposition
43
44     Ax = b
45     A = L @ L.T
46     L @ L.T @ x = b
47     L @ z = b
48     L.T @ x = z

```

```
49
50     Args:
51         L (np.ndarray): a lower triangular matrix from Cholesky decomposition
52         b (np.ndarray): right hand side of the equation
53
54     Returns:
55         (np.ndarray): the solution on the left hand side
56     """
57     z = Solve_Triangular(L, b, "lower")
58     x = Solve_Triangular(L.T, z, "upper")
59     return x
60
```

11.3.2 LU Factorisation

LU Factorisation without pivoting

LU Factorisation with row pivoting

LU Factorisation with complete pivoting

11.3.3 QR Factorisation

QR Factorisation using Gram-Schmidt algorithm

QR Factorisation using Givens rotations

QR Factorisation using Householder reflections

QR Factorisation using Householder reflections with column pivoting

11.3.4 Conversion to Hessenberg tridiagonal form using Givens rotations

11.3.5 Conversion to Hessenberg tridiagonal form using Householder reflections

11.3.6 Conversion to Hessenberg tridiagonal form using Jacobi rotations

11.3.7 Jacobi rotations to diagonal matrix

11.3.8 Arnoldi algorithm

11.3.9 Lanczos algorithm

11.3.10 Schur algorithm

11.3.11 Singular Value Decomposition (SVD)

SVD using Power Iteration

SVD using QR (orthogonal) Iteration

11.3.12 Positive semidefinite matrices

of $\mathbf{L}$ are allowed to be zero. The decomposition need not be unique, for example:

$$\begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} = \mathbf{L}\mathbf{L}^* \quad (11.8)$$

$$\mathbf{L} = \begin{bmatrix} 0 & 0 \\ \cos \theta & \sin \theta \end{bmatrix} \quad (11.9)$$

However, if the rank of $\mathbf{A}$ is r , then there is a unique lower triangular $\mathbf{L}$ with exactly r positive diagonal elements and $n - r$ columns containing all zeros.

Alternatively, the decomposition can be made unique when a pivoting choice is fixed. Formally, if $\mathbf{A}$ is an $n \times n$ positive semidefinite matrix of rank r , then there is at least one permutation matrix $\mathbf{P}$ such that $\mathbf{PAP}^T$ has a unique decomposition of the form

$$\mathbf{PAP}^T = \mathbf{L}\mathbf{L}^* \quad (11.10)$$

with

$$\mathbf{L} = \begin{bmatrix} \mathbf{L}_1 & \mathbf{0} \\ \mathbf{L}_2 & \mathbf{0} \end{bmatrix} \quad (11.11)$$

where $\mathbf{L}_1$ is a $r \times r$ lower triangular matrix with positive diagonal.

Example

$$\begin{bmatrix} 4 & 12 & -16 \\ 12 & 37 & -43 \\ -16 & -43 & 98 \end{bmatrix} = \begin{bmatrix} 2 & 0 & 0 \\ 6 & 1 & 0 \\ -8 & 5 & 3 \end{bmatrix} \begin{bmatrix} 2 & 6 & -8 \\ 0 & 1 & 5 \\ 0 & 0 & 3 \end{bmatrix} \quad (11.12)$$

11.3.13 Applications

Solution of a system of linear equations

The Cholesky decomposition is mainly used for the numerical solution of **linear equations** $\mathbf{Ax} = \mathbf{b}$. If $\mathbf{A}$ is symmetric and positive definite, then we can solve $\mathbf{Ax} = \mathbf{b}$ by first computing the Cholesky decomposition $\mathbf{A} = \mathbf{LL}^*$, then solving $\mathbf{Ly} = \mathbf{b}$ for $\mathbf{y}$ by forward substitution and finally solving $\mathbf{L}^*\mathbf{x} = \mathbf{y}$ for $\mathbf{x}$ by back substitution.

$$\begin{aligned}\mathbf{Ax} &= \mathbf{b} \\ \mathbf{LL}^*\mathbf{x} &= \mathbf{b}, \text{ substitute } \mathbf{L}^*\mathbf{x} = \mathbf{y} \\ \mathbf{Ly} &= \mathbf{b}, \text{ then solve} \\ \mathbf{L}^*\mathbf{x} &= \mathbf{y}\end{aligned}\tag{11.13}$$

For linear systems that can be put into symmetric form, the Cholesky decomposition is the method choice, for superior efficiency and numerical stability. Compared to the **LU** decomposition, it is roughly twice as efficient.

11.3.14 Matrix Inversion

A non-Hermitian matrix $\mathbf{B}$ can be inverted using the following identity, where $\mathbf{BB}^*$ will always be Hermitian:

$$\mathbf{B}^{-1} = \mathbf{B}^*(\mathbf{BB}^*)^{-1}\tag{11.14}$$

The above stated matrix relation is also called the **pseudoinverse** of a matrix. The following equality holds:

$$\mathbf{B}^*(\mathbf{BB}^*)^{-1} = (\mathbf{B}^*\mathbf{B})^{-1}\mathbf{B}^*\tag{11.15}$$

The best way to compute the **pseudoinverse** is to use **singular value decomposition (SVD)**. With $\mathbf{A}$ being $\mathbf{A} = \mathbf{USV}^*$, where $\mathbf{U}$ and $\mathbf{V}$ (both $n \times n$)

are orthogonal and $\mathbf{S}$ ($m \times n$) is **diagonal** with real, non-negative *singular values* $\sigma_{i,1} = 1, \dots, n$. We find that

$$\mathbf{A}^* = \mathbf{V}(\mathbf{S}^*\mathbf{S})^{-1}\mathbf{S}^*\mathbf{U}^* \quad (11.16)$$

If the rank r of $\mathbf{A}$ is smaller than n , the inverse of $\mathbf{S}^*\mathbf{S}$ does not exist, and one uses only the first r singular values. $\mathbf{S}$ then becomes an $r \times r$ matrix and $\mathbf{U}$, $\mathbf{V}$ shrink accordingly.

11.4 Linear Regression

11.4.1 Derivation

Linear Regression is an approach for predicting a response using a **single feature**. In linear regression we assume that two variables i. e. independent and dependent are linearly related.

Generalrly we define $\mathbf{x}$ as a **feature vector**:

$$\mathbf{x} = [x_1, x_2, \dots, x_n] \quad (11.17)$$

and $\mathbf{y}$ as **response vector**:

$$\mathbf{y} = [y_1, y_2, \dots, y_n] \quad (11.18)$$

then a predicted response:

$$h(x_i) = \beta_0 + \beta_1 \times x_i \quad (11.19)$$

where:

x_i represents the i -th observation

β_1 represents the slope, and

β_0 represents the intercept of the regression line.

To predict a new value y_i , we need to consider:

$$y_i = \beta_0 + \beta_1 \times x_i + \epsilon_i = h(x_i) + \epsilon_i \quad (11.20)$$

where:

ϵ_i is the **error** of the estimate.

The error function is therefore obtained:

$$\epsilon_i = y_i - h(x_i) \quad (11.21)$$

The β_0 and β_1 coefficients are obtained by minimizing the residual **error** of the estimate using e.g. the **least square method**:

$$\begin{aligned} J(\beta_0, \beta_1) &= \frac{1}{2n} \sum_{i=1}^n \epsilon_i^2 \\ &= \frac{1}{2n} \sum_{i=1}^n (y_i - h(x_i))^2 \\ &= \frac{1}{2n} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 \times x_i)^2 \end{aligned} \quad (11.22)$$

To find the coefficients we can derivate the cost function, first with regards to β_0 :

$$\frac{\partial J}{\partial \beta_0} \left[\frac{1}{2n} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 \times x_i)^2 \right] = 0 \quad (11.23)$$

where using the chain rule we can write:

$$\begin{aligned}
u &= y_i - \beta_0 - \beta_1 \times x_i \\
J &= \frac{1}{2n} \sum_{i=1}^n u^2 \\
\frac{\partial J}{\partial \beta_0} &= \frac{\partial J}{\partial u} \times \frac{\partial u}{\partial \beta_0} \\
\frac{\partial J}{\partial u} &= 2u \\
\frac{\partial u}{\partial \beta_0} &= -1 \\
\frac{\partial J}{\partial \beta_0} &= \frac{1}{2n} \times (-2) \times \sum_{i=1}^n (y_i - \beta_0 - \beta_1 \times x_i) \\
0 &= \frac{1}{n} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 \times x_i) \\
0 &= \frac{1}{n} \sum_{i=1}^n y_i - \frac{1}{n} \sum_{i=1}^n \beta_0 - \frac{1}{n} \beta_1 \sum_{i=1}^n x_i \\
0 &= \bar{y} - \beta_0 - \beta_1 \bar{x} \\
\beta_0 &= \bar{y} - \beta_1 \bar{x}
\end{aligned} \tag{11.24}$$

To find β_1 :

$$\frac{\partial J}{\partial \beta_1} \left[\frac{1}{2n} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 \times x_i)^2 \right] = 0 \tag{11.25}$$

$$\begin{aligned}
u &= y_i - \beta_0 - \beta_1 x_i \\
J &= \frac{1}{2n} \sum_{i=1}^n u^2 \\
\frac{\partial J}{\partial \beta_1} &= \frac{\partial J}{\partial u} \times \frac{\partial u}{\partial \beta_1} \\
\frac{\partial J}{\partial u} &= 2u \\
\frac{\partial u}{\partial \beta_1} &= -x_i \\
\frac{\partial J}{\partial \beta_1} &= \frac{1}{2n} \times (-2) \sum_{i=1}^n x_i \times (y_i - \beta_0 - \beta_1 x_i) \\
0 &= \frac{1}{n} \sum_{i=1}^n x_i (y_i - \beta_0 - \beta_1 x_i) \\
0 &= \frac{1}{n} \sum_{i=1}^n (x_i y_i - \beta_0 x_i - \beta_1 x_i^2)
\end{aligned} \tag{11.26}$$

Substituting the value of β_0 :

$$\begin{aligned}
\beta_0 &= \bar{y} - \beta_1 \bar{x} \\
0 &= \frac{1}{n} \sum_{i=1}^n (x_i y_i - (\bar{y} - \beta_1 \bar{x}) x_i - \beta_1 x_i^2) \\
0 &= \frac{1}{n} \sum_{i=1}^n (x_i y_i - \bar{y} x_i + \beta_1 \bar{x} x_i - \beta_1 x_i^2) \\
0 &= \frac{1}{n} \left(\sum_{i=1}^n (x_i y_i - \bar{y} x_i) - \beta_1 \sum_{i=1}^n (x_i^2 - \bar{x} x_i) \right) \\
\beta_1 &= \frac{\frac{1}{n} \sum_{i=1}^n (x_i y_i - \bar{y} x_i)}{\frac{1}{n} \sum_{i=1}^n (x_i^2 - \bar{x} x_i)}
\end{aligned} \tag{11.27}$$

Simplifying the formula using:

$$\begin{aligned}\bar{y} &= \frac{1}{n} \sum_{i=1}^n y_i \\ \bar{x} &= \frac{1}{n} \sum_{i=1}^n x_i\end{aligned}\tag{11.28}$$

we get:

$$\begin{aligned}\beta_1 &= \frac{\frac{1}{n} \sum_{i=1}^n x_i y_i - \bar{x} \bar{y}}{\frac{1}{n} \sum_{i=1}^n x_i^2 - \bar{x}^2} \\ \beta_1 &= \frac{\sum_{i=1}^n x_i y_i - n \bar{x} \bar{y}}{\sum_{i=1}^n x_i^2 - n \bar{x}^2}\end{aligned}\tag{11.29}$$

11.4.2 Result

So, to recapitulate, to get a linear regression coefficients for a dataset:

$$\begin{aligned}\mathbf{x} &= [x_1, x_2, \dots, x_n] \\ \mathbf{y} &= [y_1, y_2, \dots, y_n]\end{aligned}\tag{11.30}$$

we may use equations:

$$\begin{aligned}\beta_1 &= \frac{\sum_{i=1}^n x_i y_i - n \bar{x} \bar{y}}{\sum_{i=1}^n x_i^2 - n \bar{x}^2} \\ \beta_0 &= \bar{y} - \beta_1 \bar{x}\end{aligned}\tag{11.31}$$

with an error:

$$\epsilon_i = y_i - \beta_0 - \beta_1 x_i\tag{11.32}$$

- 11.5 Interpolation and Mapping of Values
- 11.6 Modal Assurance Criterion (MAC)
- 11.7 Campbell Diagram, Order Analysis
- 11.8 Power Spectral Density, Cross Spectral Density and Coherence
- 11.9 Sorting
- 11.10 Reverse Cuthill-McKee Algorithm for rearranging Sparse matrix
- 11.11 Topology Optimization
- 11.12 Generalised Eigenvalue Problem

Chapter 12

Miscellaneous

12.1 Bolt

12.1.1 Bolt pretension

The relation between the **tightening moment** and bolt **pretension force** can be characterised by the following equation:

$$M_A = F_V \left[\frac{d_2}{2} \tan(\varphi^\circ + \rho') + \mu_K \frac{d_{K,R}}{2} \right] \quad (12.1)$$

with:

$$d_{K,R} = \frac{d_k + d_i}{2} \quad (12.2)$$

$$\varphi^\circ = \arctan \left(\frac{P}{d_2 * \pi} \right) \quad (12.3)$$

$$\rho' = \arctan \left(\frac{\mu_G}{\cos \frac{\beta}{2}} \right) \quad (12.4)$$

at standard thread angle of $\beta = 60^\circ$:

$$\rho' = \arctan (\mu_G * 1.155) \quad (12.5)$$

where:

M_A ... Tightening Moment [Nmm]

F_V ... Pretension Force [N]

d_2 ... Bolt thread midiameter ((core + outer diameter) / 2) [mm]

ρ' ... Bolt thread friction angle [$^\circ$]

φ° ... Bolt thread angle of rising (pitch) [$^\circ$]

P ... Bolt thread pitch [mm]

μ_G ... Thread friction coefficient [-]

μ_K ... Friction coefficient of bolt head [-]

β ... Thread angle (angle between 'teeth') [$^\circ$]

$d_{K,R}$... Middle diameter of friction surface under bolt head [mm]

d_K ... Outer diameter of surface under bolt head [mm]

d_i ... Diameter of the hole in the material [mm]